

Universitatea
Politehnica
Bucuresti
Anul universitar
2022-2023

Adina Magda
Florea

Inteligență Artificială

Curs 1 - Introducere

- Organizare curs
- Conținut curs
- Istoric IA
- IA în prezent

Echipă

- AI-MAS Laboratory
- <https://aimas.cs.pub.ro/>



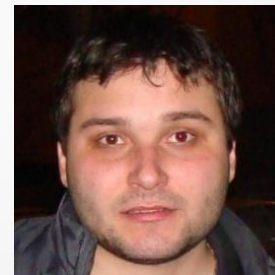
Ștefania Ghiță



Andrei Olaru



Alexandru Sorici



Mihai Trăscău



Mihai Nan



Andrei Dugășescu



Vlad Florea

Cerințe pentru promovare

- Participare la laborator: minimum 7 sedințe de laborator
- Minim 50% din punctajul de parcurs (laborator+teme de casă) și minim 50% din punctajul de la examenul final
 - Activitatea la laborator
 - Rezolvarea temelor de casă
 - Parcurgerea materialelor obligatorii
- Examen final

Notare

- Examen final: 45%
- Laborator 25%
- Teme de casă 30%
- Teste curs – bonus 10%

Materiale curs

- Slide-uri curs, Conținut laboratoare – vor fi disponibile pe Moodle
- S. Russell, P. Norvig. Artificial Intelligence: A Modern Approach, Prentice Hall, 2021 (Global Edition), <http://aima.cs.berkeley.edu/global-index.html>
- D. Poole, A. Mackworth. *Artificial Intelligence: Foundations of Computational Agents*, Cambridge University Press, 2010 – complete book online <http://artint.info/html/ArtInt.html>

Resurse AI on-line

- Tools for learning AI

<http://www.aispace.org/index.shtml>

- Direcțiile de cercetare din IA

<http://aitopics.org/>

- MIT Open Courseware

<http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-034-artificial-intelligence-fall-2010/>

- Udacity

<https://www.udacity.com/course/intro-to-artificial-intelligence--cs271>

- Diferite subiecte de IA – Coursera

<https://www.coursera.org/search?query=artificial+intelligence&categories=cs-ai>

Codul de Etica al ACS

- <https://wiki.cs.pub.ro/studenti/regulament-de-etica-al-acs>

Integritate academică

- Copiatul la examene sau la alte lucrări scrise;
- Plagiul lucrărilor realizate de alți autori, inclusiv autori anonimi sau colegi, indiferent de mediul în care acestea au fost făcute publice sau de permisiunea autorilor;
- Predarea sub semnătură proprie a unor teme sau lucrări cumpărate;
- Predarea sub semnătură proprie a unor teme sau lucrări realizate cu sprijinul unor persoane neautorizate;
- Refolosirea temelor (auto-plagiul) fără acordul explicit al profesorului;
- Falsificarea datelor sau a rezultatelor în cadrul proiectelor;
- Facilitarea oricăror dintre acțiunile de mai sus, pentru alți studenți.

Intrebări cheie

**Este posibilă simularea
comportamentului inteligent
pe calculator?**

**La ce nivel se încearca
modelarea comportamentului
inteligent?**

**Care sunt reprezentările și
tehnicile utilizate în rezolvarea
problemelor de inteligență
artificială?**

Model

- Problemele reale sunt complexe
- Trebuie să modelăm o problemă, care poate fi complexă și cu elemente incerte într-un program

Lume reală



Model



Program

Paradigmă

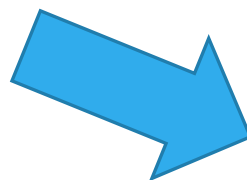
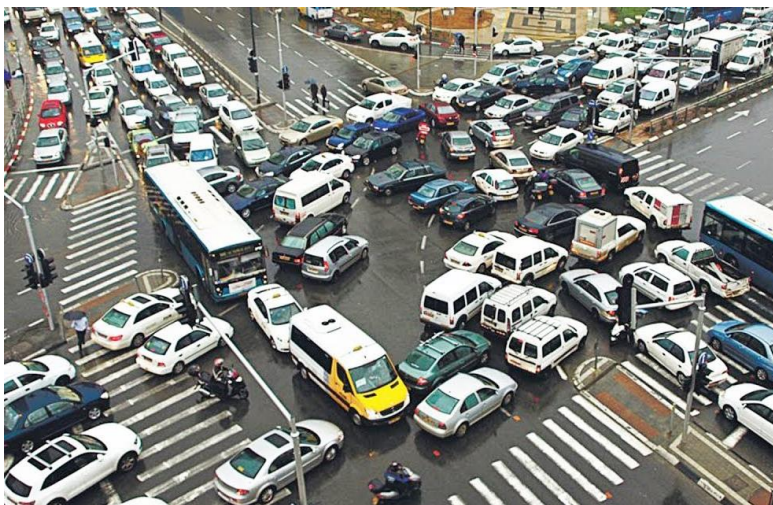
Modelare

Inferență

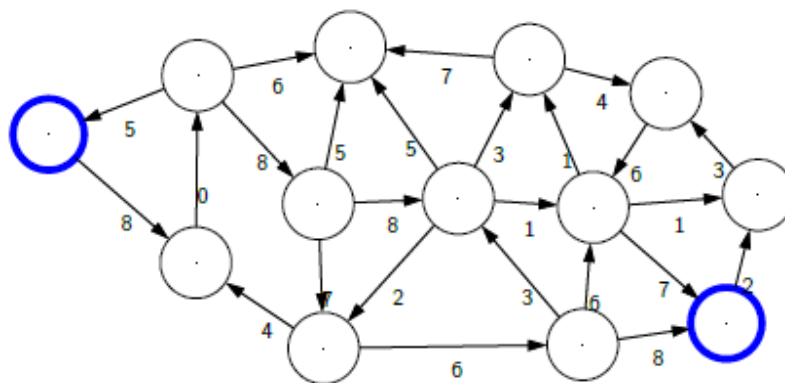
Învățare

Modelare

Problemă reală

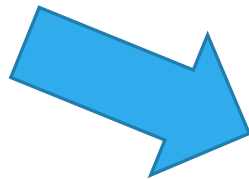
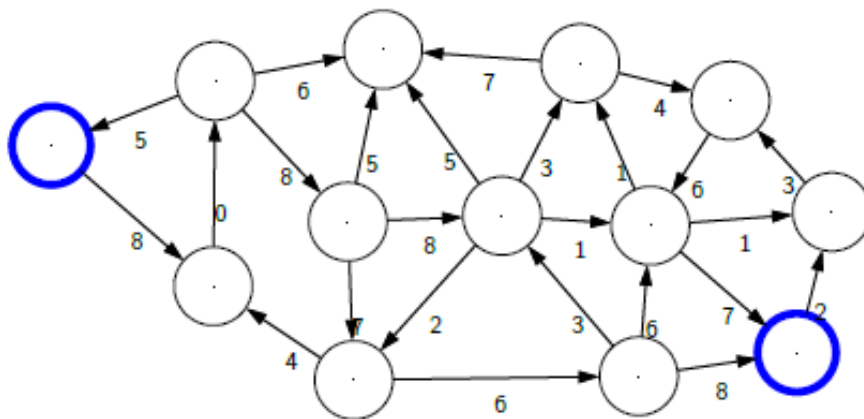


Model

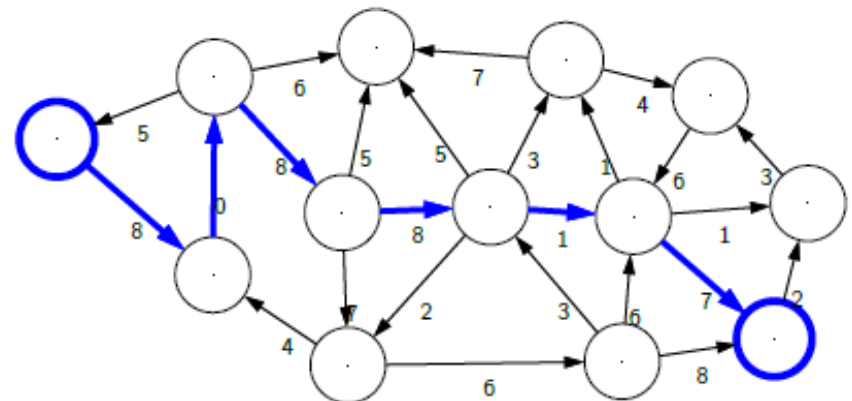


Inferență

Model

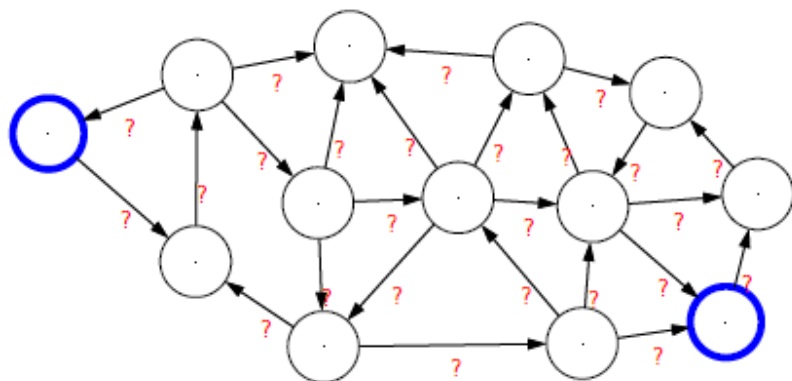


Soluție



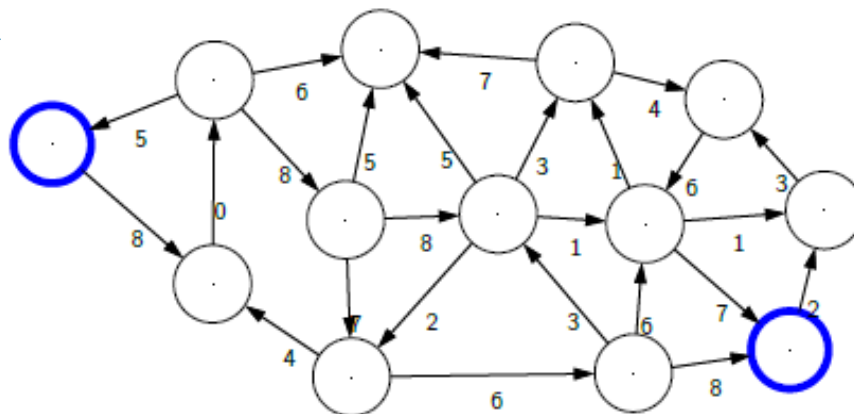
Învățare

Model fără parametri



+ date

Model cu parametri + predicție



Modele sub-simbolice

- Clasificatori liniari
- Rețele neurale
- Modele statistice
- Algoritmi genetici

Modele simbolice

- **Modele cu stări**
 - Căutări
 - Jocuri adversariale

Modele simbolice

- **Modele cu stări / variabile**

- CSP
- Rețele Bayesiene
- Modele Markov

Raționament incert

Modele simbolice

- **Modele bazate pe cunoștințe**
 - Logică simbolică
 - Ontologii

Conținut curs

- **Rezolvarea problemelor prin căutare**
 - Căutări informate
 - Problema satisfacerii restricțiilor
 - Căutare adversarială în jocuri
- **Raționament incert**
 - Cunoștințe incerte și raționament probabilistic
 - Rețele Bayesiene
 - Naive Bayes
- **Introducere în învățare automată**
 - Arbori de decizie
 - Regresie liniară
 - Regresile logistice
 - Rețele neurale
- **Prelucrarea limbajului natural**
- **Agenți și sisteme multi-agent**
- **Logică și sisteme bazate pe reguli**

Scurt istoric



LIX. No. 236.]

[October, 1950

M I N D
A QUARTERLY REVIEW
OF
PSYCHOLOGY AND PHILOSOPHY



I.—COMPUTING MACHINERY AND
INTELLIGENCE

BY A. M. TURING

1. *The Imitation Game.*

I PROPOSE to consider the question, 'Can machines think?' This should begin with definitions of the meaning of the terms 'machine' and 'think'. The definitions might be framed so as to

IA - Începuturi

- Conferința de la Dartmouth College din 1956 - primii patru mari inițiatori ai domeniului: John McCarthy, Marvin Minsky, Alen Newell si Herbert Simon.
- 1956 - 1957 A. Newell, J. Shaw si H. Simon - primul program de demonstrare automată a teoremelor, "The Logic Theorist."
- 1959 Samuel Checkers program
- Începand din 1960 apar primele programe de inteligență artificială.

IA – Inferențe și limbaje dedicate

- 1965 J. A. Robinson – rezoluția
- 1965 – DENDRAL - J. Lederberg și E. Feigenbaum. - sistem expert capabil să sintetizeze structura moleculelor organice pe baza formulelor chimice și a spectogramelor de masă
- 1959 - Limbajul Lisp (LISt Processing) - John McCarthy (Dartmouth College)
- 1972 - Limbajul Prolog (PROgrammation et LOGique) - Alain Colmerauer (universitatea Marseille-Aix)
- 1983 - Smalltalk - Goldberg, Robson

IA – Cunoștințe simbolice

- Anii '70 – importanța cunoștințelor
- **Sisteme bazate pe cunoștințe**
- **Ingineria cunoștințelor**
- Sistemul MYCIN - Buchanan, Shortliffe - sistem expert pentru diagnosticarea infecțiilor bacteriene ale sângelui, Stanford University - '74-'75
- Sisteme expert, bazate pe reguli

IA - Avânt

- Anii '80-'90 – dezamăgire
- Anii '90-'00 – relansare IA
- **IA distribuită**
- **Agenți inteligenți**
- **Sisteme multi-agent**
- Anii '90-'00 - apariția învățării profunde
- **Anii 2010 – 2021-** din ce în ce mai multe sisteme sau componente – inteligente

Modele preluate de IA

- 1801: Regresie liniară (Gauss, Legendre)
- 1936: Clasificare liniară (Fisher)
- 1956: Căutare de cost uniform (Dijkstra)
- 1957: Procese de decizie Markov (Bellman)
- 1985: Bayesian networks (Pearl)
- 1995: Support vector machines (Cortes/Vapnik)

IA – Învățare automată

Deep learning

- 1989 – CNN cu backpropagation (Yann LeCun)
- 1997 – LSTM (Sepp Hochreiter and Jürgen Schmidhuber)
- 2006 – Deep Belief Networks (Geoffrey Hinton, Ruslan Salakhutdinov)
- 2008 – use of GPU pentru DNN
- 2009 – ImageNet (Fei-Fei Li)

Deep learning

- 2011 Andrew Ng, Google Brain -Rețea neurală antrenată cu algoritmi deep learning – recunoașterea conceptelor cum ar fi pisica pe baza video din YouTube
- Yann LeCun, Facebook - deep learning pentru identificarea fețelor și a obiectelor în 350 milioane de fotografii încărcate pe Facebook
- 2012 – AlexNet – CNN pt ImageNet (Alex Krizhevsky)
- 2014 – GAN (Ian Goodfellow)

Actual

- IA este un amplificator al aplicațiilor
- IA omniprezentă
- Implicații pozitive și negative

Chat bots

A. L. I. C. E. The Artificial Linguistic Internet Computer Entity



Chatbots

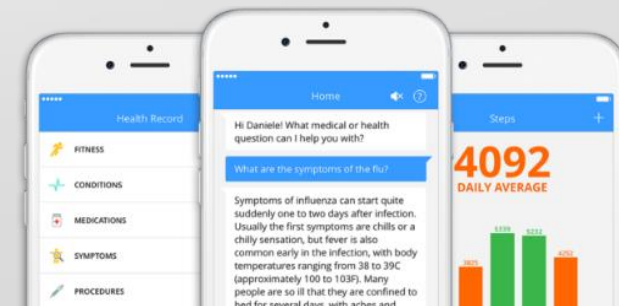
Endurance: A Companion for Dementia Patients

Casper: Helping Insomniacs Get Through the Night

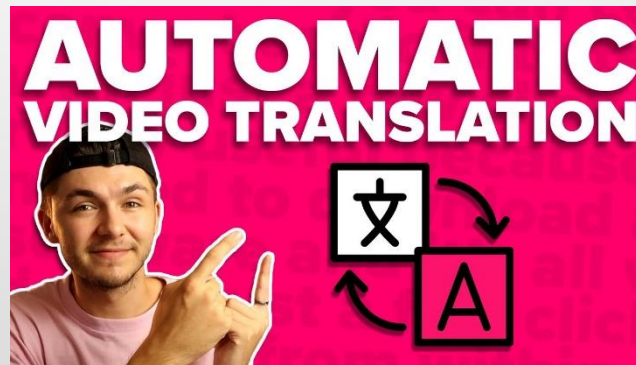
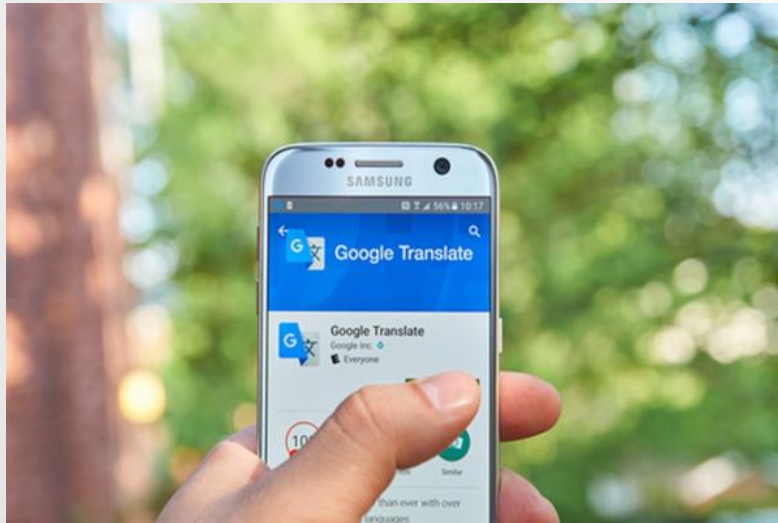
Disney: Solving Crimes with Fictional Characters

UNICEF: Helping Marginalized Communities Be Heard

MedWhat: Making Medical Diagnoses Faster



Traducere automată a textelor



Jocuri

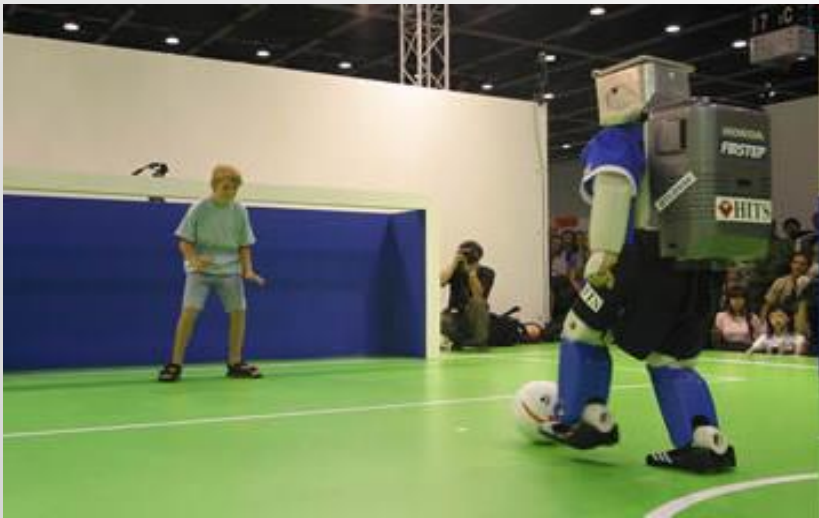
- In 1997 super-computerul *Deep Blue* l-a învins pe campionul de șah Gary Kasparov.
- Actual, calculatoarele pot juca șah mai bine decât orice campion



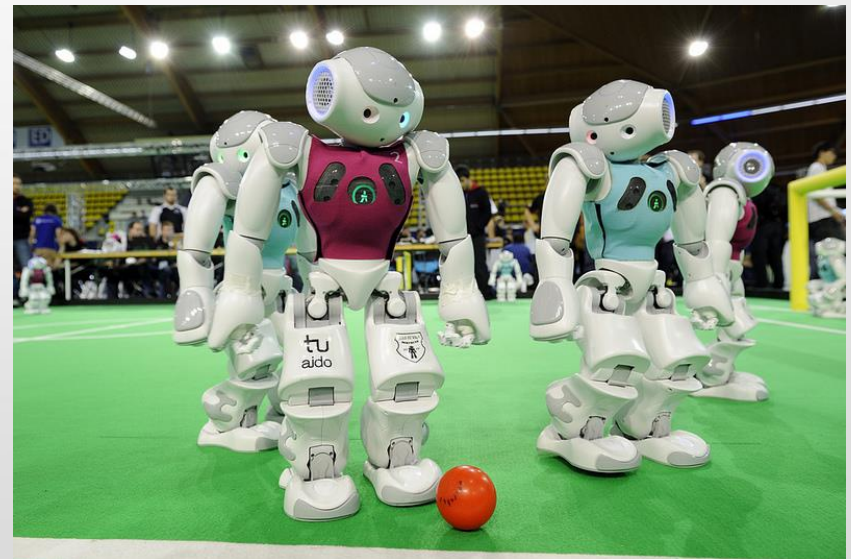
- 2016 – AlphaGo (DeepMind) – învinge un campion profesional al jocului de Go (Lee Sedol)



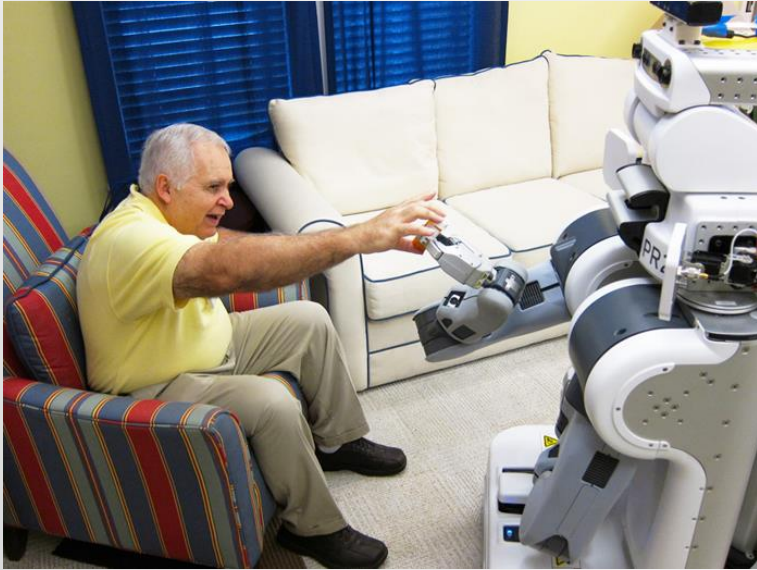
Roboți



**RoboCup Can football-playing robots
beat the
World Cup winners by 2050?**
<https://www.robocup.org/>



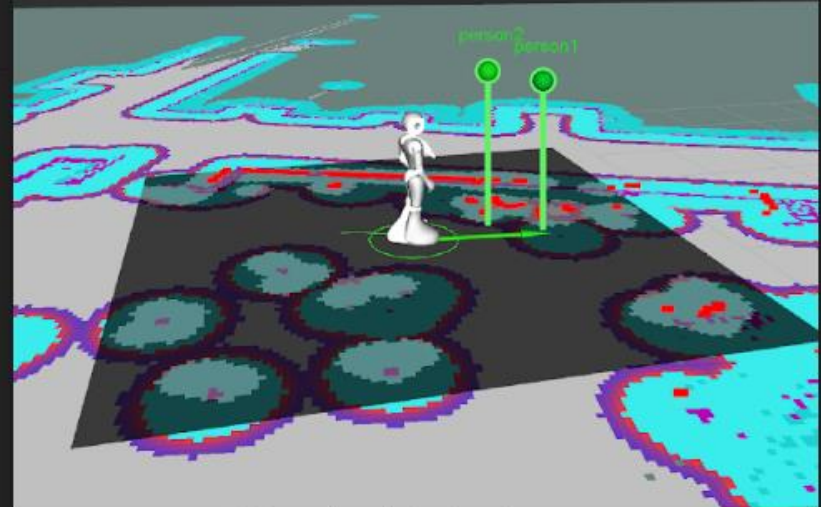
Roboți asistivi



a). Map of the environment.

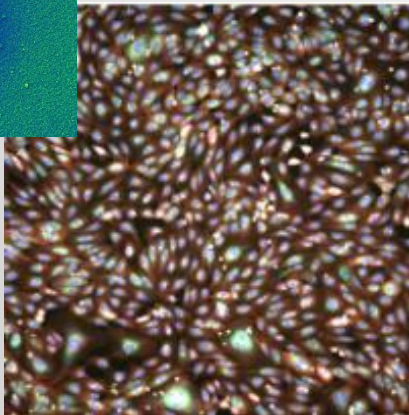
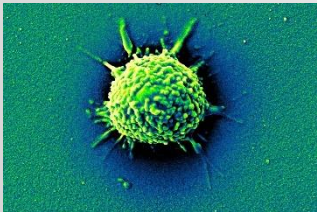


b). Detections in RGB image.

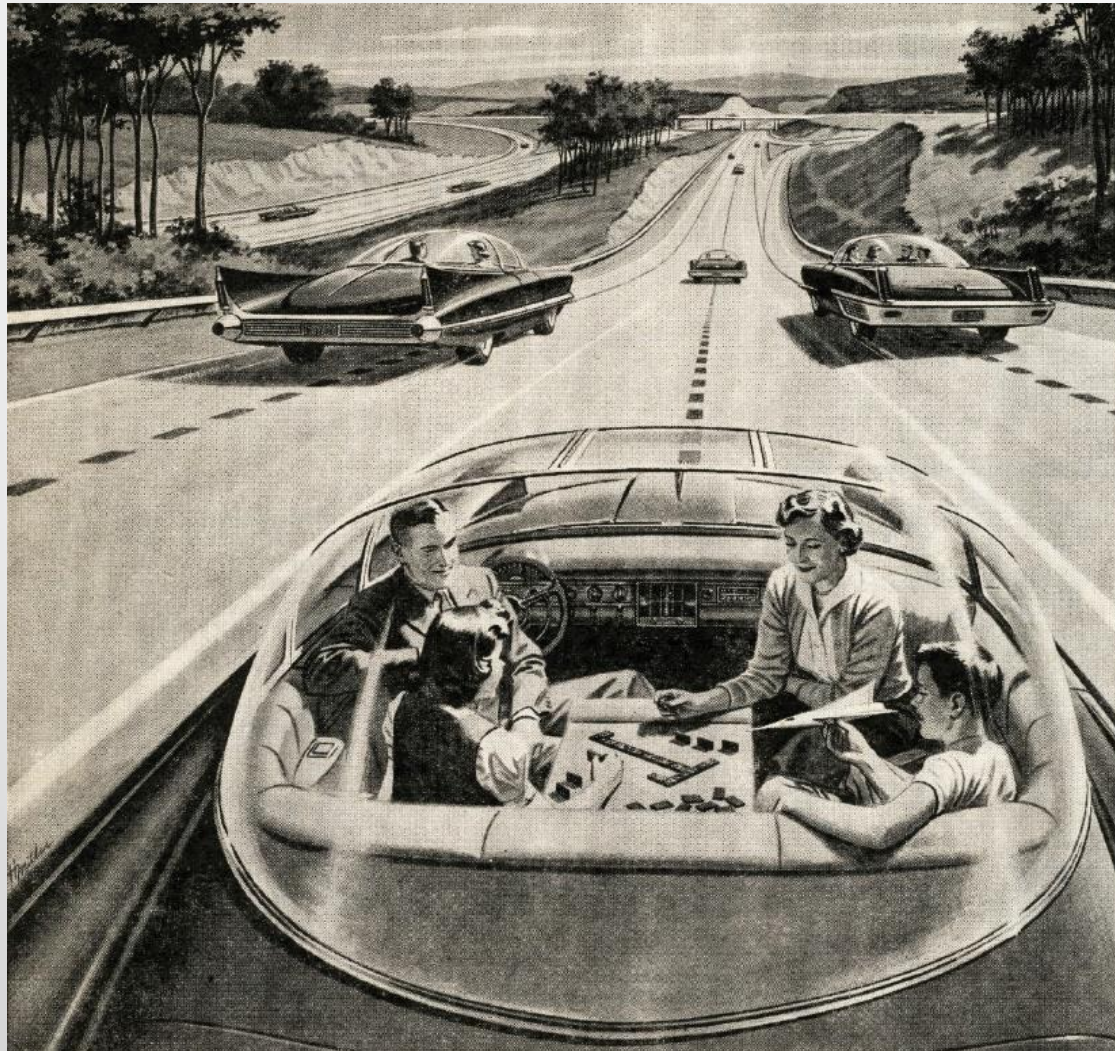


c). Detected people placed on the map.

Medicină



Maşini autonome



DARPA Grand Challenge 2004, 2005

DARPA Urban Challenge 2007



2012 – DARPA Robotics Challenge

2018-2021 – DARPA Subterranean Challenge

General Motors, Uber, Waymo, Toyota, Honda, Tesla

Waymo – 20 mil mile

General Motor Cruise – ½ mil mile

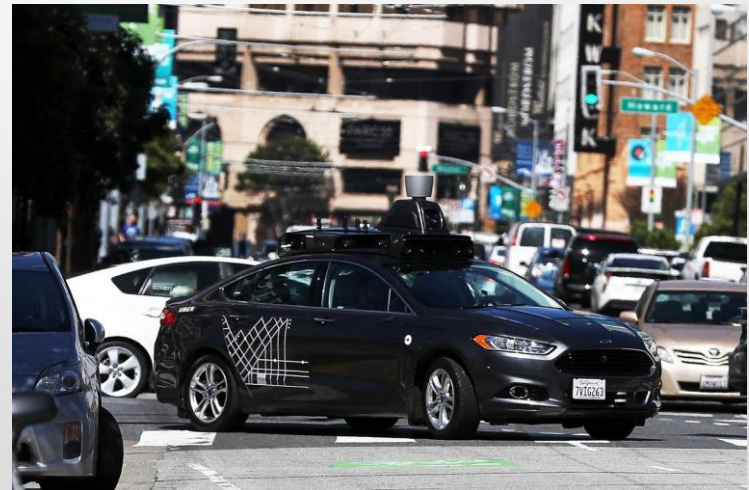
Accidente 2018: Uber, Tesla



Waymo, 2017



Google, 2015



Uber, San Francisco, 2017

Nvidia self-driving chip Xavier

General Motors Cruise

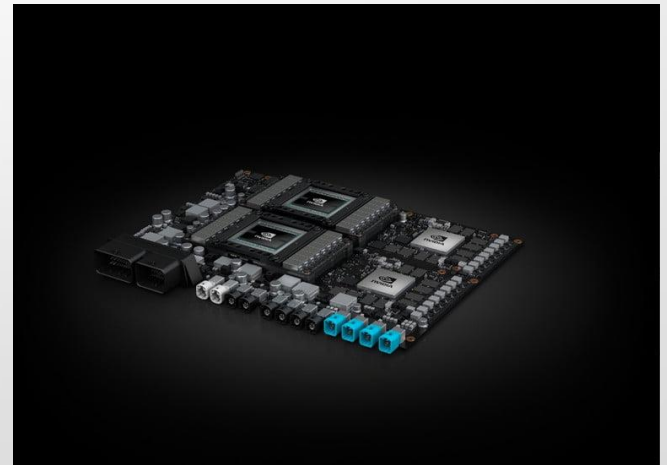
MIT + Stanford + Toyota

Volkswagen

Bosch + Daimler



General Motors' Cruise, San Francisco 2020



Xavier, 2018

Riscuri

- Consum mare de energie

Common carbon footprint benchmarks

in lbs of CO2 equivalent

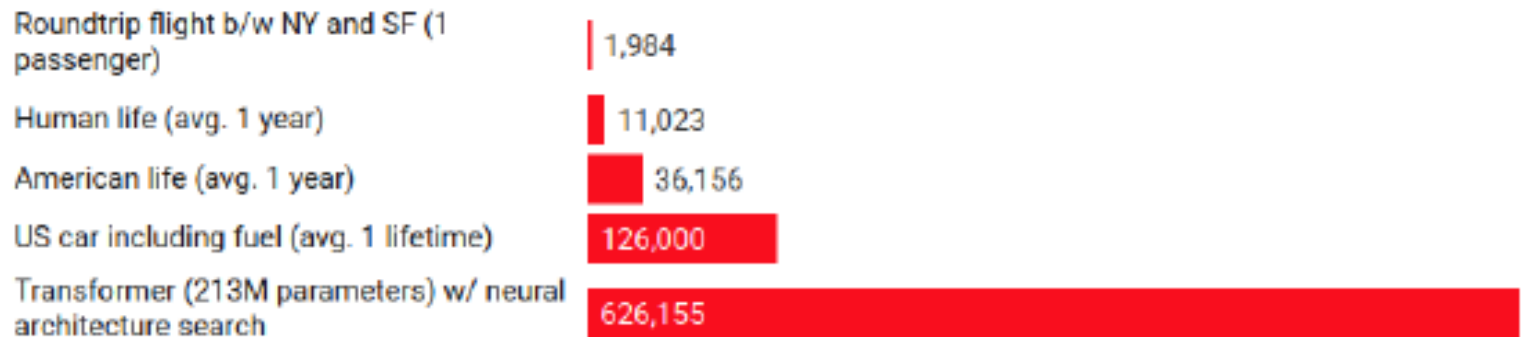
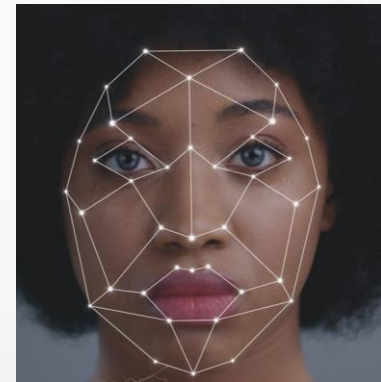


Chart: MIT Technology Review • Source: Strubell et al. • Created with Datawrapper

Riscuri

- Data bias
- Deep fakes



AutoDrive, UPB, AIMAS

https://drive.google.com/drive/folders/1puo_naqyv61Rx7jLTekZeuORE3LPuSww

Inteligență Artificială

Universitatea Politehnica Bucuresti
Anul universitar 2022-2023

Adina Magda Florea

Curs 2

Strategii de căutare

- Reprezentarea soluției problemei
- Strategii de căutare de bază
- Strategii de căutare informate
- Strategii de căutare informate cu memorie limitată
- Determinarea funcției euristice

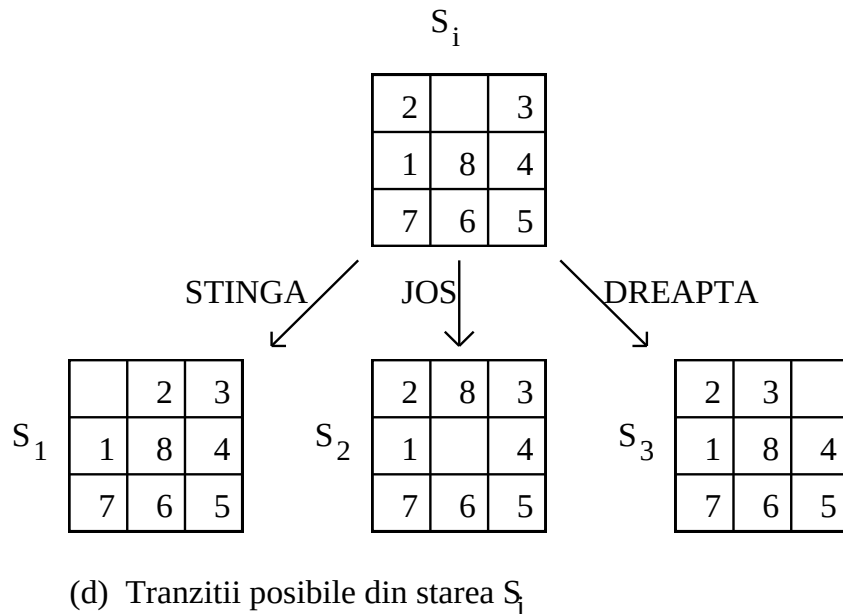
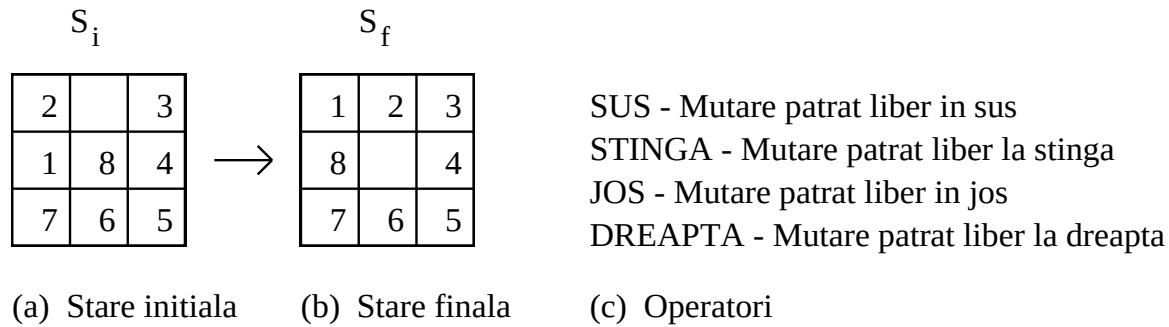
1 Reprezentarea soluției problemei

- Grafuri reprezentate explicit
- Grafuri reprezentate implicit
- Graful asociat unei probleme de cautare
- Graf neponderat sau graf ponderat

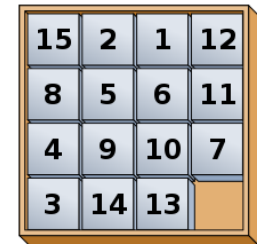
Reprezentarea soluției problemei

- Reprezentare prin spatiul starilor
- (S_i, O, S_f)
- Stare
- Spatiu de stari (S_Set)
- Stare initiala
- Stare/stari finala/finale
- Solutia problemei

8-puzzle

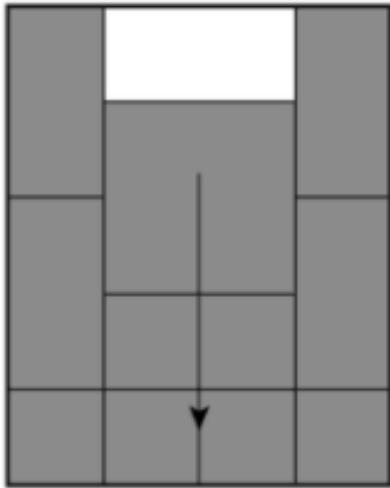


15-puzzle

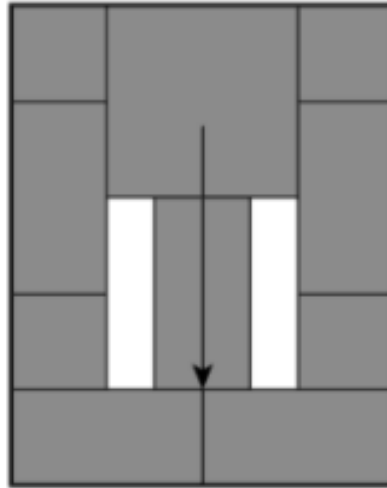


(n^2-1) -puzzle

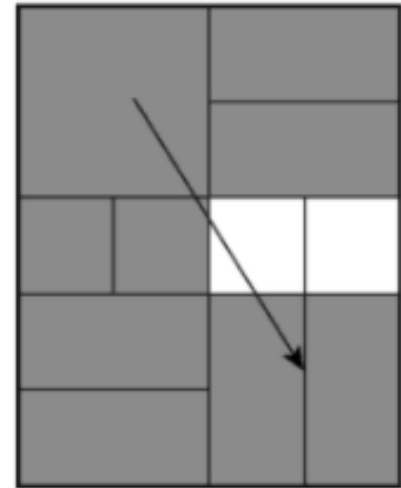
General sliding tile puzzle



Donkey' puzzle



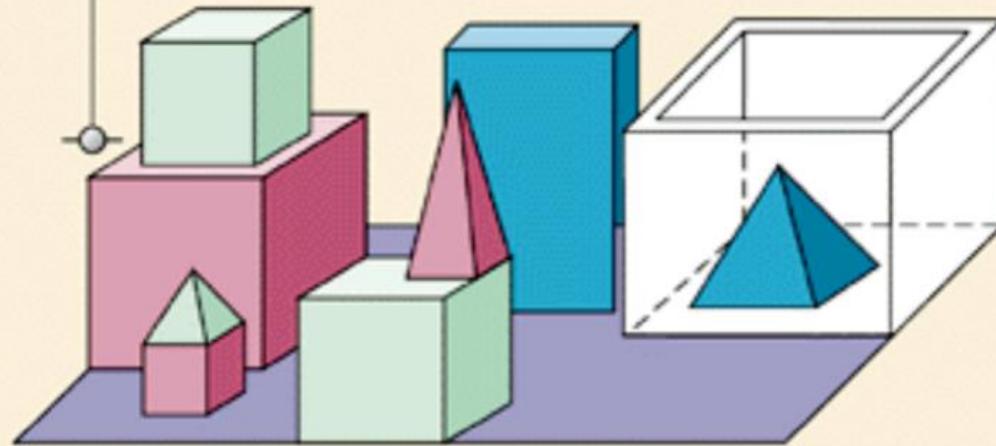
Century puzzle



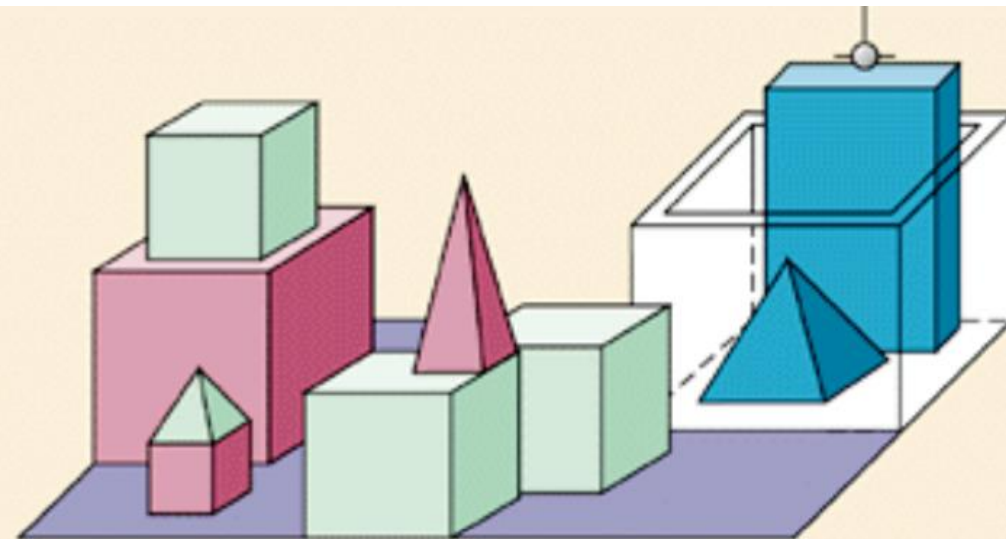
Dad' puzzle

gripper

Block world



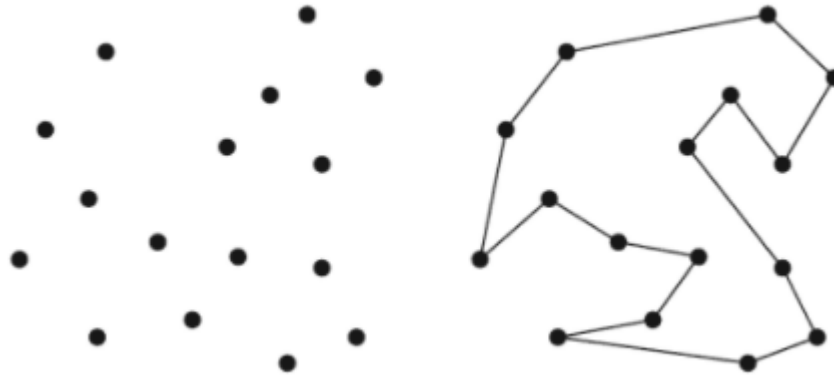
(a) "Pick up a big red block."



(b) "Find a block which is taller than the one you are holding and put it into the box."

Probleme cu cost

- Planificarea rutelor
- Problema comisvoiajorului



Caracteristicile mediului

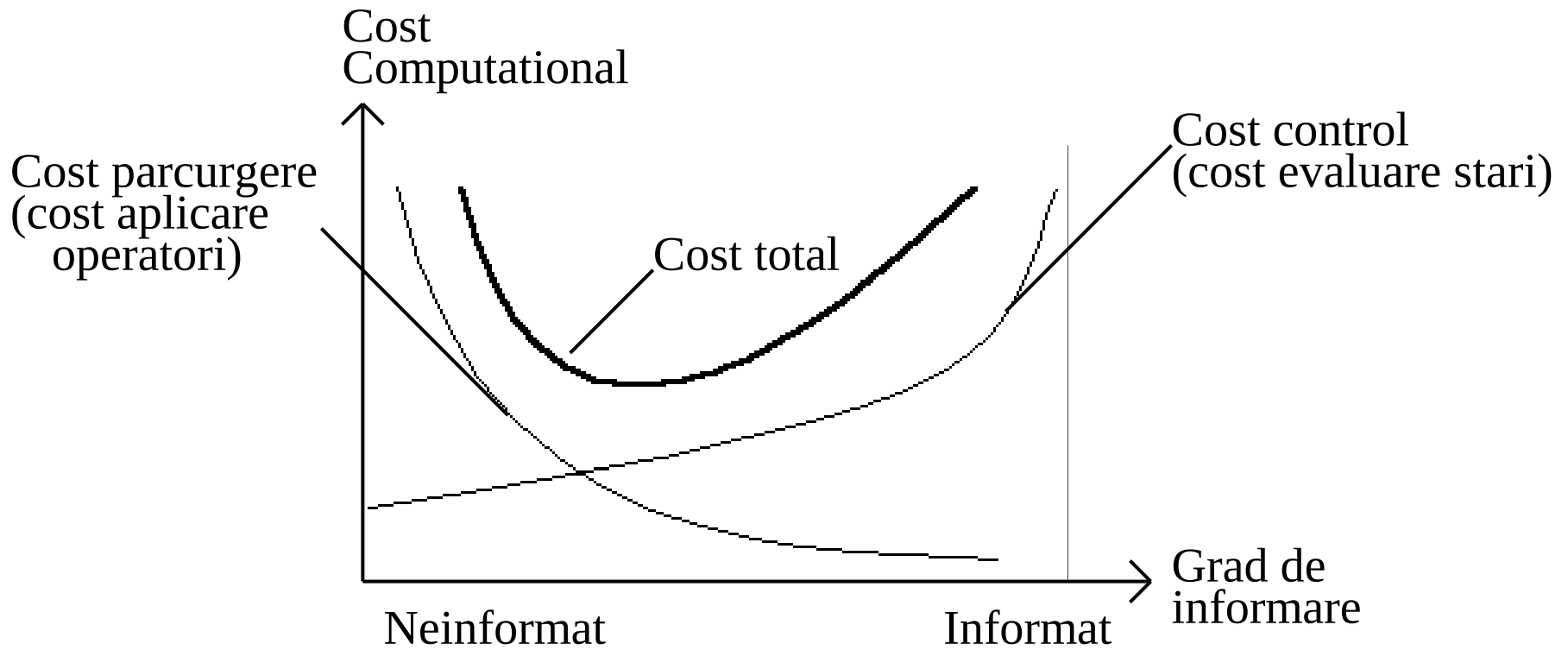
- Observabil / neobservabil
- Discret / continuu
- Finit / infinit
- Determinist / nedeterminist

2. Strategii de căutare de bază

Criterii de caracterizare

- Completitudine
- Optimalitate
- Complexitate: timp, spatiu
- Capacitatea de revenire
- Informare

Costuri ale cautarii



Căutari neinformate

- Gasirea unei cai sau a tuturor cailor, cu cost sau fara cost

Grafuri specificate explicit

- **Cautarea pe nivel** si **cautarea in adancime** - parcurgerea se face in ordinea nodurilor succesoare starii curente in cautare:
 - Cautarea pe nivel – cele mai apropiate intai
 - Cautarea in adancime – cele mai departate intai
- **Algoritmul lui Dijkstra** rezolva problema cai de cost minim daca toate costurile arcelor sunt ≥ 0
- **Algoritmul Bellman-Ford** rezolva problema cai de cost minim dar costurile arcelor pot fi si negative (de la un nod sursa la toate)
- **Algoritmul Floyd-Warshall** rezolva problema gasirii tuturor cailor de cost minim, costuri pozitive si negative dar cicluri ne-negative

Grafuri specificate implicit (probleme de cautare)

- Cautarea pe nivel (Breadth First Search - BFS)
- Cautare in adancime (Depth First Search - DFS)
- Cautare in adancime cu nivel iterativ (Iterative Deepening - ID)
- Cautare de tip backtracking
- Cautare bidirectionala

Tratarea nodurilor

- Intr-o reprezentare a solutiei problemei prin spatiul starilor *adancimea unui nod* se defineste astfel:
 - $Ad(S_i) = 0$, unde S_i este nodul stare initiala,
 - $Ad(S) = Ad(S_p) + 1$, unde S_p este nodul predecesor nodului S .
- Cele mai multe implementari bazate pe utilizarea a 2 liste: OPEN si CLOSED
- DFS – Open – stiva (LIFO)
- BFS – Open – coada (FIFO)
- In cele mai multe implementari CLOSED este implementata ca o tabela de dispersie (Hash)

Algoritm BFS / DFS(AdMax): Cautare pe nivel/in adancime in spatiul starilor

Intrari: Starea initiala S_i

Iesiri: SUCCES si solutia sau INSUCCES

1. Initializeaza listele $OPEN \leftarrow \{S_i\}$, $CLOSED \leftarrow \{\}$
2. **daca** $OPEN = \{\}$
 atunci intoarce INSUCCES
3. Elimina primul nod S din $OPEN$ si insereaza-l in $CLOSED$
4. **daca** $S \in OPEN \cup CLOSED$ (inainte de inserare S)
 atunci repeta de la 2
- 4'. **daca** $Ad(S) = AdMax$ **atunci repeta de al 2** /* pt DFS */
5. Expandeaza nodul S
 - 5.1. Genereaza toti succesorii directi S_j ai nodului S
 - 5.2. **pentru** fiecare succesor S_j al lui S **executa**
 - 5.2.1. Stabileste legatura $S_j \rightarrow S$
 - 5.2.2. **daca** S_j este stare finala
 atunci
 - i. Solutia este $(S_j, \dots, S_i)$
 - ii. **intoarce** SUCCES
 - 5.2.3. Insereaza S_j in $OPEN$, **la sfarsit / la inceput**
6. **repeta de la 2**
sfarsit.

Algoritm DFS(AdMax): Cautare in adancime cu adancime limitata

Intrari: Starea initiala S_i

Iesiri: SUCCES si solutia sau INSUCCES sau AdMax

1. Initializeaza lista OPEN $\leftarrow \{S_i\}$,
 2. **daca** OPEN = {} **atunci intoarce** INSUCCES
 3. Elimina primul nod S din OPEN
 4. **daca** S este stare finala
atunci i. Solutia este (S,..., S_i)
ii. **intoarce** SUCCES
 5. **daca** Ad(S) > AdMax **atunci intoarece** AdMax
 6. Expandeaza nodul S
 - 6.1. Genereaza toti succesorii directi S_j ai nodului S
 - 6.2. **pentru** fiecare succesor S_j al lui S **executa**
 - 6.2.1 Stabileste legatura $S_j \rightarrow S$
 - 6.2.2. **daca** $S_j \notin$ OPEN
atunci introduce S_j in OPEN la inceput
 7. **repetă de la 2**
- sfarsit.**

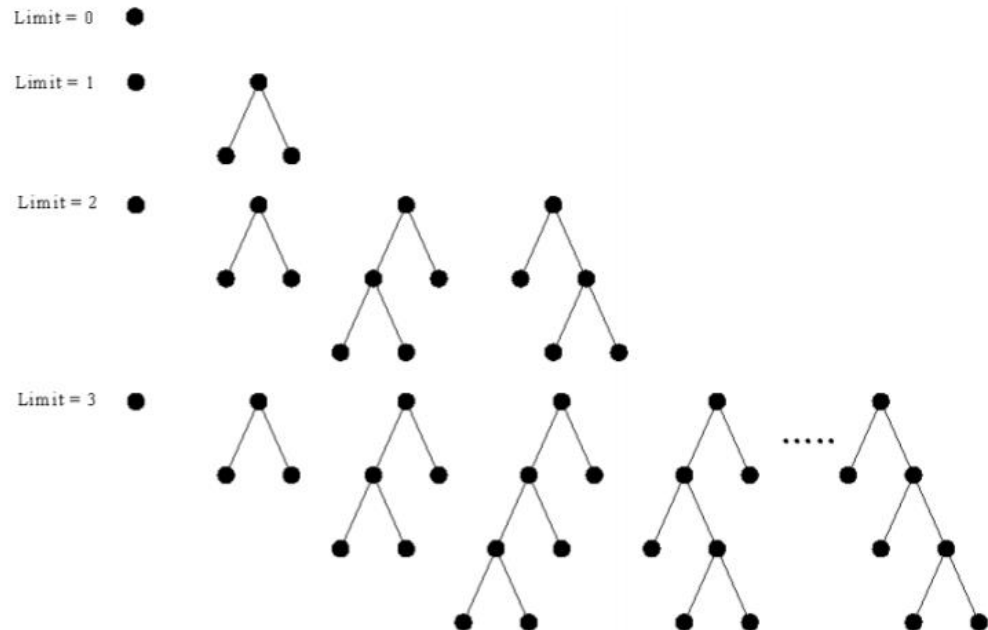
Algorithm Iterative Deepening: Cautare in adancime cu nivel iterativ

Intrari: Starea initiala S_i

Iesiri: SUCCES si solutie sau INSUCCES

1. **pentru** AdMax = 0 la inf **repeta**
 - 1.1 **rezultat** = DFS(AdMax)
 - 1.2 **daca** rezultat $\neq$ AdMax
atunci intoarece rezultat

sfarsit



Abordare hibrida

Complexitatea strategiilor de căutare

- *B* - *factorul de ramificare* al unui spatiu de cautare 8-puzzle

- Numar de miscari:

- 2 m pt colt = 8

- 3 m centru lat = 12

- 4m centru $\Rightarrow$ 24 miscari

- **$B = \text{nr. misc.} / \text{nr. poz. p. liber} = 2.67$**

- Numar de miscari:

- 1 m pt colt = 4

- 2 m centru lat = 8

- 3m centru $\Rightarrow$ 15 miscari $\Rightarrow$ **$B = 1.67$**

Complexitatea strategiilor de căutare

- **B** - *factorul de ramificare*
- **d**- adancimea celui mai apropiat nod solutie (de cost minim daca exista costuri)
- **m** – lungimea maxima a oricarei cai din spatiul de cautare

Rad – B noduri, B^2 pe niv 2, etc.

- Numarul de stari posibil de generat pe un nivel de cautare d este B^d
- T - numarul total de stari generate intr-un proces de cautare, d – adancime nod solutie

$$T = B + B^2 + \dots + B^d = O(B^d)$$

Complexitatea strategiilor de căutare

- **Cautare pe nivel**

Numar de noduri generate

$$B + B^2 + \dots + B^d = O(B^d)$$

- **Cautare in adancime**

Numar de noduri generate - B^*d – daca nodurile expandate se sterg din CLOSED

Sau B^*m cu AdMax=m

Complexitatea strategiilor de căutare

- **Cautare backtracking**

Numar de noduri generate **m**

- **Cautare in adancime cu nivel iterativ**

Numar de noduri generate

$$d*B+(d-1)*B^2+ \dots + (1)*B^d = O(B^d)$$

Complexitatea strategiilor de căutare

Criteriu	Nivel	Adanci me	Adanc. limita	Nivel iterativ	Bidirec tionala
Timp	B^d	B^d	B^m	B^d	$B^{d/2}$
Spatiu	B^d	$B * d$	$B * m$	B^d	$B^{d/2}$
Optima litate?	Da	Nu	Nu	Da	Da
Comple ta?	Da	Nu	Da daca $m \geq d$	Da	Da

B – factor de ramificare, **d** – adancimea solutiei,
m – adancimea maxima de cautare (AdMax)

3. Strategii de căutare informate

Cunostintele euristice pot fi folosite pentru a crește eficiența căutării, în general, în trei moduri:

- Selectarea nodului următor de expandat în cursul căutării
- În cursul expandării unui nod al spațiului de căutare se poate decide pe baza informațiilor euristice care dintre succesorii lui vor fi generați și care nu
- Eliminarea din spațiul de căutare a anumitor noduri generate

Căutare informata de tip "best-first"

- Evaluarea cantitatii de informatie
- Calitatea unui nod este estimata de *functia de evaluare euristica*, notata **w(n)** pentru nodul n
- O functie euristica $w(n)$ este o evaluare a unui nod care mapeaza acel nod la o valoare reala ≥ 0
- Exemple
 - Strategia de cautare "best-first"
 - Strategia de cautare a alpinistului

Algoritm BestFS: Cautare "best-first" in spatiul starilor

Intrari: Starea initiala S_i si functia $w(S)$ asociata starilor

Iesiri: SUCCES si solutia sau INSUCCES

1. Initializeaza listele $OPEN \leftarrow \{S_i\}$, $CLOSED \leftarrow \{\}$
2. Calculeaza $w(S_i)$ si asociaza aceasta valoare nodului S_i
3. **daca** $OPEN = \{\}$
atunci intoarce INSUCCES
4. Elimina nodul S cu $w(S)$ minim din $OPEN$ si insereaza-l in $CLOSED$
5. **daca** S este stare finala
atunci
 - i. Solutia este $(S, \dots, S_i)$
 - ii. **intoarce** SUCCES
6. Expandeaza nodul S
 - 6.1. Genereaza toti succesorii directi S_j ai nodului S

6.2. **pentru** fiecare succesor S_j al lui S **executa**

6.2.1 Calculeaza $w(S_j)$ si asociaza-l lui S_j

6.2.2. Stabileste legatura $S_j \rightarrow S$

6.2.3. **daca** $S_j \notin \text{OPEN} \cup \text{CLOSED}$

atunci introduce S_j in OPEN cu $w(S_j)$ asociat

6.2.4. **altfel**

i. Fie S'_j copia lui S_j din OPEN sau CLOSED

ii. **daca** $w(S_j) < w(S'_j)$

atunci

- Elimina S'_j din OPEN sau CLOSED

(de unde apare copia)

- Insereaza S_j cu $w(S_j)$ asociat in OPEN

iii. **altfel** ignora nodul S_j

7. **repetă de la 3**

sfarsit.

Cazuri particulare

- Strategia de cautare "best-first" este o generalizare a strategiilor de cautare neinformate
 - strategia de cautare pe nivel $w(S) = \text{Ad}(S)$
 - strategia de cautare in adincime $w(S) = -\text{Ad}(S)$
- Strategia de cautare de cost uniform / Dijkstra

$$w(S_j) = \sum_{k=i}^{j-1} \text{cost_arc}(S_k, S_{k+1})$$

- Minimizarea efortului de cautare – cautare euristica

$$w(S) = \text{functie euristica}$$

Căutarea soluției optime în spațiul starilor. Algoritmul A*

$w(S)$ devine **$f(S)$** cu 2 comp:

- **$g(S)$** , o funcție care estimează costul real $g^*(S)$ al căii de căutare între starea inițială S_i și starea S ,
- **$h(S)$** , o funcție care estimează costul real $h^*(S)$ al căii de căutare între starea curentă S și starea finală S_f .
- **$f(S) = g(S) + h(S)$**
- $f^*(S) = g^*(S) + h^*(S)$

Calculul lui $f(S)$

- Calculul lui $g(S)$

$$g(S) = \sum_{k=i}^n \text{cost_arc}(S_k, S_{k+1})$$

- Calculul lui $h(S)$
- Trebuie sa fie admisibila
- O functie euristica h se numeste *admisibila* daca pentru orice stare S , $h(S) \leq h^*(S)$ & $h(S_f)=0$
- Definitia stabileste **conditia de admisibilitate** a functiei h si este folosita pentru a defini **proprietatea de admisibilitate** a unui algoritm A^* .

A* - proprietatea de admisibilitate

Fie un algoritm A* care utilizeaza cele doua componente **g** si **h** ale functiei de evaluare **f**. Daca

- (1) functia **h** satisface conditia de admisibilitate
- (2) $\text{cost_arc}(S, S') \geq c$

pentru orice doua stari S, S' , unde $c > 0$ este o constanta si costul c este finit

atunci **algoritmul A* este admisibil**, adica este garantat sa gaseasca calea de cost minim spre solutie.

- Completitudine – garantat sa gaseasca solutie daca solutia exista si costurile sunt pozitive

Implementare A*

Strategia de cautare "best-first" se modifica:

...

2. Calculeaza $w(S_i) = g(S_i) + h(S_i)$ si asociaza aceasta valoare nodului S_i

3. **daca** OPEN = {}

atunci intoarce INSUCCES - *nemodificat*

4. Elimina nodul S cu $w(S)$ minim din OPEN si insereaza-l in CLOSED - *nemodificat*

.....

6.2.4. **altfel**

i. Fie S'_j copia lui S_j din OPEN sau CLOSED

ii. **daca** $g(S_j) < g(S'_j)$

atunci ...

$h(S)$ aproape de $h^*(S)$?

- Fie doi algoritmi A^* , $A1$ si $A2$, cu functiile de evaluare h_1 si h_2

$$f_1(S) = g(S) + h_1(S) \qquad f_2(S) = g(S) + h_2(S)$$

- Se spune ca algoritmul **A2 este mai informat decat** algoritmul **A1** daca pentru orice stare S cu $S \neq S_f$

$$h_1(S) \geq h_2(S)$$

h_2 domina h_1

Consistentă funcției euristice

- **Euristica consistentă**

O funcție euristica h este **consistentă** dacă

$$h(S) \leq h(S') + \text{cost_arc}(S, S') \quad \& \quad h(S_f) = 0$$

pentru orice două stări S și S' succesor al lui S din spațiul de căutare

O funcție euristica **consistentă** este **admisibilă**

Dacă **h este consistentă** un nod introdus în CLOSED nu va mai fi niciodată eliminat de acolo și reintrodus în OPEN

Relaxarea condiției de optimalitate a algoritmului A^*

- O funcție euristica h se numeste **ε -admisibila** daca

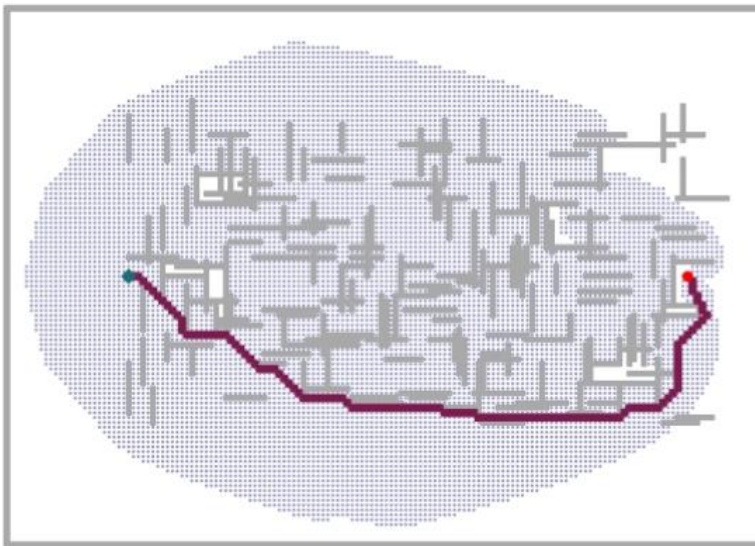
$$h(S) \leq h^*(S) + \varepsilon \quad \text{cu } \varepsilon > 0$$

- Algoritmul A^* care utilizeaza o funcție de evaluare f cu o componenta **h** ε -admisibila gaseste intotdeauna o solutie al carei cost **depaseste costul solutiei optime cu cel mult ε**
- Un astfel de algoritm se numeste *algoritm A^* ε -admisibil* iar solutia gasita se numeste *solutie ε -optimala*.

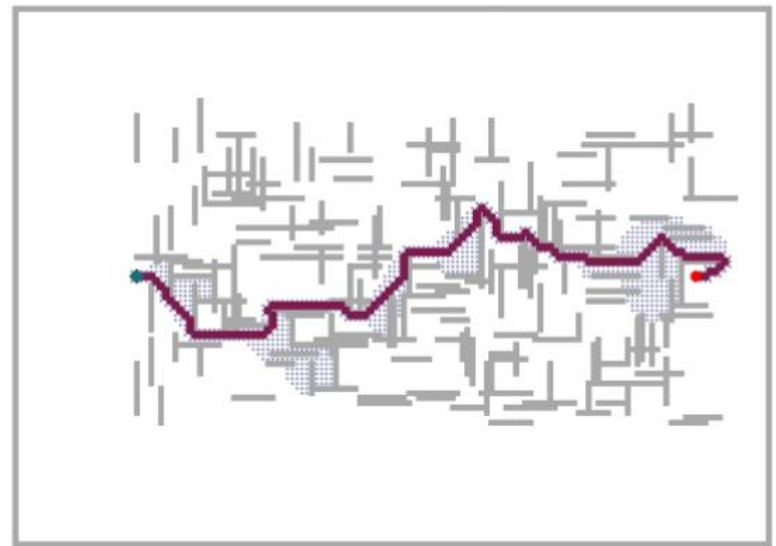
Relaxarea condiției de optimalitate

- De fapt putem utiliza o functie euristica ne-admisibila
- **A* ponderat** (Weighted A* search)

$$f(S) = g(S) + W * h(S) , \quad W > 1$$



(a)



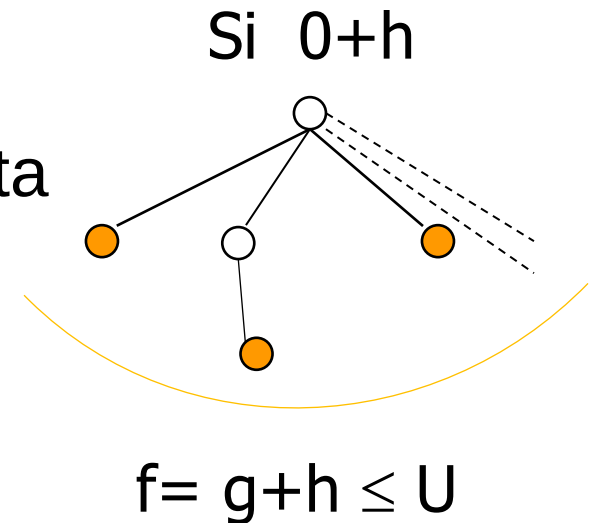
(b)

4. Strategii de căutare informată cu memorie limitată

- A^* se termina intotdeauna gasind o solutie optima si poate fi aplicat pe probleme generale
- Cu toate acestea, cantitatea de memorie necesara creste repede pe masura avansului algoritmului.
- Exemple de algoritmi pentru cautare solutiei de cost optim cu memorie limitata:
 - Beam search
 - Iterative deepening A^* (IDA*)
 - Memory bound A^* (MA*) sau Simplified MA* (SMA*)

IDA*

- Iterative deepening A*
 $f(S) = g(S) + h(S)$
- Bazat pe DFID dar utilizeaza o limita de cost U
- Mentine o limita sup U' pt urmatoarea iteratie
- Garantat sa gaseasca solutia de cost minim



Algoritm IDA*: Iterative deepening A*

Foloseste

- functia iterativa **BuclaIDA***
- functia recursiva **IDA***
- Functia **Expand** pentru generarea succesorilor unui nod
- Functia **Goal** care testeaza daca stare finala

BuclaIDA*

Intrari: Starea initiala s , functia de cost $g(s)$ si $h(s)$ euristica

Iesiri: Calea de la s la starea finala sau $\{\}$

$U' \leftarrow h(s), \quad \text{bestPath} \leftarrow \{\}$

cat timp ($\text{bestPath} = \{\}$ si $U' \neq \text{inf}$) **repeta**

$U \leftarrow U', \quad U' \leftarrow \text{inf}$

$\text{bestPath} \leftarrow \text{IDA}^*(s, 0, U)$

intoarce bestPath

sfarsit

IDA*(s, pathCost, U)

Intrari: starea s , costul caii $pathCost$, limita superioara U

lesiri: calea *Path* de la *s* la starea finala sau $\{\}$

Efect lateral: Actualizarea lui U'

daca (Goal(s)) **atunci** intoarce Path(s)

$$\text{Suc}(s) \leftarrow \text{Expand}(s)$$

pentru fiecare s' in $\text{Suc}(s)$ repeta

daca $\text{pathCost} + g(s, s') + h(s') \leq U$ /* starea este in limita U */

atunci

$$p \leftarrow \text{IDA}^*(s', \text{pathCost} + g(s, s'), U)$$

```
daca p ≠ {} atunci intoarce p      /* s-a gasit solutie */
```

altfel /* cost mai mare decat limita veche U */

daca $\text{pathCost} + g(s, s') + h(s') < U'$

```
/* cost mai mic decat noua limita */
```

```
atunci  $U' \leftarrow \text{pathCost} + g(s, s') + h(s')$  /* actualizez noua limita */
```

intoarce {

sfarsit

5. Determinarea funcției de evaluare h

- Specifica problemei, determinate manual (hand crafted)

SAU

- Generare automata de euristici
 - **Transformare abstracta** prin relaxarea unor restrictii ale problemei
 - **Pattern databases** Ne intereseaza o functie euristica $h(s)$ cat mai aproape de $h^*(s)$
- Avem mai multe euristici bune
- Pe care o alegem?
- $h(s) = \max (h_1(s), \dots h_k(s))$ – pt fiecare stare s

Exemple de funcții euristice

- **Problema comis-voiajorului**

$$h_1(S) = \text{cost_arc}(S_i, S)$$

- $h_2(S)$ = costul arborelui de acoperire de cost minim al oraselor neparcurse pana in starea S

Exemple de funcții euristice

- 8-puzzle

$$h_1(S) = \sum_{i=1}^8 t_i(S)$$

$$h_2(S) = \sum_{i=1}^8 \text{Distanța}(t_i)$$

Distanța Manhattan

$dx = |\text{nod.x} - \text{scop.x}|$

$dy = |\text{nod.y} - \text{scop.y}|$

$h(\text{nod}) = (dx + dy)$



Relaxarea condiției de optimalitate pentru 8-puzzle

- 8-puzzle

$$f_3(S) = g(S) + h_3(S) \quad h_3(S) = h_2(S) + 3 \cdot T(S)$$

$$T(S) = \sum_{i=1}^8 \text{Scor}[t_i(S)]$$

$$\text{Scor}[t_i(S)] = \begin{cases} 2 & \text{daca patratul } t_i \text{ in starea } S \text{ nu este urmat de} \\ & \text{succesorul corect din starea finala} \\ 0 & \text{pentru orice pozitie a lui } t_i \text{ diferita de centru} \\ 1 & \text{pentru } t_i \text{ aflat la centrul mozaicului} \end{cases}$$

Transformare abstracta

Variante “relaxate” ale problemei

O piesa poate fi mutata de la X la Y daca X si Y sunt adiacente si X este blank

O piesa poate fi mutata de la X la Y daca X si Y sunt adiacente

$$h_2(S) = \sum_{i=1}^8 \text{Distanta}(t_i)$$

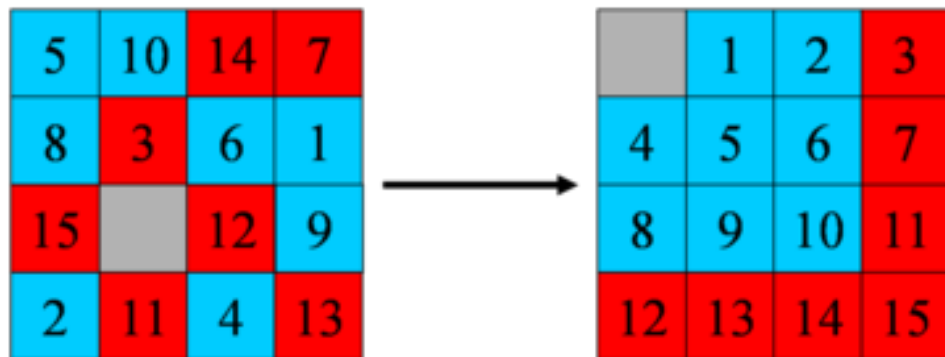
O piesa poate fi mutata de la X la Y $h_1(S) = \sum_{i=1}^8 t_i(S)$

Pattern databases

- **Pattern database pt euristici**
- Memoreaza o colectie de solutii ale unor subprobleme care trebuie rezolvate pentru a rezolva problema
- Fiecare solutie de subproblema are o functie euristica precalculata (costul cautarii) si memorata
- **Pattern** – o specificare partiala a unei stari
- **Pattern database** – multimea tuturor pattern-urilor care pot fi obtinute prin relaxari sau permutari ale target pattern

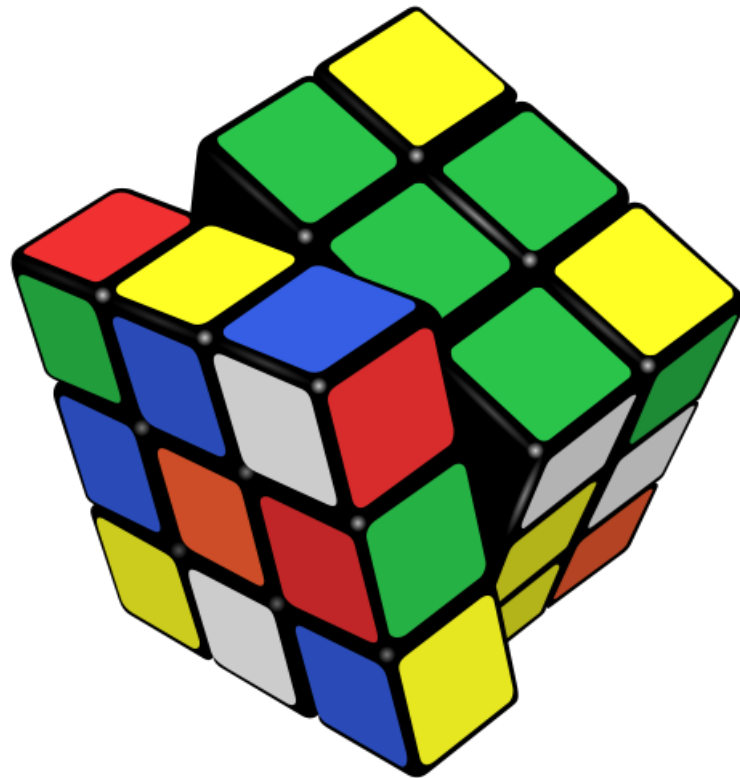
Pattern database pt euristici

- 15 puzzle
- Calculeaza costul solutiei pt fiecare instanta a unei subprobleme si memoreaza starea si cost
- Ex:
 - numarul minim de mutari necesare pt a duce in starea finala piesele rosii (pt orice configuratie a acestora)
 - Reducere cu un factor de 1000
- Costa memorie suplimentara si timp calcul pattern DB



Cubul lui Rubik

- 9 patrate cu 6 culori diferite
- Cea mai buna solutie IDA*



Cubul lui Rubik

Euristici

- ***Distanța Manhattan 3D*** =
Calculează distanța liniară între 2 puncte în R^3 prin însumarea distanțelor punctului în fiecare dimensiune
- Distanța Manhattan 3D între punctele p_1 și p_2
$$MD_{3d}(p_1, p_2) = |x_1 - x_2| + |y_1 - y_2| + |z_1 - z_2|$$
- Poate fi calculată în timp liniar
- Trebuie împartită la 8 pentru a fi admisibilă deoarece fiecare mișcare mută 4 colțuri și 4 muchii

Cubul lui Rubik

- *Dureaza mult cu IDA**
- Adancime 18 – f f mare
- **Pattern database**
- Se memoreaza intr-o tabela numarul de miscari necesare pt a rezolva colturile cubului

Invatarea euristicilor din exemple

Atributele unei stari

$x1(s)$, $x2(s)$.. si costurile asociate acelor stari

- *Ex: 8-Puzzle, 100 de stari initiale generate aleator*
- $x1(s)$ nr de cifre prost plasate
- $x2(s)$ – nr de perechi de numere care nu sunt adiacente in s fata de Sf

Inteligență Artificială

Universitatea Politehnica Bucuresti
Anul universitar 2022-2023

Adina Magda Florea

Curs 3_1

Problema satisfacerii restrictiilor (Constraint Satisfaction Problem)

1. Problema satisfacerii restrictiilor

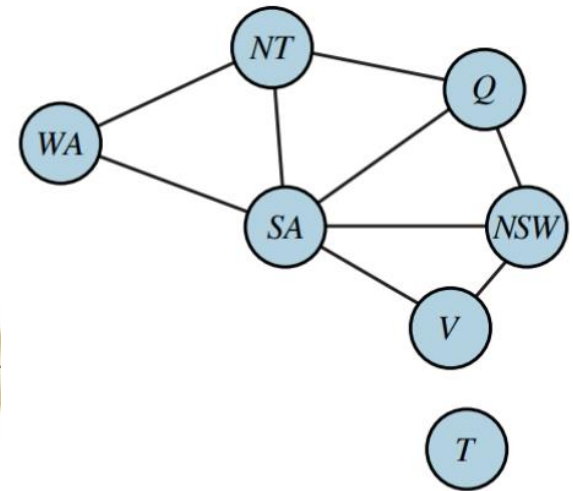
$$\{X_1 \dots X_N\} \qquad \{(X_1, x_1), \dots, (X_N, x_N)\}$$

$$D = \{D_1 \dots D_N\}$$

$$R = \{R_1 \dots R_k\}$$

- Restrictii explicite sau implicite
- Domeniul de valori
- Restrictii unare, binare, globale

Exemple



	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

$Alldiff(A1, A2, A3, A4, A5, A6, A7, A8, A9)$
 $Alldiff(B1, B2, B3, B4, B5, B6, B7, B8, B9)$
 ...
 $Alldiff(A1, B1, C1, D1, E1, F1, G1, H1, I1)$
 $Alldiff(A2, B2, C2, D2, E2, F2, G2, H2, I2)$
 ...
 $Alldiff(A1, A2, A3, B1, B2, B3, C1, C2, C3)$
 $Alldiff(A4, A5, A6, B4, B5, B6, C4, C5, C6)$
 ...

Instante ale CSP

- Determinarea unei solutii sau a tuturor solutiilor
- CSP totala
- CSP partiala
- CSP binara – graf de restrictii

CSP – problema de cautare, in NP

- Reducerea timpului (reducerea sp. de cautare)
- Metoda de baza: backtracking

2. Imbunatatirea performantelor BKT

Algoritmi care modifica spatiul de cautare prin eliminarea unor portiuni care nu contin solutii

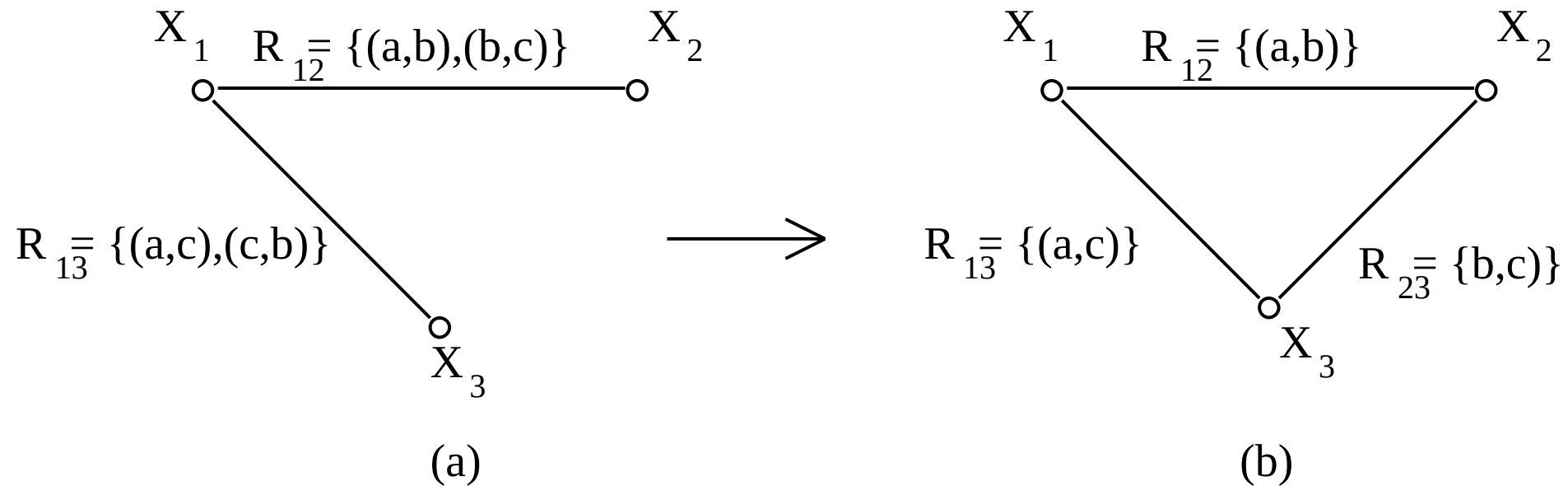
- Algoritmi de imbunatatire a consistentei reprezentarii (utilizati inainte de inceperea cautarii)
 - Consistenta locala a arcelor sau a cailor in graful de restrictii
- Algoritmi de cautare (cauta solutia si elimina portiuni din spatiul de cautare)
 - Imbunatatesc performantele rezolvarii prin reducerea numarului de teste.

Utilizarea euristicilor in cautare

Propagarea locala a restrictiilor

■ Propagarea restrictiilor - Ex.1

$$D_{X_1} = D_{X_2} = D_{X_3} = \{a, b, c\}$$



Propagarea locala a restrictiilor

Ex 2

$$V=\{X,Y,Z\} \quad D_X=D_Y=D_Z=\{1,2,3,4,5,6\}$$

$$R=\{ X < Y, Z < X-2 \}$$

$$\text{Deoarece } X < Y \text{ avem } D_X = \{1,2,3,4,5\}$$

$$D_Y = \{2,3,4,5,6\}$$

$$D_Z = \{1,2,3,4,5,6\}$$

$$\text{Deoarece } Z < X-2 \text{ avem } D_X = \{4,5\}$$

$$D_Y = \{2,3,4,5,6\}$$

$$D_Z = \{1,2\}$$

Apoi consideram din nou restrictia $X < Y$

$$\text{si avem } D_X = \{4,5\}$$

$$D_Y = \{5,6\}$$

$$D_Z = \{1,2\}$$

Propagarea locala a restrictiilor

- Combinatia de valori x si y pentru variabilele X_i si X_j este permisa de restrictia explicita $R_{ij}(x,y)$.
- Un **arc** (X_i, X_j) intr-un graf de restrictii orientat se numeste ***arc-consistent*** daca si numai daca pentru orice valoare $x \in D_i$, domeniul variabilei X_i , exista o valoare $y \in D_j$, domeniul variabilei X_j , astfel incat $R_{ij}(x,y)$.
- **Graf de restrictii orientat *arc-consistent*** – orice arc din graf este arc-consistent

AC-3: Realizarea arc-consistentei pentru un graf de restrictii

/ Intoarce fals daca inconsistentă, adevărat în caz contrar */*

Creează o coadă $Q \leftarrow \{ (X_i, X_j) \mid (X_i, X_j) \in \text{Multime arce}, i \neq j \}$

cat timp Q nu este vidă **execută**

 Elimina primul arc (X_k, X_m) din Q

dacă Verifica(X_k, X_m) **atunci**

dacă $|D_{X_k}| = 0$ **atunci întoarce** fals

$Q \leftarrow Q \cup \{ (X_i, X_k) \mid (X_i, X_k) \in \text{Multime arce}, i \neq k, m \}$

întoarce adevărat

Verifica (X_k, X_m) */* întoarce adevărat dacă se modifică D_{X_k} */*

modif $\leftarrow$ fals

pentru fiecare $x \in D_{X_k}$ **execută**

dacă nu există nici o valoare $y \in D_{X_m}$ astfel încât $R_{km}(x, y)$

atunci elimină x din D_{X_k}

modif $\leftarrow$ adevărat

întoarce **modif**

/ Modifică domeniul D_{X_k} prin efect lateral */*

Complexitate

- **N** - numarul de variabile
- **a** - cardinalitatea maxima a domeniilor de valori ale variabilelor
- **e** - numarul de restrictii.
- Algoritmului de realizare a arc-consistentei - AC-3: complexitate timp este $O(e * a^3)$
- S-a gasit si un algoritm de complexitate timp $O(e * a^2)$
– AC-4

m-Cale-consistență

- O cale de lungime m prin nodurile $i_0, \dots, i_m$ ale unui graf de restricții orientat se numește ***m-cale-consistentă*** dacă și numai dacă pentru orice valoare $x \in D_{i_0}$, domeniul variabilei i_0 și o valoare $y \in D_{i_m}$, domeniul variabilei i_m , pentru care $R_{i_0 i_m}(x, y)$, există o secvență de valori $z_1 \in D_{i_1} \dots z_{m-1} \in D_{i_{m-1}}$ astfel încât $R_{i_0 i_1}(x, z_1), \dots, R_{i_{m-1} i_m}(z_{m-1}, y)$
- **Graf de restricții orientat *m-cale-consistent* – dacă orice cale din graf este cale-consistentă**
- Algoritmul de realizare - PC-4: complexitatea timp $O(N^3 * a^3)$
- **2-cale consistentă**
- A face un graf de restricții *m-cale-consistent* are o complexitate exponențială în m

2-Cale-consistență

$$V=\{A,B,C\} \quad D_A=D_B=D_C=\{1,2,3\}$$

$$R=\{B>1, A<C, A=B, B>C-2\}$$

Putem exprima restrictiile sub forma de matrici booleene

$$\begin{array}{cccc} B>1 & A=B & A<C & B>C-2 \end{array}$$

$$\begin{array}{cccc} 000 & 100 & 011 & 110 \end{array}$$

$$\begin{array}{cccc} 010 & 010 & 001 & 111 \end{array}$$

$$\begin{array}{cccc} 001 & 001 & 000 & 111 \end{array}$$

$R_{i,j}$ – între X_i și X_j

$R_{k,k}$ – domeniu D

Consistența pe calea (i,k,j) poate fi determinată recursiv utilizând ecuația

$$R_{i,j} \leftarrow R_{i,j} \wedge (R_{i,k} * R_{k,k} * R_{k,j})$$

$$\text{De ex } R_{AC} \leftarrow R_{A,C} \wedge (R_{A,B} * R_{B,B} * R_{B,C})$$

Cale consistență (ordin 2)

/ Intari: Multime de variabile V , $n=|V|$, multime de matrici de restrictii R*

*Iesiri: Matrici de restrictii cale-consistente */*

$Y^n \leftarrow R$

repeta

$Y^0 \leftarrow Y^n$

pentru $k=1,n$ **repeta**

pentru $i=1,n$ **repeta**

pentru $j=1,n$ **repeta**

$Y_{ij}^k \leftarrow Y_{i,i}^{k-1} \wedge (Y_{i,k}^{k-1} * Y_{k,k}^{k-1} * Y_{k,j}^{k-1})$

pana $Y^n = Y^0$

intoarce Y^0

sfarsit

Este o extindere a algoritmului Floyd Warshall (all shortest path) in care minimizarea este inlocuita cu conjunctia si aduarea cu multiplicarea matricilor

3. Cautare cu BKT

- BKT – algoritmul de baza
- Imbunatatiri
- Realizarea arc-consistentei pe parcursul cautarii
- Backjumping

Backtracking

/ Intrari: variabile V, restrictii R*

*Iesiri: Atribuirii pt X si adev daca R satisfacute, fals in caz contrar */*

/ L- variabile instantiate, U – variabile neinstantiate */*

$(b, L) \leftarrow \text{BKT}(V, \{\}, R)$

daca b atunci intoarce L

intoarce fals

sfarsit

BKT(U, L, R)

daca U = {} atunci intoarce (adevarat, L)

$X \leftarrow \text{Selectie}(U)$

pentru fiecare $x \in D_X$ repeta

$b \leftarrow \text{Consistent}(L \cup \{X=x\}, R)$

daca b atunci $(b, L) \leftarrow \text{BKT}(U \setminus \{X\}, L \cup \{(X, x)\}, R)$

daca b atunci intoarce (b, L)

intoarce fals

sfarsit

$\text{Consistent}(L, R)$

pentru fiecare $r \in R$ repeta

daca $\text{Variabile}(r) \subseteq L$

atunci

daca $\text{not}(\text{Satisfacut}(r, L))$

atunci intoarece fals

intoarce adevarat

Realizarea arc-consistentei pe parcursul cautarii

- Forward checking
- **Maintaining Arc-Consistency (MAC)**
- Dupa ce s-a alocat o valoare unei variabile X , algoritmul apeleaza AC-3 dar in loc sa faca o coada cu toate arcele din problema, se introduce in coada numai arcele (X, Y_i) pentru toate variabilele Y_i neinstantiate care sunt legate de X printr-o restrictie
- AC-3 face propagarea restrictiilor si, daca un domeniu este redus la multimea vida, intoarce fals si se intra in backtracking

Backtracking cu MAC

/ Intrari: variabile V, restrictii R*

*Iesiri: Atribuire pt X si adev daca R satisfacute, fals in caz contrar */*

/ L- variabile instantiate, U – variabile neinstantiate */*

$(b, L) \leftarrow \text{BKT_MAC}(V, \{\}, R, D)$

daca b atunci intoarce L

intoarce fals

sfarsit

BKT_MAC(U, L, R, D)

daca U={} **atunci intoarce** (adevarat, L)

$X \leftarrow \text{Selectie}(U)$

pentru fiecare $x \in D_x$ **repeta**

$(b, D') \leftarrow \text{AC-3}'(L \cup \{(X, x), R, D)$

daca b atunci $(b, L) \leftarrow \text{BKT_MAC}(U \setminus \{X\}, L \cup \{(X, x)\}, R, D')$

daca b atunci intoarce (b, L)

intoarce fals

sfarsit

Backjumping

- BKT cu salt – determina nivelul in care apare eventual un conflict

Exemplu

$$V = \{A, B, C, D\} \quad D_A = D_B = D_C = D_D = \{1, 2, 3, 4\}$$

$$R = \{A > D\}$$

Backjumping

/ Intrari: variabile V, restrictii R*

*Iesiri: Atribuire pt X si adev daca R satisfacute, fals in caz contrar */*

$(n, L) \leftarrow \text{Backjump}(V, \{\}, R, 0)$ */* n – variabila intreaga */*

daca $n = |V| + 1$ **atunci intoarce** L

intoarce fals

sfarsit

Backjump(U, L, R, p)

/ Intrari: U, L, R, nivel anterior p, Iesiri: L sau nivelul de salt pt variabila conflict */*

daca $U = \{\}$ **atunci intoarce** $(|L| + 1, L)$

$X \leftarrow \text{Selectie}(U)$ $m \leftarrow 0$ */* init variabila jump */*

pentru fiecare $x \in D_X$ **repeta**

$(b, j) \leftarrow \text{Consistent}(L \cup \{(X, x, p+1)\}, R, p+1)$ */* b var booleana, j var intraga */*

daca b **atunci**

$m \leftarrow p$

$(n, L) \leftarrow \text{Backjump}(U \setminus \{X\}, L \cup \{(X, x, p+1)\}, R, p+1)$ */* n intreg */*

daca $(n \neq p+1)$ **atunci intoarce** (n, L)

altfel

$m \leftarrow \max\{m, j\}$

intoarce $(m, .)$

sfarsit

Consistent(L,R,nivel)

/ intoarce nivelul variabilei conflictuale */*

$j \leftarrow \text{nivel}$

$b \leftarrow \text{fals}$

pentru fiecare $r \in R$ **repeta**

daca Variabile(r) $\subseteq L$)

atunci

daca not(Satisfacut(r,L)

atunci

$b \leftarrow \text{adevarat}$

$j \leftarrow \min \{j, \max_k (X \in \text{Variabile}(r) \text{ si } (X,x,l) \in L \text{ si } k < l)\}$

daca b

atunci intoarce (fals, j)

intoarce (adevrat,.)

sfarsit

4. Euristici utilizate in CSP

Euristici generale

Ordonarea variabilelor – vezi functie Selectie(U)

- aleator
- **Minimum remaining value** (MRV) – se incepe cu variabila cea mai restrictionata intai (fail-first) – variabila cu cele mai putine valori legale
- **Degree heuristic** – selectie variabila care este implicata in cel mai mare numar de restrictii cu variabile neinstantiate

Ordonarea valorilor

- **Least-constrained value** – selectie valoarea care elimina cele mai putine valori din domeniul variabilelor neinstantiate cu care este legata prin restrictii (fail-last)

O solutie: variabila fail-first, valoare fail last

Toate solutiile: variabila fail-last, valoare fail first

5. CSP partiala

- Memoreaza cea mai buna solutie gasita pana la un anumit moment (gen IDA*) – *distanța* *d* fata de solutia perfecta
- Abandoneaza calea de cautare curenta in momentul in care se constata ca acea cale de cautare nu poate duce la o solutie mai buna
- NI - numarul de inconsistente gasite in "cea mai buna solutie" depistata pana la un moment dat – *limita necesara*

CSP partiala

- *S - limita suficiente* - specifica faptul ca o solutie care violeaza un numar de S restrictii (sau mai putine), este acceptabila.
- PBKT(Cale, Distanta, Variabile, Valori)
 - Semnificatie argumente
 - Rezultat: GATA sau CONTINUA
- variabile globale: CeaMaiBuna, NI, S

Algoritm: CSP Partiala
PBKT(Cale, Distanta, Variabile, Valori)

/ intoarce GATA sau CONTINUA */*

1. **daca** Variabile = {}
atunci

1.1 CeaMaiBuna $\leftarrow$ Cale

1.2 NI $\leftarrow$ Distanta

1.3 **daca** $NI \leq S$ **atunci** intoarce GATA
altfel intoarce CONTINUA

2. **altfel**

2.1 **daca** Valori = {} **atunci** CONTINUA

/ s-au incercat toate valorile si se revine la var ant. */*

2.2 **altfel**

2.2.1 **daca** Distanta $\geq$ NI

atunci intoarce CONTINUA

/ revine la var ant pentru gasirea unei solutii mai bune*/*

2.2.2 altfel

```
i. Var ← first(Variabile)
ii. Val ← first(Valori)
iii. DistNoua ← Distanta
iv. Cale1 ← Cale
v. cat timp Cale1 <> {} si DistNoua < NI executa
    - (VarC, ValC) ← first(Cale1)
    - daca Rel(Var,Val,VarC,ValC) = fals
    - atunci DistNoua ← DistNoua + 1
    - Cale1 ← Rest(Cale1)
vi. daca DistNoua < NI si
    PBKT(Cale+(Val,Var),DistNoua,Rest(Variabile),ValoriNoi)
        = GATA
/* ValoriNoi - domeniul de valori asociat primei variabile din Rest(Variabile) */
atunci intoarce GATA
altfel intoarce
    PBKT(Cale,Distanta,Variabile,Rest(Valori))
```

sfarsit

Inteligență Artificială

Universitatea Politehnica Bucuresti
Anul universitar 2022-2023

Adina Magda Florea

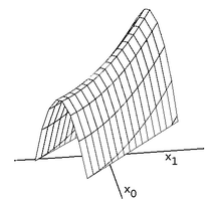
Curs 3

Strategii de cautare

- Cautari locale
- Cautari on-line

1. Strategii de cautare locala

- **Cautari locale** – opereaza asupra starii curente generand succesorii
- Calea catre solutie nu are importanta
 - Gasire solutie – CSP - nu conteaza calitatea solutiei
- Gasire solutie optima – functie de evaluare sau de cost
- Probleme in gasirea solutiei optime pe baza de cautari locale in functie de forma spatiului de cautare
- Maxim global, maxim local, platou, umar



Cautari locale

Caracteristici

- Folosesc putina memorie
- Gasesc solutii destul de bune in spatii finite f mari si chiar in spatii infinite
- Folosite si pt probleme de optimizare (cu fct obiectiv)
- Algoritmi pentru spatii de cautare discrete
 - Hill climbing
 - Simulated annealing

Algorithm: **Hill climbing** (greedy local search)

/* intoarce o stare cu functia de evaluare un maxim local */

1. $S \leftarrow$ stare initiala
2. Genereaza $S_j = \text{Succ}(S)$, toti succesorii starii S
3. $S' \leftarrow S_j$ cu $\text{Eval}(S_j)$ maxim dintre toti succesorii S_j
4. **daca** $\text{Eval}(S') \leq \text{Eval}(S)$ **atunci intoarce** S
5. $S \leftarrow S'$
6. **repeta** de la 2
 sfarsit

- Cautarea se orienteaza permanent in directia cresterii valorii;
- Se termina cand ajunge la un maxim (nici un succesor nu are valoarea mai mare)

Hill Climbing

- Problema 8 regine
- S – tabla completa
- Succesori – toate starile posibil de generat prin mutarea a 1 regina intr-un alt patrat in coloana
- o stare are $8 \times 7 = 56$ succesori
- Fct de evaluare = nr de perechi de regine care se ataca
- Minim global = 0
- 86% instante nu gaseste solutia (3 pasi)
- 14% gaseste soluta (4 pasi)
- Spatiul starilor $8^8 \approx 17$ milioane de stari

Hill Climbing - imbunatatiri

- Miscari laterale – se spera sa fie “umar” si nu “platou” (continui si pentru $\text{Eval}(S') = \text{Eval}(S)$)
- Daca “platou” – risc de bucla infinita – necesita limita in cautare
- Cu aprox. 100 miscari laterale – problema 8 regine – 94% instante pt care se gaseste solutia

Hill Climbing - imbunatatiri

Hill climbing stochastic

- Dintre starile succesoare cu $\text{Eval}(S_j) \geq \text{Eval}(S)$, se alege aleator un S_j

First choice hill climbing

- Genereaza aleator succesori pana gaseste $\text{Eval}(S_j) \geq \text{Eval}(S)$, continua cautarea cu S_j

Random restart Hill Climbing

- Repeta diferite HC cu stari initiale generate aleator
- Daca fiecare HC are o probabilitate de succes de $p \rightarrow 1/p$ repetitii
- 8 regine $p=0.14$ – 7 iteratii (6 esec si 1 succes)
- 3 mil regine – cateva secunde

Simulated annealing

- Simuleaza un proces fizic (calirea)
- T – temperatura
- Scaderea gradientului – solutii de cost minim
- Alege o miscare la intamplare
- Daca starea este mai buna, cauta in continuare de la aceasta
- Altfel alege starea mai "proasta" cu o probabilitate
- Scade probabilitatea de selectie a starilor mai "proaste" pe masura ce temperatura scade

Algorithm: **Cautare Simulated Annealing**

```
1.  $T \leftarrow$  temperatura initiala
2.  $S \leftarrow$  stare initiala
3.  $v \leftarrow \text{Eval}(S)$ 
2. cat timp  $T > \text{temp finala}$  executa
    2.1 pentru  $i=1, n$  executa
         $S' \leftarrow \text{Succ}(S)$ 
         $v' \leftarrow \text{Eval}(S')$ 
        daca  $v' < v$  atunci  $S \leftarrow S'$ 
        altfel  $S \leftarrow S'$  cu prob.  $\exp(-(v'-v)/T)$ 
            (in rest  $S$  nemodificat)
    2.2  $T \leftarrow 0.95 * T$ 
sfarsit
```

Local beam search

- **Local beam search** - incepe cu K stari generate aleatory, la fiecare pas se genereaza toti succesorii si se aleg K cei mai buni
- **Stochastic local beam search** – la fel dar alege stochastic K succesorii

Cautari locale in spatii continue

- Factor de ramificare infinit
- First choice HC, Simulated annealing
- **Problema:** dorim sa plasam 3 supermarket-uri pe o harta a.i. suma patratelor distantelor de la fiecare comuna de pe harta la cel mai apropiat supermarket sa fie minima.
- Spatiul starilor este definit prin coordonatele supermarket-urilor

$$(x_1, y_1) (x_2, y_2) (x_3, y_3)$$

- (spatiu n -dimensional cu n variabile)
- O miscare in acest spatiu – miscarea unui sm pe harta
- C_i - multimea de orase care sunt cel mai aproape de sm i in starea curenta

$$f(x_1, y_1, x_2, y_2, x_3, y_3) = \sum_{i=1,3} \sum_{c \in C_i} (x_i - x_c)^2 + (y_i - y_c)^2$$

Cautari locale in spatii continue

Cum putem gasi solutia?

VARIANTE

- Discretizare spatiu – discretizarea vecinatatii fiecărei stări (de ex mutam câte un sm o dată fie în direcția x fie în direcția y cu câte o cantitate fixă $M \Rightarrow 12$ stări succesoare)
- Utilizarea gradientului

Si aplicam HC prin actualizarea stării curente cu $x + \text{constanta} \times \text{gradient}$

2. Strategii de cautare on-line

- Cautare “offline”
- Cautare “on-line” – cautare + actiune
- Exemplu: robot care investigheaza mediul
- **Cautare online:** pentru fiecare actiune, agentul primeste perceptia care ii spune in ce stare a ajuns.
Construieste Rezultat[s,a]
- **Cautari locale online:**
 - HC, nu pot random restart,
 - Adauga memorie la HC – memoreaza cea mai buna estimare curenta

Cautare on-line

- Caracteristici
- Metode
 - On-line DFS
 - Programare dinamica asincrona (ADP)
 - Learning Real-Time A* (LRTA*)
 - Cautare cu tinta mobila

2.1 On-line DFS

- Agentul nu are un model al mediului, il descopera pe masura ce il parcurge
- Nodurile sunt expandate pe baza localitatii lor

On-line DFS

On-line DFS(s') intoarce o actiune

var globale: s, a , starea si actiunea anterioare, initial nule

Rezultat – o tabela care mapeaza (s, a) la s' , initial vida

Neincercate – o tabela care mapeaza s la o lista de actiuni neincercate

Non-revenite – o tabela ce mapeaza s la o lista de stari la care nu s-a revenit inca

daca s' este stare scop **atunci intoarce** stop

daca s' este stare noua **atunci** Neincercate[s'] $\leftarrow$ actiuni(s') /* Neincercate null pt s' */

daca $s \neq \text{null}$ **atunci**

Rezultat[s, a] $\leftarrow s'$

adauga s la inceputul lui Non-revenite[s']

daca Neincercate[s'] este vida **atunci**

daca Non-revenite[s'] este vida **atunci intoarce** stop

$a \leftarrow$ o actiune b a.i. Rezultat[s', b] = pop(Non-revenire[s'])

$s' \leftarrow \text{null}$

altfel $a \leftarrow$ pop(Neincercate[s'])

$s \leftarrow s'$

intoarce a

2.2 Programare dinamica asincrona (ADP)

Programare dinamica

Descompunerea problemei in subprobleme nedistincte

Principiul optimalitatii – o cale este optima $\leftrightarrow$ orice segment (subcale) a acesteia este optima

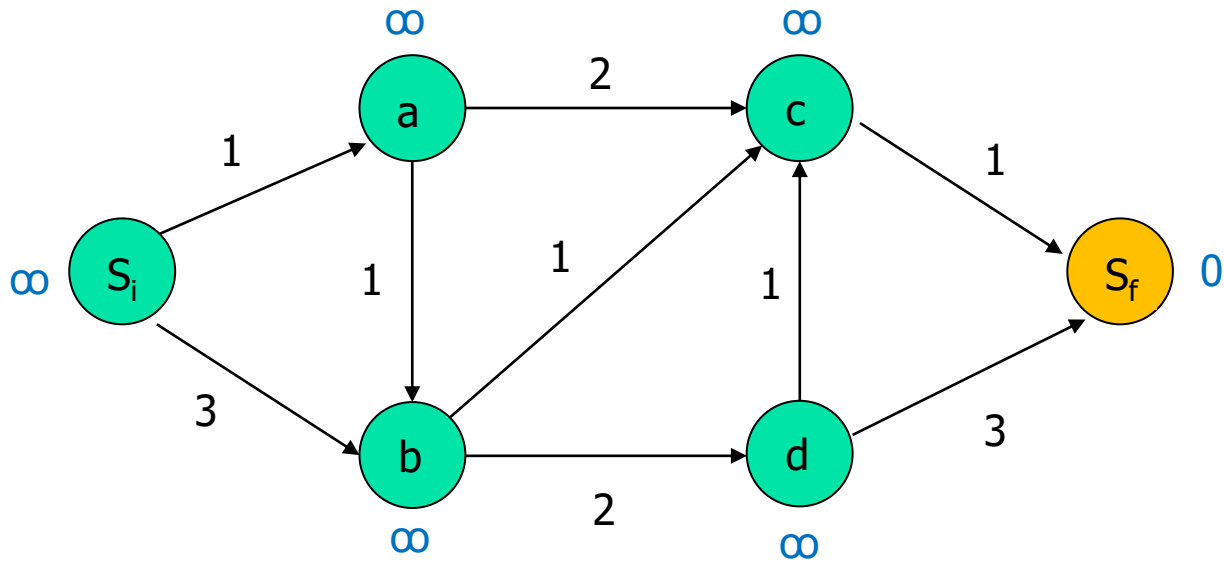
ADP: Daca se cunoaste h^* pt fiecare nod, calea de cost minim poate fi obtinuta repetand procedura:

- Pentru fiecare nod succesori j a nodului curent i calculeaza $f^*(j) = c(i,j) + h^*(j)$
- Mergi la starea j pt care $f^*(j)$ este minim

Programare dinamica asincrona

Presupunem urmatoarea situatie

- Pentru fiecare nod i exista un proces care corespunde lui i
- Fiecare proces inregistreaza $h(i)$ – estimarea lui $h^*(i)$
- Valoarea initiala a lui $h(i)$ este arbitrara cu exceptia nodurilor stare finala
- Fiecare proces i actualizeaza $h(i)$:
 - pentru fiecare vecin j - calculeaza $f(j)=c(i,j) + h(j)$
 - $h(i) \leftarrow \min_j f(j)$



ADP

$$f(j) = c(i, j) + h(j)$$

Programare dinamica asincrona

Ordinea de actualizare este arbitrara

Daca costul arcelor este pozitiv, converge la valorile reale

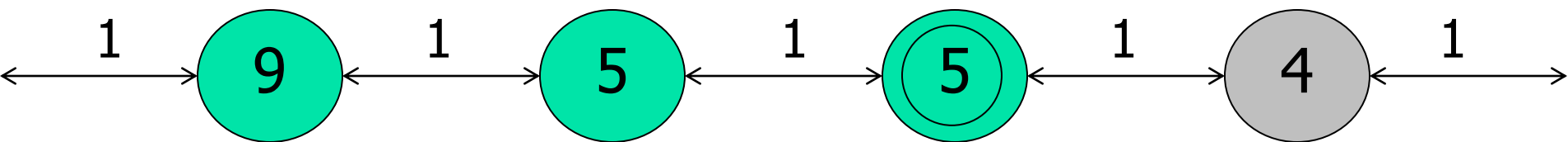
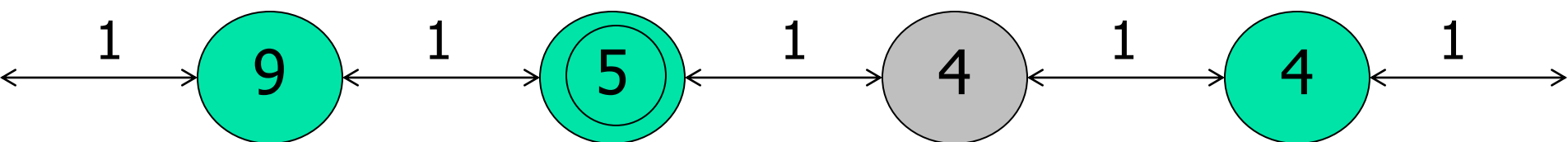
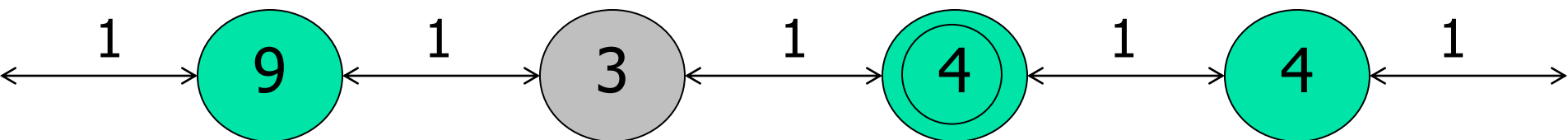
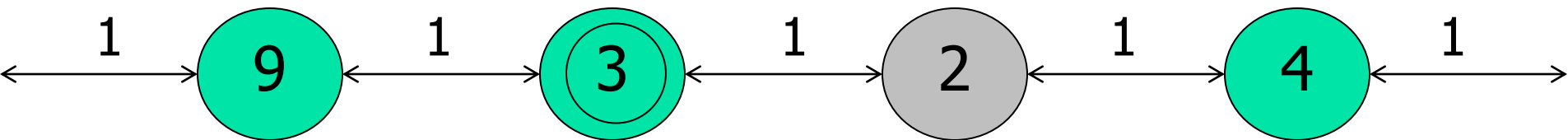
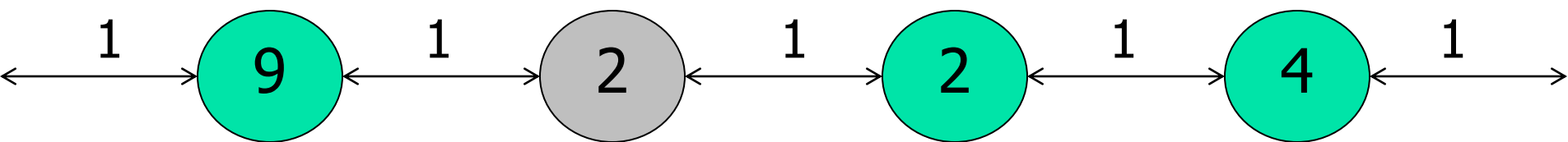
Spatiu de stari mare – nepractic

Foloseste ca fundament pt A^* in timp real

- Modific A^* pentru a putea fi utilizat on-line
- De ce nu pot folosi direct A^* ?

2.3 Learning Real-Time A* (LRTA*)

- Intrepatrunde calculul miscarii urmatoare cu executia miscarilor si alege miscarile intr-un timp constant
- Construiește și actualizează o tabelă ce conține estimări euristice ale distanței fiecărei stări la starea scop.
- Actualizează estimarea de cost a stării pe care a parasit-o și alege mișcarea (aparent) cea mai bună
- Initial, intrările în această tabelă corespund unor evaluări euristice sau 0 și sunt subestimări
- Prin explorarea repetată a spațiului, valorile sunt actualizate până când ajung, într-un final, la valorile corecte



Learning Real-Time A* (LRTA*)

Agentul considera numai nodul curent

Agent in nodul i

Agentul inregistreaza distanta estimata pentru un nod

1. Lookahead

- Pentru fiecare vecin j a lui i calculeaza

$$f(j) = c(i, j) + h(j)$$

$h(j)$ – estimarea caii $j \rightarrow S_f$

$c(i, j)$ – cost arc $i \rightarrow j$

Learning Real-Time A* (LRTA*)

2. Actualizeaza h

- Actualizeaza h estimat a nodului i

$$h(i) \leftarrow \min_j f(j)$$

3. Selecteaza actiunea si memoreaza

- Mergi la starea j cu valoarea **minima** $f(j)$ – actiunea a
- Memoreaza $i \rightarrow_a j$

Valorile initiale ale lui h trebuie sa fie admisibile

Learning Real-Time A* (LRTA*)

- Agentul aflat in starea s' face backtrack intr-o stare anterior vizitata s in momentul in care estimarea rezolvarii problemei din starea s' + costul de intoarcere in s este *mai mic* decat costul de a merge inainte din starea s' .
- In LRTA* meritul fiecarui nod este calculat relativ la pozitia curenta (nu la starea initiala).
- Este suficient sa memoram *tabela hash* H cu h a nodurilor deja vizitate

Learning Real-Time A* (LRTA*)

Algoritm LRTA*(s') intoarce o actiune

intrari: s', o perceptie care identifica starea curenta

variabile globale: Rezultat: o tabela indexata dupa stari si actiuni, initial vida

H – o tabela cu estimarea starilor, initial vida

s, a – starea anterioara si actiunea anterioara, initial nule

daca s' este stare scop **atunci intoarce** stop

daca s' este stare noua ($s' \notin H$) **atunci** $H[s'] \leftarrow h(s')$

daca s $\neq$ null **atunci**

Rezultat[s,a] $\leftarrow$ s'

$H[s] \leftarrow \min_{b \in \text{Actiuni}(s)} \mathbf{LRTA}^*\mathbf{Cost}(s, b, \text{Rezultat}[s,b], H)$

a $\leftarrow$ o actiune b din Actiuni(s') care minimizeaza $\mathbf{LRTA}^*\mathbf{Cost}(s', b, \text{Rezultat}[s',b], H)$

s $\leftarrow$ s'

intoarce a

Agentul selecteaza o actiune in fct de valorile starilor invecinate,
care sunt actualizate pe masura ce agentul exploreaza spatiul

Algoritm LRTA*Cost(s,a,s', H) intoarce o estimare de cost

daca s' nu este definita **atunci intoarce** h(s)

altfel intoarce $c(s,a,s') + H[s']$

Learning Real-Time A* (LRTA*)

- Intr-un spatiu de cautare finit cu costuri pozitive, in care exista o cale de la orice stare S la S_f si se utilizeaza estimari admisibile nenegative, LRTA* este complet (va ajunge in final in starea scop)
- In plus « invata » solutia optima in timp

Inteligență Artificială

Universitatea Politehnica Bucuresti
Anul universitar 2022-2023

Adina Magda Florea

Curs 4

Strategii de căutare

- Căutare adversarială în jocuri

1. Strategii de căutare în jocuri

- Teoria jocurilor – teoria deciziei pt agenți care interacționează
- Căutări specifice datorita acestei particularități
- S : set de stări cu S_0
- N – număr de jucători
- A – mulțime de acțiuni
- $f: S \times A \rightarrow S$ (f sau next)
- $Q: S \rightarrow \mathbb{R}^N$ – funcția de utilitate / recompensa
- $J: S \rightarrow (1, 2, \dots, N)$ – jucătorul care joacă

2 Jocuri cu 2 adversari

- Jocuri ce implică doi adversari ($N=2$)
 - Jucător
 - Adversar
- Algoritmul Minimax
- Algoritmul Alfa-Beta
- Algoritmul Monte Carlo Tree Search

Minimax

- 1944 - von Neumann si Morgenstern descriu cum un proces, numit Minimax, este capabil sa identifice rezultatul final al unui joc si sa aleaga mutarea cea mai buna pentru orice stare de joc, în cazul jocurilor cu informatie perfecta.
- Bazele teoretice ale algoritmului – 1928 - von Neumann
- **Non-cooperative Game Theory** / 2 players zero-sum games
- Valoarea **maxmin** a J1 este cea mai mare valoare pe care J1 poate spera sa o obtina fara sa stie actiunea lui J2
$$\max_{J1} \min_{J2} u_{J2}(a_{J1}, a_{J2})$$
- **Combinatorial game theory** / sequential games with perfect information
- **Algoritmul Minimax**

Minimax

- Jucător – MAX
- Adversar – MIN
- Etichetez fiecare nivel din AJ cu **MAX** (jucator) și **MIN** (adversar)
- Etichetez frunzele cu scorul jucatorului
- Parcurg AJ
 - dacă nodul parinte este **MAX** atunci i se atribuie valoarea maxima a succesorilor sai;
 - dacă nodul parinte este **MIN** atunci i se atribuie valoarea minima a succesorilor sai.

Minimax pentru spații de căutare investigate până la o adancime n

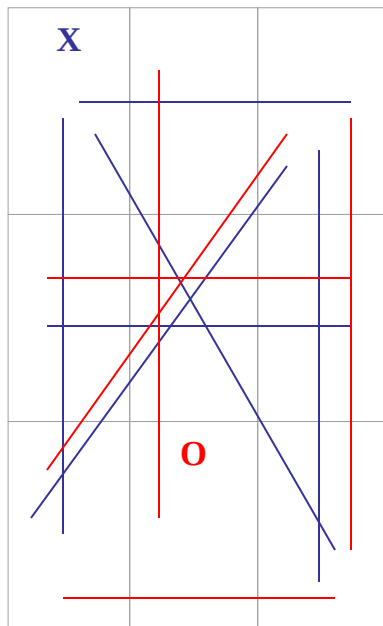
- Spațiul de cautare este foarte mare sau infinit, nu poate fi investigat exhaustiv
- Algoritmul Minimax până la o adancime n
- $nivel(S)$
- O funcție euristica de evaluare a unui nod $eval(S)$

Exemplu de funcție de evaluare

Jocul de Tic-Tac-Toe (X si O)

- Funcție de estimare euristica **eval(S)** - conflictul existent în starea **S**.
- $\text{eval}(S) = \text{numarul total posibil de linii castigatoare ale lui MAX în starea S} - \text{numarul total posibil de linii castigatoare ale lui MIN în starea S}$.
- Dacă **S** este o stare din care **MAX** poate face o mișcare cu care castiga, atunci $\text{eval}(S) = \infty$ (o valoare foarte mare)
- Dacă **S** este o stare din care **MIN** poate castiga cu o singura mutare, atunci $\text{eval}(S) = -\infty$ (o valoare foarte mica).

eval(S) în Tic-Tac-Toe



X are 6 linii castigatoare posibile

O are 5 linii castigatoare posibile

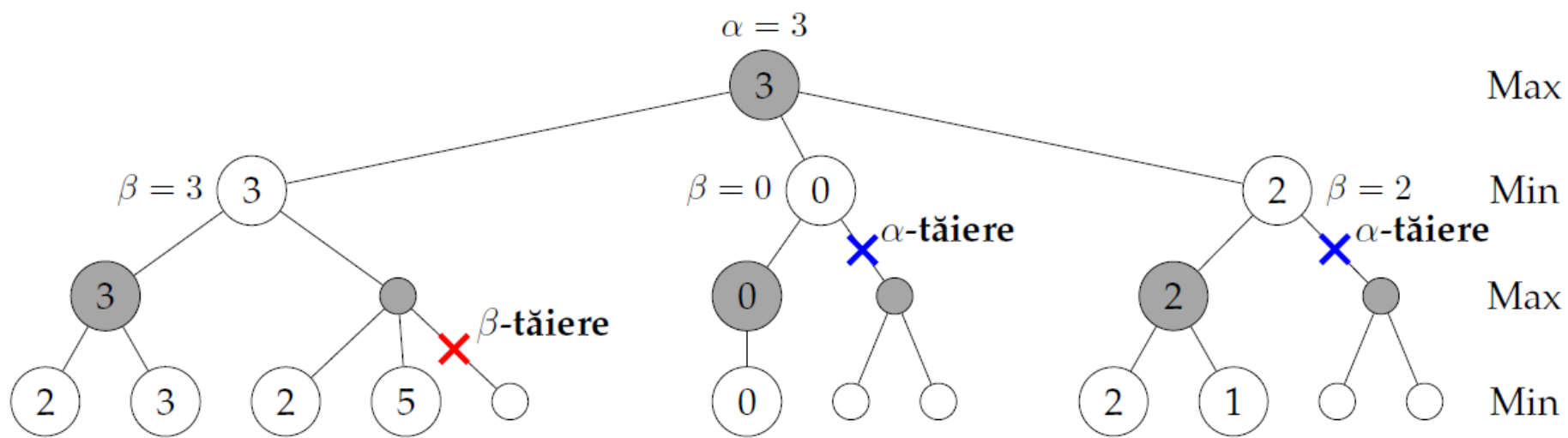
$$\text{eval}(S) = 6 - 5 = 1$$

Algoritmul tăierii alfa-beta

- Este posibil sa se obțină decizia corecta a algoritmului **Minimax** fara a mai inspecta toate nodurile din spatiului de cautare pana la un anumit nivel.
- Procesul de eliminare a unei ramuri din arborele de cautare se numeste *taierea arborelui de cautare (pruning)*.
- Alpha-beta pruning (Knuth and Moore, 1975)

Algoritmul tăierii alfa-beta

- Fie α cea mai buna valoare (cea mai mare) gasita pentru **MAX** si β cea mai buna valoare (cea mai mica) gasita pentru **MIN**.
- Algoritmul **alfa-beta** actualizeaza α si β pe parcursul parcurgerii arborelui si elimina investigarile subarborilor pentru care α sau β sunt mai proaste.
- Terminarea cautarii (taierea unei ramuri) se face dupa doua reguli:
 - **α -taieri** - În cazul în care exista, pentru un nod **Min**, o actiune ce are asociata o valoare $v \leq \alpha$, atunci putem renunta la expandarea subarborelui sau, deoarece **Max** poate atinge deja un câstig mai mare, dintr-un subarboare precedent.
 - **β -taieri** - În cazul în care exista, pentru un nod de tip **Max**, o actiune ce are asociata o valoare $v \geq \beta$, atunci putem renunta la expandarea subarborelui sau, deoarece **Min** a limitat deja castigul lui **Max** la β



Algoritm: **Alfa-beta**

MAX(S, α , β) { intoarce valoarea maxima a unei stari. }

0. daca S este nod final **atunci intoarce** scor(S)

1. daca nivel(S) = n **atunci** intoarce eval(S)

2. altfel

2.1 pentru fiecare succesori S_j al lui S **executa**

 2.1.1 $\alpha \leftarrow \max(\alpha, \text{MIN}(S_j, \alpha, \beta))$

 2.1.2 **daca** $\alpha \geq \beta$ **atunci** intoarce β

2.2 intoarce α

sfarsit

MIN(S, α , β) { intoarce valoarea minima a unei stari. }

0. daca S este nod final **atunci intoarce** scor(S)

1. daca nivel(S) = n **atunci** intoarce eval(S)

2. altfel

2.1 pentru fiecare succesori S_j al lui S **executa**

 2.1.1 $\beta \leftarrow \min(\beta, \text{MAX}(S_j, \alpha, \beta))$

 2.1.2 **daca** $\beta \leq \alpha$ **atunci** intoarce α

2.2 intoarce β

sfarsit

Algoritmul tăierii alfa-beta

- Eficienta algoritmului depinde semnificativ de ordinea de examinare a starilor
- Se recomanda o ordonare euristica a succesorilor, eventual de generat numai primii cei mai buni succesori
- Poate reduce semnificativ timpul de cautare
- *De exemplu – incepe cu cea mai “simpla” miscare sau favorizeaza nodurile cu taieri B*

Imbunatatiri

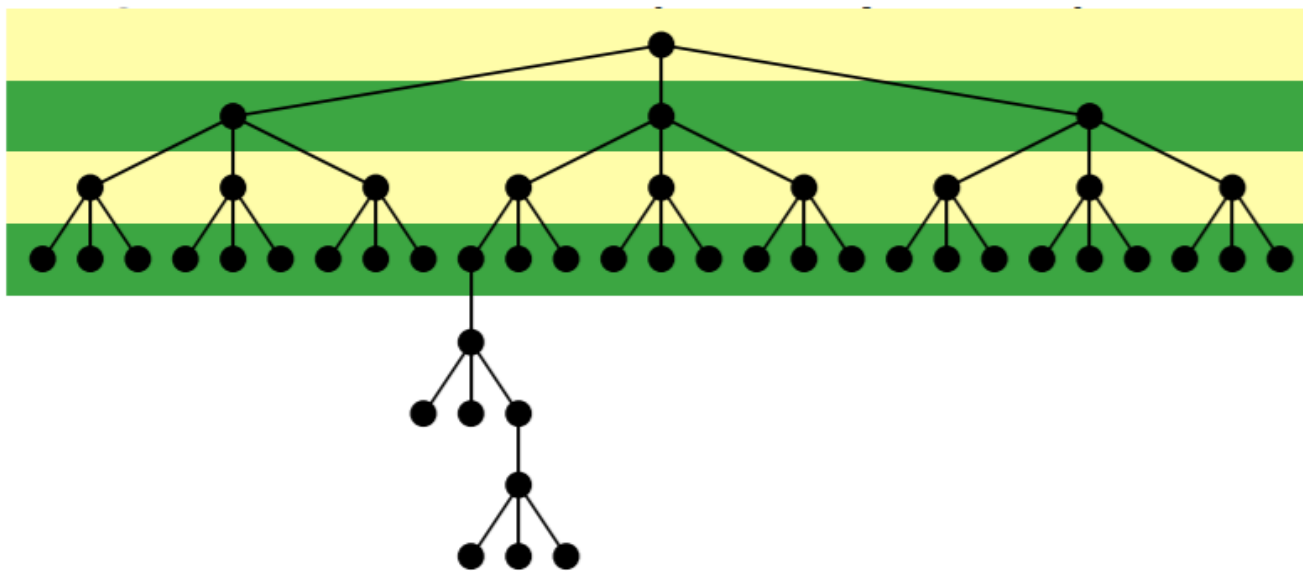
- Timp limitat pentru executarea unei miscari – anytime algorithm
- Foloseste **Iterative Deepening**
- incepe cu ply 1 si obtine cea mai buna miscare
- apoi ply 2 folosind cele mai bune stari cf evaluarii anterioare
- continua cresterea ply pana la expirarea timpului

Imbunatatiri

- Diferite secvente de mutari pot duce la aceleasi pozitii
- Mai multe pozitii de joc pot fi functional echivalente (de ex pozitiiile simetrice)
- **Memoize** – tabela hash cu pozitiiile de joc pentru a obtine:
 - Estimari ale nodurilor
 - Cea mai buna miscare dintr-un nod

Imbunatatiri

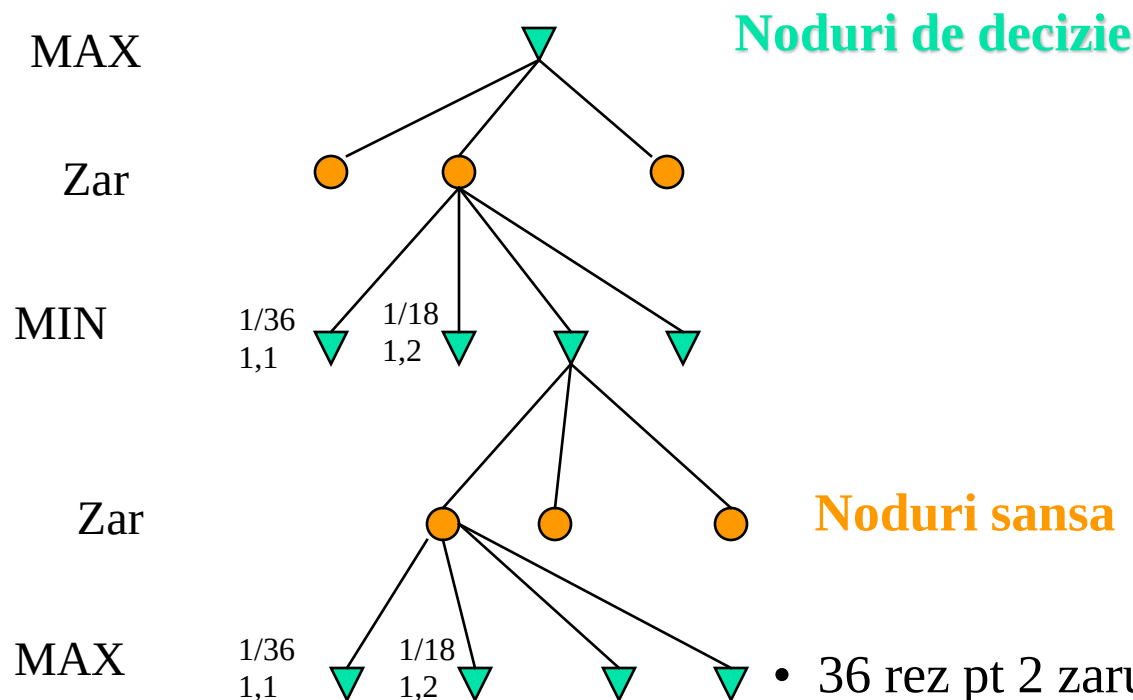
- **Efectul de orizont** – caută mai mult decât limita de cautare pentru anumite pozitii



Jocuri cu elemente de șansă

- Jucatorul nu cunoaste mișcările legale ale oponentului
- 3 tipuri de noduri:
 - MAX
 - MIN
 - Șansă (chance nodes)

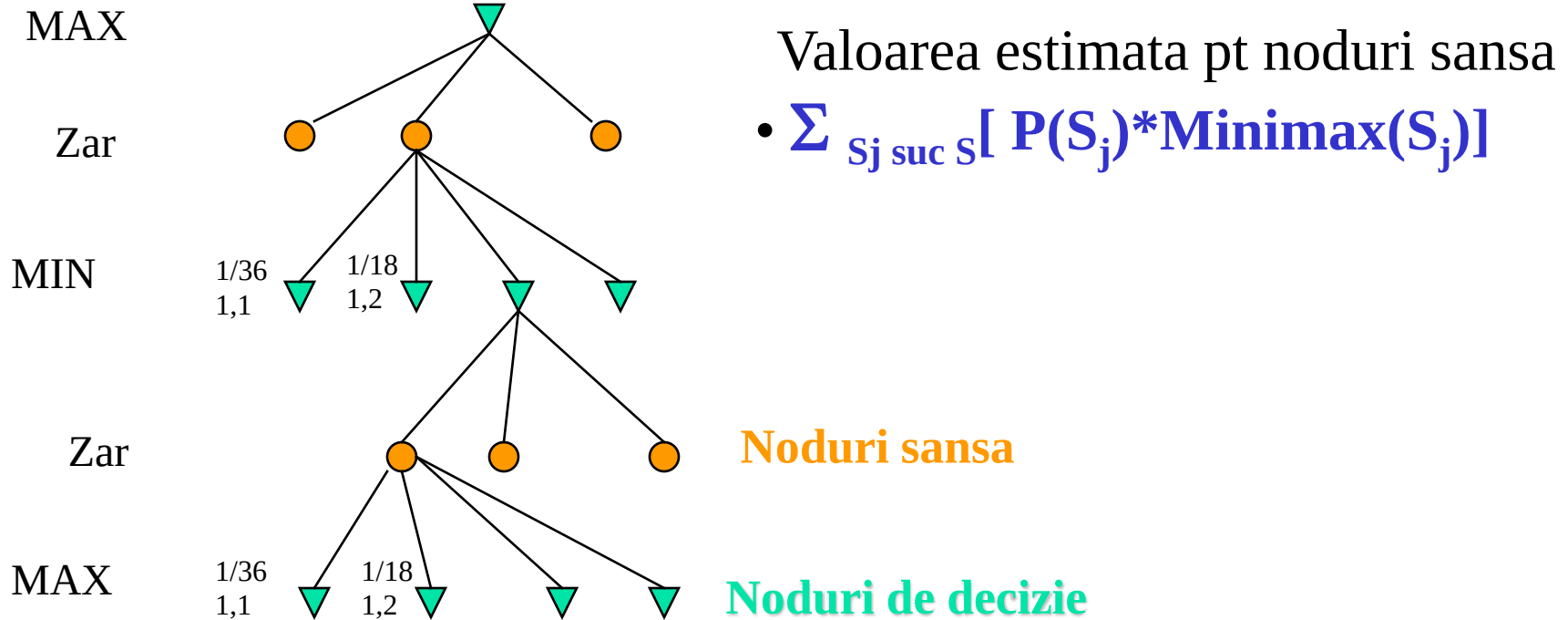
Noduri șansă – ramurile care pleaca dintr-un nod șansa indica
posibile rezultate ale sansei (de exemplu zar)



- 36 rez pt 2 zaruri, toate la fel de probabile
- 21 noduri distincte (3-4 la fel cu 4-3)
- Zaruri egale (6 dist) - $\rightarrow 1/36$ pt 1-1
- Zaruri diferite (15 dist) $\rightarrow 1/18$ pt zaruri diferite

Functia de evaluare

- scor – nod terminal
- max din Minimax succesori - MAX
- min din Minimax succesori - MIN
- $\Sigma [P(S_j) * \text{Minimax}(S_j)]$ succesori - SANSA



3 Jocuri cu mai mulți jucători

Exemplu: Chinese checkers (table chinezesti)

- Chinese checkers nu este un joc de origine chineza, ci este o variatie moderna si simplificata, aparuta în Germania prin 1892, a celebrului joc american Halma, inventat de George H. Monks în 1880.
- Abia în momentul în care a devenit cunoscut în Statele Unite ale Americii, a primit denumirea Chinese checkers

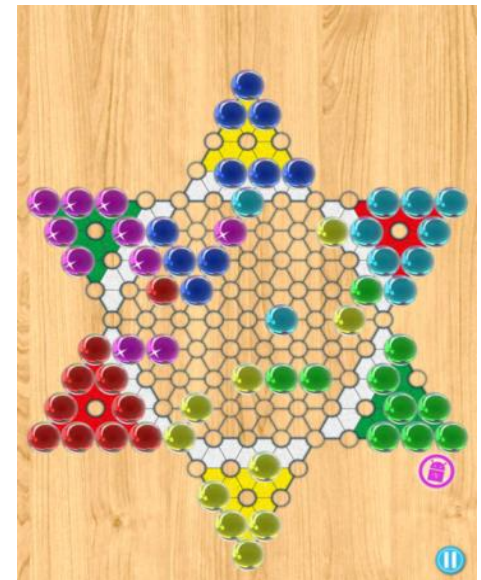
George Howard Monks



Jocuri cu mai mulți jucători

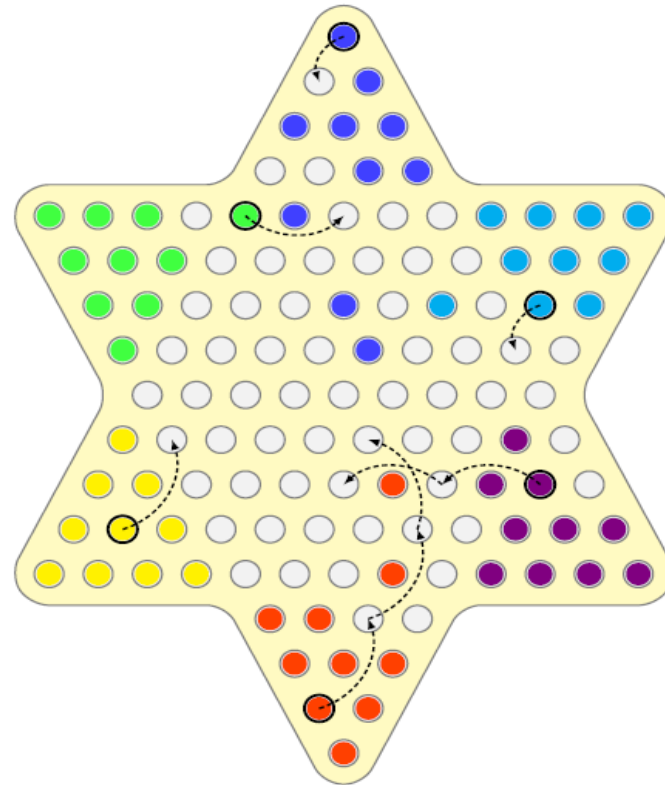
Chinese checkers

- Este un joc cu informație perfectă pentru 2-6 jucători
- Scopul este să deplasăm 10 piese dintr-o poziție de start într-o poziție finală cât mai repede.



■ Chinese checkers

- Piesele se muta prin deplasare intr-o pozitie alaturata sau sarind peste piese alaturate daca exista un loc liber. Se poate sari peste orice numar de piese si se pot inlantui mai multe sarituri. Piesele nu sunt eliminate dupa sarituri.



Jocuri cu mai mulți jucători

Nu exista algoritmi buni consacrați

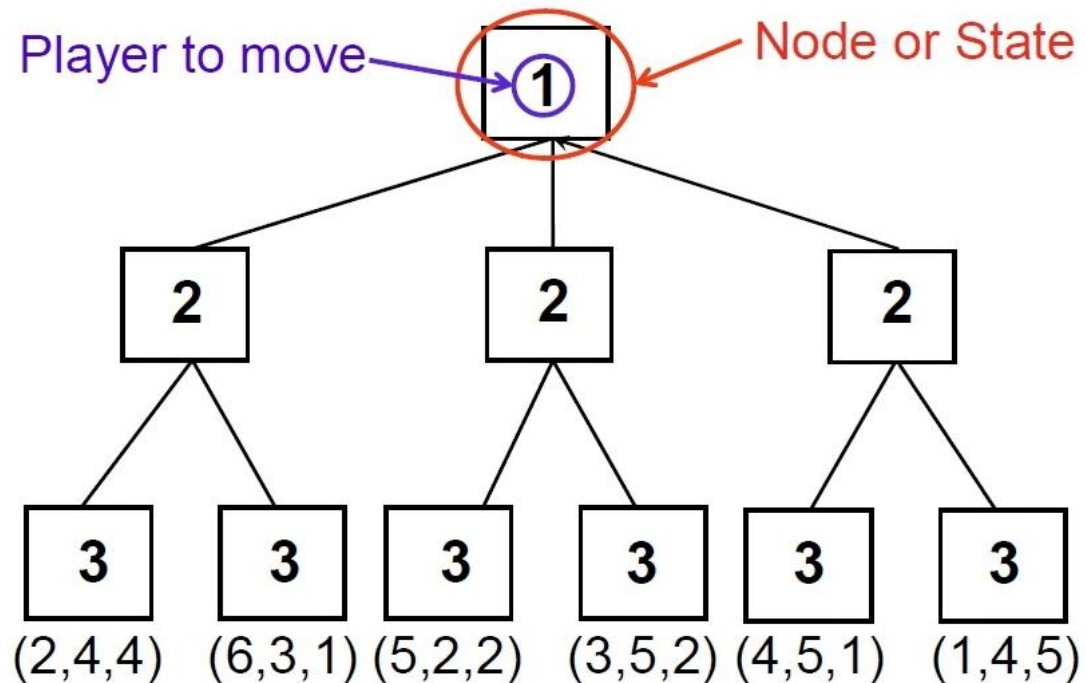
In general, 2 strategii

- **Maxⁿ** - generalizare a Minimax pt **n** jucatori
- **Paranoic** – reduce la joc cu 2 jucatori in care se presupune ca toti ceilalti colaboreaza impotriva jucatorului simulat

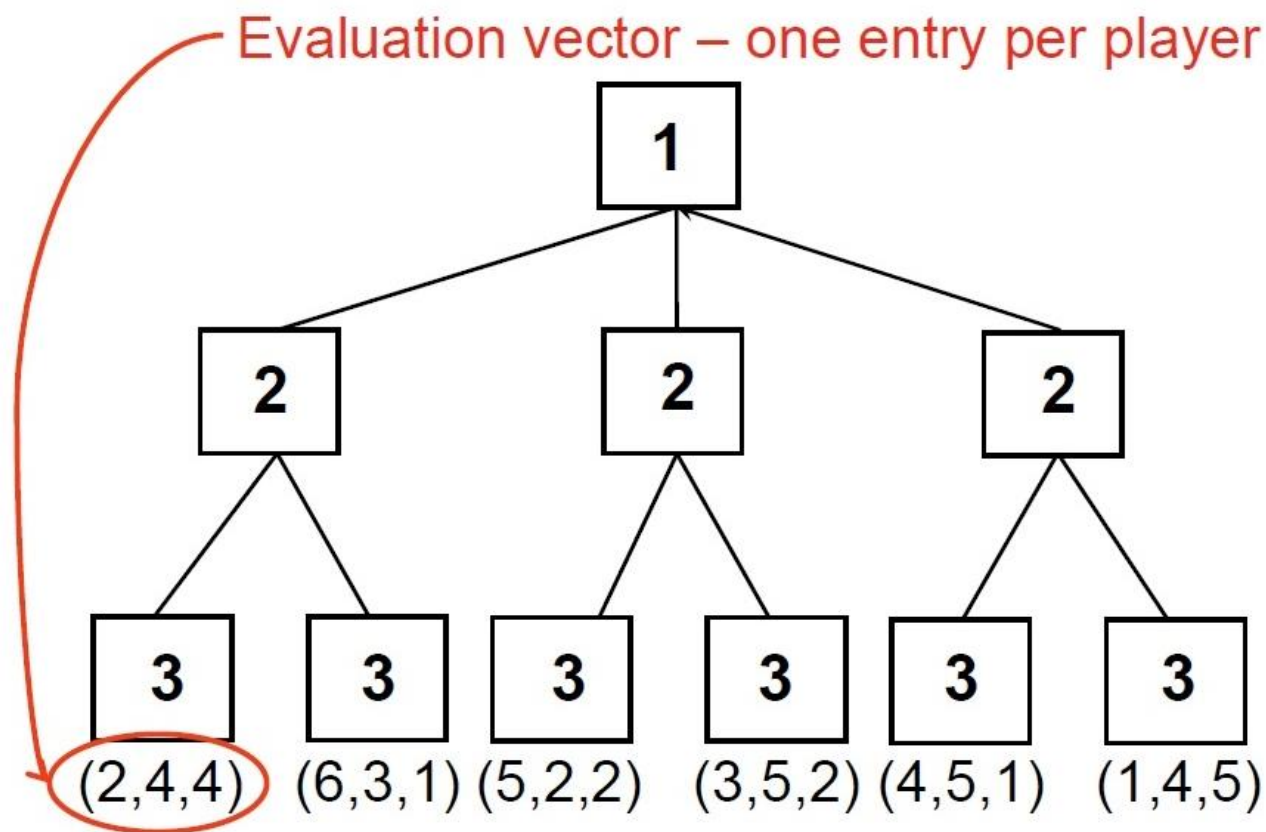
Presupunem jocuri cu informatie perfecta

Jocuri cu mai mulți jucători

- Functia de evaluare – dependenta de joc
- Regula de decizie in parcurgerea arborelui de cautare – generica

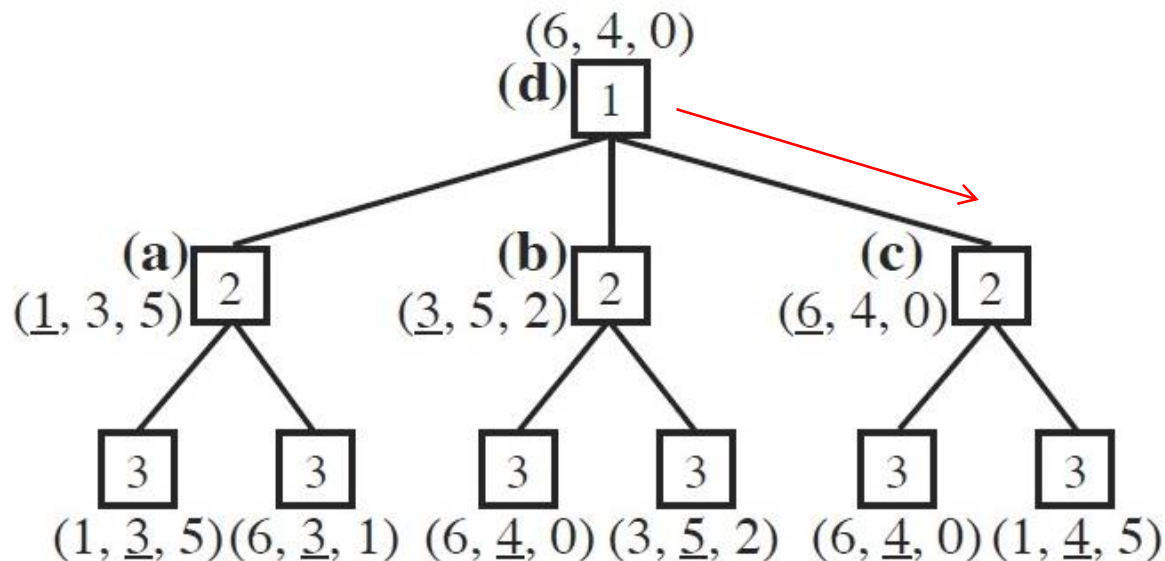


Jocuri cu mai mulți jucători



Strategie Maxⁿ

- Generalizarea Minimax pentru n jucatori
- Frunzele arborelui de joc sunt n -tuple, in care elementul pe pozitia i este scorul jucatorului i .
- Pentru nodurile din interior, valoarea Maxⁿ a unui nod in care jucatorul i muta este valoarea Maxⁿ a succesoriului pentru care a i -a componenta din vector este maxima.



Strategie Maxⁿ

Maxn(Nod, Juc)

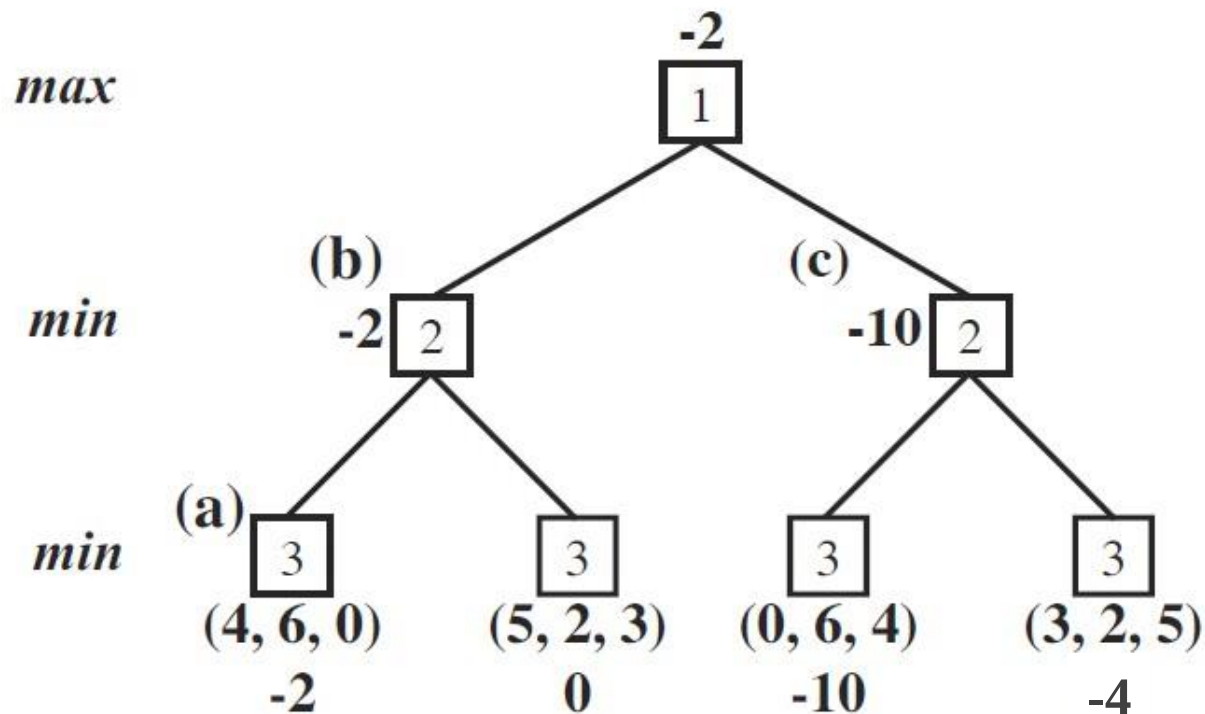
1. **daca** Nod este nod final **atunci intoarce** scor(Nod[Juc])
2. **daca** nivel(Nod) = n **atunci** intoarce eval(Nod[Juc])
3. **altfel**
 - 3.1 $P \leftarrow \text{Prim_succesor}(\text{Nod})$
 - 3.2 Best [Juc] $\leftarrow \text{Maxn} (P, \text{Juc_Urm})$
 - 3.3 **pentru** fiecare succesor $S_j \neq P$ al lui Nod **executa**
 - Curent $\leftarrow \text{Maxn}(S_j, \text{Juc_Urm})$
 - **if** Curent[Juc] > Best [Juc]
then Best $\leftarrow$ Curent
4. Intoarce Best [Juc]

Strategie Maxⁿ

- Pot exista multe valori egale Maxⁿ intr-un arbore
- Rezultatul poate depinde drastic de felul in care se face alegerea
- E.g., (2,3,3) vs. (2,1,7)
- Alpha-beta in adancime nu poate fi aplicat
- Maxⁿ – shallow pruning, Korf, 1991 (analog to alpha-beta dar cu performante proaste)

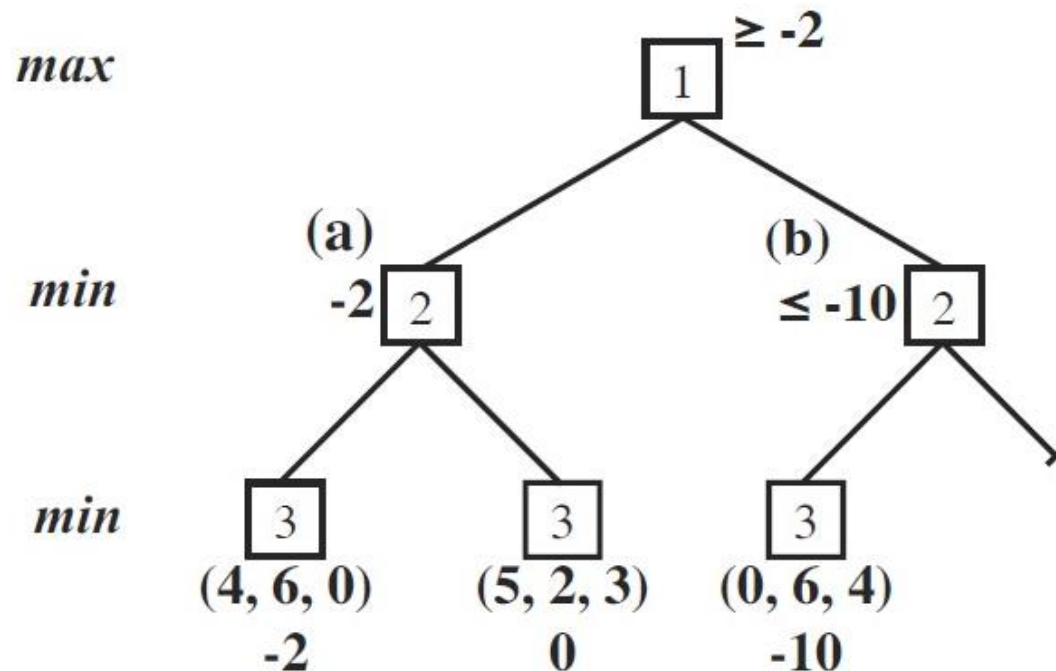
Strategie Paranoid

- **Paranoic** – reduce jocul la 2 jucatori si se poate aplica Minimax



Strategie Paranoic

- La fel ca la Minimax:
 - Exista o unica valoare
 - Se poate utiliza Alpha-Beta



Strategie Paranoic

- Pe masura ce numarul jucatorilor creste beneficiul adus de taiere scade
- Pt jocuri cu 3-6 jucatori, adancimea este cu 20-50% mai mare decat la Max^n
- Presupunerea Paranoic este foarte pesimista
- Exista cazuri in care greseste; in aceasta situatie, cu cat se cauta mai adanc cu atat este mai proasta estimarea

4 Monte Carlo Tree Search

- MCTS - algoritm probabilistic care utilizeaza o serie de simulari aleatoare pentru a expanda selectiv arborele de joc
- Reprezinta o metoda buna pentru luarea deciziilor în probleme cu un spatiu de cautare mare
- Este un fel de best-first search ghidat de rezultatele unei simulari Monte-Carlo
- Metoda se bazeaza pe 2 ipoteze:
 - Adevarata **valoare a unei actiuni** (mutare in joc) poate fi aproximata utilizand simulari aleatoare
 - Valorile astfel obtinute pot fi utilizate pentru a **ajusta politica de selectie** spre o cea mai buna strategie
- MCTS (Coulom, 2006)

Monte Carlo Tree Search

- Baza metodei este o **unda de joc** (“**playout**”)
- **Playout** = un joc rapid jucat cu mutari dintr-o anumita stare pana la sfarsitul jocului, obtinandu-se castig/pierdere sau un scor
- Fiecarui nod parcurs i se asociaza un merit
- In varainta cea mai simpla acest merit este un procent de castig = de cate ori s-a castigat daca s-a pornit unda din acel nod

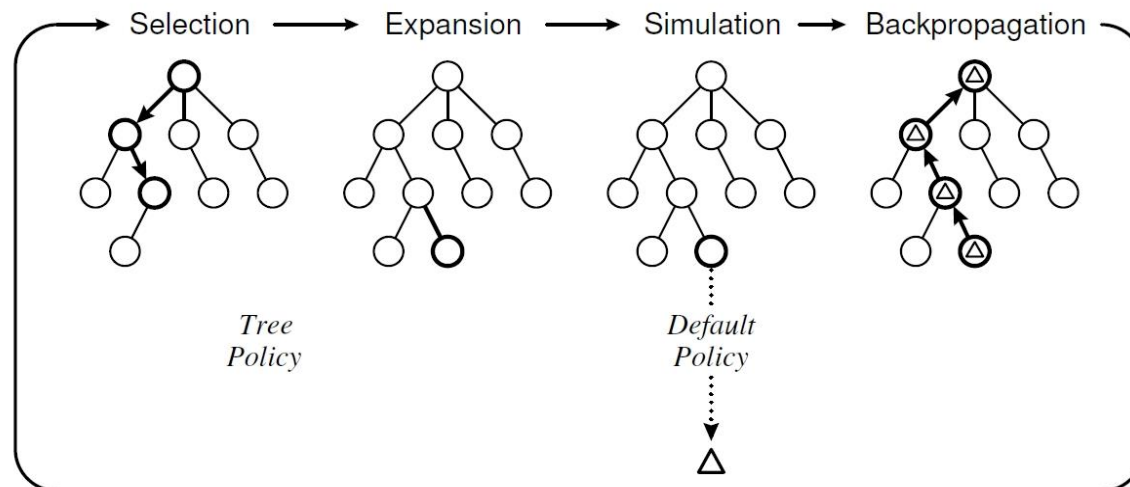
Monte Carlo Tree Search

- Algoritmul construiește progresiv un **arbore de joc partial**, ghidat de rezultatele explorărilor anterioare ale acestui arbore
- Arborele este utilizat pentru a **estima valoarea mișcărilor**, estimările devenind din ce în ce mai bune pe măsura ce arborele este construit
- Algoritmul implică construirea iterativă a arborelui de căutare până când o anumită cantitate de efort s-a atins, și întoarce cea mai bună acțiune găsită

Monte Carlo Tree Search

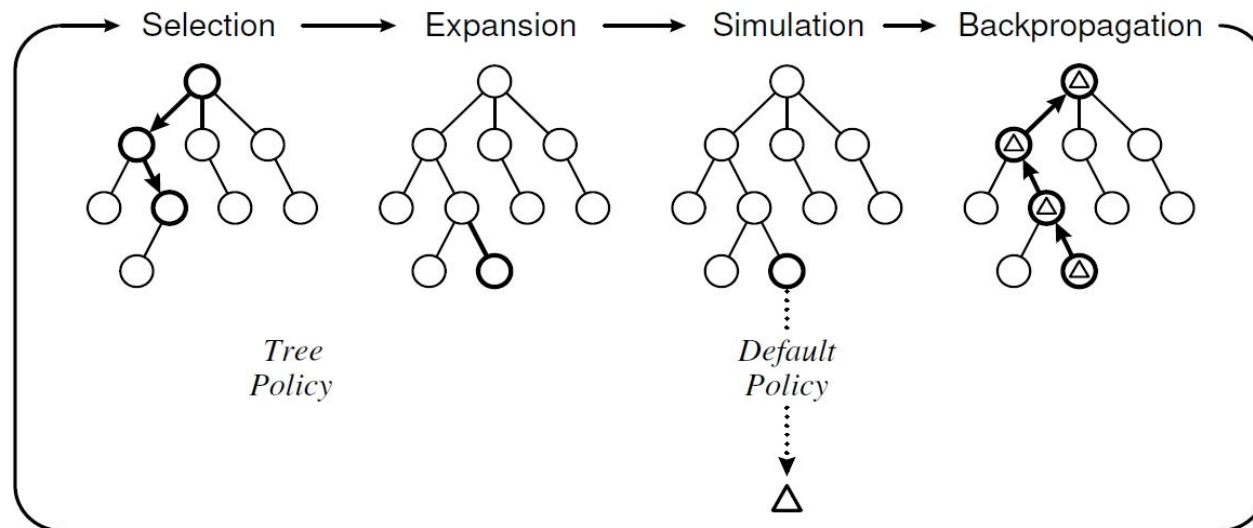
Pentru fiecare iteratie se aplica 4 pasi:

- **Selectie** – Pornind de la radacina o politica de selectie a copiilor este aplicata recursiv pana cand se gaseste cel mai interesant nod neexpandat (un nod E care are un copil ce nu este inca parte a arborelui)
- **Expandare** – Un nod copil (sau mai multe noduri copii) a lui E este adaugate in arbore cf. actiunilor disponibile



Monte Carlo Tree Search

- **Simulare** – Se executa o simulare de la nodul/nodurile noi cf. politicii implicite pentru a obtine un rezultat R
- **Backpropagation** – rezultatul simulatii este propagat inapoi catre nodurile care au fost parcurse si se actualizeaza valorile acestora



Monte Carlo Tree Search

Algoritm MCTS(Radacina) **intoarce** cea mai buna miscare

Creaza nodul radacina v_0 cu starea s_0

cat timp nu s-au epuizat resursele **repeta**

$v_1 \leftarrow \text{TreePolicy}(v_0)$

$\Delta \leftarrow \text{DefaultPolicy}(v_1)$

$\text{BackUp}(v_1, \Delta)$

intoarce $a(\text{BestChild}(v_0))$

- **TreePolicy** – construiește un nod frunza din nodurile aflate deja în arborele de cautare; nodul la care se ajunge cu TreePolicy este v_1
- **DefaultPolicy** – joacă jocul dintr-o stare neterminală v_1 pentru a produce o estimare a valorii / recompensa (pană stare terminală)
- **$a(\text{BestChild}(v_0))$** – acțiunea care selectează cel mai bun copil al lui v_0

Algoritm MCTS(Radacina) **intoarce** cea mai buna miscare

Creaza nodul radacina v_0 cu starea s_0

cat timp nu s-au epuizat resursele **repeta**

$v_1 \leftarrow \text{TreePolicy}(v_0)$

$\Delta \leftarrow \text{DefaultPolicy}(v_1)$

$\text{BackUp}(v_1, \Delta)$

intoarce $a(\text{BestChild}(v_0))$

TreePolicy(v) **intoarce** un nod

cat timp v este nod neterminal **executa**

daca v nu este complet expandat **atunci** $v \leftarrow \text{Expand}(v)$; **break**

altfel $v \leftarrow \text{BestChild}(v)$

intoarce v

Expand(v) **intoarce** un nod

alege $a \in \text{actiunile neincercate inca din } A(v)$ /* $A(v)$ act legale in v */

adauga un copil nou v' la v cu $v' = \text{next}(v, a)$ /* aleator sau in fct de merit */

intoarce v'

Algoritm MCTS(Radacina) **intoarce** cea mai buna miscare

Creaza nodul radacina v_0 cu starea s_0

cat timp nu s-au epuizat resursele **repeta**

$v_1 \leftarrow \text{TreePolicy}(v_0)$

$\Delta \leftarrow \text{DefaultPolicy}(v_1)$

$\text{BackUp}(v_1, \Delta)$

intoarce $a(\text{BestChild}(v_0))$

DefaultPolicy(s) **intoarce** recompensa

cat timp s este nod neterminal **executa**

alege aleator $a \in A(s)$ /* sau cf unei politici de selectie */

$s \leftarrow \text{next}(s, a)$

intoarce recompensa pentru starea s

BackUp(v, Δ)

cat timp v nu este null **executa**

$N(v) \leftarrow N(v) + 1$ /* numar vizitari nod v */

$Q(v) \leftarrow Q(v) + \Delta(v)$ /* recompensa nod */

$v \leftarrow p$ /* p parintele lui v */

Monte Carlo Tree Search

Ce strategii se folosesc pt fiecare pas?

- **Selectia**
- Problema exploatarei vs explorare
 - Selectam stari din care s-a castigat des si au fost parcurse de multe ori
 - Ori selectam stari cu putine simulari anterioare
- Diferite strategii propuse in literatura

Monte Carlo Tree Search

Exemplu de strategie pentru selectie

Fie I multimea de noduri succesoare nodului curent p . Se selecteaza copilul K a nodului p care satisface formula

$$K \in \arg \max_{i \in I} \left(\overset{\text{Exploatare}}{\frac{Q(i)}{N(i)}} + C * \overset{\text{Explorare}}{\sqrt{\frac{2 * \ln N(p)}{N(i)}}} \right)$$

$Q(i)$ - recompensa nodului i

$N(i)$ – numarul de vizitari a nodului i

$N(p)$ – numarul de vizitari a nodului p (parinte)

C – coeficient experimental

BestChild(v) intoarce K

Monte Carlo Tree Search

Expandarea

Se genereaza un nou de unde se va incepe simularea

Simularea

- Total aleator, sau pseudoaleator, sau in functie de o politica

Backpropagation

- Diferite metode

$$V_p = \frac{V_{med} * W_{med} + V_r * N_r}{W_{med} * N_r}$$

$$\Delta(v) = V_p$$

V_{med} – media valorilor nodurilor copii

W_{med} – ponderea acestei medii

V_r – mutarea cu cel mai mare numar de simulari

N_r – numarul de ori de care s-a jucat V_r

Monte Carlo Tree Search

- Converge spre valorile Minimax, dar lent
- Cu toate acestea mai eficient decat AlfaBeta
- Este un algoritm de tip anytime
- Algoritmul nu gaseste întotdeauna cea mai buna mutare, dar are, în general, un succes rezonabil în cazul alegerii mutarilor care duc la sanse mari de câstig
- Poate sa nu detecteze o ramura care conduce la pierdere (din cauza componentei aleatoare)
- Light playouts – aleator
- Heavy playouts – euristici/politici pentru selectarea miscarii urmatoare (nu strict aleator)

Monte Carlo Tree Search

MoGo

A participat in 30 de turnee intre 2006 si 2010 (9 x 9 GO)

A castigat contra profesionistilor

- MoGo invinge pe campionul Myungwan Kim, august 2008, utilizand MCTS
- **FUEGO** - 2009 – a invins mai multi campioni la 9 x 9 GO



Monte Carlo Tree Search

2009 – **MoHex** devine campion la jocul **Hex**

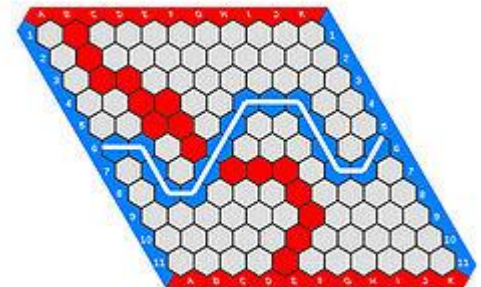
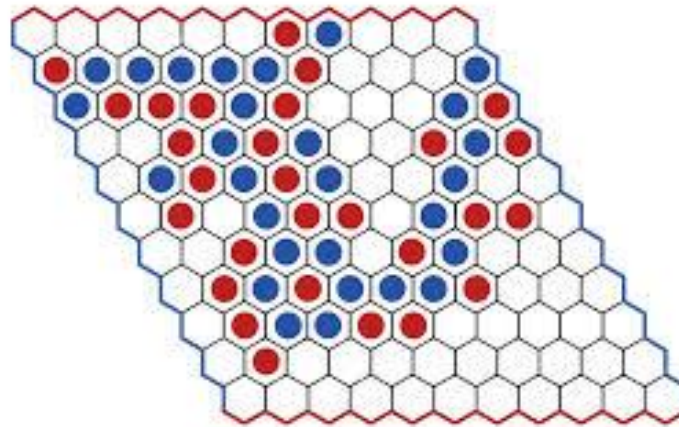
11 x 11

13 x 13

14 x 14

19 x 19

Joc inventat in 1940
de Piet Hein and John Nash



5 Invatarea functiei de evaluare pentru selectie

- Functia de evaluare – depinde de parametrii starii si o serie de ponderi ale acestora

$$Eval_w(u)$$

- Daca avem evaluari corecte ale unor stari $Eval^*(u)$ putem aplica **metoda scaderii gradientului** pentru a reduce **eroarea**

$$L = (Eval_w(u) - Eval^*(u))^2$$

$$\Delta w = \alpha (Eval^*(u) - Eval_w(u)) * \nabla Eval_w(u)$$

Q-Learning

Temporal credit assignment problem

Q-Learning

- **Funcție acțiune-valoare** = atribuie o utilitate estimată executării unei acțiuni într-o stare
- $Q: A \times S \rightarrow U$
- Folosește o matrice (stare, acțiune) - valoare

Q-Learning

Foloseste **ecuatia lui Bellman**

$Q(a,s) \leftarrow$

$$Q(a,s) + \underline{\alpha}(R(s) + \delta \max_{a'} Q(a', s') - Q(a,s))$$

calculata dupa fiecare tranzitie din s in s' .

α - viteza de invatare

Q-Learning

1. Initializeaza δ si matricea $\mathbf{R}$
2. Initializeaza $\mathbf{Q}$ la 0 sau aleator
3. Pentru fiecare episod repeta
 - Selecteaza starea initiala $\mathbf{s}$ aleator
 - **cat timp** $\mathbf{s}$ nu este stare scop **repeta**
 - obtine recompensa $\mathbf{R}(\mathbf{s})$
 - selecteaza o actiune $\mathbf{a}$ din starea $\mathbf{s}$
 - executa trecerea in $\mathbf{s}'$ cu $\mathbf{a}$
 - $\mathbf{Q}(\mathbf{a}, \mathbf{s}) \leftarrow$
 $\mathbf{Q}(\mathbf{a}, \mathbf{s}) + \alpha(\mathbf{R}(\mathbf{s}) + \delta \max_{\mathbf{a}'} \mathbf{Q}(\mathbf{a}', \mathbf{s}') - \mathbf{Q}(\mathbf{a}, \mathbf{s}))$
 - $\mathbf{s} \leftarrow \mathbf{s}'$

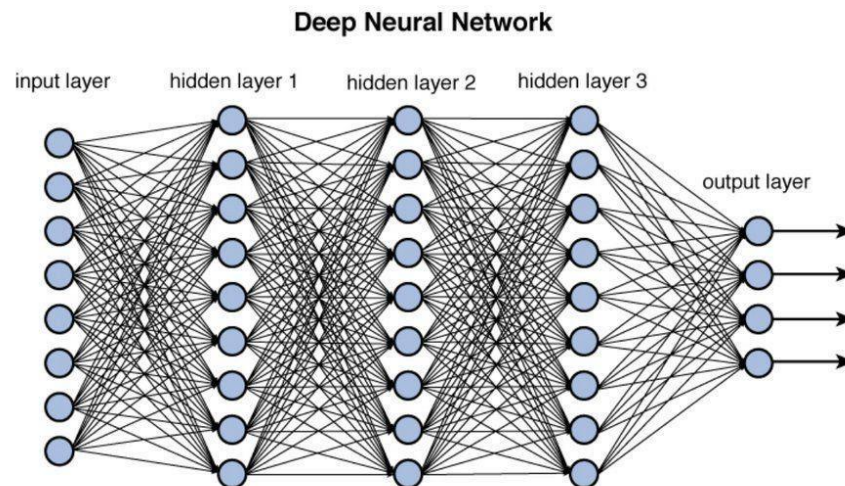
Q-Learning

Utilizare Q

1. $s \leftarrow$ starea initiala
2. gaseste actiunea **a** care maximizeaza $Q(a,s)$
3. Executa **a**
4. **Repeta** de la 2 **pana** stare scop

Deep Q-Learning

- Combina Q-Learning cu DNN
- Reteaua neurala invata estimarea perechilor stare-actiune numai pe baza descrierii starii si a recompensei, fara a sti regulile de joc

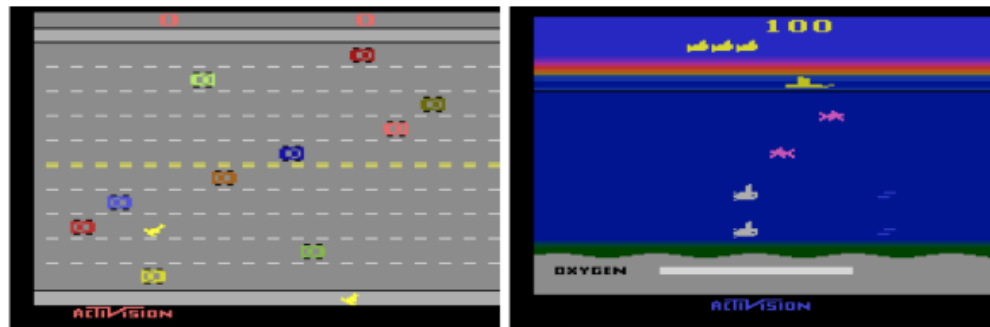


Atari Deep Q-Learning

- Deep learning – extrage caracteristici de nivel inalt din imagini

Atari

- O retea convolutionala este antrenata sa invete evaluarea stare-actiune pt a aplica RL
- Intrare: pixelii din imagine
- Iesire: valori Q



Two classic Atari 2600 video games, “Freeway” and “Seaquest”, from the Arcade Learning Environment

Atari Deep Q-Learning

- Selectează acțiunea
- Acțiunea trimisă emulatorului de joc care modifică starea și scorul jocului
- Agentul primește o imagine (vectori de pixeli) și recompensă = schimbarea scorului
- Consideră o secvență de stări și observații

$$S_t = s_1, a_1, s_2, a_2, \dots, a_{t-1}, s_t$$

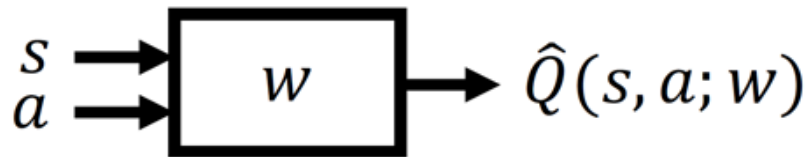
și învață strategia în funcție de astfel de secvențe

Atari Deep Q-Learning

- Foloseste un Q-network cu parametrii w care minimizeaza functia de eroare (**Loss**)

$$L(w) = \text{Estimare } [y - Q(s,a,w)]^2$$

$$y = R(s) + \delta \max_{a'} Q(s',a')$$



AlphaGo

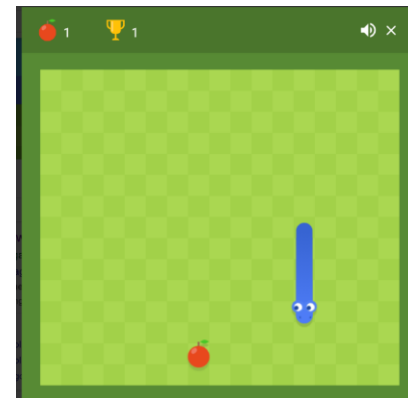
- Octombrie 2015 - AlphaGo joaca impotriva campionului european Fan Hui si castiga cu un scor de 5-0
- In 2016 castiga in fata lui Lee Sedol, detinator a 18 titluri mondiale
- A obtinut cel mai mare scor
- A inventat mutari
- Foloseste Deep Q-Learning

AlphaGo

- Foloseste:
 - MCTS
 - 2 rețele neurale adanci: una pentru a invata politica de selectie si una pentru a invata evaluarea nodurilor
 - Algoritmul de Q-Learning – initializat cu politica invatata si imbunatatita in functie de recompensele viitoare

Un exemplu simplu

- **Jocul Snake**
- Se dau sistemului parametrii legati de stare si recompensa
- Nu necesita reguli de joc pt invatarea politicii
- Trebuie sa maximizeze recompensa



Un exemplu simplu

■ Tabela Q

Stare	Right	Left	Up	Down
1	0	0.31	0.12	0.87
2	0.98	-0,21	0.01	0.14

Se actualizeaza cu Q-Learning si ecuatia lui Bellman

$Q(a,s) \leftarrow$

$$Q(a,s) + \alpha(R(s) + \delta \max_{a'} Q(a', s') - Q(a,s))$$

1. Initializeaza Q aleator
2. Initializeaza δ si matricea R
3. Pentru fiecare episod repeta

Obține starea curenta s

Executa o actiune a (aleator sau selectata de RNA)

Obține recompensa $R(s)$

Executa trecerea in s' cu a

$Q(a,s) \leftarrow$

$$Q(a,s) + \alpha(R(s) + \delta \max_{a'} Q(a', s') - Q(a,s))$$

$s \leftarrow s'$

Reprezentare stare pentru retea

11 variabile boolene

- Daca este pericol in jurul sarpelui (Right, Left, Straight)
- Daca sarpele se misca Up, Down, Left, Right
- Daca mancarea este Up, Down, Left, Right
- RNA estimeaza actiunea cea mai buna pentru o anumita stare incercand sa maximizeze recompensa pe baza functiei de eroare (**Loss**)

$$L_i(\theta_i) = [R(s) + \delta \max_{a'} Q(s', a') - Q(s, a, \theta_i)]^2$$

Inteligență Artificială

Universitatea Politehnica Bucuresti
Anul universitar 2022-2023

Adina Magda Florea

Curs nr. 5

Reprezentarea cunostintelor incerte

- Teoria probabilitatilor
- Retele Bayesiene

1. Teoria probabilitatilor

1.1 Cunostinte incerte

- **Teoria probabilitatilor** → un grad numeric de **incredere** sau **plauzibilitate** a afirmatiilor in $[0,1]$
- Valorile variabilelor pot fi din diferite domenii si acestor valori li se adauga un grad de **incredere/plauzibilitate**
- **Fuzzy logic** → grad numeric de **adevar** / al adevarului
- Valorile variabilelor arata un grad de **adevar** in $[0,1]$
- Gradul de **incredere** $\neq$ gradul de **adevar**

1.2 Definitii TP

- **Probabilitatea unui eveniment incert A** este masura gradului de incredere sau plauzibilitatea produceri unui eveniment
- Camp de probabilitate, S
- **Probabilitate neconditionata** (apriori) - inaintea obtinerii de probe pt o ipoteza / eveniment
- **Probabilitate conditionata** (aposteriori) - dupa obtinerea de probe

Exemple

$$P(\text{Carie}) = 0.1$$

$$P(\text{Vreme} = \text{Soare}) = 0.7$$

$$P(\text{Vreme} = \text{Ploaie}) = 0.2 \quad P(\text{Vreme} = \text{Nor}) = 0.1$$

Vreme - **variabila aleatoare**

- **Distributie de probabilitate**

Definitii TP - cont

- **Probabilitate conditionata** (aposteriori) - $P(A|B)$

$$P(\text{Carie} \mid \text{Dur_d}) = 0.8$$

Măsura probabilității producerii unui eveniment A este o funcție $P:S \rightarrow R$ care satisface **axiomele**:

- $0 \leq P(A) \leq 1$
- $P(S) = 1$ (sau $P(\text{adev}) = 1$ și $P(\text{fals}) = 0$)
- $P(A \vee B) = P(A) + P(B) - P(A \wedge B)$

$$P(A \vee \sim A) = P(A) + P(\sim A) - P(\text{fals}) = P(\text{adev})$$

$$\Rightarrow P(\sim A) = 1 - P(A)$$

Definitii TP - cont

Evenimente **mutual exclusive**

Moneda – cap/pajura – mutual exclusive

Zar – 1, 2, 3 – mutual exclusive

Evenimente **exhaustive**

Moneda – cap/pajura – mutual exhaustive

Zar – 1, 2, 3, 4, 5, 6 – mutual exhaustive

Definitii TP - cont

A si B **mutual exclusive** →

$$P(A \vee B) = P(A) + P(B)$$

$$P(e_1 \vee e_2 \vee e_3 \vee \dots e_n) = \\ P(e_1) + P(e_2) + P(e_3) + \dots + P(e_n)$$

$e(a)$ – multimea de evenimente atomice mutual exclusive si exhaustive in care apare **a**

$$P(a) = \sum_{e_i \in e(a)} P(e_i)$$

1.3 Regula produsului

Probabilitatea conditionata de producere a evenimentului A in conditiile producerii evenimentului B

- $P(A|B) = P(A \wedge B) / P(B)$

$P(A \wedge B) = P(A|B) * P(B)$ – regula
produsului

Regula produsului

- A si B sunt **evenimente independente** daca

$$P(A|B) = P(A)$$

- In acest caz regula produsului devine

$$P(A \wedge B) = P(A) * P(B)$$

- Daca A si B sunt **evenimente independente conditional** fiind dat C

$$P(A \wedge B|C) = P(A|C) * P(B|C)$$

1.4 Teorema lui Bayes

$P(A|B) = P(A \wedge B) / P(B)$ – regula produsului

$$P(A|B) = P(A \wedge B) / P(B)$$

$$P(B|A) = P(A \wedge B) / P(A)$$



$$P(B|A) = P(A|B) * P(B) / P(A)$$

APosteriori = Plauzibilitate x APriori / Evidenta

Teorema lui Bayes

$$P(B|A) = P(A|B) * P(B) / P(A)$$

- Daca **B** si **~B** sunt mutual exclusive si exhaustive, probabilitatea de producere a lui A in conditiile producerii lui B se poate scrie

$$P(A) = P(A \wedge B) + P(A \wedge \sim B) = P(A|B)*P(B) + P(A| \sim B)*P(\sim B)$$

$$P(B|A) = \frac{P(A | B) * P(B)}{[P(A|B)*P(B) + P(A| \sim B)*P(\sim B)]}$$

Teorema lui Bayes - generalizare

Notand cu B - h, A - e, rescriu formula precedenta

$$P(h|e) = P(e | h) * P(h) / [P(e|h)*P(h) + P(e| \sim h)*P(\sim h)]$$

Generalizarea la mai multe ipoteze h_i

Daca h_i mutual exclusive si exhaustive, $i=1,k$

$$P(h_i|e) = \frac{P(e|h_i) \cdot P(h_i)}{\sum_{j=1}^k P(e|h_j) \cdot P(h_j)}, \quad i = 1, k$$

Teorema lui Bayes generala

Generalizarea la mai multe ipoteze h_i si probe e_i

h_i – evenimente / ipoteze probabile ($i=1,k$);

$e_1, \dots, e_n$ – probe (evenimente)

$P(h_i)$

$P(h_i | e_1, \dots, e_n)$

$P(e_1, \dots, e_n | h_i)$

$$P(h_i | e_1, e_2, \dots, e_n) = \frac{P(e_1, e_2, \dots, e_n | h_i) \cdot P(h_i)}{\sum_{j=1}^k P(e_1, e_2, \dots, e_n | h_j) \cdot P(h_j)}, \quad i = 1, k$$

Teorema lui Bayes - cont

$$P(h_i|e_1, e_2, \dots, e_n) = \frac{P(e_1, e_2, \dots, e_n|h_i) \cdot P(h_i)}{\sum_{j=1}^k P(e_1, e_2, \dots, e_n|h_j) \cdot P(h_j)}, \quad i = 1, k$$

Daca $e_1, \dots, e_n$ sunt ipoteze independente conditional fiind dat h_j atunci

$$P(e|h_j) = P(e_1, e_2, \dots, e_n|h_j) = P(e_1|h_j) \cdot P(e_2|h_j) \cdot \dots \cdot P(e_n|h_j), \quad j = 1, k$$

PROSPECTOR – sistem expert pentru consultari privind exploatare miniere si evaluarea resurselor

1.5 Inferente pe baza Distributiei de Probabilitate si a Teoremei lui Bayes

Distributie de probabilitate

P(Carie, Dur_d)

	Dur_d	~Dur_d
Carie	0.04	0.06
~Carie	0.01	0.89

$$P(\text{Carie}) = 0.04 + 0.06 = 0.1$$

$$P(\text{Carie} \vee \text{Dur_d}) = 0.04 + 0.01 + 0.06 = 0.11$$

$$P(\text{Carie}|\text{Dur_d}) = P(\text{Carie} \wedge \text{Dur_d}) / P(\text{Dur_d}) = 0.04 / 0.05$$

Inferente din DP si TB

	Dur_d		~Dur_d	
	Evid	~Evid	Evid	~Evid
Carie	0.108	0.012	0.072	0.008
~Carie	0.016	0.064	0.144	0.576

Distributie de probabilitate

P(Carie, Dur_d, Evid)

$$P(\text{Carie}) = 0.108 + 0.012 + 0.072 + 0.008 = 0.2$$

$$P(\text{Carie} \vee \text{Dur_d}) = 0.108 + 0.012 + 0.072 + 0.008 + 0.016 + 0.064 = 0.28$$

P(Carie, Dur_d, Vreme) – o tabela cu $2 \times 2 \times 3 = 12$ intrari
Distributie de probabilitate completa

Inferente din DP si TB

	Dur_d		~Dur_d	
	Evid	~Evid	Evid	~Evid
Carie	0.108	0.012	0.072	0.008
~Carie	0.016	0.064	0.144	0.576

$$P(\text{Carie} \mid \text{Dur_d}) = P(\text{Carie} \wedge \text{Dur_d}) / P(\text{Dur_d})$$

$$P(\sim\text{Carie} \mid \text{Dur_d}) = P(\sim\text{Carie} \wedge \text{Dur_d}) / P(\text{Dur_d})$$

$\alpha = 1 / P(\text{Dur_d})$ – constanta de normalizare a distributiei

$$P(\text{Carie} \mid \text{Dur_d}) = \alpha P(\text{Carie} \wedge \text{Dur_d}) =$$

$$\alpha [P(\text{Carie} \wedge \text{Dur_d} \wedge \text{Evid}) + P(\text{Carie} \wedge \text{Dur_d} \wedge \sim\text{Evid})] =$$

$$\alpha [<0.108, 0.016> + <0.012, 0.064>] = \alpha <0.12, 0.08> = <0.6, 0.4>$$

Chiar daca nu cunoastem α , adica $P(\text{Dur_d})$, putem calcula α

$$\alpha = 1/(0.12+0.08) = 1/0.2$$

Inferente din DP si TB

Generalizare – procedura generala de inferenta bazata pe DP

Interogare asupra lui X

X – variabila de interogat (in ex anterior Carie)

E – lista de variabile probe (in ex anterior Dur_d)

e – lista valorilor observate pt aceste variabile E

Y – lista variabilelor neobservate (restul) (in ex anterior Evid)

$$\mathbf{P(Carie \mid Dur_d)} = \alpha [P(Carie \wedge Dur_d \wedge Evid) + P(Carie \wedge Dur_d \wedge \sim Evid)]$$

Insumarea se face peste toate combinatiile de valori ale variab. neobservate Y

$$\mathbf{P(X \mid e)} = \alpha \mathbf{P(X, e)} = \alpha \sum_{Y=y} \mathbf{P(X, e, Y)}$$

Inferente din DP si TB

$$\mathbf{P}(X \mid e) = \alpha \mathbf{P}(X, e) = \alpha \sum_{Y=y} \mathbf{P}(X, e, Y)$$

Avand DP ecuatia poate da raspuns la
interogari cu variabile discrete

Complex computational

n var Bool – tabela $O(2^n)$

- timp $O(2^n)$

1.6 Independenta conditionala

- Evenimentele **A** si **B** sunt independente conditional fiind dat un eveniment **C** daca

Stiind ca C apare, aparitia lui A nu influenteaza aparitia lui B si aparitia lui B nu influenteaza aparitia lui A

$$P(A|C) = P(A|B,C)$$

- A** si **B** sunt independente conditional fiind dat un eveniment **C** daca, pentru orice valoare a lui C:

- distributia de probabilitate a lui A este aceeaasi pentru orice valoare ar lua B
- distributia de probabilitate a lui B este aceeaasi pentru orice valoare ar lua A

$$P(A,B|C) = P(A|C)*P(B|C)$$

Independenta conditionala

$$\mathbf{P}(\text{cauza} \mid \text{efect}) = \mathbf{P}(\text{efect} \mid \text{cauza}) * \mathbf{P}(\text{cauza}) / \mathbf{P}(\text{efect})$$

$$\mathbf{P}(y \mid x_1, \dots, x_n) = P(x_1, \dots, x_n \mid y) * \mathbf{P}(y) / \mathbf{P}(x_1, \dots, x_n)$$

$$\mathbf{P}(y \mid x_1, \dots, x_n) = \alpha * P(x_1, \dots, x_n \mid y) * \mathbf{P}(y)$$

Daca $\mathbf{x}_1, \dots, \mathbf{x}_n$ sunt independente conditional fiind dat y atunci

$$P(x_1, \dots, x_n \mid y) = \prod_i P(x_i \mid y)$$

$$\begin{aligned} \mathbf{P}(\text{Cauza} \mid \text{Efect}_1, \text{Efect}_2, \dots) = \\ \alpha \mathbf{P}(\text{Cauza}) * \prod_i \mathbf{P}(\text{Efect}_i \mid \text{Cauza}) \end{aligned}$$

1.7 Model Bayesian naiv

$$\mathbf{P}(\text{Cauza}_y \mid \text{Efect}_1, \text{Efect}_2, \dots) = \alpha \mathbf{P}(\text{Cauza}_y) * \prod_i \mathbf{P}(\text{Efect}_i \mid \text{Cauza}_y)$$

Ipoteza de independenta
conditionala / naiva

$$\text{Cauza_MAP} = \operatorname{argmax}_y \mathbf{P}(\text{Cauza}_y) * \prod_i \mathbf{P}(\text{Efect}_i \mid \text{Cauza}_y)$$

Model Bayesian naiv

Model Bayesian naiv

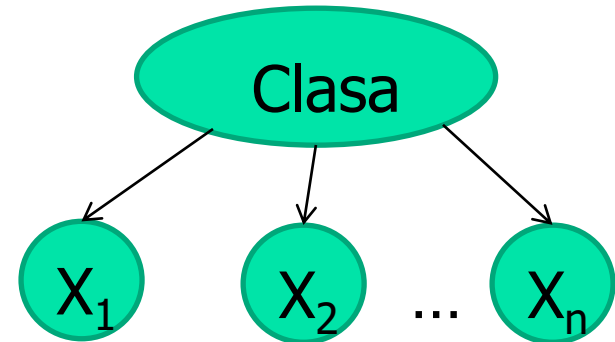
Vector de caracteristici $x_1, \dots, x_n$ si o serie de clase C_k

$$\mathbf{P}(C_k \mid x_1, x_2, \dots) = \alpha \mathbf{P}(C_k) * \prod_i \mathbf{P}(x_i \mid C_k)$$

$$C_MAP = \operatorname{argmax}_k \mathbf{P}(C_k) * \prod_i \mathbf{P}(x_i \mid C_k)$$

Model Bayesian naiv

MAP – Maximum a Posteriori



1.8 Modele grafice probabiliste

- Fiecare nod reprezinta o variabila aleatoare si fiecare legatura reprezinta o relatie probabilistica
 - Retele Bayesiene – modele grafice orientate - permit reprezentare compacta a DP si punerea in evidenta a independentei conditionale
 - Modele Markov – lanturi Markov – stari si tranzitii probabilistice

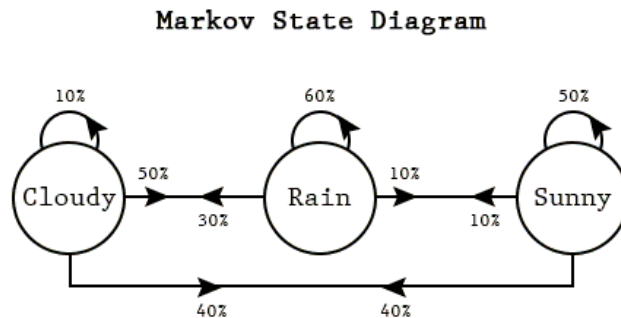
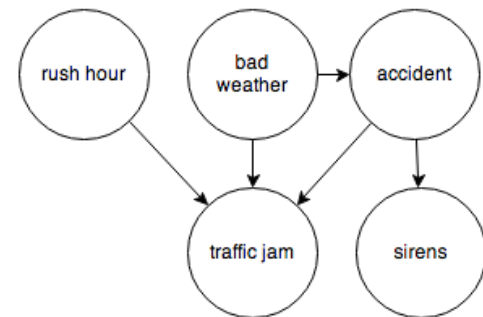


Figure 2



2 Retele Bayesiene

- Reprezinta dependente intre variabile aleatoare
- Specifica distributiei de probabilitate
- Simplifica calculele
- Au asociata o reprezentare grafica convenabila
- DAG care reprezinta relatiile cauzale intre variabile
- Pe baza structurii retelei se pot realiza diverse tipuri de inferente
- Calcule complexe in general dar se pot simplifica pentru structuri particulare

2.1 Structura retelelor Bayesiene

O RB este un DAG in care:

- Nodurile reprezinta variabilele aleatoare
- Legaturile orientate $X \rightarrow Y$: X are o influenta directa asupra lui Y , $X = \text{Parinte}(Y)$
- Fiecare nod are asociata o tabela de probabilitati conditionate care cuantifica efectul parintilor asupra nodului
 - $P(X_i \mid \text{Parinti}(X_i))$

Structura retelelor Bayesiene - cont

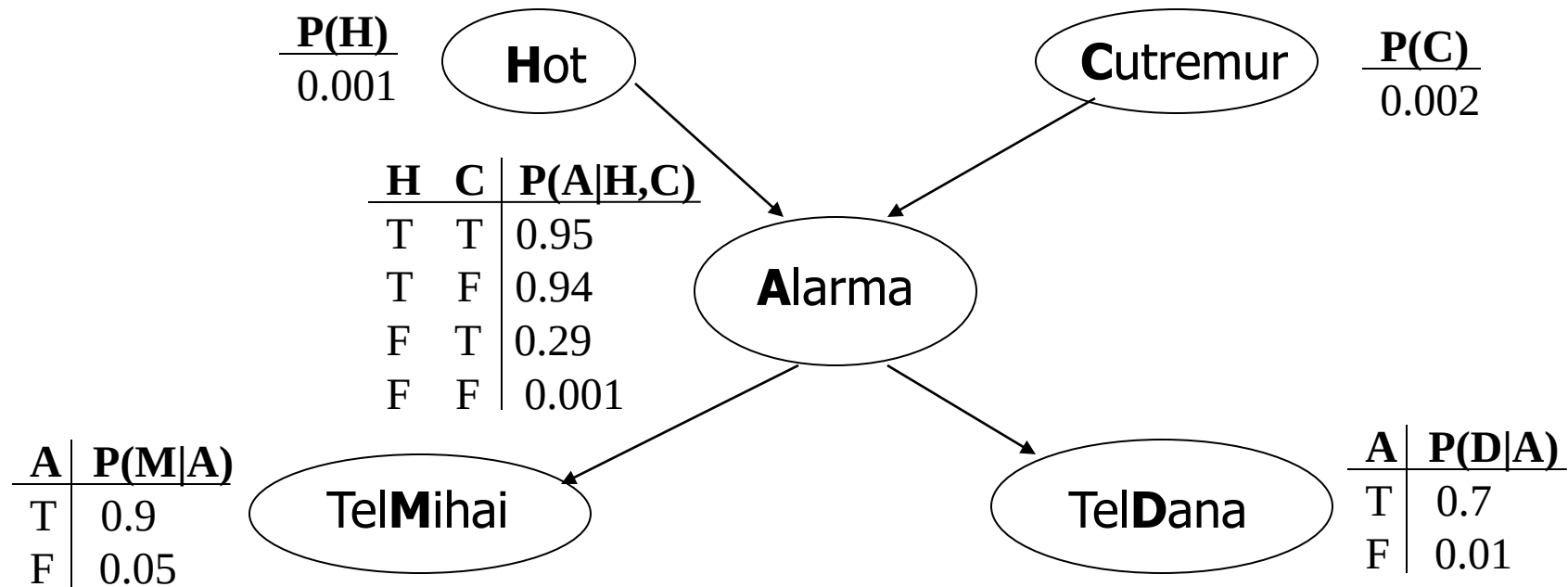


Tabela de probabilitati conditionate	H	C	P(A H, C)	
			T	F
	T	T	0.95	0.05
	T	F	0.94	0.06
	F	T	0.29	0.71
	F	F	0.001	0.999

Structura retelelor Bayesiene

In general X (Cauza) $\rightarrow Y$ (Efect)

- Stabilesc topologia
- Specifica distributia de probabilitati conditionate
- Combinarea topologiei si distributia de probabilitati conditionate este suficienta pentru a specifica (implicit) intreaga DPC
- DPC poate raspunde la interogari
- Si RB la fel, mai eficient

2.2 Semantica retelelor Bayesiene

- Reprezentare a distributiei de probabilitate
- Specificare a independentei conditionale – constructia retelei
- Fiecare valoare din distributia de probabilitate poate fi calculata ca:

$$P(X_1=x_1 \wedge \dots X_n=x_n) = P(x_1, \dots, x_n) = \prod_{i=1,n} P(x_i \mid \text{parinti}(x_i))$$

unde $\text{parinti}(x_i)$ reprezina valorile specifice ale variabilelor $\text{Parinti}(X_i)$

2.3 Construirea rețelei

Cum sa construim o retea a.i. RB/DPC sa fie o buna reprezentare?

Ecuatia $\mathbf{P}(\mathbf{x}_1, \dots, \mathbf{x}_n) = \prod_{i=1, n} \mathbf{P}(\mathbf{x}_i \mid \text{parinti}(\mathbf{x}_i))$ implica anumite relatii de independenta conditionala care pot ghida construirea rețelei

$$\mathbf{P}(X_1=\mathbf{x}_1 \wedge \dots X_n=\mathbf{x}_n) = \mathbf{P}(\mathbf{x}_1, \dots, \mathbf{x}_n) =$$

$$\mathbf{P}(\mathbf{x}_n \mid \mathbf{x}_{n-1}, \dots, \mathbf{x}_1) * \mathbf{P}(\mathbf{x}_{n-1}, \dots, \mathbf{x}_1) = \dots =$$

$$\mathbf{P}(\mathbf{x}_n \mid \mathbf{x}_{n-1}, \dots, \mathbf{x}_1) * \mathbf{P}(\mathbf{x}_{n-1} \mid \mathbf{x}_{n-2}, \dots, \mathbf{x}_1) * \dots * \mathbf{P}(\mathbf{x}_2 \mid \mathbf{x}_1) * \mathbf{P}(\mathbf{x}_1) =$$
$$\prod_{i=1, n} \mathbf{P}(\mathbf{x}_i \mid \mathbf{x}_{i-1}, \dots, \mathbf{x}_1) - \text{valabila in general}$$

- DPC daca, pt fiecare variabila X_i din RB

$$\mathbf{P}(X_i \mid X_{i-1}, \dots, X_1) = \mathbf{P}(\mathbf{x}_i \mid \text{Parinti}(X_i)) \text{ cu conditia ca}$$
$$\text{Parinti}(X_i) \subseteq \{ X_{i-1}, \dots, X_1 \}$$

Construirea rețelei

Pt fiecare variabila X_i din RB

$P(X_i \mid X_{i-1}, \dots, X_1) = P(x_i \mid \text{Parinti}(X_i))$ cu
conditia ca

$$\text{Parinti}(X_i) \subseteq \{X_{i-1}, \dots, X_1\}$$

- O RB este o reprezentare corecta a domeniului cu conditia ca fiecare nod sa fie independent conditional de nondescendenti, fiind dati parintii lui

Construirea rețelei

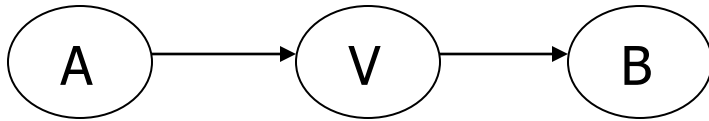
- Condiția poate fi satisfăcută prin etichetarea nodurilor într-o ordine consistentă cu DAG
- Intuitiv, părinții unui nod X_i trebuie să fie toate acele noduri $X_{i-1}, \dots, X_1$ care influențează direct X_i .

Construirea rețelei - cont

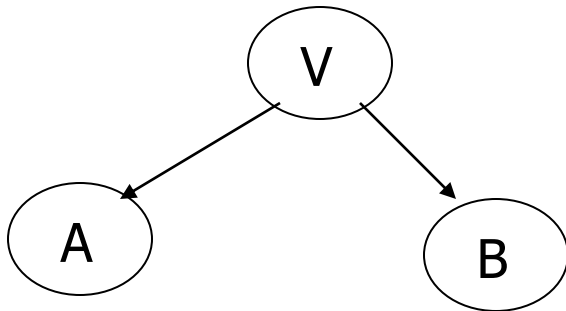
- Alege o multime de variabile aleatoare relevante care descriu problema
- Alege o ordonare a acestor variabile
- **cat timp** mai sunt variabile **repeta**
 - (a) alege o variabila X_i si adauga un nod corespunzator lui X_i
 - (b) atribuie $\text{Parinti}(X_i) \leftarrow$ un set minim de noduri deja existente in retea a.i. proprietatea de independenta conditionala este satisfacuta
 - (c) defineste tabela de probabilitati conditionate pentru X_i

Deoarece fiecare nod este legat numai la noduri anterioare $\rightarrow$
DAG

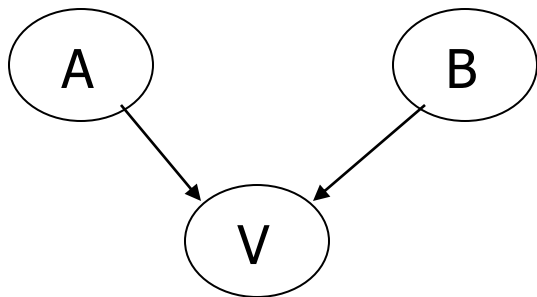
2.4 Inferente probabilistiche



$$P(A \wedge V \wedge B) = P(A) * P(V|A) * P(B|V)$$

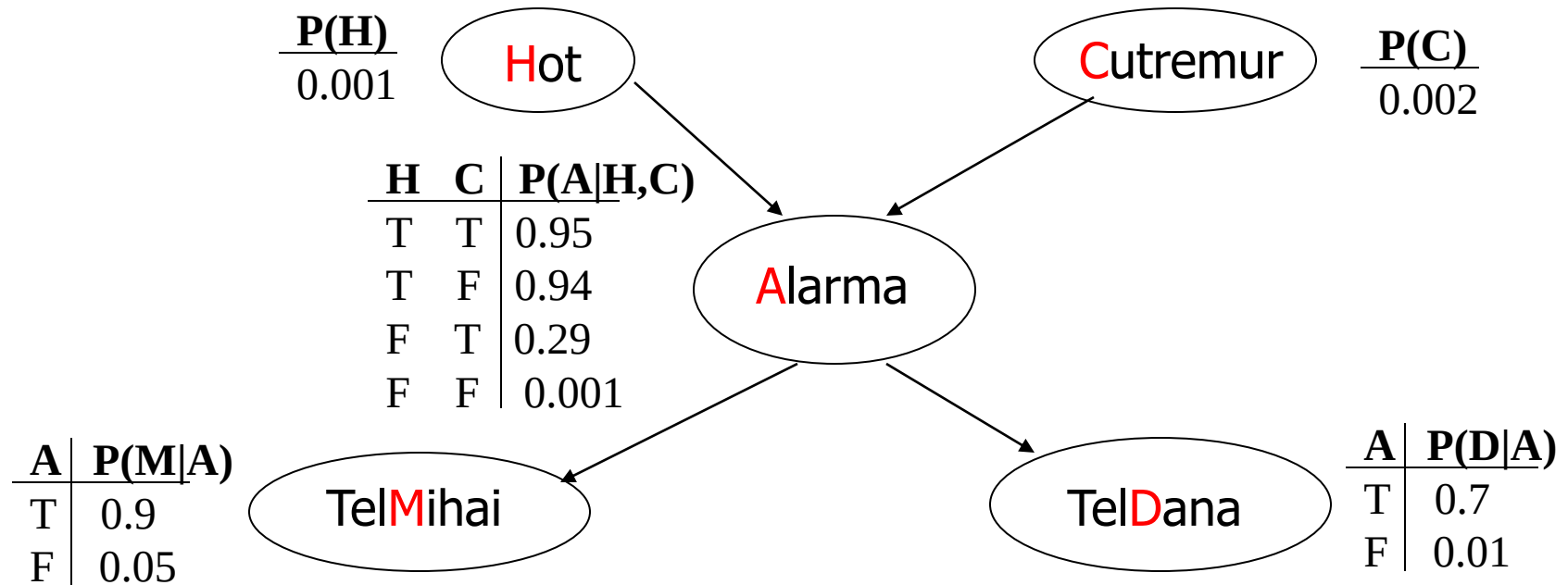


$$P(A \wedge V \wedge B) = P(V) * P(A|V) * P(B|V)$$



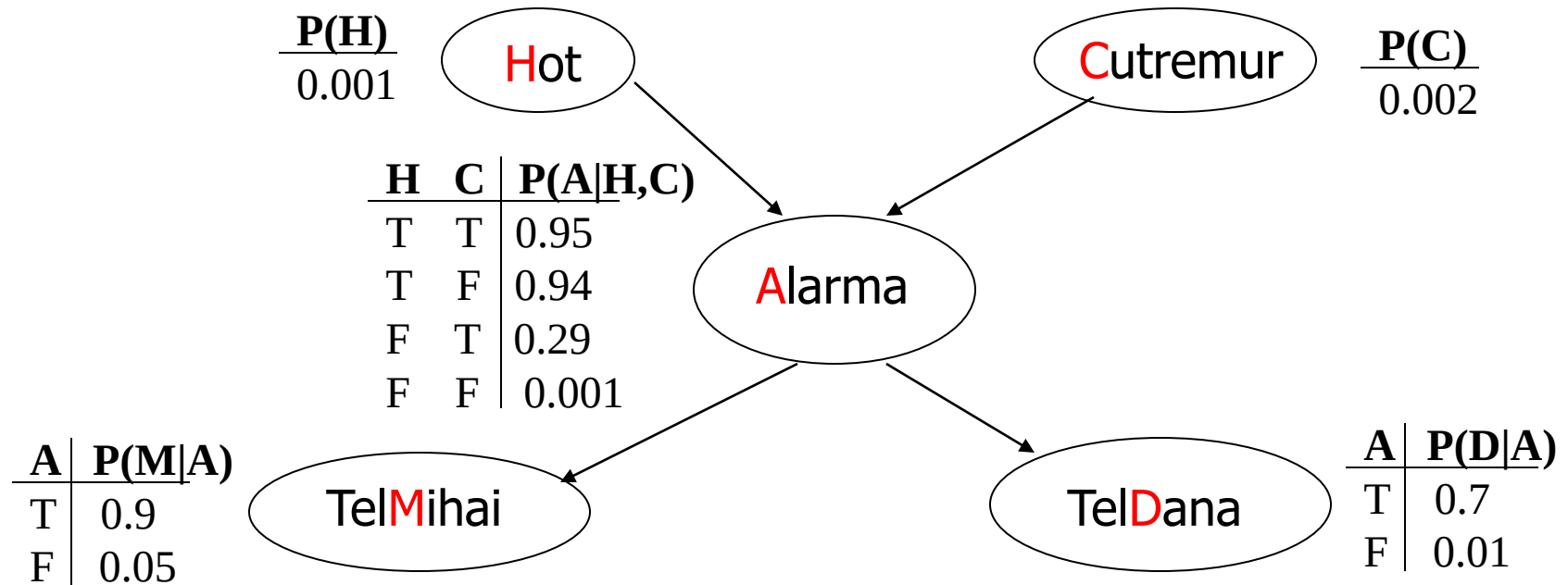
$$P(A \wedge V \wedge B) = P(A) * P(B) * P(V|A,B)$$

Inferente probabilistică



$$\begin{aligned}
 &P(M \wedge D \wedge A \wedge \sim H \wedge \sim C) = \\
 &P(M|A) * P(D|A) * P(A|\sim H \wedge \sim C) * P(\sim H) \wedge P(\sim C) = \\
 &0.9 * 0.7 * 0.001 * 0.999 * 0.998 = 0.00062
 \end{aligned}$$

Inferente probabilistice



$$\begin{aligned}
 P(A|H) &= P(A|H,C) * P(C|H) + P(A|H,\sim C) * P(\sim C|H) \\
 &= P(A|H,C) * P(C) + P(A|H,\sim C) * P(\sim C) \\
 &= 0.95 * 0.002 + 0.94 * 0.998 = 0.94002
 \end{aligned}$$

Inteligență Artificială

Universitatea Politehnica Bucuresti
Anul universitar 2022-2023

Adina Magda Florea

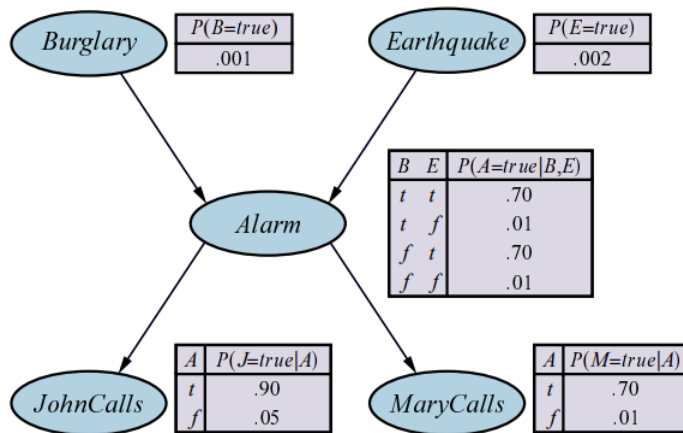
Curs nr. 6

Cunoștințe probabilistice – reprezentare și inferențe

- Independența în rețele Bayesiene
 - D-separare
- Inferențe exacte în rețele Bayesiene
 - Inferențe probabilistice
 - Inferențe prin enumerare
 - Eliminarea variabilelor
- Inferențe aproximative în rețele Bayesiene

1. Reprezentare compacta si ordonarea nodurilor

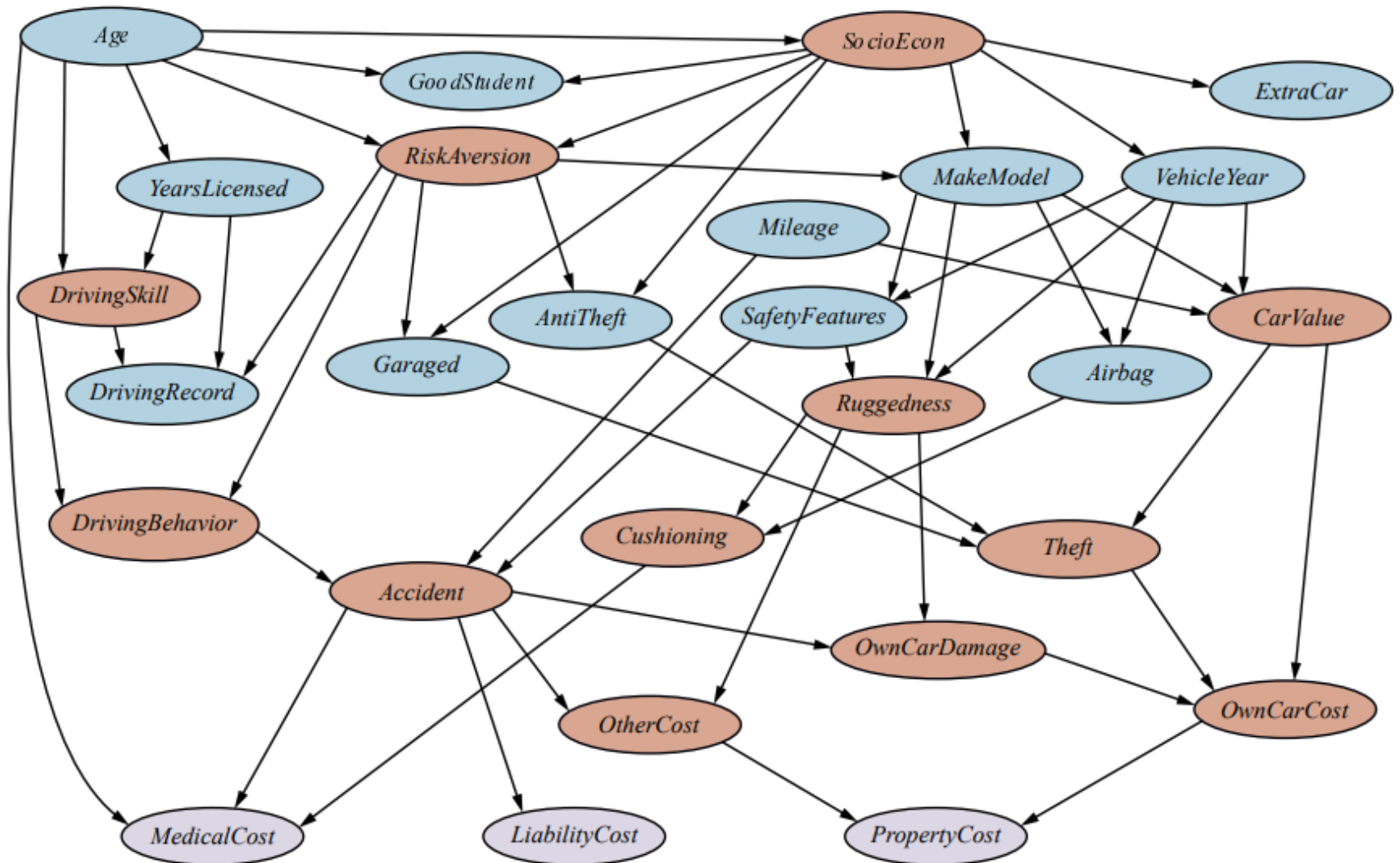
- BN – **reprezentare structurata local** (sparse systems) – fiecare componenta a structurii interactioneaza numai cu un numar limitat de alte componente
- Din cele **n** variabile aleatoare din BN putem presupune ca cel mult **k** alte variabile influenteaza o variabila
- $2^k * n$ numere versus 2^n numere



$n=5, k=2, 2^2*5=20$ vs $2^5=32$ numere

$n=30, k=5, 2^5*30=960$ nr vs 2^{30}

Asigurarea auto



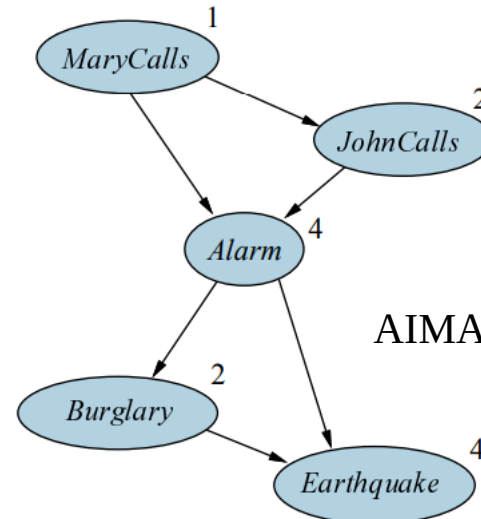
Ordonarea nodurilor

- Reteaua anterioara - ordonare: Burglary, Earthquake, Alarm, JohnCalls, MaryCalls
- Ordonare: **MaryCalls, JohnCalls, Alarm, Burglary, Earthquake**
- Adaug MaryCalls – nu are parinti
- Adaug JohnCalls – probabil influentat de MaryCalls
- Adaug Alarm – influentat de MaryCalls si JohnCalls
- Adaug Burglary – influentat de Alarm
- Adaug Earthquake – daca alarma suna exista posibilitatea sa fie cutremur, dar daca stim ca este hot (lucru care explica alarma) probabilitatea sa fie cutremur este mult mai mica – deci este influentat si de Alarm si de Burglary

■ Model cauzal vs

Model de diagnosticare

Model causal – **reprezentare mai buna**



AIMA, Russel, Norvig

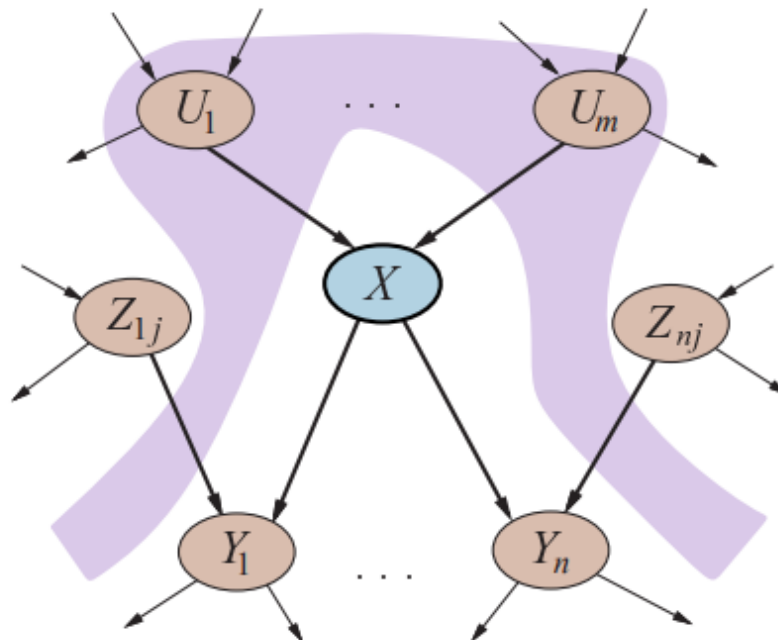
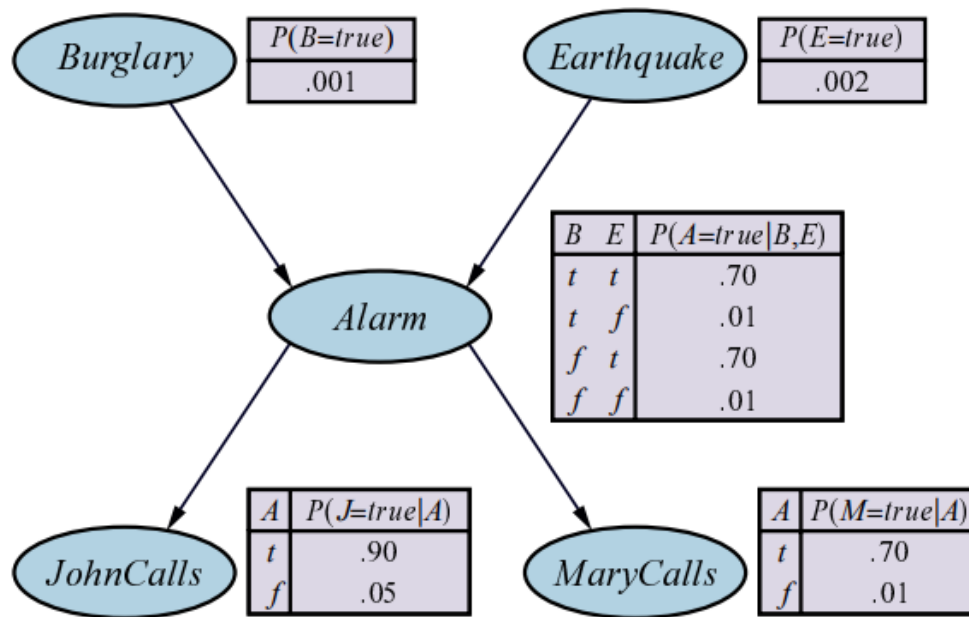
2. Independența în RB

- Evenimentele **X** și **Y** sunt **independente conditional** fiind dat un eveniment **Z** dacă

Știind că **Z** apare, apariția lui **X** nu influențează apariția lui **Y** și apariția lui **Y** nu influențează apariția lui **X**

- Structura de graf RB codifică anumite relații de independență:

fiecare nod este independent conditional de nondescendenți, fiind dați părinții lui



AIMA, Russel, Norvig

Independența in RB

- Mai sunt si alte relatii de independenta?
- Doua sau mai multe relatii de independenta conditionala pot conduce la o noua relatie folosind un mecanism bazat pe axiome ale grafurilor
- **D-separarea** este un concept care surprinde astfel de relatii de independenta conditionala derivate
- Z **d-separa** pe X si Y intr-un DAG daca X si Y sunt independente conditional fiind dat Z dpv al axiomelor grafului.
- $(X \perp Y \mid Z)$ – X este independent conditional de Y fiind dat Z

D-separare

D-separare

- Cand nu este indeplinita $(X \perp Y \mid Z)$?

Conexiuni directe $X \rightarrow Y$

- Se pot influenta indiferent de Z

Conexiuni indirecte: X si Y nu sunt direct conectate dar exista un lant intre ele (trail)

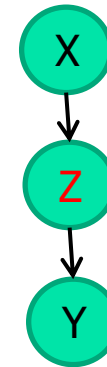
D-separare

- Fie cazul a 3 noduri

Caz a: Efect cauzal

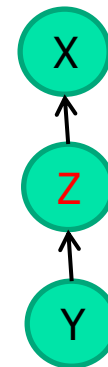
Lantul cauzal $X \rightarrow Z \rightarrow Y$

- X nu influenteaza Y via Z daca Z observat



Caz b: Efect evidenta

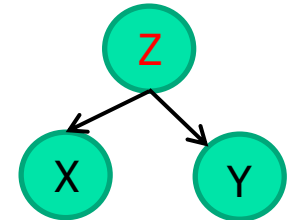
- $Y \rightarrow Z \rightarrow X$
- Identic cu cazul **a**:
- Y nu influenteaza X via Z daca Z observat
- **daca $(X \perp Y \mid Z)$ nu este indeplinit atunci nici $(Y \perp X \mid Z)$ nu este indeplinit**



D-separare

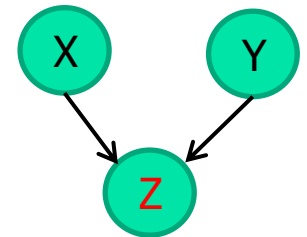
Caz c: Cauza comuna

- $X \leftarrow Z \rightarrow Y$
- X nu influenteaza pe Y via Z daca Z observat



Case d: Efect comun

- $X \rightarrow Z \leftarrow Y$
- Influenta nu poate merge de-a lungul lantului $X \rightarrow Z \leftarrow Y$ daca Z nu este observat



Deci X nu influenteaza pe Y via Z daca Z NU este observat

D-separare

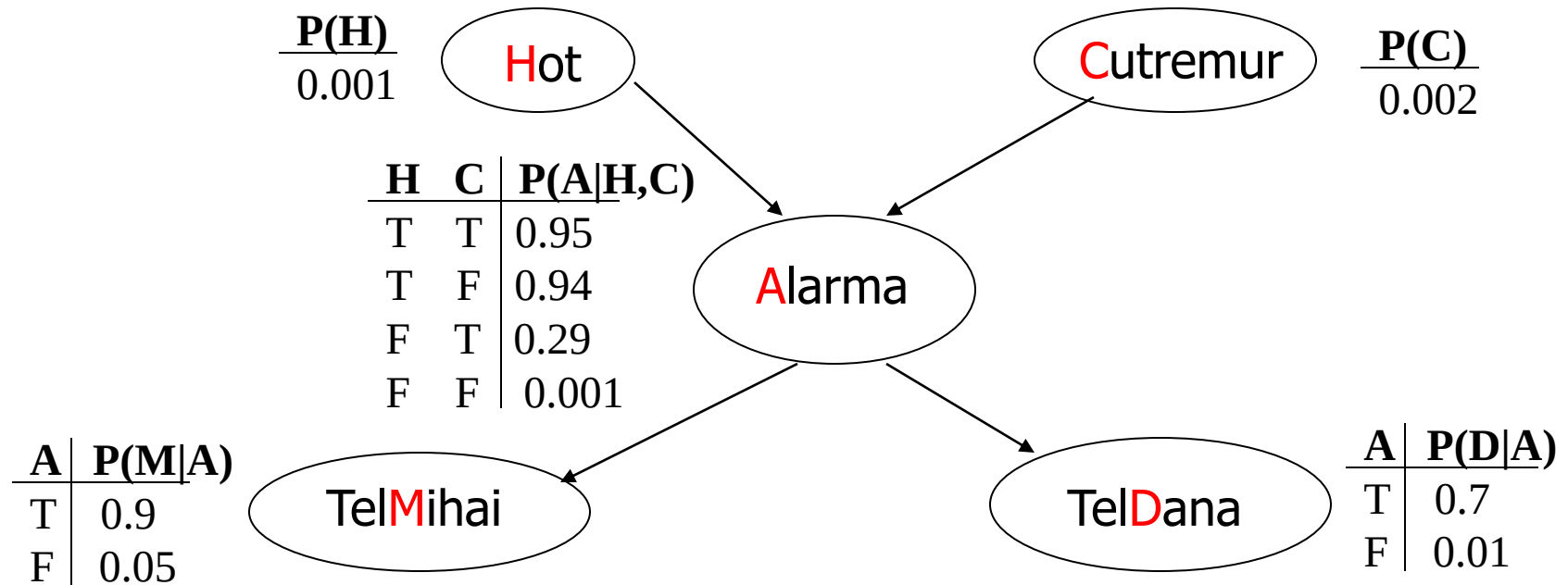
Daca influenta poate curge de la X la Y via Z spunem ca lantul $X \leftrightarrow Z \leftrightarrow Y$ este activ

- Efect cauzal $X \rightarrow Z \rightarrow Y$ – activ $\leftrightarrow$ Z nu este observat
- Efect evidenta $Y \rightarrow Z \rightarrow X$ – activ $\leftrightarrow$ Z nu este observat
- Cauza comuna $X \leftarrow Z \rightarrow Y$ – activ $\leftrightarrow$ Z nu este observat
- Efect comun $X \rightarrow Z \leftarrow Y$ - activ $\leftrightarrow$ fie Z fie unul din descendentii lui Z este observat
- **Influenta probabilistica = flux in graf** – cand influenta din X poate curge prin Z catre Y a.i. sa afecteze probabilitatea lui Y,
- In acest caz X nu este conditional independent de Y fiind dat Z

3. Inferențe exacte în RB

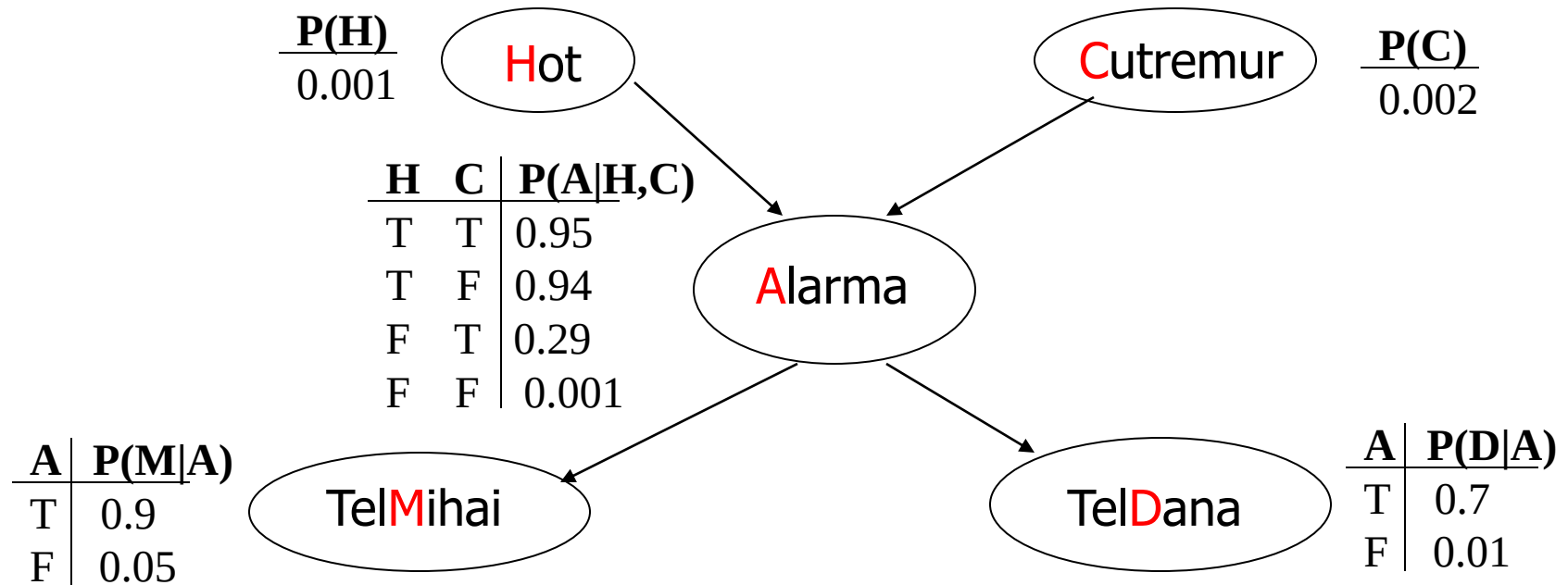
- Cum rezolvăm probleme cu RB?
- Cum se execută, algorithmic, inferențele probabilistice?

3.1 Inferențe probabilistice



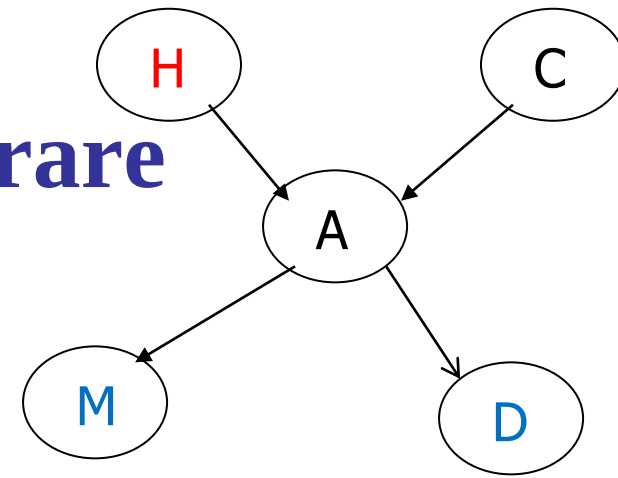
$$\begin{aligned}
 &P(M \wedge D \wedge A \wedge \sim H \wedge \sim C) = \\
 &P(M|A) * P(D|A) * P(A|\sim H \wedge \sim C) * P(\sim H) \wedge P(\sim C) = \\
 &0.9 * 0.7 * 0.001 * 0.999 * 0.998 = 0.00062
 \end{aligned}$$

Inferențe probabilistice



$$\begin{aligned}
 P(A|H) &= P(A|H,C) * P(C|H) + P(A|H,\sim C) * P(\sim C|H) \\
 &= P(A|H,C) * P(C) + P(A|H,\sim C) * P(\sim C) \\
 &= 0.95 * 0.002 + 0.94 * 0.998 = 0.94002
 \end{aligned}$$

3.2 Inferență prin enumerare



X – variabila de interogare (**H**)

E – lista de variabile probe (**M,D**)

e – lista valorilor observate pt aceste variabile E

Y – lista variabilelor neobservate (restul)

$$\mathbf{P}(X \mid e) = \alpha \mathbf{P}(X, e) = \alpha \sum_Y \mathbf{P}(X, e, Y)$$

Insumarea se face peste toate combinatiile de valori ale variab. neobservate Y

$$\mathbf{P}(\mathbf{H}|\mathbf{M},\mathbf{D}) = \alpha \mathbf{P}(H,M,D) =$$

$$\alpha \sum_C \sum_A \mathbf{P}(H)\mathbf{P}(C)\mathbf{P}(A|H,C)\mathbf{P}(M|A)\mathbf{P}(D|A)$$

Inferență prin enumerare

$$P(\mathbf{H}|\mathbf{M},\mathbf{D}) = \alpha \sum_C \sum_A P(H)P(C)P(A|H,C)P(M|A)P(D|A)$$

$$\begin{aligned} P(H|M,D) &= \alpha P(H) \sum_C P(C) \sum_A P(A|H,C)P(M|A)P(D|A) \\ &= \alpha \langle 0.00059224, 0.0014919 \rangle =_{\text{aprox}} \langle 0.284, 0.716 \rangle \end{aligned}$$

Complexitate spatiu – $O(n)$

Complexitate timp $O(2^n)$

Inferență prin enumerare - algoritm

algoritm Enumerare(X,e,rb) intoarce distributie X

X – var de interogare

e – valori observate pt E

rb – Retea Bayesiană cu var $\{X\} \cup E \cup Y$

1. $Q(X) \leftarrow$ o distributie X, initial vida
 2. **pentru** fiecare valoare x_i a lui X **repeta**
 $Q(x_i) \leftarrow \text{EnumToate}(\text{rb.Vars}, \mathbf{e}_{x_i})$
 unde $\mathbf{e}_{x_i}$ este **e** extins cu $X=x_i$
 3. **intoarce** Normalizare($Q(x)$)
- sfarsit**

$$\mathbf{P}(X \mid e) = \alpha \mathbf{P}(X, e) = \alpha \sum_Y \mathbf{P}(X, e, Y)$$

Inferență prin enumerare - algoritm

algoritm EnumToate(Vars,e) intoarce un numar real

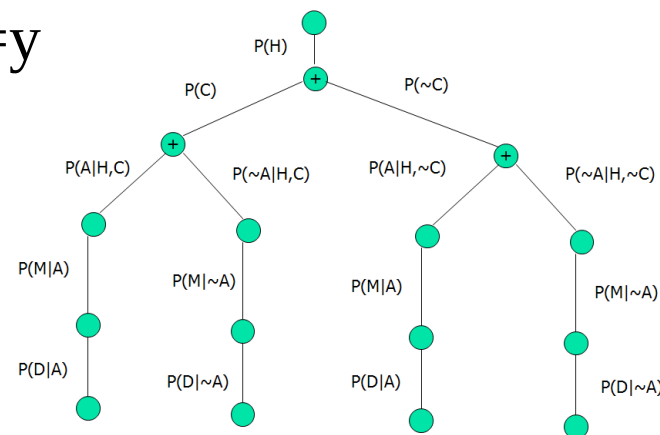
1. daca $\text{Vars} = []$ atunci intoarce 1.0
2. $Y \leftarrow \text{first}(\text{Vars})$
3. **daca** Y are valoare y in e
atunci intoarce $P(y|\text{parinti}(Y)) * \text{EnumToate}(\text{Rest}(\text{Vars}),e)$
altfel intoarce

$$\sum_y P(y|\text{parinti}(Y)) * \text{EnumToate}(\text{Rest}(\text{Vars}),e_y)$$

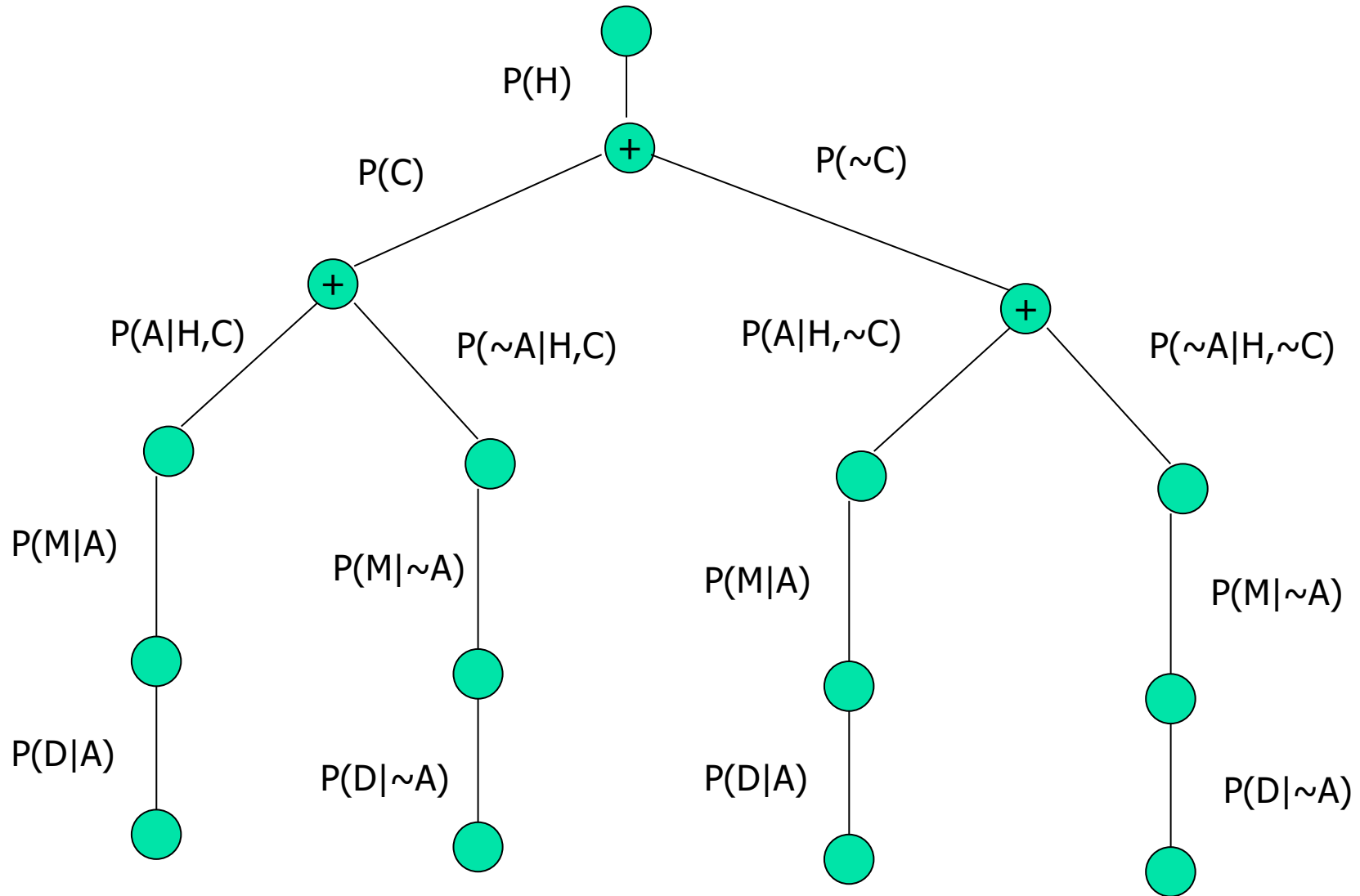
unde e_y este e extins cu $Y=y$

sfarsit

$$P(\mathbf{H}|\mathbf{M},\mathbf{D}) = \alpha \sum_C \sum_A P(H)P(C)P(A|H,C)P(M|A)P(D|A)$$



$$\mathbf{P}(X \mid e) = \alpha \mathbf{P}(X, e) = \alpha \sum_Y \mathbf{P}(X, e, Y)$$



$$\mathbf{P}(\mathbf{H}|\mathbf{M}, \mathbf{D}) = \alpha \sum_C \sum_A P(H)P(C)P(A|H, C)P(M|A)P(D|A)$$

3.3 Eliminarea variabilelor (Variable elimination)

Pt a elimina calculele duplicate – se calculeaza o data si se salveaza rezultatul

Idee – se calculeaza ecuatia de la dreapta la stanga

$$P(H|M,D) = \alpha \sum_C \sum_A P(H) P(C) P(A|H,C) P(M|A) P(D|A)$$

$f1(H) \quad f2(C) \quad f3(A,H,C) \quad f4(A) \quad f5(A)$

$$f5(A) = [P(D|A) \quad P(D|\sim A)] = [.07 \quad .001]$$

$$f4(A) = [P(M|A) \quad P(M|\sim A)] = [.09 \quad .05]$$

f3(A,H,C) va fi o matrice de 2 x 2 x 2

$$[P(A|H,C) \quad P(\sim A|H,C) \quad P(A|H, \sim C) \quad P(\sim A|H, \sim C) \quad \dots \quad P(\sim A|\sim H, \sim C)]$$

Eliminarea variabilelor

$$P(H|M,D) = \alpha \sum_C \sum_A \frac{P(H) P(C) P(A|H,C) P(M|A) P(D|A)}{f_1(H) f_2(C) f_3(A,H,C) f_4(A) f_5(A)}$$

Insumam peste A in f3, f4, f5 – rezulta un factor **f6(H,C)**

$$\mathbf{f6(H,C)} = \sum_A \mathbf{f3(A,H,C)} \times \mathbf{f4(A)} \times \mathbf{f5(A)} =$$

$$\mathbf{f3(a,H,C)} \times \mathbf{f4(a)} \times \mathbf{f5(a)} + \mathbf{f3(\sim a,H,C)} \times \mathbf{f4(\sim a)} \times \mathbf{f5(\sim a)}$$

Insumam peste C – rezulta un factor **f7(H)**

$$P(H|M,D) = \alpha \mathbf{f1(H)} \times \sum_C \mathbf{f2(C)} \times \mathbf{f6(H,C)}$$

$$\mathbf{f7(H)} = \sum_C \mathbf{f2(C)} \times \mathbf{f6(H,C)} = \mathbf{f2(c)} \times \mathbf{f6(H,c)} + \mathbf{f2(\sim c)} \times \mathbf{f6(H,\sim c)}$$

$$P(H|M,D) = \alpha \mathbf{f1(H)} \times \mathbf{f7(H)}$$

Avem nevoie de 2 operatii

- **Pointwise product** **x**
- **Insumarea unei variabile dintr-un produs de factori**

Implementarea operațiilor

Pointwise product

$$(\mathbf{f}_1 \times \mathbf{f}_2)(X_1, \dots, X_i, Y_1, \dots, Y_j, Z_1, \dots, Z_k) = \\ \mathbf{f}_1(X_1, \dots, X_i, Y_1, \dots, Y_j) \times \mathbf{f}_2(Y_1, \dots, Y_j, Z_1, \dots, Z_k)$$

Insumarea unei variabile dintr-un produs de factori

- Insumarea peste **X1**, rezultatul este un factor peste $X_2, \dots, X_i$

$$(\sum_{X_1} \mathbf{f})(X_2, \dots, X_i) = \mathbf{f}(X_1=v_1, \dots, X_i) + \dots + \mathbf{f}(X_1=v_k, \dots, X_i)$$

Implementarea operațiilor - exemplu

Pointwise product

$f1(A,B) \times f2(B,C) = f3(A,B,C)$ are $2^{1+1+1} = 8$ intrari

A	B	f1(A,B)	B	C	f2(B,C)	A	B	C	f3(A,B,C)
a	a	.3	a	a	.2	a	a	a	.3 x .2 = .06
a	f	.7	a	f	.8	a	a	f	.3 x .8 = .24
f	a	.9	f	a	.6	a	f	a	.7 x .6 = .42
f	f	.1	f	f	.4	a	f	f	.7 x .4 = .28
						f	a	a	.9 x .2 = .18
						f	a	f	.9 x .8 = .72
						f	f	a	.1 x .6 = .06
						f	f	f	.1 x .4 = .04

Implementarea operațiilor - exemplu

Insumarea unei variabile dintr-un produs de factori

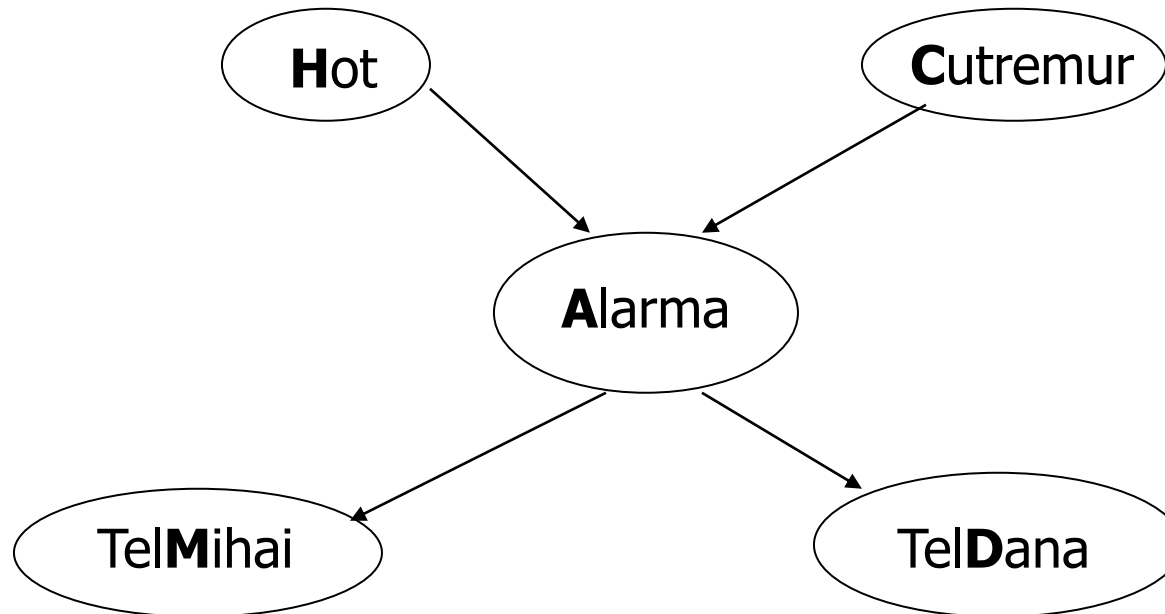
- Insumarea peste A din $f_3(A,B,C)$

$$f(B,C) = \sum_A f_3(A,B,C) = f_3(a,B,C) + f_3(\sim a,B,C) =$$

$$\begin{bmatrix} .06 & .24 \\ .42 & .28 \end{bmatrix} + \begin{bmatrix} .18 & .72 \\ .06 & .04 \end{bmatrix} = \begin{bmatrix} .24 & .96 \\ .48 & .32 \end{bmatrix}$$

A	B	C	$f_3(A,B,C)$
a	a	a	$.3 \times .2 = .06$
a	a	f	$.3 \times .8 = .24$
a	f	a	$.7 \times .6 = .42$
a	f	f	$.7 \times .4 = .28$
f	a	a	$.9 \times .2 = .18$
f	a	f	$.9 \times .8 = .72$
f	f	a	$.1 \times .6 = .06$
f	f	f	$.1 \times .4 = .04$

3.4 Tipuri de inferență



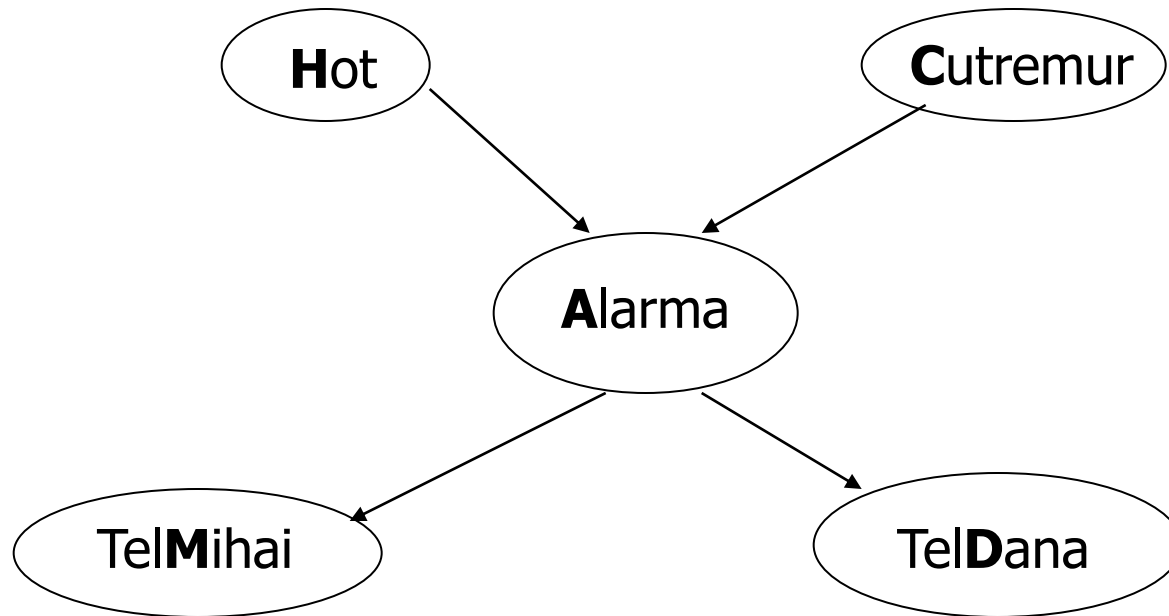
Inferente de diagnosticare (efect $\rightarrow$ cauza)

$P(\text{Hot} \mid \text{TelMihai})$

Inferente cauzale (cauza $\rightarrow$ efect)

$P(\text{TelMihai} \mid \text{Hot}), P(\text{TelDana} \mid \text{Hot})$

Tipuri de inferență



Inferente intercauzale (intre cauza si efecte comune)

$$P(\text{Hot} \mid \text{Alarma} \wedge \text{TelDana})$$

Inferente mixte

$$P(\text{Alarma} \mid \text{TelMihai} \wedge \sim \text{Cutremur}) \rightarrow \text{diag} + \text{cauzal}$$

$$P(\text{Hot} \mid \text{TelMihai} \wedge \sim \text{Cutremur}) \rightarrow \text{diag} + \text{intercauzal}$$

4. Inferențe de aproximare în RB

- Inferențe exacte în RB:
 - Inferența prin enumerare
 - Eliminarea variabilelor (eficientizare)

Asa cum s-a aratat anterior, exprimam probabilitatea conditionata ca probabilitate neconditionata, apoi insumam peste variabilele neobservate

$$\mathbf{P}(X \mid e) = \alpha \mathbf{P}(X, e) = \alpha \sum_Y \mathbf{P}(X, e, Y)$$

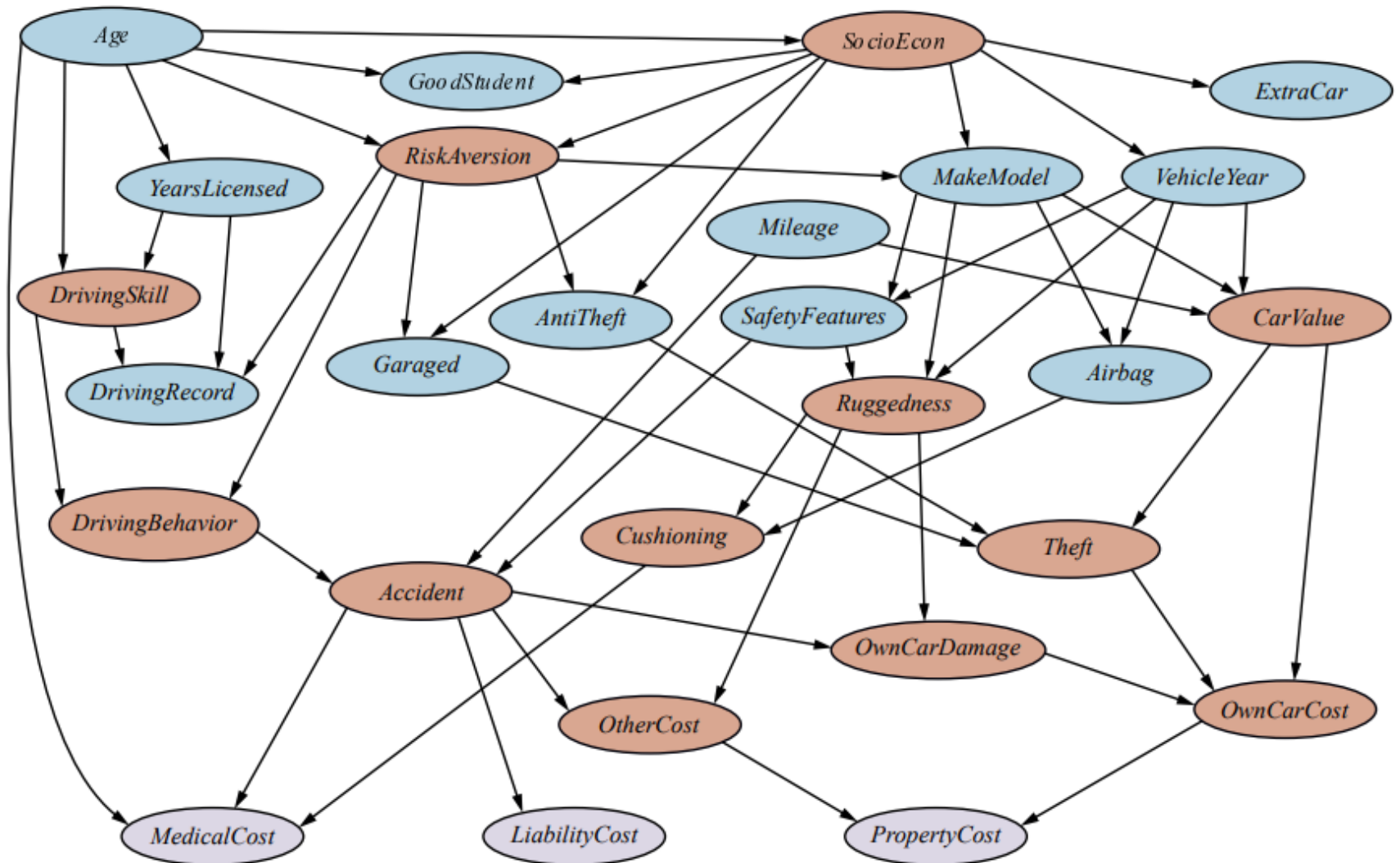
Inferențe de aproximare în RB

Inferente exacte in RB foarte costisitoare

- Complexitate spatiu – $O(n)$
- Complexitate timp $O(2^n)$ – pt n variabile booleene

Pentru rețele complexe aceste metode devin de multe ori nepractice

Asigurarea auto



Inferențe de aproximare

Inferente de aproximare

- Algoritmi de esantionare aleatoare, cunoscuti si ca algoritmi Monte Carlo
- Oferă raspunsuri aproximative care depind de numarul de esantioane care se genereaza
- Ne intereseaza esantionare pentru calculul probabilitatilor conditionate

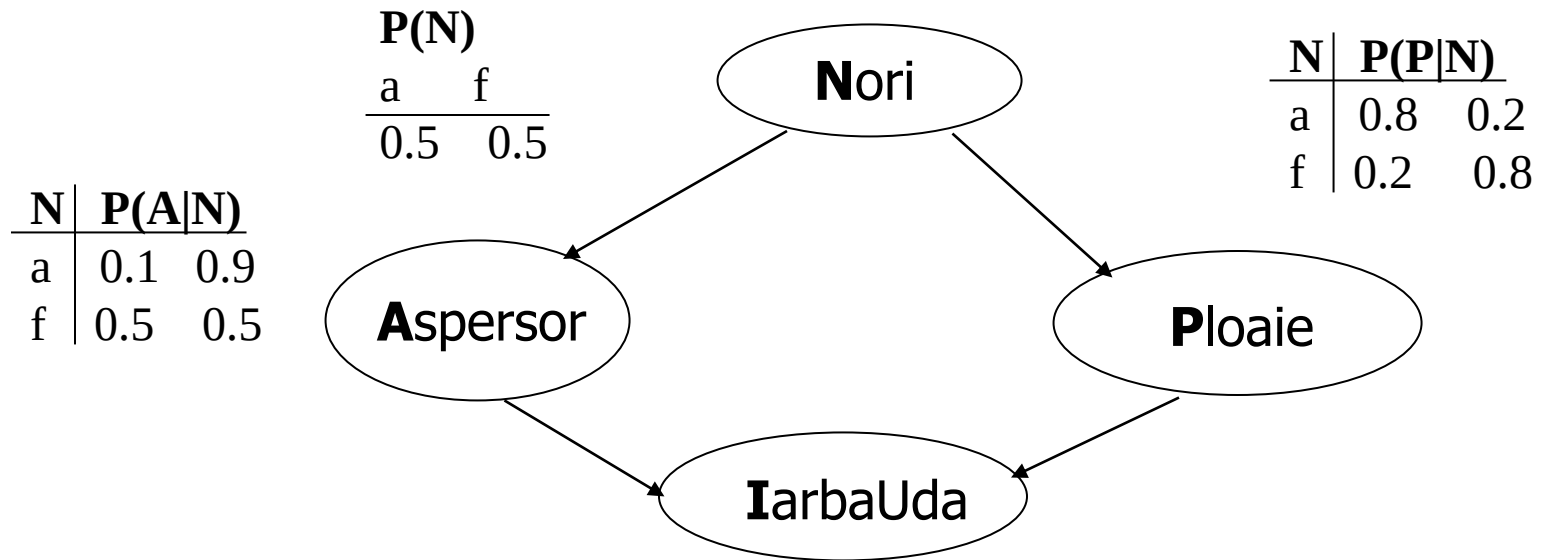
Esantionare directa

Esantionare cu lanturi Markov

4.1 Eșantionare directă

- Se generează esantioane dintr-o distribuție de probabilitate cunoscută
- Cel mai simplu proces de esantionare într-o RB generează esantioane/evenimente din rețea fără nici o probă
- Ideea este de a esantiona fiecare variabilă pe rând în ordine topologică
- Distribuția de probabilitate din care se obțin valorile unei variabile din esantion este condiționată de valorile care au fost deja atribuite părinților

RB exemplu



A	P	P(I A, P)	
		a	f
a	a	0.99	0.01
a	f	0.9	0.1
f	a	0.9	0.1
f	f	0.1	0.9

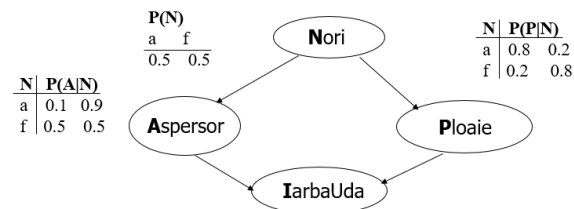
Generarea eșantionului

Presupun ordinea [*Nori*, *Aspersor*, *Ploaie*, *IarbaUda*]

1. Esantion $\mathbf{P(Nori)} = \langle 0.5 \ 0.5 \rangle$ **a**
2. Esantion din $\mathbf{P(Aspersor|Nori=a)} = \langle 0.1 \ 0.9 \rangle$ **f**
3. Esantion $\mathbf{P(Ploaie|Nori=a)} = \langle 0.8 \ 0.2 \rangle$ **a**
4. Esantion din $\mathbf{P(IarbaUda|Aspersor=f, Ploaie=a)} = \langle 0.9 \ 0.1 \rangle$ **a**

Esantionarea este [**a**, **f**, **a**, **a**]

RB exemplu



A	P	P(I A, P)	
		a	f
a	a	0.99	0.01
a	f	0.9	0.1
f	a	0.9	0.1
f	f	0.1	0.9

Algoritm de eşantionare

algoritm Esantionare(rb) intoarce esantion

intrari: rb specificand d.p. $P(X_1, \dots, X_N)$

$x \leftarrow$ esantion cu n elemente

pentru fiecare variabila X_i din $X_1, \dots, X_N$ **repeta**

$x[i] \leftarrow$ un esantion aleator din $P(X_i | \text{parinti}(X_i))$

intoarce x

sfarsit

Algoritm de eşantionare

- Intr-un algoritm de esantionare, raspunsurile sunt calculate prin numararea numarului de esantioane generate
- Sa presupunem ca avem N esantionari

$N_{PS}(x_1, \dots, x_n)$ – de cate ori a aparut $x_1, \dots, x_n$ in multimea de esantioane

- Se presupune ca acest numar va converge chiar la probabilitate

$$\lim_{N \rightarrow \infty} (N_{PS}(x_1, \dots, x_n)/N) = P(x_1, \dots, x_n)$$

- Fie S_{PS} probabilitatea ca un eveniment(esantion) specific sa fie generat de algoritmul de esantionare

$$S_{PS}(x_1, \dots, x_n) = \prod_{i=1, n} P(x_i | \text{parinti}(X_i)) = P(x_1, \dots, x_n)$$

deoarece fiecare pas de esantionare depinde de valorile parintilor

Algoritm de eşantionare

$$S_{PS}(x_1, \dots, x_n) = \prod_{i=1, n} P(x_i | \text{parinti}(X_i)) = P(x_1, \dots, x_n)$$

$$\lim_{N \rightarrow \infty} (N_{PS}(x_1, \dots, x_n) / N) = S_{PS}(x_1, \dots, x_n) = P(x_1, \dots, x_n)$$

De exemplu sa consideram esantionul

[a, f, a, a] generat anterior

Probabilitatea de esantionare a acestui eveniment

$$S_{PS}(a, f, a, a) = 0.5 \times 0.9 \times 0.8 \times 0.9 = 0.324$$

Pentru N foarte mare, putem presupune ca 32,4% din esantioane vor fi egale cu acesta

4.2 Eliminarea eşantioanelor nepotrivite

Eliminarea esantioanelor (*Rejection sampling*)

- Se refera la situatia in care avem de estimat o probabilitate conditionata $P(X|\mathbf{e})$
- Se genereaza esantioanele din distributia de probabilitate specificata de retea
- Apoi se elimina acele esantioane care nu se potrivesc cu probele $\mathbf{e}$
- In final estimarea lui $P(X=x|\mathbf{e})$ este obtinuta prin numararea numarului de aparitii $X=x$ in esantioanele ramase

Eliminarea eşantioanelor nepotrivite

- Dorim sa estimam

$$P(\text{Ploaie}|\text{Aspersor} = a)$$

- Generam, de ex, 100 esantioane
- Presupunem ca din cele 100 esantioane 73 au Aspersor = f, le eliminam
- Raman 27 cu Aspersor = a
- Din acestea 8 au Ploaie=a si 19 au Ploaie=f
- **$P(\text{Ploaie}|\text{Aspersor} = a) = \text{Normalizare}(<8, 19>) = <0.296, 0.704>$**

Rapunsul corect este $<0.3, 0.7>$

- Pe masura ce colectam mai multe esantioane, raspunsul va converge catre valoarea reala

Algoritm de eşantionare cu eliminare

algoritm EsantionareEliminare(X, e, rb, N)

intoarce o estimare $P(X|e)$

intrari: rb specificand d.p. $P(X_1, \dots, X_N)$

X – variabila de interogare

e – valorile observate pentru variabilele E

N – numarul total de esantioane generate

variabile locale VN – vector care numara aparitiile valorilor lui X

pentru $j \leftarrow 1, N$ **repeta**

$y \leftarrow$ Esantionare (rb)

daca y este consistent cu e **atunci**

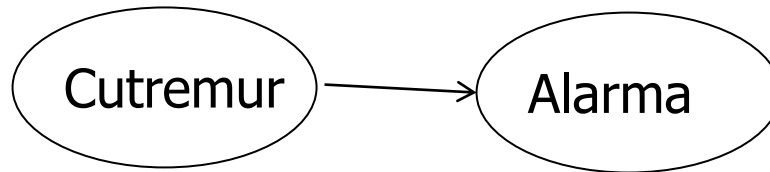
$VN[x] \leftarrow VN[x] + 1$ unde x este valoarea lui X in y

intoarce Normalizare(VN)

sfarsit

Eliminarea eşantioanelor nepotrivite

- Principala problema a acestei abordari este faptul ca elimina prea multe esantioane



- Esantioane pt $P(\text{Cutremur}|\text{Alarma}=a)$

~~Cutremur = f, Alarma = f~~

~~Cutremur = f, Alarma = f~~

~~Cutremur = f, Alarma = f~~

Cutremur = a, Alarma = a

- Procentul de esantioane consistent cu probele scade exponential pe masura ce creste numarul de probe, deci problematic pentru probleme complexe

4.3 Eșantionarea ponderată

Esantionare ponderata (*Likelihood weighting*)

- Elimina ineficienta potentiala a eliminarii esantioanelor prin generarea numai a acelor **evenimente** (esantioane) care sunt **consistente cu probele e**
- Se fixeaza valorile variabilelor probe **E** si se esantioneaza numai pentru celelalte variabile
- Acest lucru garanteaza faptul ca fiecare esantion generat este consistent cu probele; dar nu toate evenimentele sunt la fel de importante

Eșantionarea ponderată

- Înainte de a le numara, fiecare eveniment este ponderat cu o **plauzibilitate** (likelihood) pe care eșantionul o acorda probei
- **Plauzibilitatea (likelihood)** este măsurată ca **produsul probabilităților conditionate pentru fiecare variabilă probă, fiind dați părinții ei**

Eșantionarea ponderată - Exemplu

$P(\text{Ploaie}|\text{Nori}=a, \text{IarbaUda}=a)$

si ordonarea $[\text{Nori}, \text{Aspersor}, \text{Ploaie}, \text{IarbaUda}]$

Initial $w=1.0$

Se genereaza un esantion

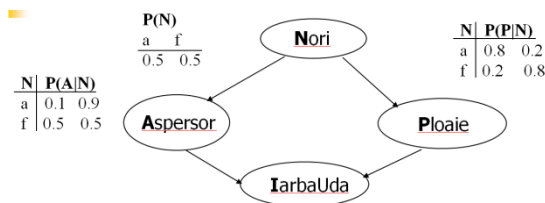
1. *Nori* este o variabila proba cu valoarea **a**

In consecinta **$w \leftarrow w \times P(\text{Nori}=a) = 0.5$**

2. *Aspersor* nu este o variabila proba deci esantionam

$P(\text{Aspersor}|\text{Nori}=a) = (0.1, 0.9)$ - presupun **f**

3. Similar $P(\text{Ploaie}|\text{Nori}=a) = (0.8, 0.2)$ - presupun **a**



A	P	P(I A, P)	
		a	f
a	a	0.99	0.01
a	f	0.9	0.1
f	a	0.9	0.1
f	f	0.1	0.9

Eșantionarea ponderată - Exemplu

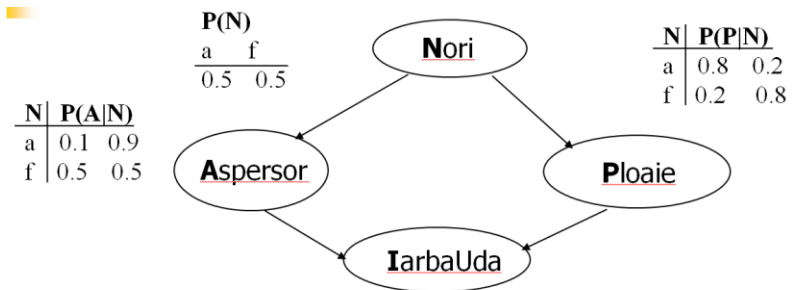
$P(\text{Ploaie}|\text{Nori}=\text{a}, \text{IarbaUda}=\text{a})$

și ordonarea $[\text{Nori}, \text{Aspersor}, \text{Ploaie}, \text{IarbaUda}]$

4. *IarbaUda* este o variabilă probă cu valoarea **a**

$$w \leftarrow w \times P(\text{IarbaUda}=\text{a} \mid \text{Aspersor}=\text{f}, \text{Ploaie}=\text{a}) = 0.9 \times 0.5 = 0.45$$

Eșantionarea ponderată întoarce un eșantion **[a,f,a,a]** cu ponderea **0.45** în condițiile $\text{Ploaie}=\text{a}$



A P		P(I A, P)	
		a	f
a	a	0.99	0.01
a	f	0.9	0.1
f	a	0.9	0.1
f	f	0.1	0.9

Algoritm de eşantionare ponderată

algoritm EsantionarePonderata(X, e, rb, N)

intoarce o estimare $P(X|e)$

intrari: rb specificand d.p. $P(X_1, \dots, X_N)$

X – variabila de interogare

e – valorile observate pentru variabilele E

N – numarul total de esantioane generate

variabile locale W – vector care numara aparitiile valorilor lui X ,
ponderate

pentru $j \leftarrow 1, N$ **repeta**

$x, w \leftarrow \text{Esantion-Ponderat}(rb, e)$

$W[x] \leftarrow W[x] + w$ unde x este valoarea lui X in x

intoarce Normalizare(W)

sfarsit

Algoritm de eşantionare ponderată

algoritm Esantion-Ponderat(rb,e)

intoarce un esantion si o pondere

$w \leftarrow 1$

$\mathbf{x} \leftarrow$ un esantion cu n elemente initializate din e

pentru fiecare variabila X_i din $X_1, \dots, X_n$ **repeta**

daca X_i este o variabila proba cu valoarea x_i in e

atunci $w \leftarrow w * P(X_i = x_i \mid \text{parinti}(X_i))$

altfel $\mathbf{x}[i] \leftarrow$ un esantion aleator din $P(X_i \mid \text{parinti}(X_i))$

intoarce $\mathbf{x}, w$

sfarsit

Inteligență Artificială

Universitatea Politehnica Bucuresti
Anul universitar 2022-2023

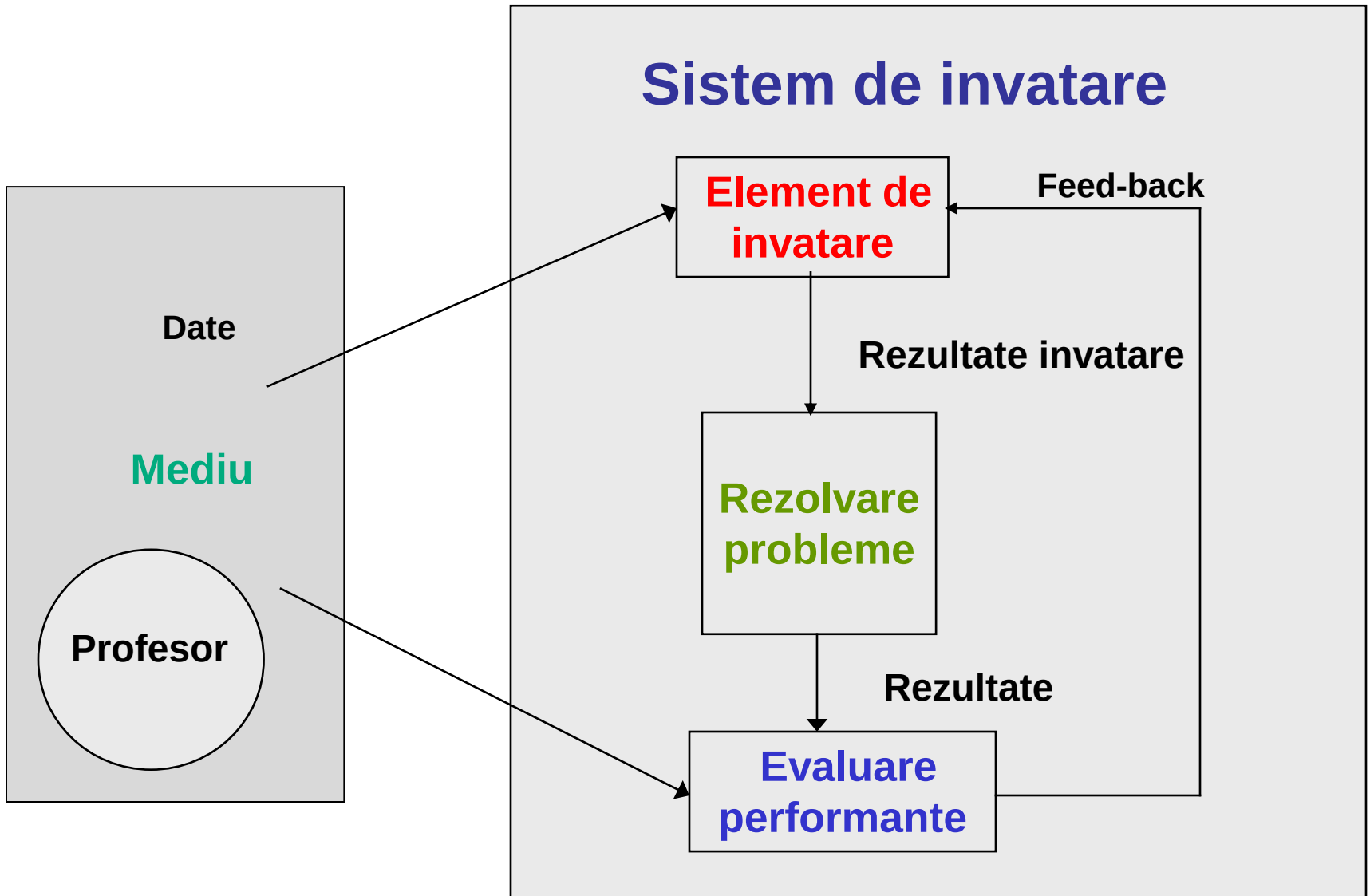
Adina Magda Florea

Curs 7

- Definitii
- Tipuri de invatare automata (ML)
- Caracteristici importante in ML
- Metrice de performanta
- Arbori de decizie (decision trees)
- Paduri aleatoare (random forests)

1. Definiții

- Învățarea este procesul prin care un sistem își îmbunătățește performanțele (Herbert Simon)
- Învățarea este achiziția cunoștințelor explicite
- Învățarea este achiziția deprinderilor de rezolvare a problemelor
- Învățarea este formarea teoriilor, formarea ipotezelor și inferența inductivă
- ML – observă cantități mari de date și caută șabloane pentru predicție



Invatare automata

De ce sa invete?

- Taskuri definite prin exemple
- Relatii / corelatii in cantitati mari de date
- Mediu in schimbare
- Date diferite, cu zgomot
- Cantitate de cunostinte prea mare pentru a fi reprezentate explicit

Denumiri utilizate

- Instanță
- Concept
- Concept vizat (target concept)
- Clasă de ipoteze
- Mulțimea de învățare (Training set)
- Mulțimea de test (Test set)

Denumiri utilizate

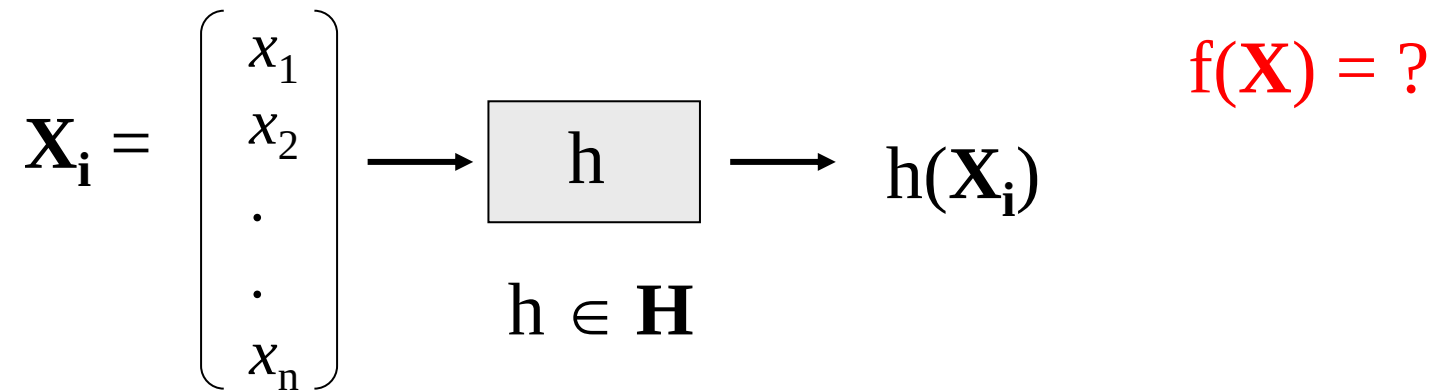
- T – vector de intrare, vector sablon, vector de caracteristici, esantioane, exemple, instanțe
- x_i – caracteristici, attribute, variabile de intrare, componente
- x_i – pot fi valori reale, valori numerice intregi, valori simbolice, valori booleene
- $f(X_i)$
 - valori reale: h – functie de esantionare
 - valori simbolice: h – clasificator
 - booleene: 1 – instanta pozitiva, 0 – instanta negativa

2. Tipuri de invatare

- **Învățare supervizată** – determinarea ipotezei de invatare pe baza unor date etichetate
 - Simbolica sau non-simbolica
- **Învățare nesupervizată** - determinarea ipotezei de invatare / a unei structuri pe baza unor date neetichetate
 - Simbolica sau non-simbolica
- ***Învățarea prin recompensa***

Model simplu

$T = \{\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_m\}$ – **multime de invatare**



■ **Invatare supervizata** – se cunosc $f(\mathbf{X}_1), \dots, f(\mathbf{X}_m)$

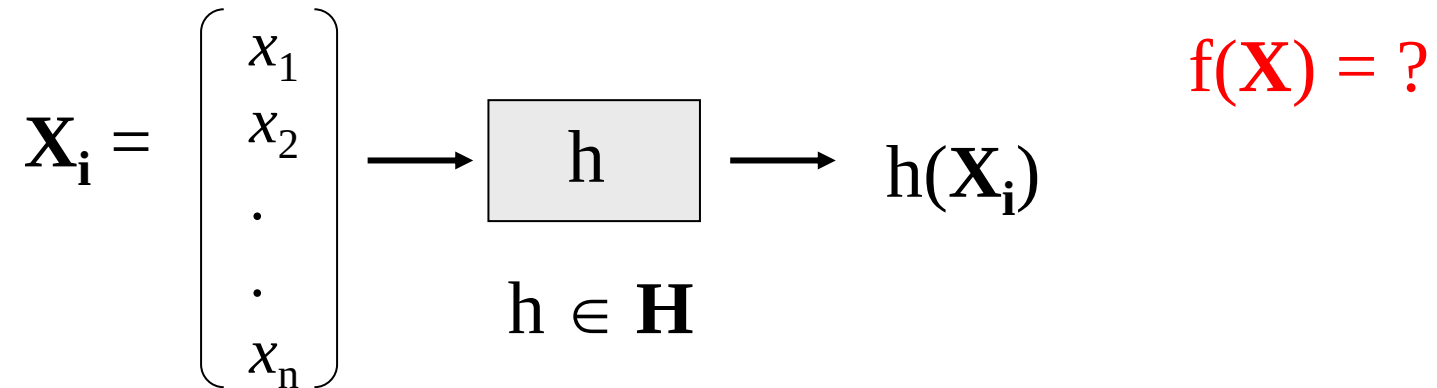
Gasim **h** a.i. $h(\mathbf{X}_i) = f(\mathbf{X}_i)$, $i=1, m \rightarrow h(\mathbf{X}_i) = f(\mathbf{X}_i)$, $\forall i$

Clasificare – **f** - valori discrete – grupeaza exemple

Regresie – **f** – valori continue, estimeaza sau prezice valori

Model simplu

$T = \{X_1, X_2, \dots, X_m\}$ – **multime de invatare**



- **Invatare ne-supervizata** – NU se cunosc $f(X_1), \dots, f(X_m)$

Imparte T in submultimi – clase

Se poate vedea tot ca invatarea unei functii – val f = numele submultimii careia ii apartine X_i

- **Invatare prin recompensa** – Se cunosc recompensele pentru valorile $h(X_1), \dots, h(X_m)$, Nu se cunosc valorile lui f

Metode de invatare supervizata

Clasificare

- Arbori de decizie
- Invățare mulțime (ensemble learning): păduri aleatoare, ADA boost
- SVM
- Clasificare probabilistica (Naïve Bayes)
- K-Nearest neighbours (k-NN pt clasificare)
- Retele neurale

Metode de invatare supervizata

Regresie

- Regresie liniara
- Arbori de regresie
- SVR
- K-Nearest neighbours (k-NN pt regresie)

Clasificare vs regresie



■ Caine / Pisica

Clasificare

- Dan 10.000 lei
- George 3.000 lei
- Vlad 15.000 lei
- Maria 5.000 lei

credit de 30.000 lei
credit de 10.000 lei
credit de 50.000 lei
cat credit?

Regresie

Clasificare vs regresie

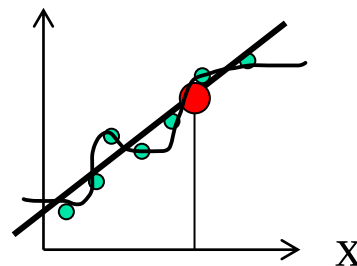
- Reprezentam ipotezele de invatare sub o forma care sa permita ca iesirea algoritmului sa fie o eticheta (clasa): arbori, retele neurale, etc.

Clasificare

- Reprezentam ipoteza de invatare ca o functie liniara

$$h(x) = a_0 + a_1 x$$

- Cat este valoarea y pt punctul rosu?



Regresie

Metode de invatare nesupervizata

- Clustering
 - k-means clustering (grupare), k-means ierarhic (grupare ierarhica)
- Retele neurale
 - Retele neurale cu auto-organizare
 - Autoencoders
 - Deep belief networks

Metode de invatare nesupervizata

- Expectation maximization (EM)
 - HMM – algoritmul Baum-Welch
 - Retele Bayesiene – estimarea parametrilor in cazul datelor incomplete
 - Soft clustering – clasele se pot suprapune
- Principal Component Analysis (PCA)

3. Caracteristici importante in ML

- **Regimuri de invatare:**

- Batch
- Incremental

- **Zgomot:**

- zgomot intrari (de ex valorile atributelor)
- zgomot iesiri (alterare iesiri)

- **Ipoteza favorita a inductiei** (Inductive bias)

- O multime de presupuneri (explicite sau implicite) pe care algoritmul de invatare se bazeaza pentru a obtine un model (a generaliza) din setul de invatare

Inductive bias

- Spatiu de ipoteze restrictionat (Restricted Bias)
 - Functii liniare sau polinomiale de ordin mic
- Ipoteza preferata (Preference Bias)
 - Occam's razor
 - Nerest neighbours
 - Independenta conditionala maxima
 - Margine maxima
 - CNN, Deep RL agents

Occam's Razor

Principiul lamei lui Occam (*lex parsimoniae*)

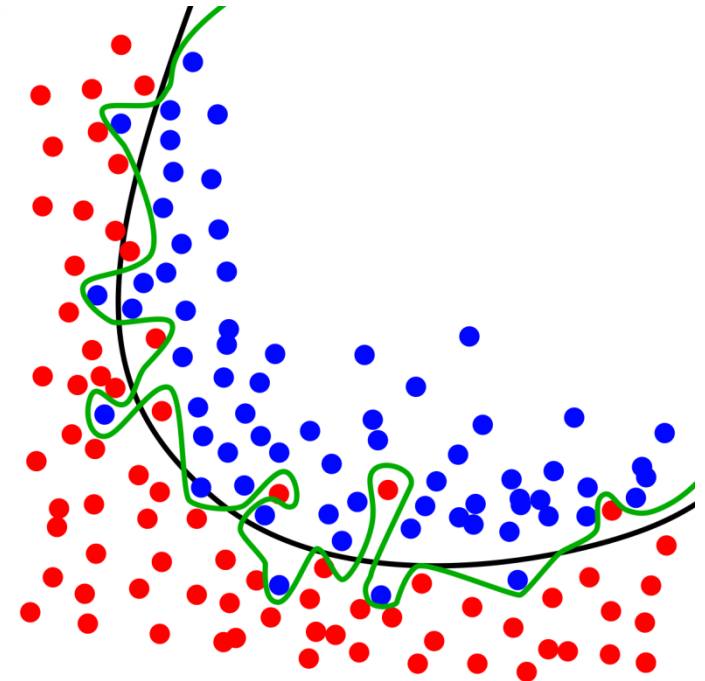
- prefera explicatiile simple celor complexe
- selecteaza ipoteza care implica cele mai putine presupuneri, intre ipoteze similare/egal probabile
- Wiliam of Occam, 1285 – 1347
- filozof englez

"non sunt multiplicanda entia praeter necessitatem"



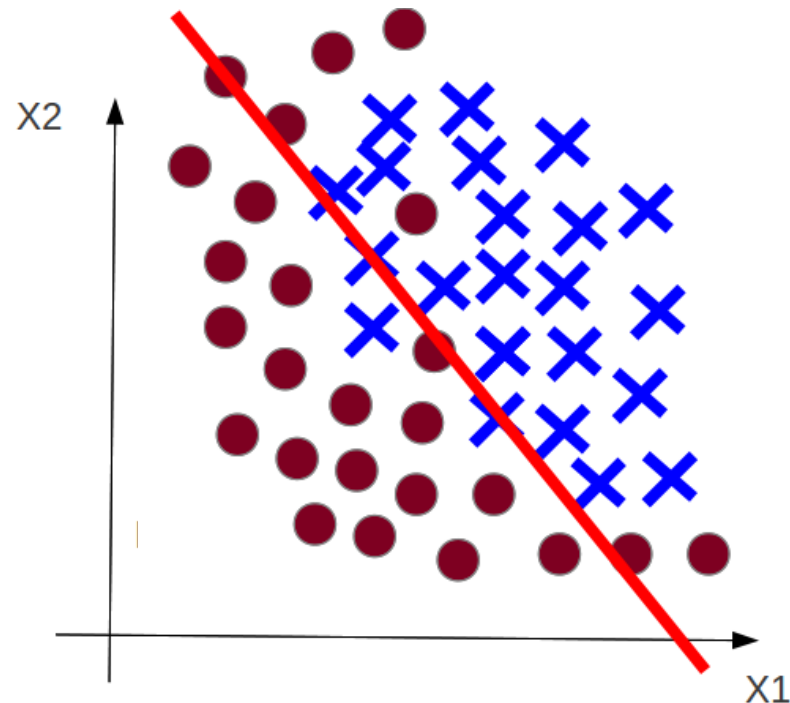
Caracteristici importante

- **Overfitting** = modelul include zgomot sau sabloane de date nerelevante
- Nu mai poate generaliza
- Validare încrucișată
- Regularizare



Caracteristici importante

- **Underfitting** = modelul nu se potrivește nici datelor de învățare și nici nu poate generaliza la date noi



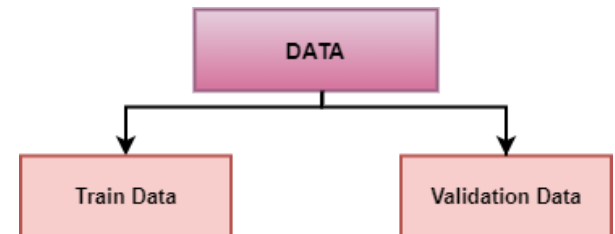
Caracteristici importante

- **Regularizare**
- Evită overfitting
- Se poate aplica aproape oricărui model ML
- Simplifică modelele cu complexitate prea mare prin adăugarea de termeni de penalizare în funcția obiectiv

Caracteristici importante

Cross-validation

- Antrenarea se face pe subseturi de date de învățare și evaluare pe restul
- Metode:
 - Hold out
 - K-fold cross validation
 - Leave-P-out



4. Metrice de performanță

Regresie

- **Mean Absolute Error (MAE)** = media diferenței absolute între valori corecte și valori prezise
- **Root Mean Square Error (RMSE)** = rădăcina pătrată a mediei diferențelor pătratelor între valori corecte și valori prezise

Metri de performanță pt clasificare

Clasificare

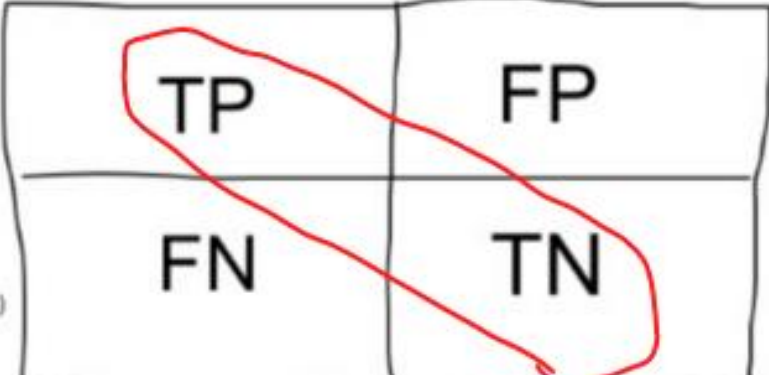
- **Confusion matrix** – corectitudinea și acuratețea modelului

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Metrici de performanță pt clasificare

- **Accuracy**– utilă dacă clasele sunt distribuite uniform

		Actual	
		Positives(1)	Negatives(0)
Predicted	Positives(1)	TP	FP
	Negatives(0)	FN	TN



$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

Metrici de performanță pt clasificare

■ Precision (hit)

		Actual	
		Positives(1)	Negatives(0)
Predicted	Positives(1)	TP	FP
	Negatives(0)	FN	TN

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Precision

Metri de performanță pt clasificare

■ Recall sau Sensitivity (misses)

		Actual	
		Positives(1)	Negatives(0)
Predicted	Positives(1)	TP	FP
	Negatives(0)	FN	TN

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Recall or Sensitivity

5 Arbori de decizie

- ID3 – in jur de 1960s
- C4.5 (Quinlan):
- Permite attribute numerice
- Trateaza cazurile valorilor lipsa
- Trateaza cazul valorilor cu zgomot
- C4.5 – unul din cei mai cunoscuti algoritmi de invatare
- Ultima versiune de cercetare: C4.8, implementata in Weka
- Versiunea comerciala: C5.0

Invatarea inductiva prin AD

- Vede invatarea ca achizitia cunostintelor structurate
- Reprezentarea cunostintelor = **arbori de decizie** (AD)
- Problema de invatare = **clasificare**
- **Invatare supervizata**
- Strategie = invatare batch (ne-incrementala)
- AD se construiesc pornind de la radacina spre frunze
= *Top Down Induction of Decision Tree*

Conditii pentru o invatare "corecta"

Problema: identifica personaje de film
"bune" sau "rele" dupa modul in care arata

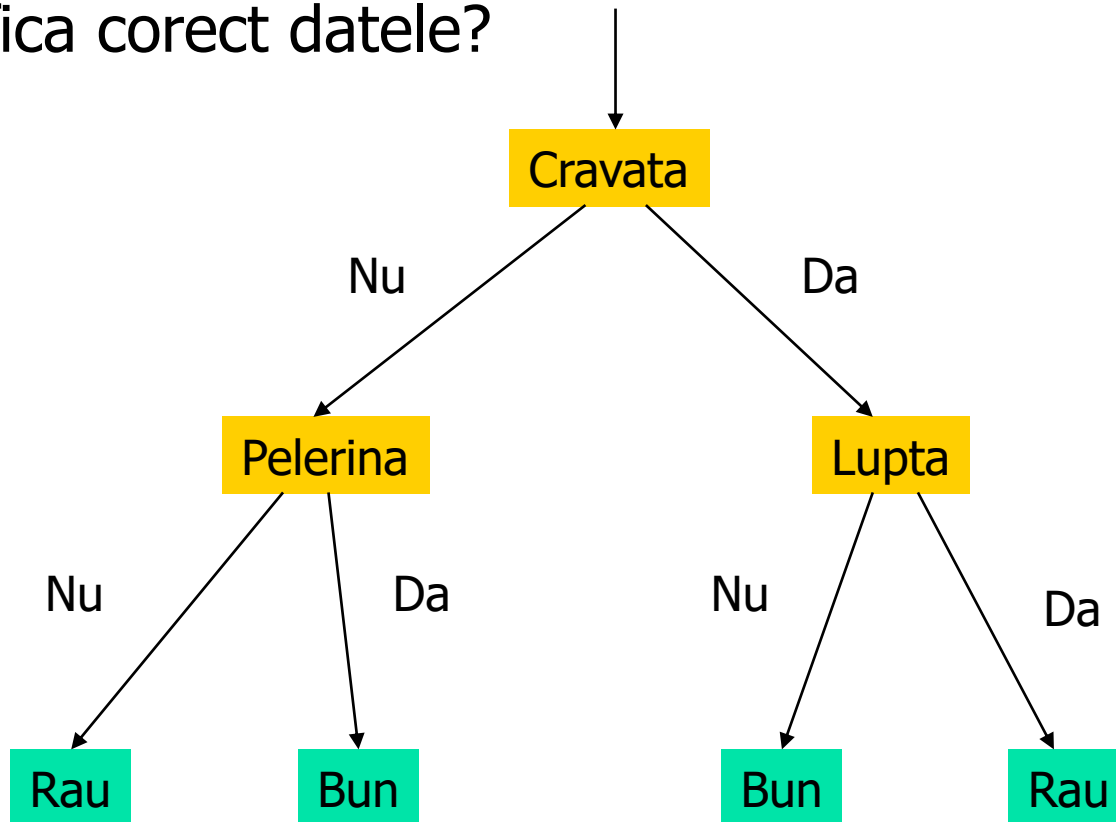


Conditii pentru o invatare "corecta"

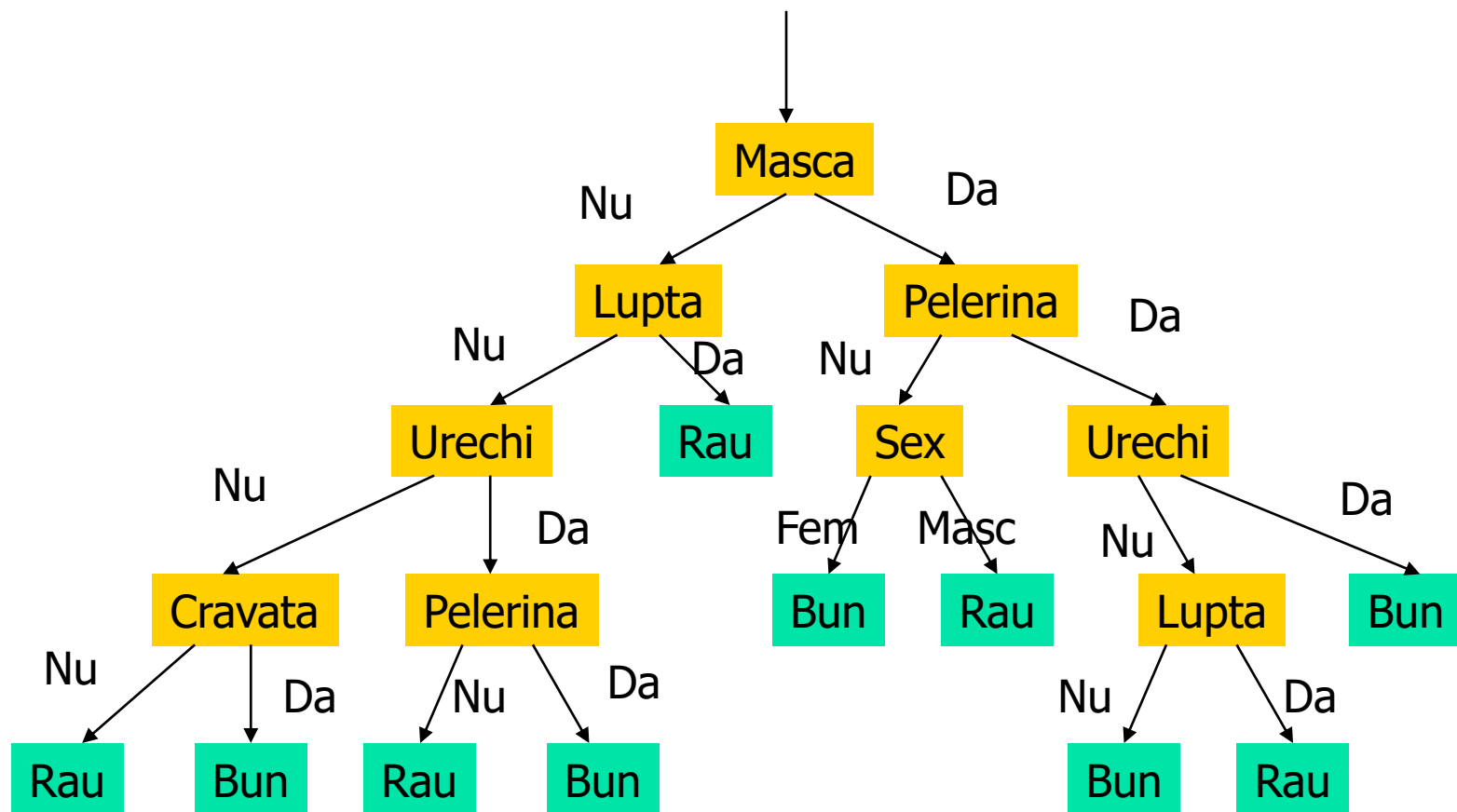
Atribute / Instante	Sex	Masca	Pelerina	Cravata	Urechi	Lupta	Clasa
	Set de invatare						
Batman	Masc	Da	Da	Nu	Da	Nu	Bun
Robin	Masc	Da	Da	Nu	Nu	Nu	Bun
Alfred	Masc	Nu	Nu	Da	Nu	Nu	Bun
Penguin	Masc	Nu	Nu	Da	Nu	Da	Rau
Catwoman	Fem	Da	Nu	Nu	Da	Nu	Rau
Joker	Masc	Nu	Nu	Nu	Nu	Nu	Rau
	Date de test						
Batgirl	Fem	Da	Da	Nu	Da	Nu	??
Fred	Masc	Da	Nu	Nu	Nu	Nu	??

Conditii pentru o invatare "corecta"

Clasifica corect datele?



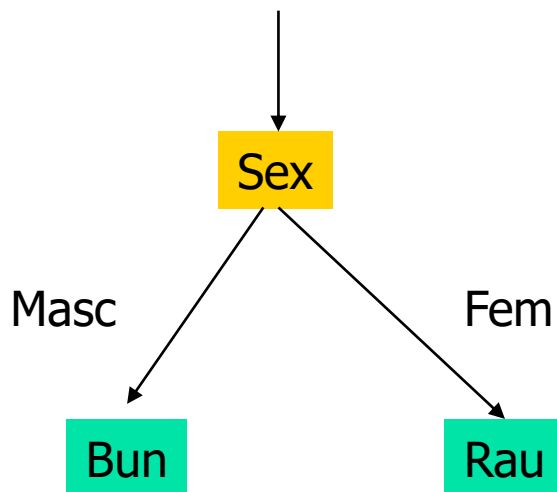
Conditii pentru o invatare "corecta"



Clasifica corect datele?

Conditii pentru o invatare "corecta"

Clasifica corect datele?



Aleg prima varianta (cf. lamei lui Occam)

Conditii pentru o invatare "corecta"

- Clasificatoarele trebuie sa fie suficient de "expresive" pentru a fi in concordanta cu setul de invatare
- Dar clasificatoarele care au o complexitate prea mare pot duce la overfitting

ID3 (Quinlan)

- Univers de obiecte U descrise in termenii unei colectii de attribute $\{A\}$
- Fiecare **atribut** masoara o caracteristica importanta a unui obiect $o \in U$
- Domeniul de valori attribute $D_A =$ discret, simbolic (ulterior extins)
- Fiecare obiect apartine unui **clase** dintr-o multime de clase mutual exclusive $\{C\}$
- Se da **setul de invatare** (SI)
- **Problema** = obtinerea unor **reguli de clasificare** / construirea unui **AD** care clasifica corect nu numai $\forall o \in SI$ dar si $\forall o \in U$

ID3 (Quinlan)

- Structura iterativa – fereastra din SI
- S-au gasit AD corecti in cateva iteratii pt 30 000 obiecte cu 50 attribute
- Empiric s-a aratat ca iterativ se obtin arbori mai buni decat daca s-ar construi din tot SI
- Utilizare AD
- Reguli de decizie

ID3 (Quinlan)

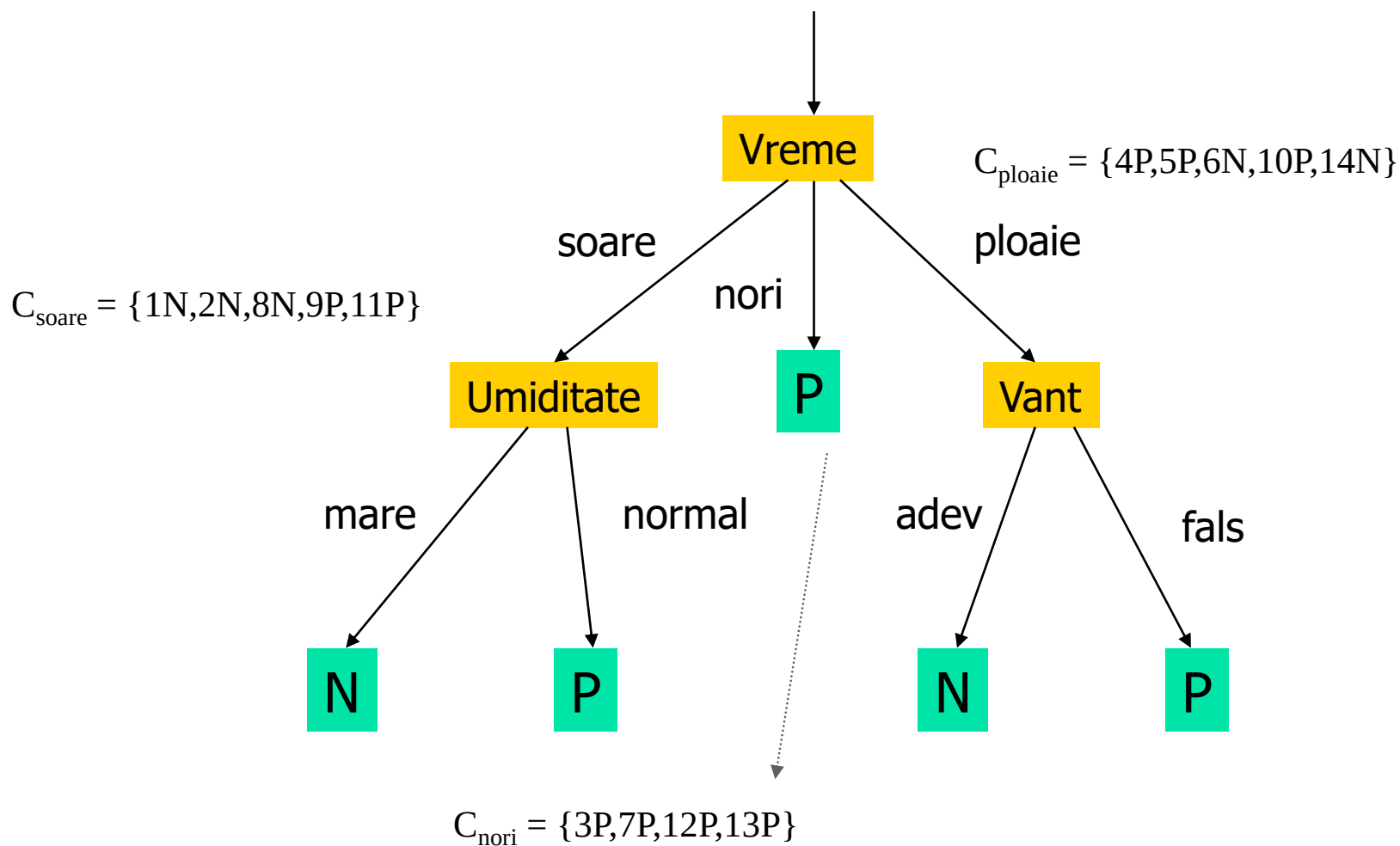
Metoda de constructie

- C = multimea de obiecte / ex inv. din SI
- A^k – atribut test cu valori $a^k_1, \dots, a^k_v$
- "divide-and-conquer"
- Impartirea/expandarea AD se opreste cand toate obiectele din C apartin unei aceleiasi clase
- Se termina intotdeauna (in cazul cel mai nefavorabil, cate un obiect in fiecare clasa)

ID3 – Exemplu

No.	Atribute				Clasa
	Vreme	Temperatura	Umiditate	Vant	
1	soare	cald	mare	fals	N
2	soare	cald	mare	adev	N
3	nori	cald	mare	fals	P
4	ploaie	placut	mare	fals	P
5	ploaie	racoare	normal	fals	P
6	ploaie	racoare	normal	adev	N
7	nori	racoare	normal	adev	P
8	soare	placut	mare	fals	N
9	soare	racoare	normal	fals	P
10	ploaie	placut	normal	fals	P
11	soare	placut	normal	adev	P
12	nori	placut	mare	adev	P
13	nori	cald	normal	fals	P
14	ploaie	placut	mare	adev	N

ID3 – Exemplu



ID3 – Arbore minim

- Din acelasi SI se pot contrui diferiti AD
- Cum se poate obtine cel mai mic arbore (lama lui Occam) ?
- = Cum selectez atributul din radacina unui arbore?

ID3 – Cum selectez A?

- Multimea de exemple C cu un numar de ex pozitive $p \in P$ si exemple negative $n \in N$

Se presupune ca:

- (1) Orice AD corect va clasifica obiectele proportional cu reprezentarea lor in C

Un obiect (exemplu) $o \in C$ va fi clasificat:

- $\in P$ cu probabilitatea $p/(p+n)$
 - $\in N$ cu probabilitatea $n/(p+n)$
- (2) Cand un AD este utilizat pentru a clasifica obiecte, acesta intoarce o clasa $\Rightarrow$

AD poate fi vazut ca o sursa a unui mesaj ' P ' sau ' N ' avand informatia necesara pentru a genera acest mesaj

Teoria informatiei ofera criteriul

- Pentru un univers de mesaje

$$M = \{m_1, m_2, \dots, m_n\}$$

si o probabilitate $p(m_i)$ de aparitie a fiecarui mesaj, **continutul informational** $I(M)$ al mesajelor din M se defineste astfel:

$$I(M) = \sum_{i=1}^n -p(m_i) \log_2(p(m_i))$$

Selectia testului (atributului)

- Fie multimea de obiecte C cu $p \in P$ si $n \in N$
- **Continutul de informatie** $I(AD_{p,n})$ este

$$I(AD_{p,n}) = -\frac{p}{p+n} \log_2 \frac{p}{p+n} - \frac{n}{p+n} \log_2 \frac{n}{p+n}$$

- Selecteaza A in radacina; valori $A \in \{a_1, \dots, a_v\}$
- Fie C_i cu $p_i \in P$ si $n_i \in N$, $i=1, v$
- Continutul de informatie pentru fiecare C_i este
 $I(AD_{p_i, n_i})$, $i=1, v$

Selectia testului (atributului)

- Dupa selectarea lui A in radacina, **cantitatea de informatie necesara pentru a termina constructia arborelui** este suma ponderata a continutului de informatie din toti subarborii

$$E(A) = \sum_{i=1}^v \frac{p_i + n_i}{p + n} I(AD_{pi,ni})$$

- unde ponderea ramurii i este fractiunea de obiecte din C care apartin lui C_i ;
- v este numarul de valori ale lui A

Selectia testului (atributului)

- **Castigul informational** al unui atribut A obtinut prin selectia acestuia ca radacina a arborelui de decizie este:

$$G(A) = I(AD_{p,n}) - E(A)$$

- Se selecteaza A cu castig informational maxim
- Recursiv pentru a forma AD corespunzatori multimilor $C_1 \dots C_v$

Calcul G(A) pt Ex

- 14 exemple, $9 \in P$, $5 \in N$

- $I(AD_{p,n}) = 0.940$ bits

$$-\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14}$$

- vreme

- soare - $2 \in P$, $3 \in N \Rightarrow I(AD_{p1,n1}) = 0.971$

- nori - $4 \in P$, $0 \in N \Rightarrow I(AD_{p2,n2}) = ?$

- ploaie - $3 \in P$, $2 \in N \Rightarrow I(AD_{p3,n3}) = ?$

- $E(vreme) = 0.694$ bits

$$\frac{5}{14} I(AD_{p1,n1}) + \frac{4}{14} I(AD_{p2,n2}) + \frac{5}{14} I(AD_{p3,n3})$$

- $G(vreme) = 0.940 - 0.694 = 0.246$ bits

- $G(temperatura) = 0.029$ bits

- $G(umiditate) = 0.151$

- $G(vant) = 0.048$ bits

$$C_{\text{soare}} = \{1N, 2N, 8N, 9P, 11P\}$$

$$C_{\text{nori}} = \{3P, 7P, 12P, 13P\}$$

$$C_{\text{ploaie}} = \{4P, 5P, 6N, 10P, 14N\}$$

Generalizare la mai multe clase

- **Continutul de informatie**

$$I(Arb) = \sum_{i=1}^v - p(Cl = C_i) * \log_2 p(Cl = C_i)$$

- **Cantitatea de informatie necesara pentru a termina constructia arborelui**

$$E(A) = \sum_{i=1}^v \frac{|C_i|}{|C|} I(C_i)$$

- **Castigul informational**

$$G(A) = I(Arb) - E(A)$$

Algoritm ID3

functie **ind-arbore** (set-invatare, atribute, default)

1. **daca** set-invatare = vid **atunci intoarce** frunza etichetata cu *default* sau "Failure" Caz 1 – ex inv lipsa
2. **daca** toate exemplele din set-invatare sunt in aceeaasi clasa **atunci intoarce** o frunza etichetata cu acea clasa Bine = recunoaste
3. **daca** atribute este vida **atunci intoarce** o frunza etichetata cu disjunctia tuturor claselor din set-invatare Caz 2 – atr inadecvate
4. selecteaza un atribut A, creaza nod pt A si eticheteaza nodul cu A
5. sterge A din atribute → atribute1
6. **m** = cea mai frecventa clasa (set-invatare)
7. **pentru** fiecare valoare V a lui A **repeta**
 - fie partitie_V multimea exemplurilor din set-invatare, cu valoarea V pentru A
 - creaza $\text{nod}_V = \text{ind-arbore}(\text{partitie}_V, \text{atribute1}, m)$
 - creaza legatura nod A – nod_V etichetata cu V

sfarsit

Complexitate

- In fiecare nod cu exceptia frunzelor trebuie aflat G (castig informational) pt fiecare atribut A
- G depinde de valorile p_i si n_i pentru fiecare valoare a_i a lui $A \Rightarrow$ fiecare obiect din C trebuie investigat (clasa, valoare A) $\Rightarrow$
- $O(|C| * |A|)$, $|A|$ - nr attribute
- Pentru fiecare iteratie, **complexitatea ID3**
- $O(|C| * |A| * |AD|)$, unde $|AD|$ - numar de noduri interne AD

Cazuri speciale

Caz 1. Nu exista obiecte $o \in C$ pentru care $A=A_j$

(exemple de invatare lipsa)

- ID3 eticheteaza frunzele cu "*null*" sau "*Failure*" – deci nu clasifica in aceste noduri
- **Solutie**
 - Generalizeaza si se atribuie frunzei clasa cu cea mai mare frecventa de aparitie in C (cea mai frecventa)

Cazuri speciale: Zgomot

Caz 2. Informatia din SI este afectata de zgomot
(attribute inadecvate)

- Zgomot
 - valori de atribut ale obiectelor din C afectate de zgomot
 - clasificare incorecta a obiectelor din C
- Erorile din C (zgomotele) pot duce la 2 probleme:
 - AD cu complexitate mai mare decat este necesar (a)
 - attribute inadecvate (b)

Cazuri speciale: Zgomot

Modificari necesare ID3 pt a trata zgomotul

- (a) Trebuie **sa decida daca testarea unor attribute** suplimentare va creste sau nu acuratetea predictiva a AD
- (b) Trebuie sa poata **lucra cu attribute inadecvate**

Cum se realizeaza (a)

Solutie

- $G(A) > \text{prag } \alpha$ absolut sau relativ
- α suficient de mare pt a elimina attribute nerelevante - dar elimina si attribute relevante pt cazul fara zgomot

Cazuri speciale: Zgomot attribute

Cum se realizeaza (b – attribute inadecvate)

- Trebuie produsa o eticheta pt C_i dar obiectele nu sunt in aceeasi clasa

Solutia 1

- Se utilizeaza notiunea de apartenenta la o clasa cu o anumita probabilitate, de ex. $p_i/(p_i+n_i)$

Solutia 2

- Eticheteaza cu clasa cea mai numeroasa: P daca $p_i > n_i$, N daca $p_i < n_i$, oricare (P sau N) daca $p_i = n_i$

Cazuri speciale: Extinderi C4.5

Caz 3. Valori necunoscute de atribute

3.1 Valori de atribute lipsa in setul de invatare

Solutia 1

- Atribuire valoarea cu cea mai mare frecventa

Solutia 2

- Foloseste probabilitati pt a determia distributia de probabilitate a valorilor lui A in C in functie de apartenenta la o clasa

$$prob(A = A_i | clasa = P) = \frac{prob(A = A_i \wedge clasa = P)}{prob(clasa = P)}$$

si alege valoarea cu cea mai mare probabilitate

Cazuri speciale: attribute lipsa SI

Solutia 3

- Construiești un AD pt a învăța valorile atributelor lipsa
- $C' \subset C$, C' cu valori pt A
- În C' clasa este privită ca un atribut cu valori P sau N
- Valorile lui A – clasele de învățat
- Obține AD' utilizat apoi pentru a clasifica obiectele din $C-C' \Rightarrow$ determin valoarea atributului A
- Solutia 3 > Solutia 2 > Solutia 1

Cazuri speciale: Extinderi C4.5

Caz 4. Atribute cu multe valori

- $A_1 \dots A_n$ - f multe valori simbolice sau valori numerice / continue, sau chiar valori aleatoare
- Castig mare $\rightarrow$ arbore cu 1 nivel

Solutia 1 (valori numerice)

- Partitionez in intervale $(A_i + A_{i+1})/2$, fiecare interval o valoare

Cazuri speciale: A cu multe valori

Exemplificare Solutia 1 (valori numerice)

- Partitionez in intervale $(A_i + A_{i+1})/2$, fiecare interval o valoare
- Ordonez valorile atributului din setul de invatare

64	65	68	69	70	71	72	72	75	75	80	81	83	85
da	nu	da	da	da	nu	nu	da	da	da	nu	da	da	nu
64.5	66.5			70.5		72	73.5			77.5	80.5		84

Cazuri speciale: Extinderi C4.5

Solutia 2 (valori numerice)

Pt fiecare A_i , $i=1,m$ imparte obiectele in $(-\infty, A_i]$ si

$(A_i, +\infty) \Rightarrow$ partitie P_i

- Pentru fiecare P_i calculeaza castigul informational si selecteaza partitia cu castig informational maxim

Cazuri speciale: A cu multe valori

Solutia 3 (valori simbolice)

- Utilizeaza **Informatia de separare** = cantitatea de informatie necesara pentru a determina valoarea unui atribut A intr-un set de invatare C
- Fie $P_{A,C}$ *distributia de probabilitate* a valorilor lui A

$$P_{AC} = (\frac{|A_1|}{|C|}, \dots, \frac{|A_v|}{|C|})$$

- **Informatia de separare**

$$ISep(A) = -\sum_{i=1}^v \frac{p_i + n_i}{p + n} \log_2 \frac{p_i + n_i}{p + n}$$

Cazuri speciale: A cu multe valori

$$GR(A) = \frac{G(A)}{ISep(A)} \quad \text{sa fie cat mai mare}$$

- Dar GR poate sa nu fie intotdeauna definita ($ISep=0$) sau sa tinda sa favorizeze $ISep$ mici
- Selecteaza dintre attributele cu $G(A)$ mare (peste medie) acelea cu $GR(A)$ cel mai bun
- $G(\text{vreme}) = 0.246 \text{ bits}$ ←
- $G(\text{temperatura}) = 0.029 \text{ bits}$
- $G(\text{umiditate}) = 0.151 \text{ bits}$ ←
- $G(\text{vant}) = 0.048 \text{ bits}$
- $ISep(\text{vreme}) = 1.578, \quad ISep(\text{umiditate}) = 1 \text{ bit}$
- $GR(\text{vreme}) = 0.246/1.578=0.156$ ←
- $GR(\text{umiditate}) = 0.151/1.0 = 0.151$

Cazuri speciale: A cu multe valori

- Utilizeaza diverse metode de invatare pentru a forma clase din aceste valori sau a grupa aceste valori in grupuri (cluster), clase care sa devina valori discrete (sau sa produca mai putine valori) pentru atribut

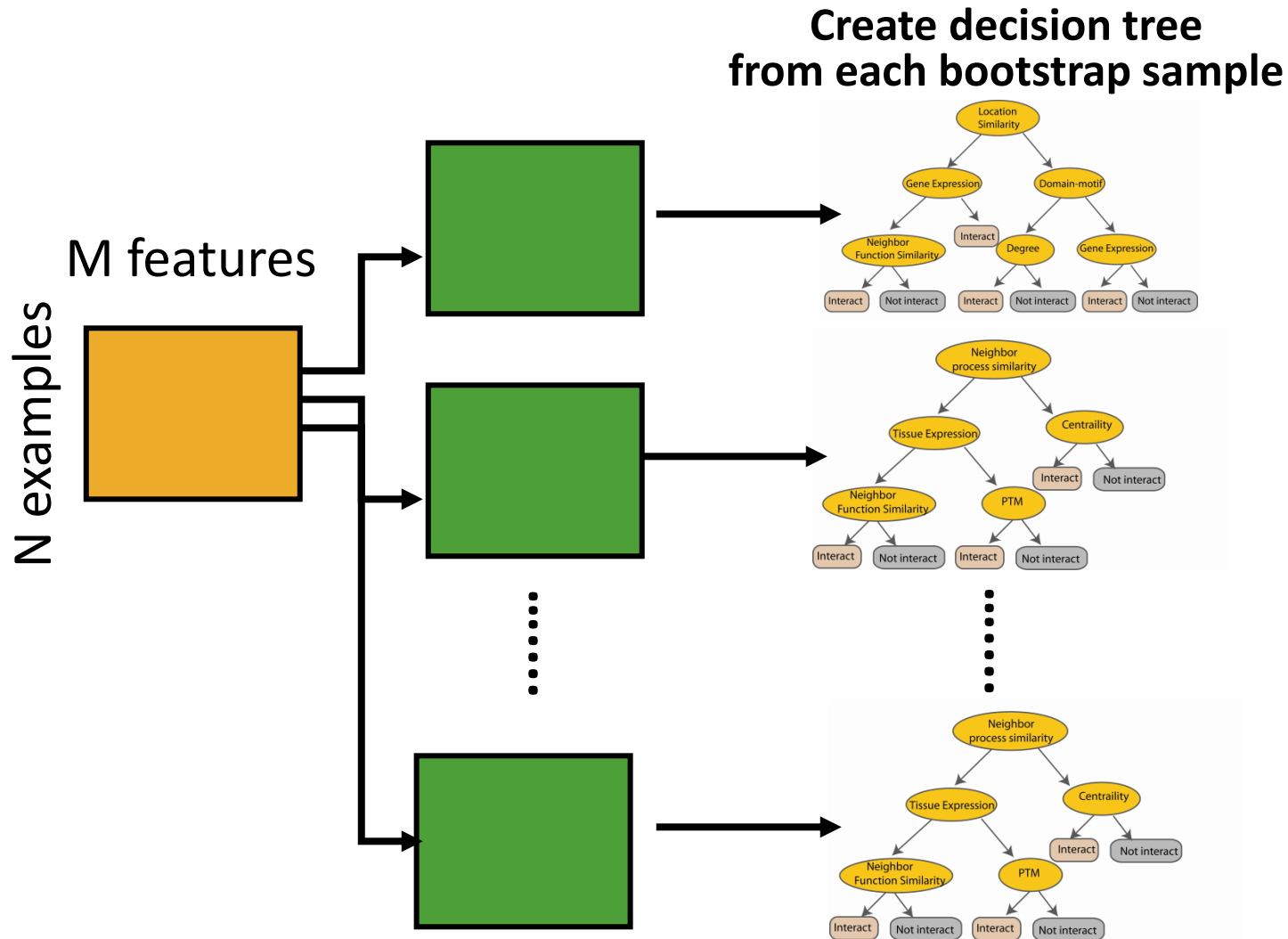
6. Random forests

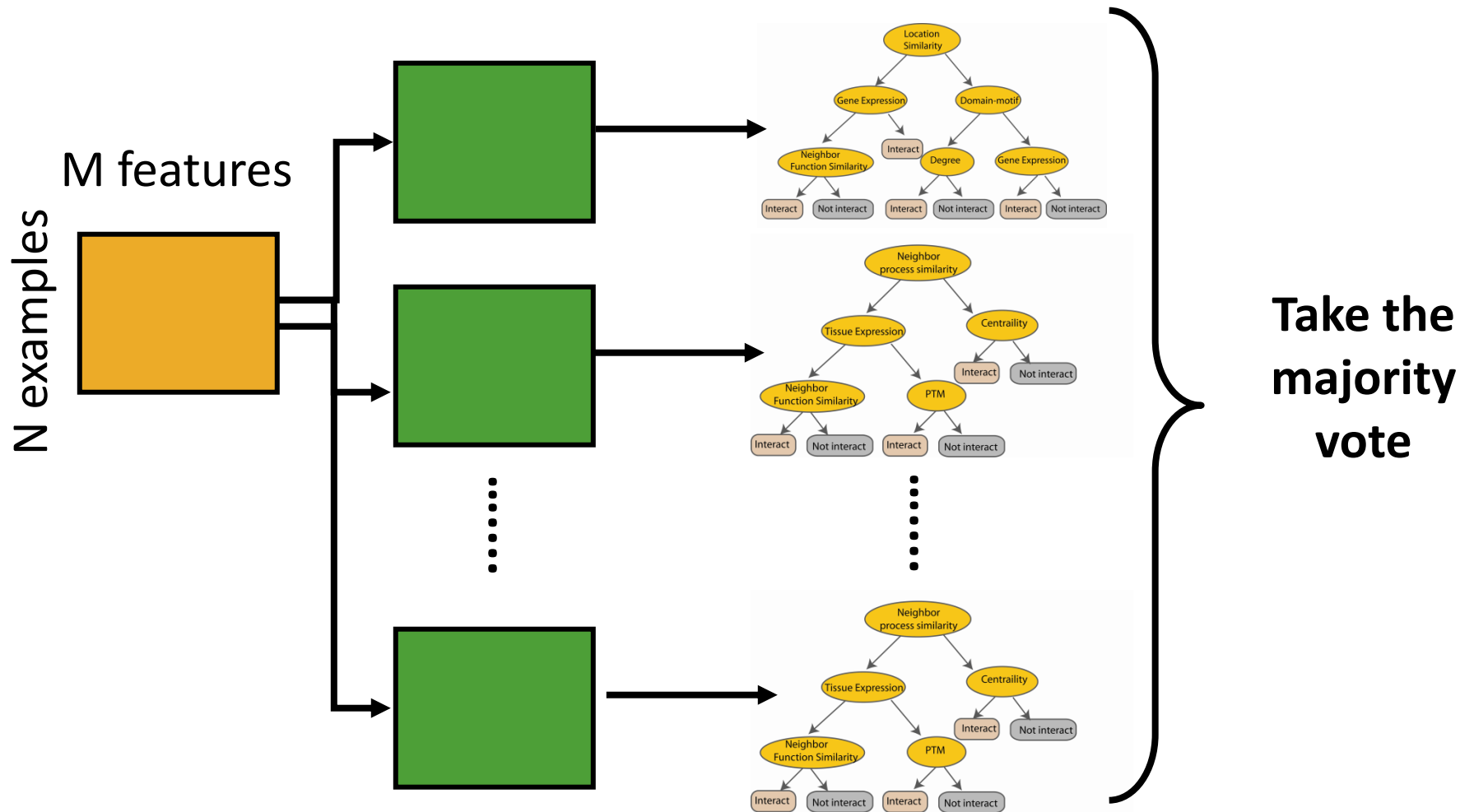
- Ipoteza metodei: combina mai multe modele de invatare a conceptului pentru a creste acurateta invatarii
- **Bagging** – obtine o medie a unor modele de invatare (fiecare poate fi afectat de zgomot sau de impartire favorizanta - biased)
- **Idee** - Selectam aleator din multimea de invatare o submultime de exemple si tot aleator selectam o submultime de attribute, apoi construim, pentru fiecare selectie cate un arbore (**random selection of attributes and random selection of examples**)
- Obtinem o **padure de arbori**

Random forests

- N – numărul de exemple de învățare, M numărul de atribute
- Se vor selecta în fiecare arbore m atribute ($m < M$) și n ($n < N$) exemple, aleator, din setul de învățare (bootstrap sample)
- Restul exemplurilor se folosesc pentru evaluare
- Fiecare arbore este complet construit

Random forests





Random forests

- Pentru o padure de arbori, si o instanta necunoscuta se clasifica instanta cu fiecare arbore
- Fiecare arbore aduce un vot, se considera clasificarea data de votul majoritar

Avantajele metodei

- Acuratete mare de invatare
- Reduce influenta unui model favorizant (biased)
- Mai putin sensibil la zgomote
- Totusi are tendinta la overfitting pentru anumite seturi de date

Inteligență Artificială

Universitatea Politehnica Bucuresti
Anul universitar 2022-2023

Adina Magda Florea



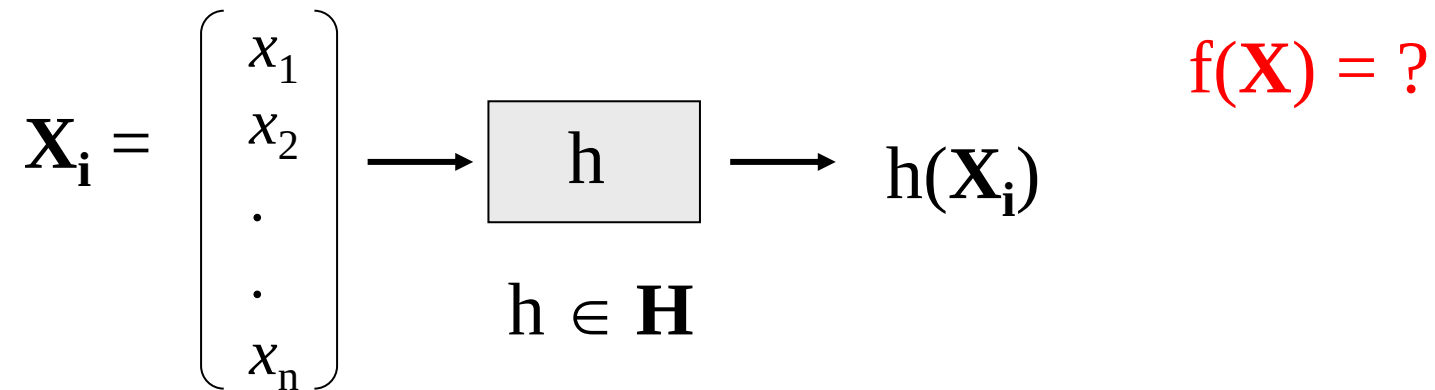
Curs nr. 8

- Regresie liniara
- Regresie liniara cu atribute multiple
- Arbori de regresie



Model simplu

$T = \{X_1, X_2, \dots, X_m\}$ – **multime de invatare**



■ **Invatare supervizata** – se cunosc $f(X_1), \dots, f(X_m)$

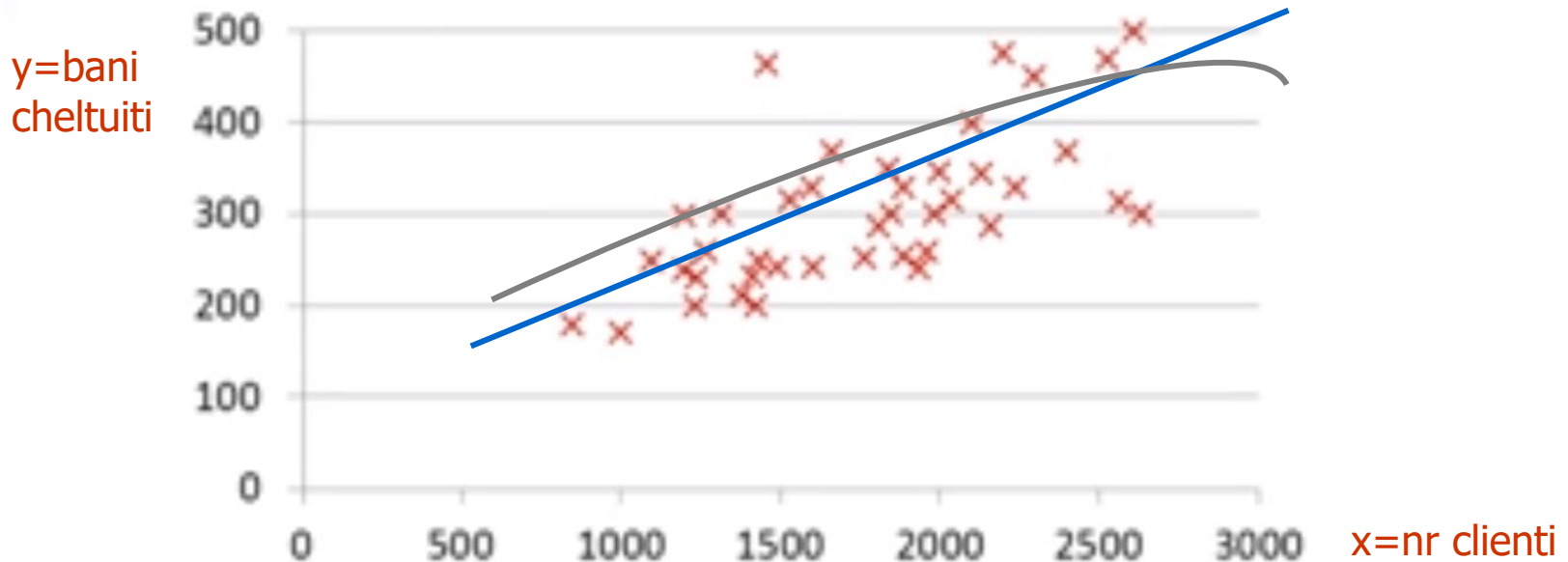
Gasim **h** a.i. $h(X_i) = f(X_i)$, $i=1, m \rightarrow h(X_i) = f(X_i)$, $\forall i$

Clasificare – **f** - valori discrete – grupeaza exemple

Regresie – **f** – valori continue, estimeaza sau prezice valori

1 Regresie liniara

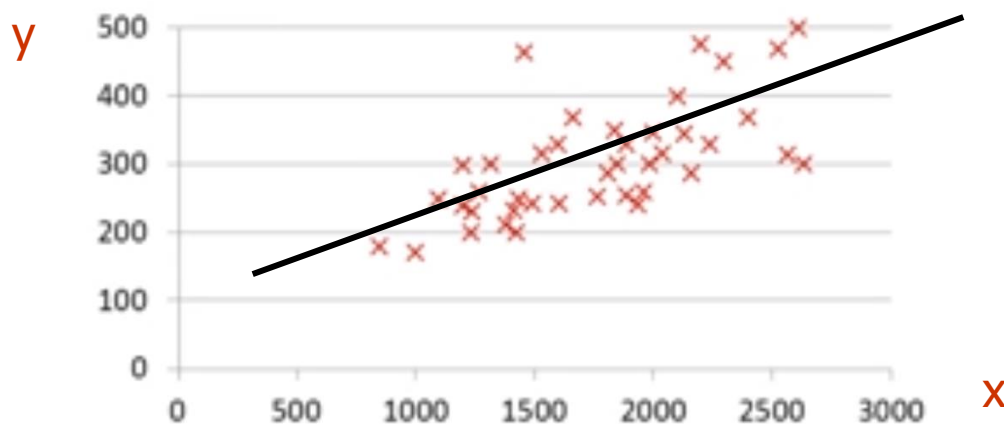
- Algoritm de invatare supervizata
- *Invata valori continue pe baza exemplelor de invatare*



Regresie liniara

- Cum se reprezinta h?
- In acest exemplu - regresie liniara cu 1 variabila (1 atribut x)

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$





Funcție de cost

- n – număr exemple de învățare
- x – variabilă de intrare / atribut
- y – variabilă de ieșire (ieșirea cunoscută din exemple)
- Cum alegem parametrii θ_0 și θ_1 pentru a găsi modelul?
- Trebuie să minimizăm **o funcție de cost**



Funcție de cost

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Funcție de cost – eroare patratică

$$J(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (h(x^i) - y^i)^2$$

Trebuie sa minimizam

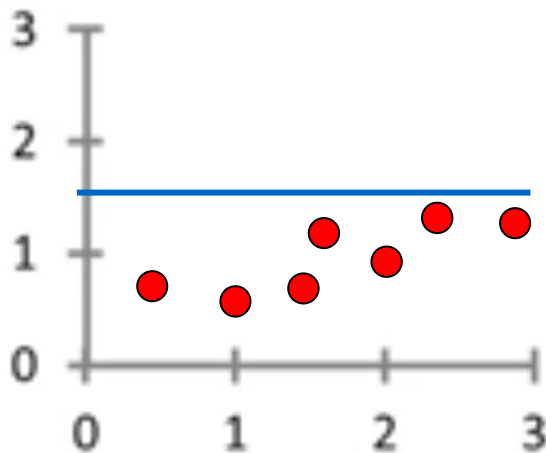
$$\min_{\theta_0 \text{ si } \theta_1} J(\theta_0, \theta_1)$$

Cum minimizam o astfel de funcție de cost?

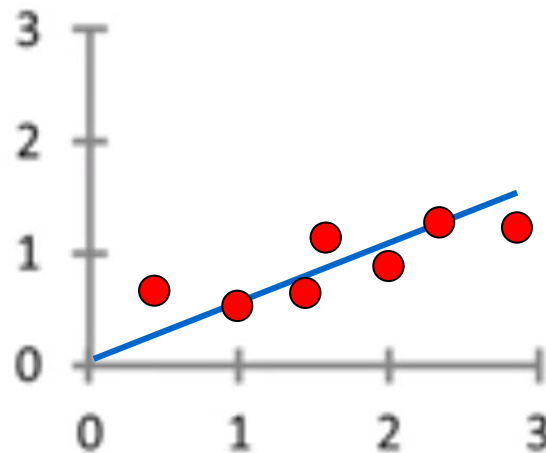
Funcție de cost

Trebuie sa alegem parametrii astfel incat $h_{\theta}(x)$ sa fie cat mai aproape de pct (x,y)

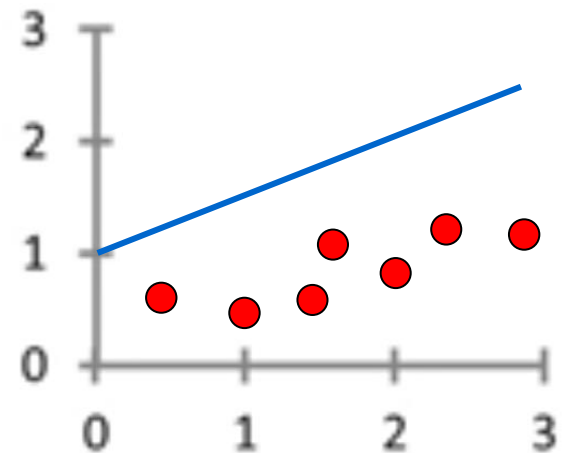
$$h_{\theta}(x) = \theta_0 + \theta_1 x$$



$$\theta_0 = 1.5$$
$$\theta_1 = 0$$



$$\theta_0 = 0$$
$$\theta_1 = 0.5$$



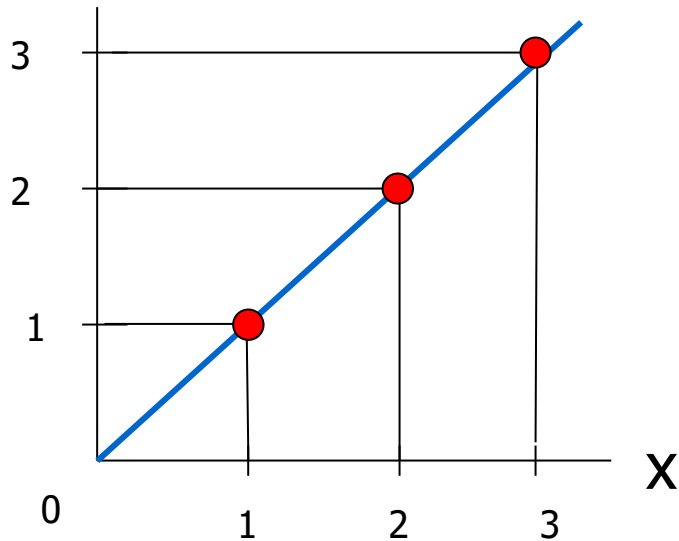
$$\theta_0 = 1$$
$$\theta_1 = 0.5$$

Presupunem numai drepte cu $\theta_0=0$

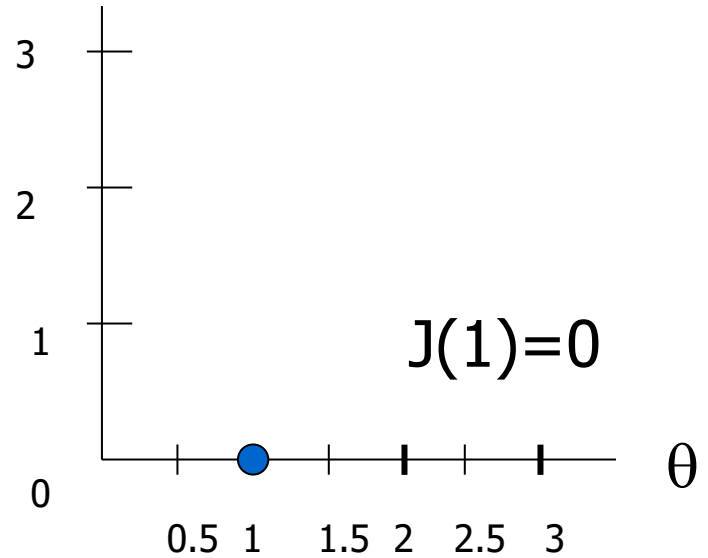
$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

$$\theta_1 = \mathbf{1}, \theta_0 = 0$$

y, h



$$J(\theta_1)$$

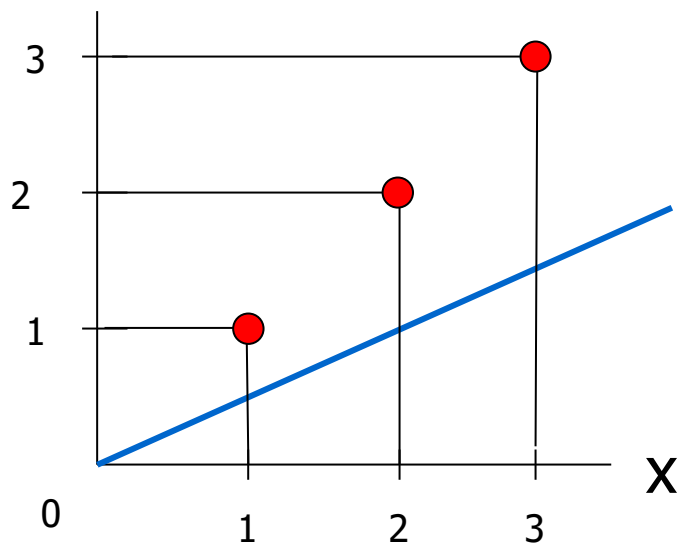


$$h_{\theta}(x) = \theta_1 x$$

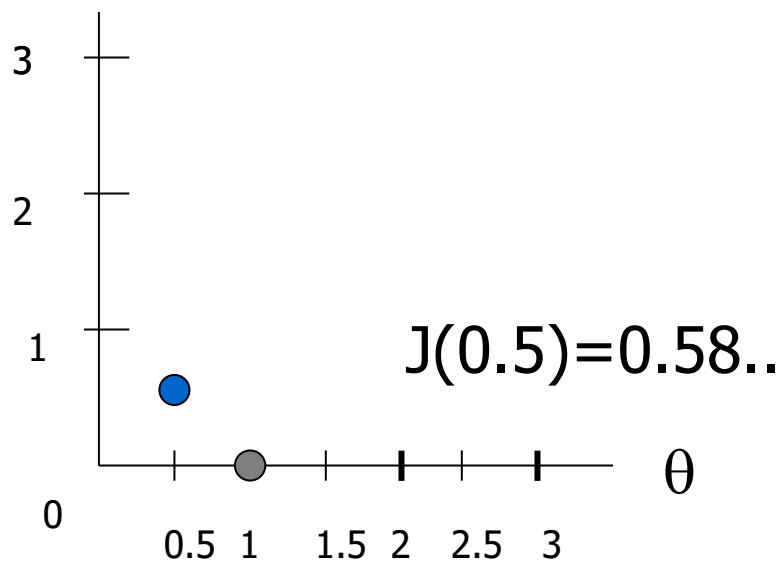
$$J(\theta_1) = \frac{1}{2n} \sum_{i=1}^n (h(x^i) - y^i)^2$$

$$\theta_1 = 0.5, \theta_0 = 0$$

y, h



$J(\theta_1)$

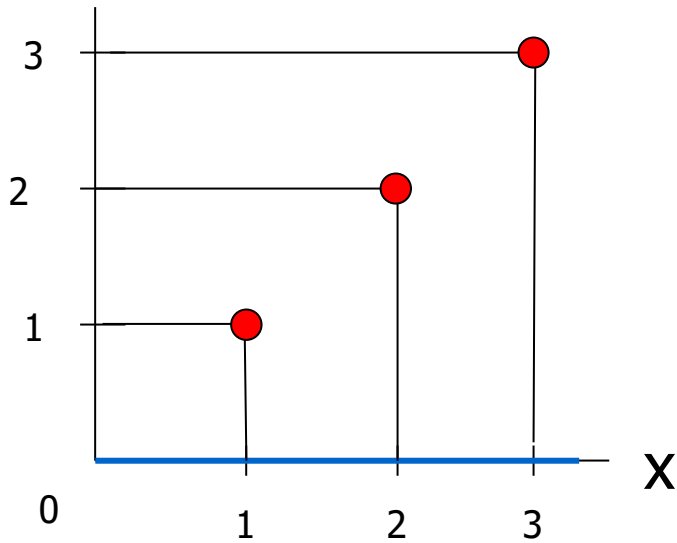


$$h_{\theta}(x) = \theta_1 x$$

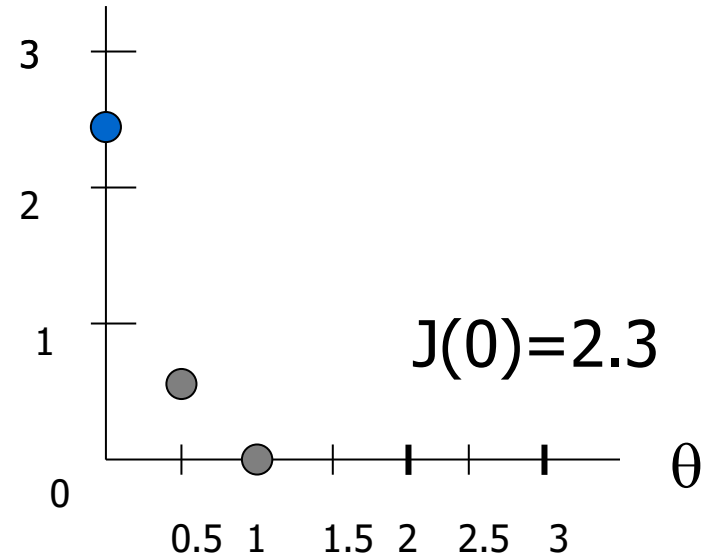
$$J(\theta_1) = \frac{1}{2n} \sum_{i=1}^n (h(x^i) - y^i)^2$$

$$\theta_1 = \mathbf{0}, \theta_0 = 0$$

y, h



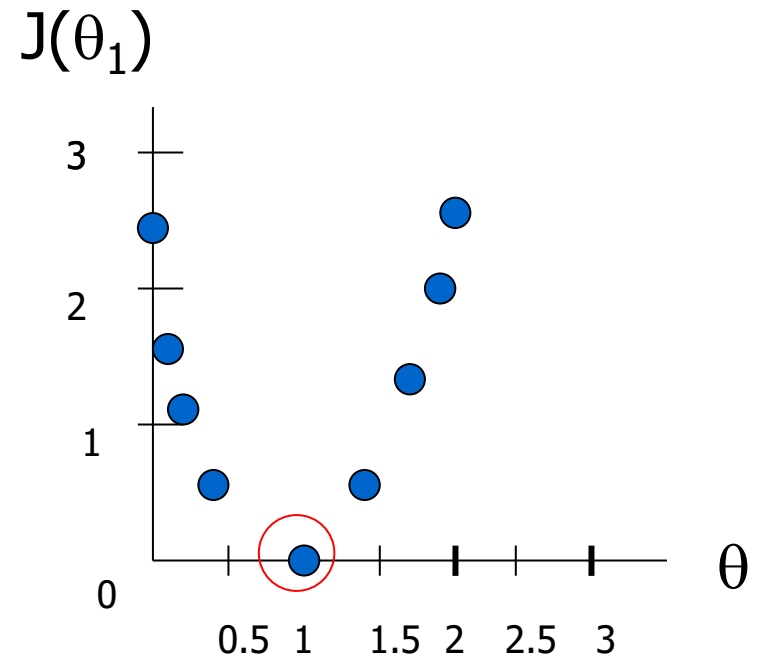
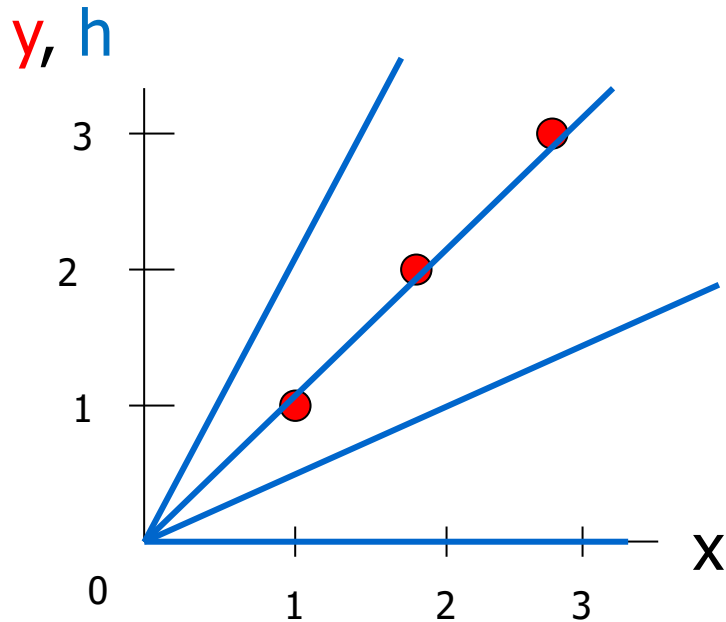
$J(\theta_1)$



$$h_{\theta}(x) = \theta_1 x$$

$$J(\theta_1) = \frac{1}{2n} \sum_{i=1}^n (h(x^i) - y^i)^2$$

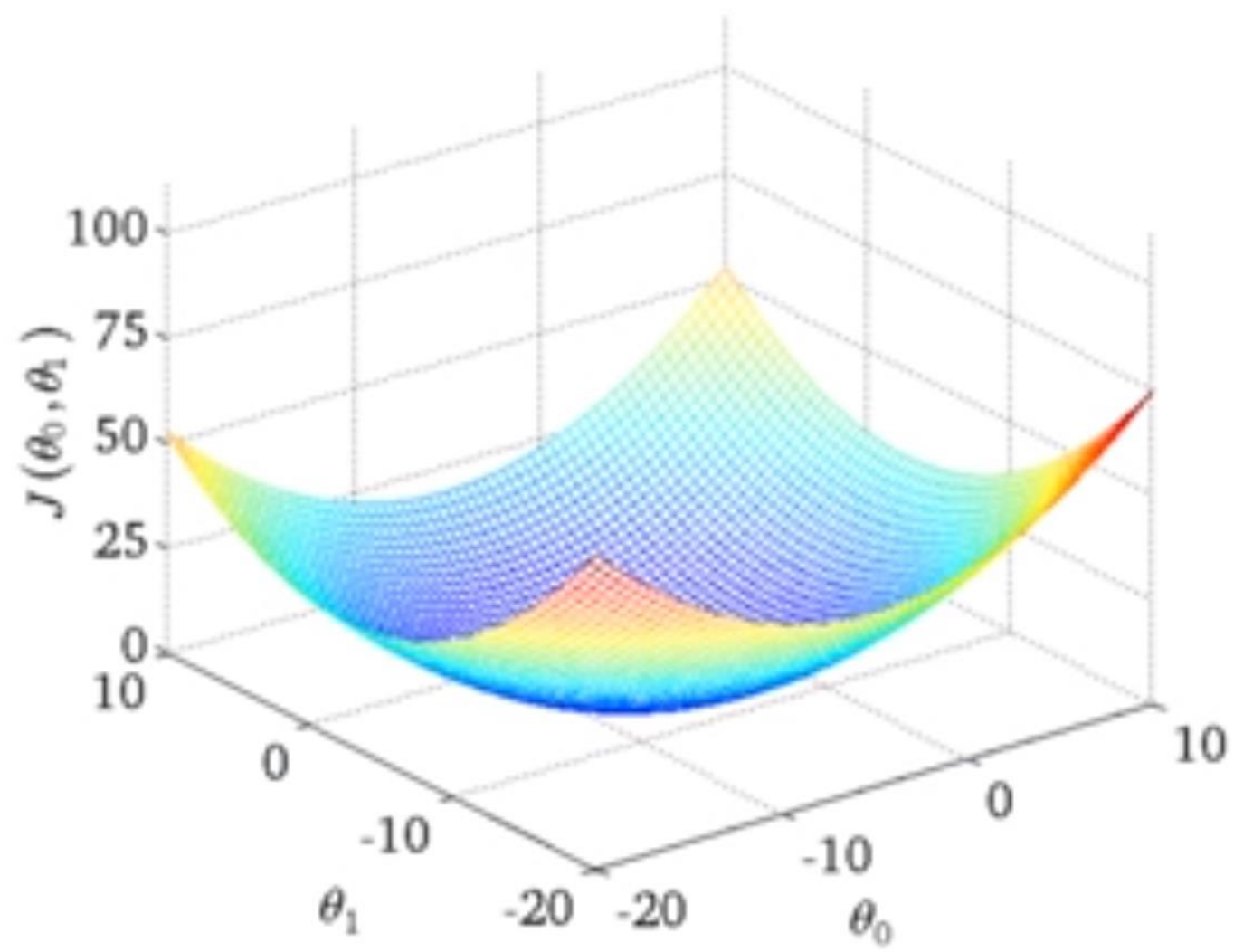
$$\theta_1 = 1, 0.5, 0, \dots \theta_0 = 0$$



$$h_{\theta}(x) = \theta_1 x$$

$$J(\theta_1) = \frac{1}{2n} \sum_{i=1}^n (h(x^i) - y^i)^2$$

Pr fiecare valoare θ_1 avem o valoare diferita a $J(\theta_1)$





Scaderea gradientului (Gradient descent)

- *Metoda iterativa pentru gasirea minimului unei functii*
- Pentru a afla **minimul** unei functii, se fac pasi proportionali cu negativul derivatei/gradientului functiei in punctul curent
- Sau proportionali cu pozitivul derivatei/gradientului pentru **maxim**
- Minime, respectiv maxime locale – nu garanteaza gasirea valorii optime globale



Gradient descent

Minimizarea functiei de cost $J(\theta_0, \theta_1)$

Initializeaza θ_0 θ_1

repetă până la convergență

pentru $j=0$ și $j=1$ (simultan) **execută**

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

■ α – viteza de învățare

$$h(x) = \theta_0 + \theta_1 x$$

$$\min_{\theta_0 \text{ și } \theta_1} \frac{1}{2n} \sum_{i=1}^n (h(x^i) - y^i)^2$$



Gradient descent

Minimizarea functiei de cost $J(\theta_0, \theta_1)$

Initializeaza $\theta_0 \theta_1$

repetă până la convergență

$$\theta_0 \leftarrow \theta_0 - \alpha \frac{1}{n} \sum_{i=1}^n (h(x^i) - y^i)$$

$$\theta_1 \leftarrow \theta_1 - \alpha \frac{1}{n} \sum_{i=1}^n (h(x^i) - y^i) x^i$$

n – număr exemple de învățare



Gradient descent

- Pe masura ce ne apropiem de minimul local, se vor face pasi din ce in ce mai mici
- Regim de invatare de tip batch – utilizeaza toate exemplele de invatare
- α – viteza de invatare
- Daca α prea mica – converge incet
- Daca α prea mare poate sa nu gaseasca minimul
- Daca initializam $\theta_0 \theta_1$ chiar cu valorile minimului – J nu se mai modifica (derivata partiala 0)



Regresie liniara multi-variabile

Consideram un numar de attribute $x_1 \dots x_m$

Minimizarea functiei de cost $J(\theta_0, \theta_1, \dots \theta_m)$

Initializeaza θ_j

repetă până la convergență

pentru $j=0$ la m (simultan) **execută**

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1, \dots \theta_m)$$



Regresie liniara multi-variabile

$$h(\mathbf{x}) = \theta_0 + \theta_1 x_1 \dots \theta_m x_m$$

Consideram un atribut suplimentar $\mathbf{x}_0$ care are valoarea **1** pentru orice exemplu

$$\mathbf{x} = \begin{bmatrix} x_0 \\ \dots \\ x_m \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \dots \\ \theta_m \end{bmatrix}$$

$$h(\mathbf{x}) = \boldsymbol{\theta}^T \mathbf{x}$$



Regresie liniara multi-variabile

$$h(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \dots + \theta_m x_m$$

$$h(\mathbf{x}) = \boldsymbol{\theta}^T \mathbf{x}$$

$$J(\theta_0, \theta_1, \dots, \theta_m) = \frac{1}{2n} \sum_{i=1}^n (h(x^i) - y^i)^2$$

$$\min_{\theta_0 \dots \theta_m} J(\theta_0, \theta_1, \dots, \theta_m)$$



Regresie liniara multi-variabile

Minimizarea functiei de cost $J(\theta_0, \theta_1, \dots, \theta_m)$

Initializeaza $\theta_0, \theta_1, \dots, \theta_m$

repetă până la convergență

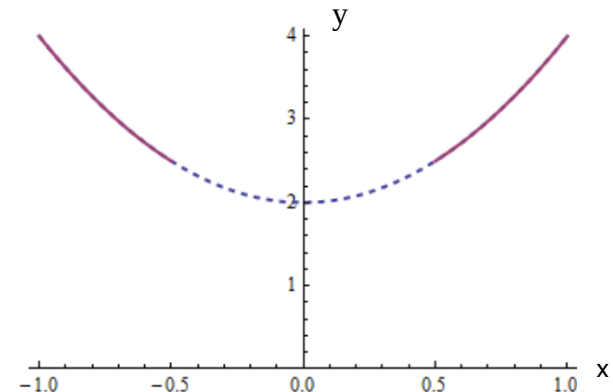
$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^n (h(x^i) - y^i) x^i$$

- Actualizeaza simultan θ_j cu $j=1, m$
- Functia de cost pentru regresie liniara este o functie convexa

Regresie liniara multi-variabile

- Functia de cost are **un singur minim global**?
- Daca **functia de cost este strict convexa** (daca domeiul parametrilor este strict convex, i.e., nu exista restrictii asupra coeficientilor, nu exista regularizare, etc.) atunci scaderea gradientului va avea **o solutie unica care va fi optima global**
- In caz contrar pot exista mai multe solutii,
- Exemplu

$y = (1-x)^2 + (-1-x)^2 \quad |x| \geq 1/2$ solutii $x = \pm 1/2$
 $x \in (-\infty, -1/2] \cup [1/2, \infty)$ - domeniu nonconvex





Scalarea valorilor de atribute

- Pentru ca GD sa converga repede este bine ca valorile de atribute sa fie cam in acelasi interval

Ex: Suprafata casa $\in [0-1000] \text{ m}^2$

Numar camere 1-7

- GD converge foarte incet daca intervale mult diferite
- Scalam, de ex

$$x_1 = \text{sup}/2000 \quad x_2 = \text{nr cam}/7$$

- La fel pentru valori foarte mici, de ex

$$x_1 \in [1..1000] \text{ si } x_2 \in [-0.000.1..+0.0001]$$



2 Arbori de regresie

- **Arborii de regresie** sunt arbori de decizie dar pentru valori continue ale atributelor / variabile
- Realizeaza predictie
- Modelele predictive de regresie liniare sunt **modele globale** in care o singura formula predictiva trebuie (se presupune) sa fie valabila in tot spatiul
- Daca datele de intrare au multe attribute care interactioneaza in moduri complicate, detectarea unui singur model global poate fi **problematica**



Arbori de regresie

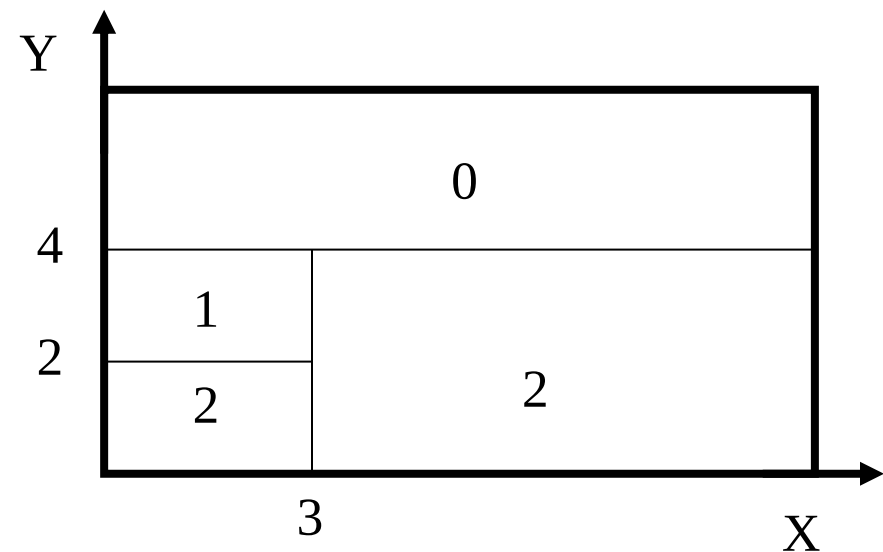
- O metoda pentru a realiza regresie neliniara este partitionarea spatiului in regiuni mici in care interactiunile sunt mai simple
- ***Partitionare recursiva*** pana se ajunge la portiuni ale spatiului care sunt suficient de mici pentru a le putea modela prin modele simple
- Modelul are astfel **2 parti**: **partitionarea recursiva** si un **model simplu pentru fiecare partitie**



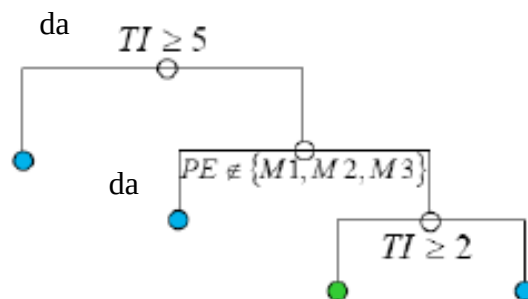
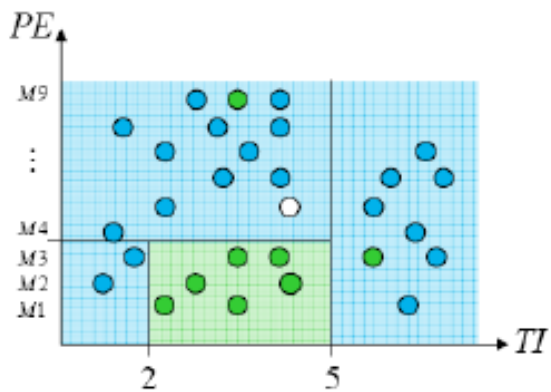
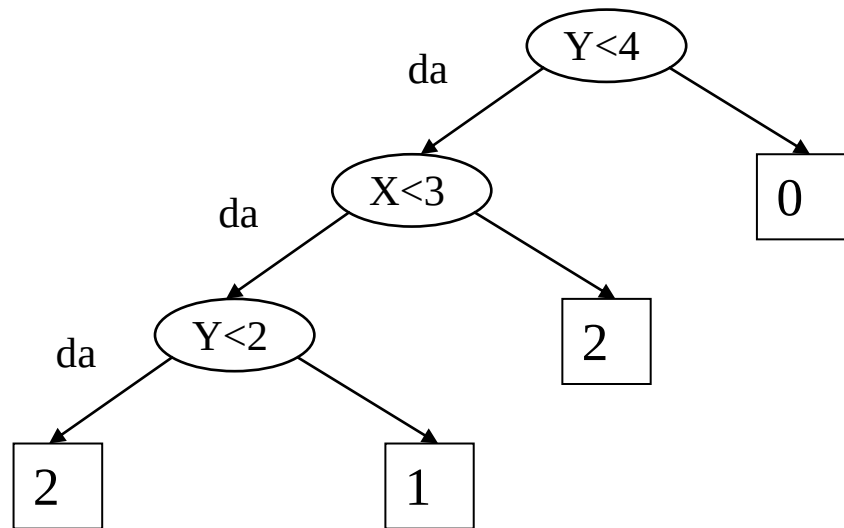
Arbori de regresie

- **Arborii de regresie** utilizeaza arbori de decizie pentru a reprezenta partitionarea
- **Arbori de regresie** sunt arbori de decizie cu valori continue
- **Frunzele** corespund unor regiuni din spatiul de intrare (vecinatati) care sunt **similare ca iesiri si cu intrari alaturate**
- Arborii de predictie / regresie sunt un fel de *nearest-neighbourhood adaptiv*

Funcție $f(X,Y)$



Reprezentare $f(X,Y)$





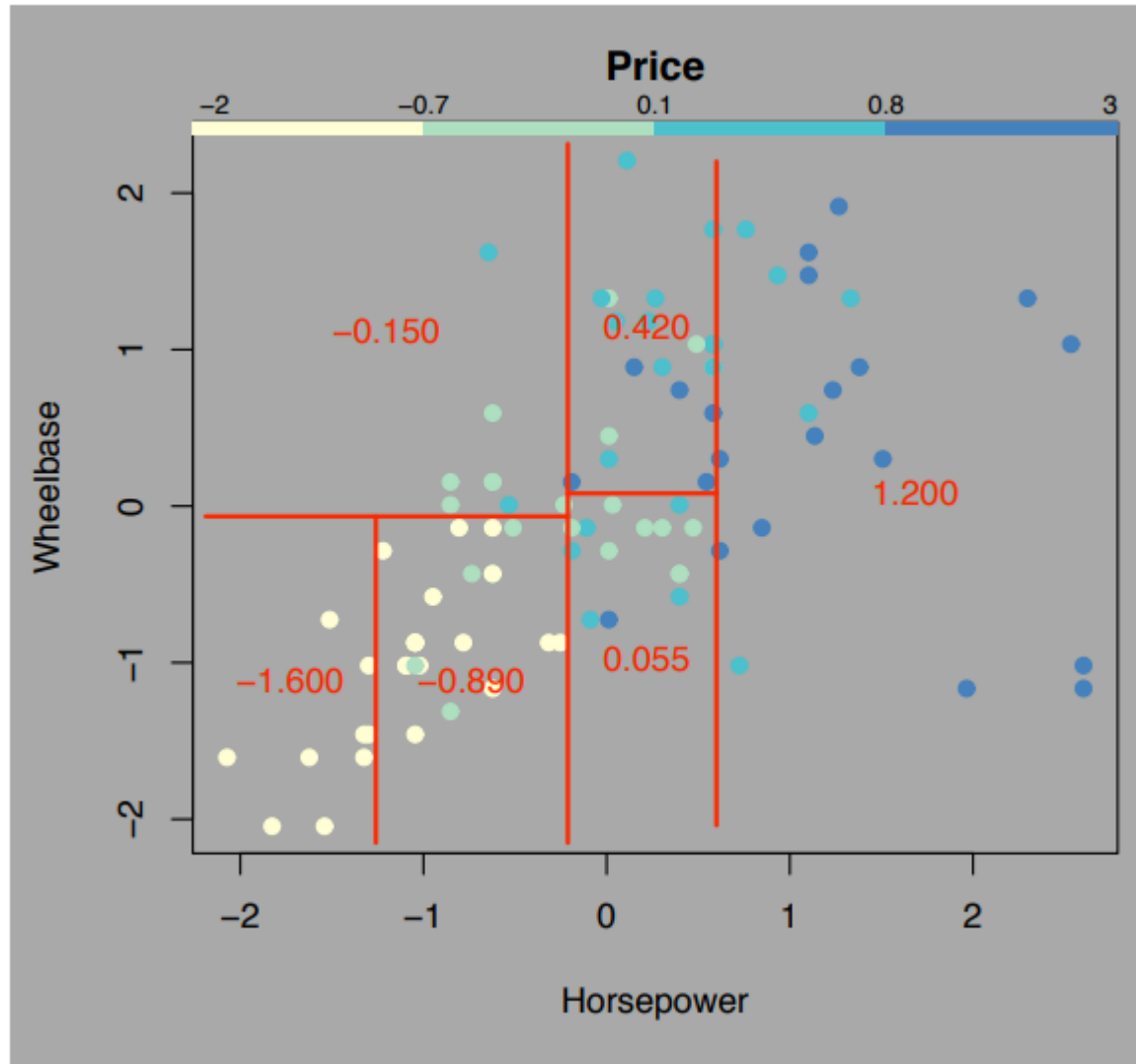
Arbori de regresie

- O frunza $(x_1, y_1) \dots (x_c, y_c)$ Exemplele din regiunea corespunzatoare frunzei

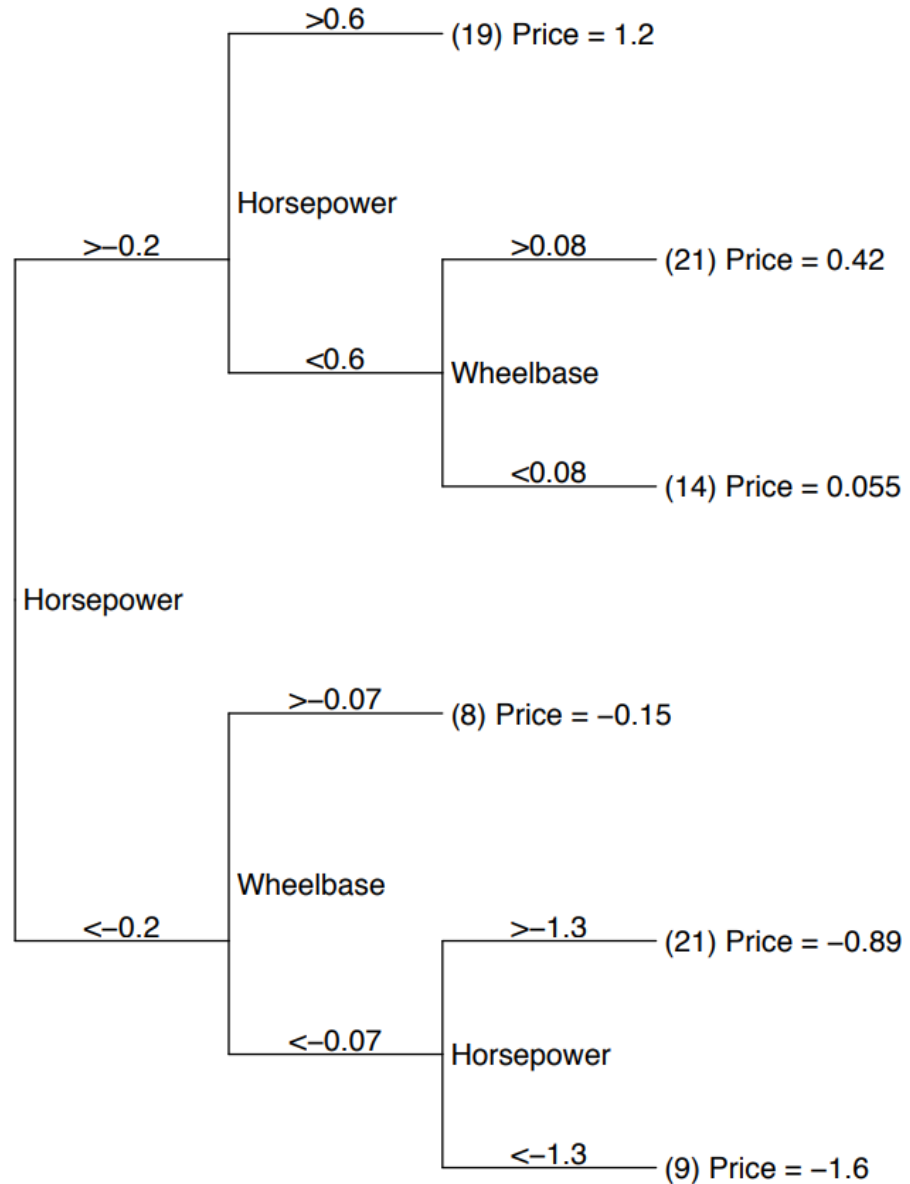
- $y_{\text{est}} = \frac{1}{n_c} \sum_{i=1}^{n_c} y_i$ Modelul pentru regiunea corespunzatoare frunzei

- Flexibilitatea si eficienta unui arbore sunt date de cate frunze are = in cate celule partitionam

Estimare cost
automobiile 1990
in functie de cai-putere
si apartament

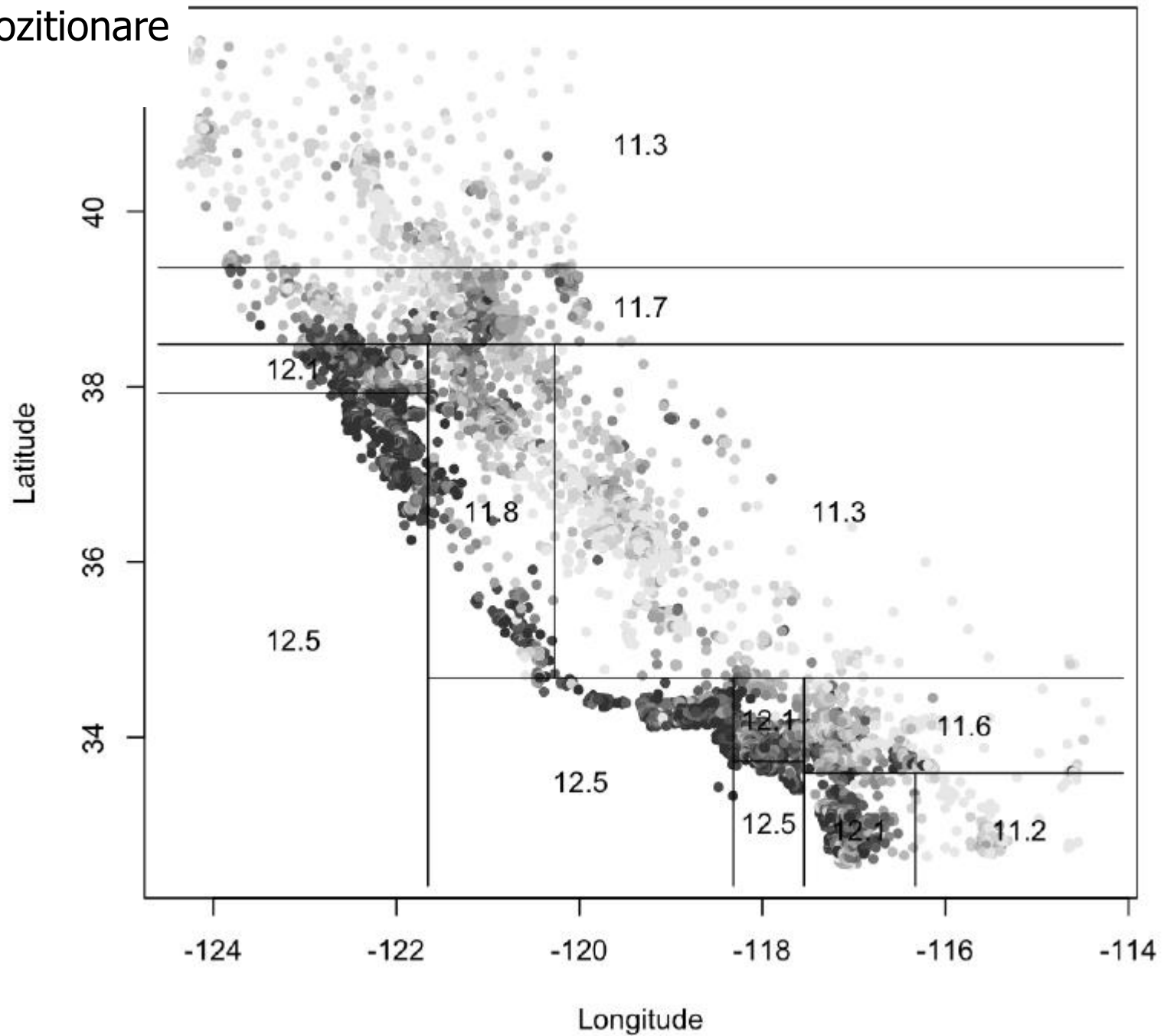


Arbore de regresie - Estimare cost automobilele 1990 in functie de cai-putere si apatament

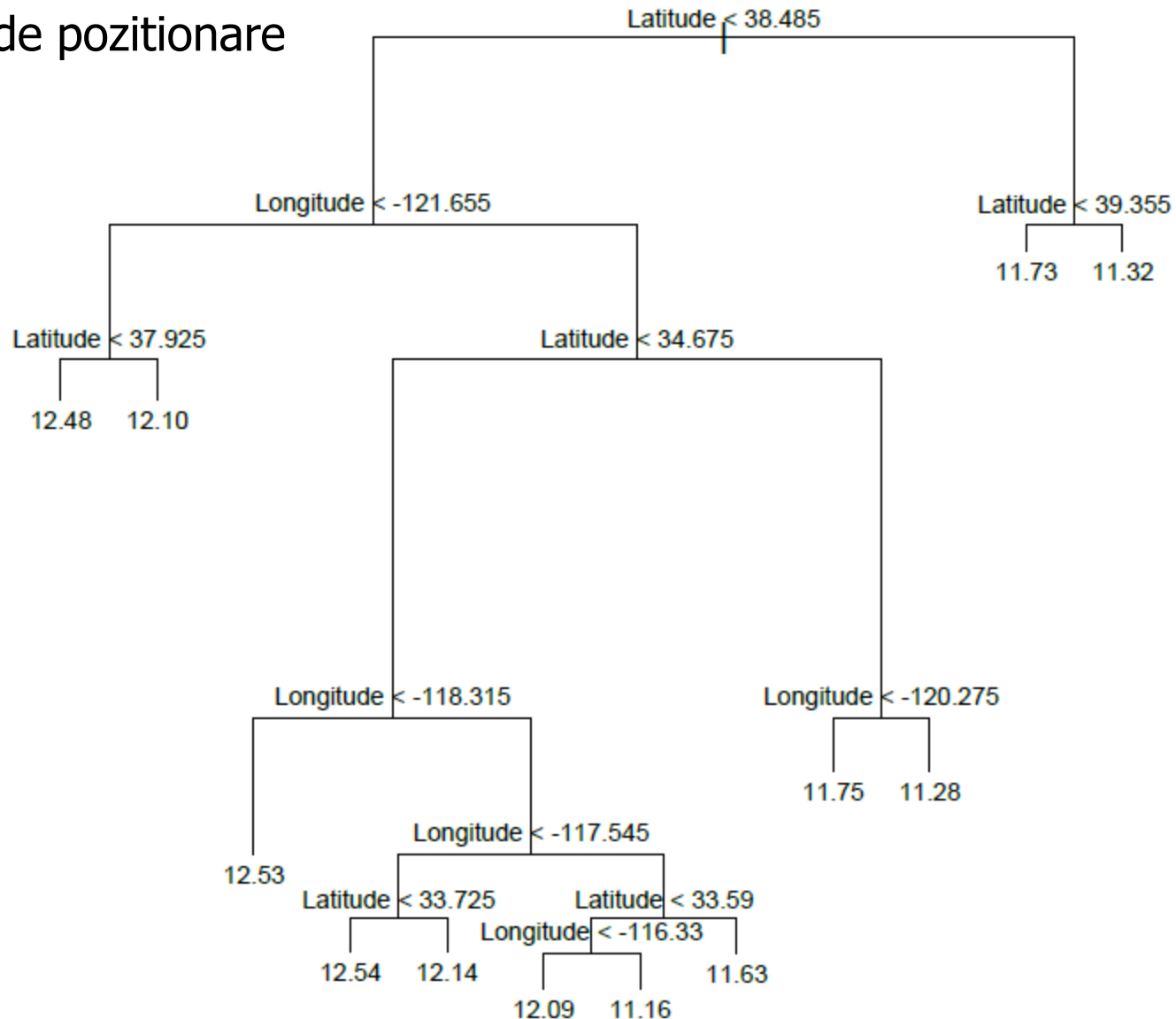


All features have been standardized to have zero mean and unit variance
Credit Cosma Shalizi

Estimare cost case
in functie de pozitionare
pe harta

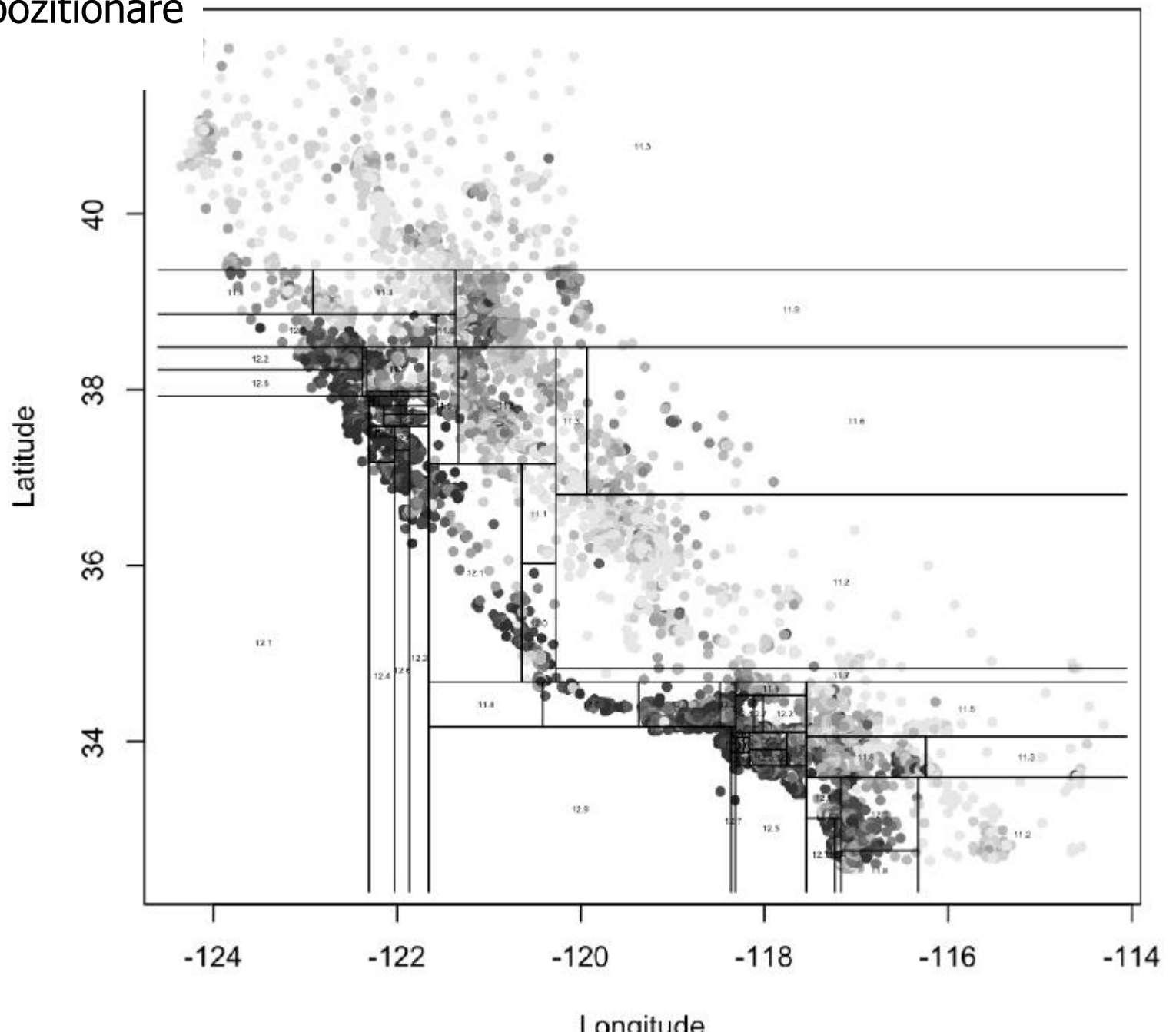


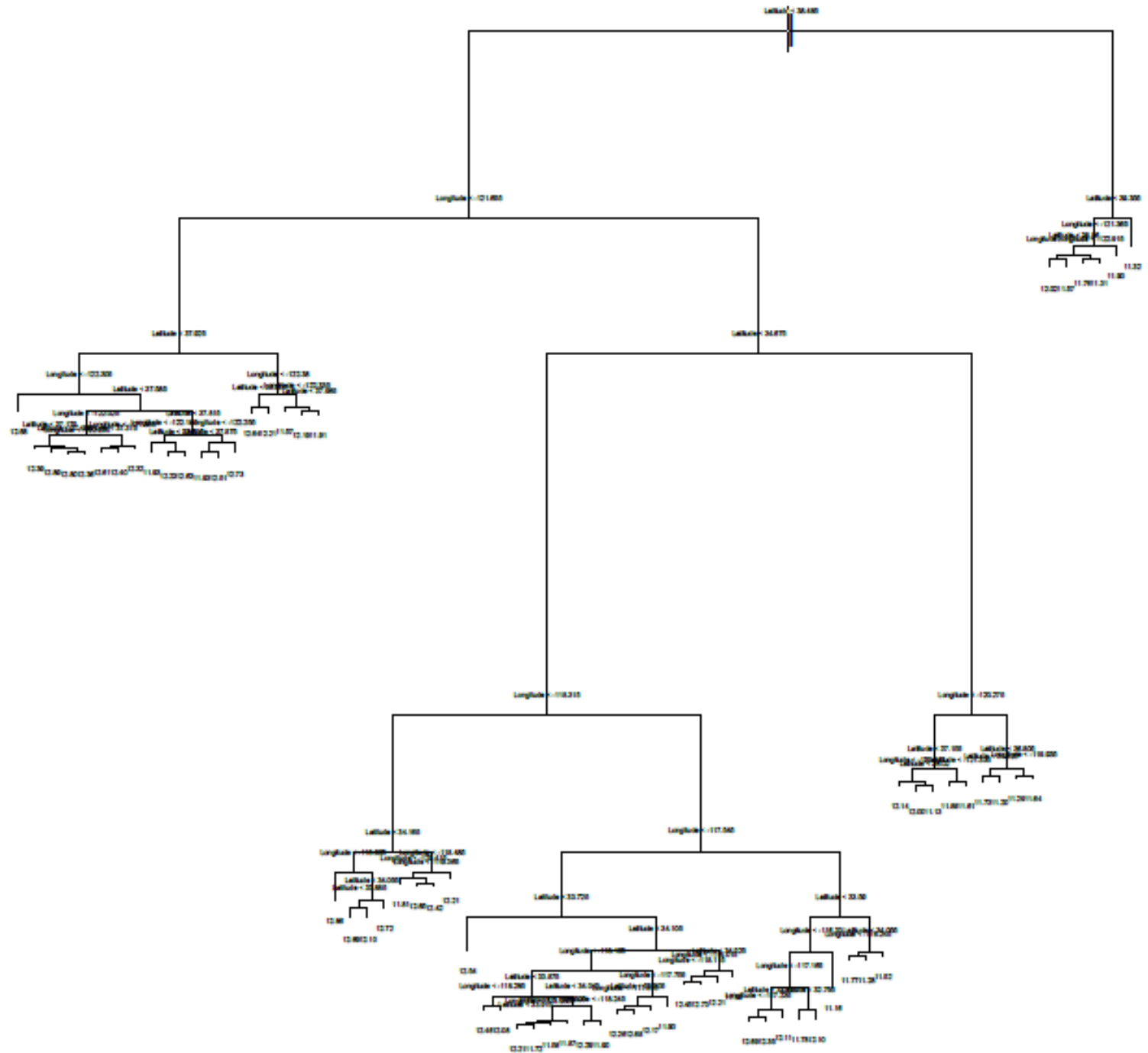
Arbore de regresie - Estimare cost case in functie de pozitionare pe harta



All features have been standardized to have zero mean and unit variance

Estimare cost case
in functie de pozitionare
pe harta







Criteriul de divizare

- Cat de mult cresc arborele?
- Fiecare nod are asociat un anumit numar de puncte
- Adaugarea unui nod trebuie sa reduca eroarea
- RMS- Root Mean Square

De ex RMS 0.41 primul arbore, RMS 0.32 al doilea

- $$\text{RMSError} = \sqrt{\frac{\sum_{i=1,n} (y'_i - y_i)^2}{n}}$$



Criteriul de oprire

- Care este criteriul de oprire?
 - fiecare nod contine mai putin de q puncte
sau
 - divizarea creste informatia cu mai putin de o
valoare de prag = scade eroarea cu mai putin de
valoarea de prag
- Frecvent se utilizeaza patrutul erorilor pentru un
arbore T in loc de $RMSError$



Criteriul de oprire

- Suma erorilor patratice pentru un arbore T este

$$S = \sum_{c \in \text{leaves}(T)} \sum_{i \in c} (y_i - m_c)^2$$

$$m_c = \frac{1}{n_c} \sum_{i \in c} y_i$$

- Trebuie sa minimizam S



Construirea arborelui de regresie

1. Incepe cu 1 nod care contine toate punctele
2. Calculeaza m_c si S pentru acel nod
3. **daca** toate punctele din nod au aceeasi valoare pentru variabilele de intrare atunci **stop**
altfel cauta dintre toate diviziunile binare ale variabilelor pe aceea care reduce S cat mai mult
4. **daca** scaderea in $S < \text{prag}$ sau unul din nodurile obtinute prin diviziune are **mai putin de q** puncte atunci **stop**
altfel creeaza 2 noduri si repeta de la 3

Inteligență Artificială

Universitatea Politehnica Bucuresti
Anul universitar 2022-2023

Adina Magda Florea



Curs nr. 9

- Regresie logistica
- Invatare “Ensemble”



1. Regresie logistica

Invatare supervizata

Utilizata in clasificare

- Regresie logistica binomiala (binara)
- Regresie logistica multinomiala (multi-clasa)

- Am putea utiliza regresia liniara cu
$$h_{\theta}(x) \geq 0.5 \quad y=1$$
$$h_{\theta}(x) < 0.5 \quad y=0$$
- Nu este o idee buna

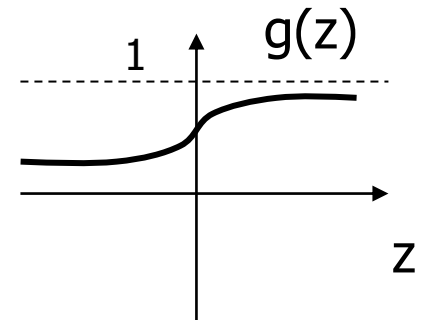
Ipoteza de invatare

$$0 \leq h_{\theta}(x) \leq 1$$

$$h(\mathbf{x}) = g(\boldsymbol{\theta}^T \mathbf{x})$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

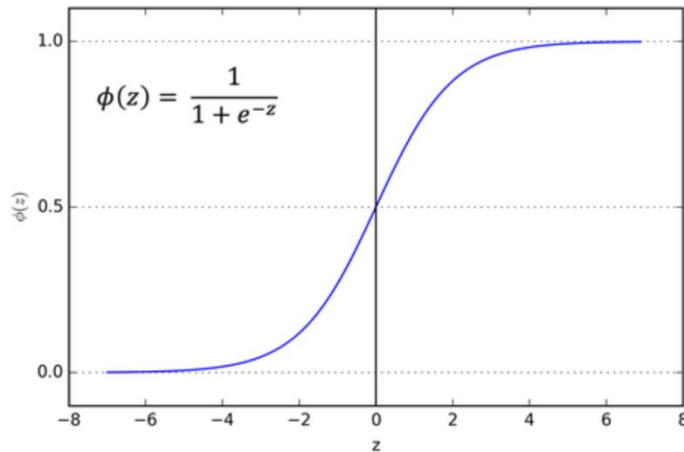
$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$



$h_{\theta}(x)$ estimeaza probabilitatea ca $y=1$ pentru o intrare x

$$h_{\theta}(x) = P(y=1 \mid x; \theta)$$

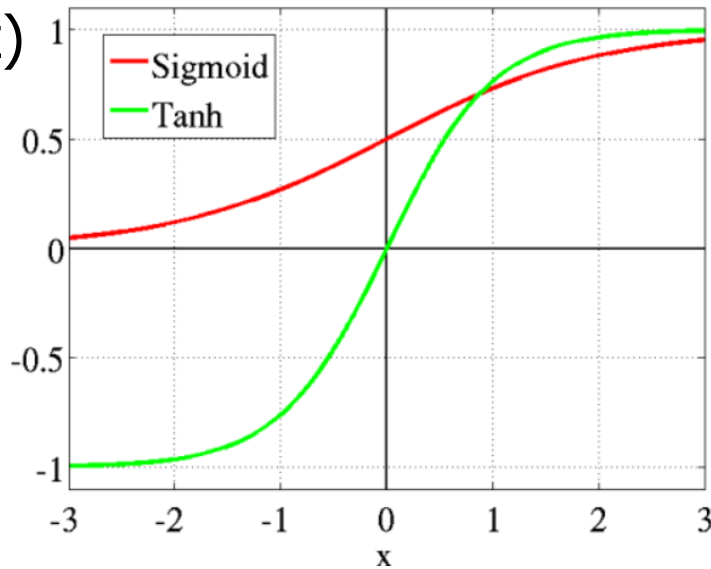
$\sigma(z)$



$$\sigma(z) = 1 / (1 + e^{-z})$$

Sigmoid

$\tanh(z)$



$$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$$

Tangenta hiperbolica



Ipoteza de invatare

$0 \leq h_{\theta}(x) \leq 1$ presupunem o relatie liniara intre 2
atribute x_1 si x_2 si consideram probabilitatea
clasei y

$$p = P(y=1 \mid x; \theta)$$

$$l = \ln \frac{p}{1-p} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 \quad \text{“log odds” } (l)$$

$$\frac{p}{1-p} = e^{\theta_0 + \theta_1 x_1 + \theta_2 x_2}$$

$$p = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2)}}$$



Funcția de cost

În regresie liniară avem funcția de cost

$$J(\theta_0, \theta_1, \dots, \theta_m) = \frac{1}{2n} \sum_{i=1}^n (h(x^i) - y^i)^2$$

În regresie logistică

$$h_{\theta}(\mathbf{x}) = \theta^T \mathbf{x}$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Dacă am utiliza aceeași funcție de cost aceasta nu ar mai fi convexă (deci nu un unic minim global)

Avem nevoie să definim o altă funcție de cost



Funcția de cost

$$\text{Cost}(h_{\theta}(\mathbf{x}), y) = -\log(h_{\theta}(\mathbf{x})) \quad \text{dacă } y=1$$
$$-\log(1-h_{\theta}(\mathbf{x})) \quad \text{dacă } y=0$$

Inseamna ca dacă $h_{\theta}(\mathbf{x}) = 0$ costul tinde la infinit deci
predictia este $P(y=1 \mid \mathbf{x}; \theta) = 0$

Scriem compact

$$\text{Cost}(h_{\theta}(\mathbf{x}), y) = -y \log(h_{\theta}(\mathbf{x})) - (1-y) \log(1-h_{\theta}(\mathbf{x}))$$

$$J(\theta) = -\frac{1}{2n} \sum_{i=1}^n y^i \log(h_{\theta}(\mathbf{x}^i)) + (1-y^i) \log(1-h_{\theta}(\mathbf{x}^i))$$

MLE – Maximum Likelihood Estimation



Clasificare

Pentru a face o predictie (a clasifica) trebuie sa determinam parametrii θ

$\min_{\theta} J(\theta)$ cu $J(\theta) = -\frac{1}{2n} \sum_{i=1}^n y^i \log(h_{\theta}(\mathbf{x}^i)) + (1-y^i) \log(1-h_{\theta}(\mathbf{x}^i))$

Aplicam scaderea gradientului

Initializeaza θ_j

repetă până la convergență

pentru $j=0$ la m (simultan) execută

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1, \dots, \theta_m)$$



Scaderea gradientului

Initializeaza $\theta_0, \theta_1, \dots, \theta_m$

repetă până la convergență

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^n (h(x^i) - y^i) x^i$$

- Actualizează simultan θ_j cu $j=0, m$

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

($h_{\theta}(x) = \theta^T x$ pt regresie liniară)



Diferente

Regresie liniara

- Distributia este Gausiana
- Valorile prezise sunt continue

Regresie logica

- Distributia este Bernoulli/Binomială
- Valorile prezise sunt probabilitati pentru
apartenenta (sau non-apartenenta) la o clasa

(Distributie Bernoulli – 1 eveniment cu iesire 1/0

Distributie categoriala – 1 eveniment cu k iesiri

Distributie binomiala – n evenimente cu iesire 1/0

Distributie multinomiala – n evenimente cu k iesiri posibile)



Algoritmi de optimizare

- Scaderea gradientului
- Scaderea conjugata a gradientului
- Broyden–Fletcher–Goldfarb–Shanno (BFGS) - metoda quasi-Newtoniana (tip hill climbing)
(quasi-newtoniana – daca Hessiana este prea complicat de calculat, aceasta este actualizata prin analiza iterativa a vectorilor de gradient)
- Limited memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS)

BFGS nu neecsita stabilirea manuala a α



Clasificare multi-clasa

Clasificare “**one-versus-all**”, K clase

Clasa **1** $h^1_{\theta}(x) = P(Y=1 \mid X; \theta_1)$ – determina setul de parametrii θ_1

Clasa **2** $h^2_{\theta}(x) = P(Y=2 \mid X; \theta_2)$ – determina setul de parametrii θ_2

etc.

Pentru o instanta necunoscuta x , determina $h^i_{\theta}(x)$ pt fiecare clasa si alege clasa pentru care $h^i_{\theta}(x)$ are valoarea maxima



Clasificare multi-clasa

Pentru K clase, se executa regresie logistica pentru K-1 modele de regresie logistica binara, din care o clasa este aleasa ca pivot si celelalte K-1 sunt clasificate in functie de pivot. Daca clasa K este aleasa ca pivot atunci:

$$\ln \frac{\Pr(Y_i = 1)}{\Pr(Y_i = K)} = \theta_1 \cdot \mathbf{X}_i$$

$$\ln \frac{\Pr(Y_i = 2)}{\Pr(Y_i = K)} = \theta_2 \cdot \mathbf{X}_i$$

.....

$$\ln \frac{\Pr(Y_i = K - 1)}{\Pr(Y_i = K)} = \theta_{K-1} \cdot \mathbf{X}_i$$



Overfitting

Solutii pt minimizare

1) **Reducerea numarului de attribute**

- Manual
- Algoritmi de selectie a modelului

Poate reduce overfitting dar elimina informatie

2) **Regularizare**

Pastrez toate attributele dar reduc dimensiunea parametrilor $\theta_1, \dots, \theta_m$

Potrivita daca avem multe attribute, fiecare contribuind cu putin/la fel la predictie



2. Regularizare

Tehnica de a minimiza overfitting

Idee = sa pastram θ mici, duce la modele mai simple

$$J(\theta_0, \theta_1, \dots, \theta_m) = \frac{1}{2n} \left[\sum_{i=1}^n (h(x^i) - y^i)^2 + \varphi \sum_{j=1}^m \theta_j^2 \right]$$

$$\theta_j \leftarrow \theta_j - \alpha \left[\frac{1}{n} \sum_{i=1}^n (h(x^i) - y^i) x^i + \frac{\varphi}{m} \theta_j \right]$$

Termen de
regularizare

pt $j=1, m$ si

$$\theta_0 \leftarrow \theta_0 - \alpha \frac{1}{n} \sum_{i=1}^n (h(x^i) - y^i) x^i$$

φ - Parametru de
regularizare



Regularizare

Se poate rescrie

$$\theta_j \leftarrow \theta_j \left(1 - \alpha \frac{\varphi}{m}\right) - \alpha \frac{1}{n} \sum_{i=1}^n (h(x^i) - y^i) x^i$$

pt $j=1,m$

Primul termen de obicei mai mic ca 1, are efectul reducerii parametrilor

Al doilea termen actualizeaza parametrii conform scaderii gradientului



Observatii

- Regularizarea se aplica atat la regresie logistica cat si la regresie liniara.

Formula de actualizare a θ_j , $j=1,m$ este aceeaasi (evident difera $h(x)$)

- Regresia logistica poate beneficia, la fel ca cea liniara, de scalarea valorilor de atribute (vezi Curs 8)



3. Invatarea “Ensemble”

- Genereaza mai multe ipoteze din acelasi set de date
- Combina mai multe ipoteze de invatare

Metode

- Naïve Bayes
- Bagging – Random forests
- Boosting – construiește incremental o multime de ipoteze prin antrenarea fiecărei noi instante de model a.i. sa ia in considerare exemplele gresit clasificate de modelul anterior - Adaboost

Adaboost

- N exemple de invatare $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$, M etape

Invatare

Initializare $D_i^1 = \frac{1}{N}$ pt $i=1, N$

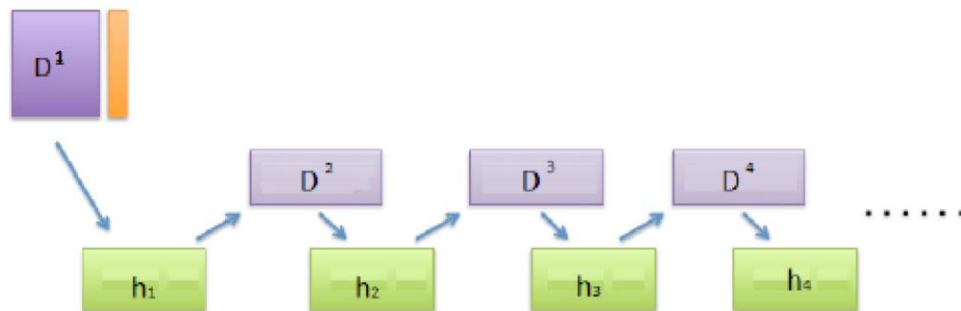
pentru $j=1, M$ **repeta**

1. Construiești un model h_j din setul de invatare utilizand distributia D^j
2. Definesti eroarea ponderata a modelului j

$$\varepsilon_j = \sum_{i: h_j(x_i) \neq y_i} D_i^j$$



Eroarea ponderata a modelului h_j





Adaboost

3. Obține o ipoteză h_j care minimizează eroarea ε_j și satisface condiția $\varepsilon_j < 1/2$

$$\alpha_j = \frac{1}{2} \log \left(\frac{1 - \varepsilon_j}{\varepsilon_j} \right)$$

$$D_i^{j+1} = e^{-y_i h_j(x_i) \alpha_j} D_i^j$$

$$D_i^{j+1} = \frac{D_i^{j+1}}{\sum_{i=1}^N D_i^{j+1}}$$

Predictie

$$H(x) = \text{sign} \left[\sum_{j=1}^M \alpha_j h_j(x) \right]$$



Boosting

<https://towardsdatascience.com/understanding-adaboost-2f94f22d5bfe>

https://medium.com/mlreview/gradier_boosting-from-scratch-1e317ae4587d

Inteligență Artificială

Universitatea Politehnica Bucuresti
Anul universitar 2022-2023

Adina Magda Florea

Curs nr. 12

Prelucrarea limbajului natural

- Prelucrare LN pentru comunicare
- Prelucrare LN pt achizitia cunostintelor

Prelucrare LN pentru comunicare

1 Comunicare

- *Definitie:* schimbul intentional de informatie generat de producerea si perceperea semnelor dintr-un sistem partajat de semne conventionale

 - Componentele comunicarii
 - intentie
 - generare
 - sinteza

Emitator

 - perceptie
 - analiza
 - desambiguare
 - incorporare
- Receptor

Acte de comunicare

J. Austin - How to do things with words, 1962, J. Searle - Speech acts, 1969

Un act de comunicare:

- **locutie** = fraza spusă de locutor
- **illocutie** = înțelesul dorit să fie comunicat de locutor (performativă)
- **prelocutie** = acțiunea care rezultă din locutie

Maria i-a spus lui Ion: "Te rog închide ușa"

locutie

illocutie continut

prelocutie: **ușa închisă**

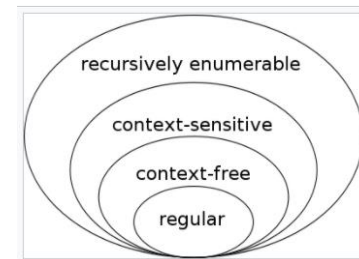
Categorii ilocutionale

- **Asertive**
- **Directive**
- **Permisive**
- **Prohibitive**
- **Declarative**
- **Expresive**

2 Modele limbaj

- Modelele n-gram - bazate pe secvențe de caractere, cuvinte
- Modele de limbaj bazate pe structura gramaticală
- Categoriile lexicale (parts of speech): substantiv, adjectiv, etc
- Categoriile sintactice: grup substantival, grup verbal, etc.
- Structuri de fraze

Modele limbaj



<u>Gramatica /</u> <u>Clasa limbaj</u>	Automat	Reguli de <u>productie</u>
<u>Rekursiv numarabile</u>	<u>Masina Turing</u>	$\gamma \rightarrow \alpha$
<u>Dependente de context</u>	<u>Masina Turing</u> <u>linear marginita</u>	$\alpha A \beta \rightarrow \alpha \gamma \beta$
<u>Independente de context</u>	Automate Push down	$A \rightarrow \alpha$
Regulate	Automate cu <u>stari</u> finite	$A \rightarrow a$ $A \rightarrow aB$

Fernando Pereira: "*The older I get, the further down the Chomsky hierarchy I go*"

Definire limbaj

- Lexicon, categorii deschise si inchise
- Analiza lexicala
- Analiza sintactica (pars oratoris)
- Gramatici
- Terminale, neterminale
- Reguli de rescriere/productii (LHS $\rightarrow$ RHS)
- Analiza semantica + analiza pragmatica

3 Analiza lexicala

Gramatici regulate

$G=(N,\Sigma,R,S)$

N este multimea de neterminale

Σ este multimea de terminale

R este multimea de reguli de rescriere

$R:\{ X \rightarrow a \text{ sau } X \rightarrow aY \mid X,Y \in N \text{ si } a \in \Sigma \}$

S – simbolul de start

- Genereaza Lexiconul

4 Analiza sintactica

Gramatici independente de context

$G=(N,\Sigma,R,S)$

N este multimea de neterminale

Σ este multimea de terminale

R este multimea de reguli de rescriere

$$R:N \rightarrow (N \cup \Sigma)^*$$

S – simbolul de start

Gramatica pt sintaxa

Lexicon

$S \rightarrow NP VP$

$S \rightarrow Aux NP VP$

$S \rightarrow VP$

$NP \rightarrow Pronoun$

$NP \rightarrow Proper-Noun$

$NP \rightarrow Det Nominal$

$Nominal \rightarrow Noun$

$Nominal \rightarrow Nominal Noun$

$Nominal \rightarrow Nominal PP$

$VP \rightarrow Verb$

$VP \rightarrow Verb NP$

$VP \rightarrow Verb NP PP$

$VP \rightarrow Verb PP$

$VP \rightarrow VP PP$

$PP \rightarrow Preposition NP$

$Det \rightarrow that \mid this \mid a$

$Noun \rightarrow book \mid flight \mid meal \mid money$

$Verb \rightarrow book \mid include \mid prefer$

$Pronoun \rightarrow I \mid she \mid me$

$Proper-Noun \rightarrow Houston \mid TWA$

$Aux \rightarrow does$

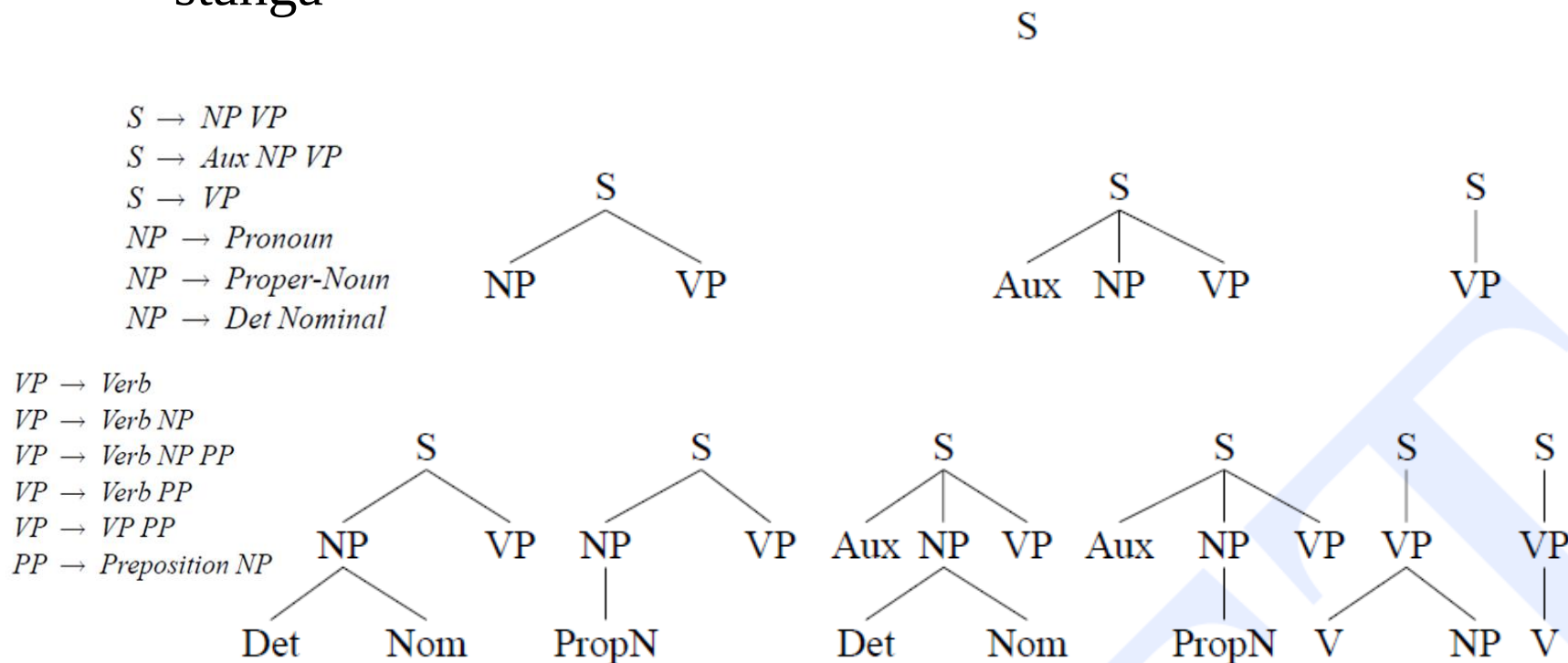
$Preposition \rightarrow from \mid to \mid on \mid near \mid through$

Care este arborele de derivare/sintactic
al frazei **Book that flight?**

Analiza Top-down

Book that flight

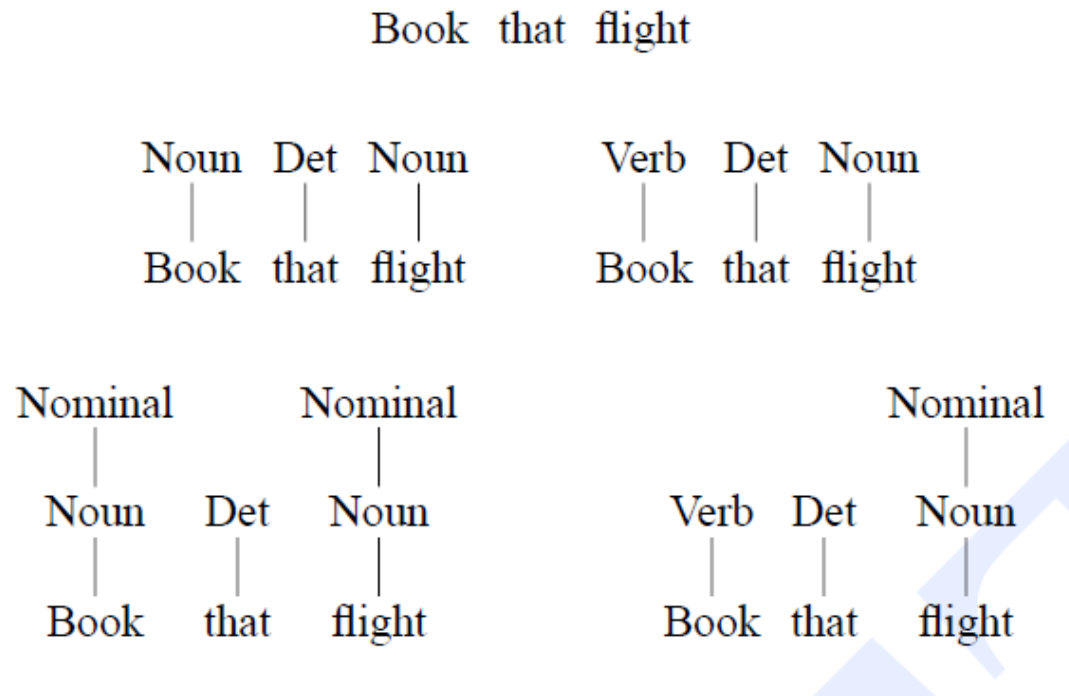
- Analiza top-down cauta arborele de derivare incepand de la S catre frunze
- Pe fiecare nivel se expandeaza neterminalul cel mai din stanga



Analiza Bottom-Up

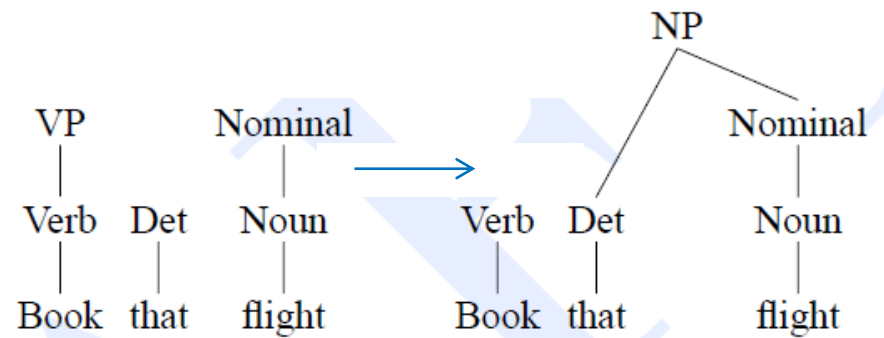
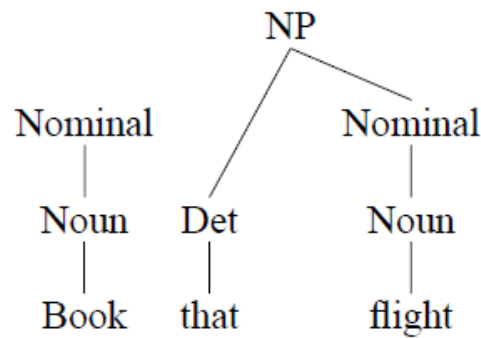
Book that flight

- Porneste de la cuvintele din fraza si cauta sa obtina S
- Folosita si la analizoarele “shift-reduce” pt limbaje de programare



<i>Det</i> → <i>that</i> <i>this</i> <i>a</i>
<i>Noun</i> → <i>book</i> <i>flight</i> <i>meal</i> <i>money</i>
<i>Verb</i> → <i>book</i> <i>include</i> <i>prefer</i>

S → *NP VP*
S → *Aux NP VP*
S → *VP*
NP → *Pronoun*
NP → *Proper-Noun*
NP → *Det Nominal*
Nominal → *Noun*
Nominal → *Nominal Noun*
Nominal → *Nominal PP*
VP → *Verb*
VP → *Verb NP*
VP → *Verb NP PP*
VP → *Verb PP*
VP → *VP PP*
PP → *Preposition NP*



$S \rightarrow NP VP$

$S \rightarrow Aux NP VP$

$S \rightarrow VP$

$NP \rightarrow Pronoun$

$NP \rightarrow Proper-Noun$

$NP \rightarrow Det Nominal$

$Nominal \rightarrow Noun$

$Nominal \rightarrow Nominal Noun$

$Nominal \rightarrow Nominal PP$

$VP \rightarrow Verb$

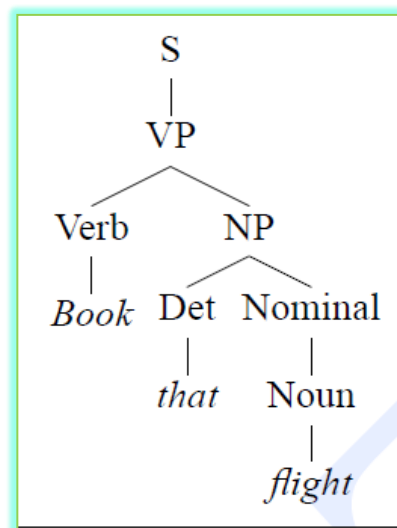
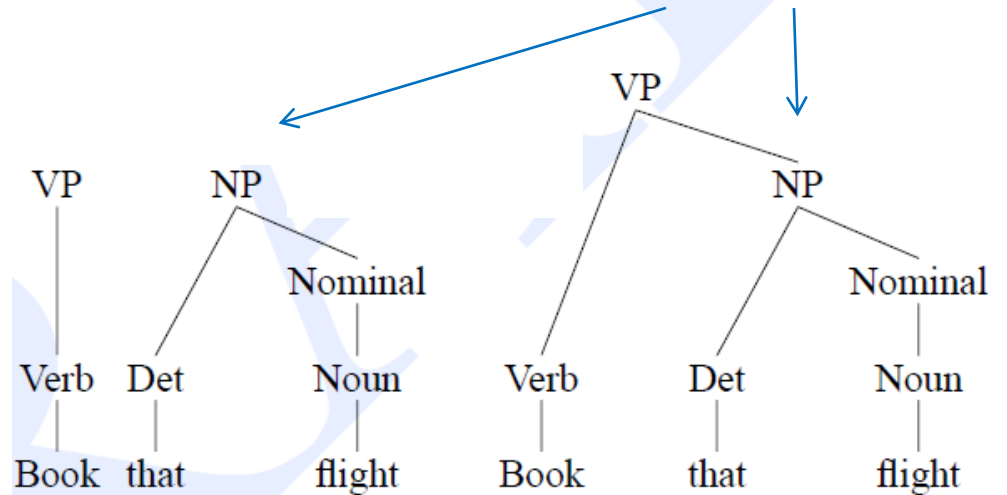
$VP \rightarrow Verb NP$

$VP \rightarrow Verb NP PP$

$VP \rightarrow Verb PP$

$VP \rightarrow VP PP$

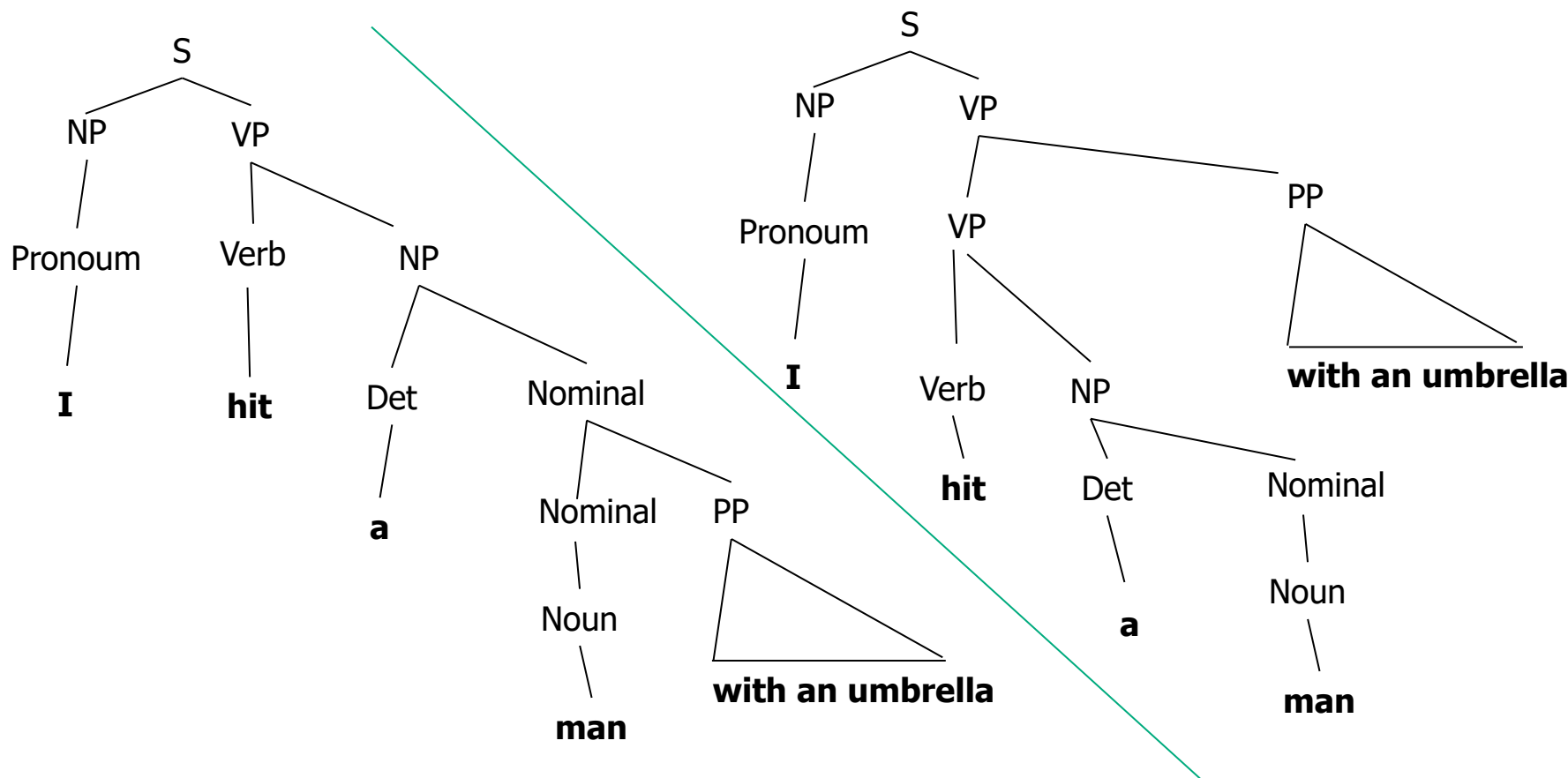
$PP \rightarrow Preposition NP$



Book that flight

Problema ambiguitatii sintactice

- Ambiguitate sintactica sau **ambiguitate structurala**
- Gramatica genereaza mai multi arbori de analiza



Problema ambiguitatii sintactice

Ambiguitate structurala

- **Ambiguitate de asociere** (attachment ambiguity) – un constituent poate fi atasat in arborele de derivare in mai mult de 1 loc
 - I hit a man with an umbrella
 - We saw the Eiffel Tower flying over Paris
- **Ambiguitate de coordonare** (coordination ambiguity) - parti de fraza care pot fi cuplate diferit de conjunctii cum ar fi “and”
 - Old men and women
 - He speaks good French and English

Analiza sintactica pt LN

- Ambiguitatea structurala poate genera multi arbori sintactici
(Old men and women speak good French and English)
- Pentru anumite gramatici, numarul posibil de arbori poate fi exponential in raport cu numarul de cuvinte din fraza
- O solutie – Gramaticile independente de context probabiliste

GICP (PCFG)

- Gramatici independente de context probabiliste (GICP)

VP $\rightarrow$ Verb [0.70]
 | Verb NP [0.30]

- Determinare probabilitati
 - de proiectant
 - pe baza de treebanks (fraze deja analizate corect), de ex. Penn Treebank (<https://en.wikipedia.org/wiki/Treebank>)

Grammatici regulate probabiliste

■ Lexicon

Noun → **breeze** [0.10] | **wumpus** [0.15] | **ball** [0.15] ...

Verb → **is** [0.10] | **see** [0.10] | **smells** [0.10] | **hit** [0.10]...

Adjective → **right** [0.10] | **left** [0.10] | **smelly** [0.15] ...

Adverb → **here** [0.05] | **there** [0.05] | **ahead** [0.02] ...

Pronoun → **me** [0.10] | **you** [0.03] | **I** [0.10] | **it** [0.10] ...

RelPronoun → **that** [0.40] | **who** [0.20] ...

Name → **John** [0.1] | **Mary** [0.01] ...

Article → **the** [0.40] | **a** [0.30] | **an** [0.10] ...

Preposition → **to** [0.20] | **in** [0.10] | **on** [0.05] ...

Conjunction → **and** [0.50] | **or** [0.10] | **but** [0.20]...

GIC probabiliste - sintaxa

■ Sintaxa

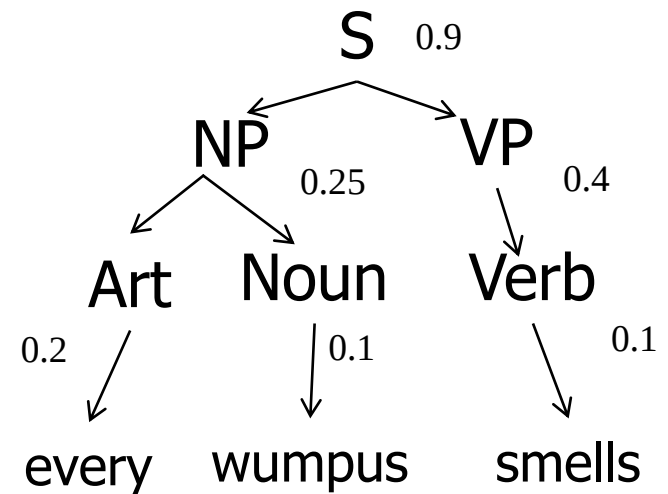
S → NP VP	[0.90]	I feel a breeze
S Conjunction S	[0.10]	I feel a breeze and it stinks
NP → Pronoun	[0.30]	I
Name	[0.10]	John
Noun [0.10]	[0.10]	pit
Article Noun	[0.25]	the wumpus
NP PP	[0.10]	the wumpus in 1,3
NP RelClause	[0.05]	the wumpus that is smelly
VP → Verb	[0.40]	stinks
VP NP	[0.35]	feel a breeze
VP Adjective	[0.05]	smells dead
VP PP	[0.10]	is in 1,3
VP Adverb	[0.10]	go ahead
PP → Preposition NP	[1.00]	to the east
RelClause → RelPronoun VP	[1.00]	that is smelly

Analiza sintactica cu GICP

- Fiecare arbore de derivare intr-o GICP are asociata o probabilitate $P(t)$

Ex: $0.9 \times 0.25 \times 0.2 \times 0.1 \times 0.4 \times 0.1 = 0.00018$

$S \rightarrow NP \ VP$	[0.90]
$NP \rightarrow Art \ Noun$	[0.25]
$Art \rightarrow every$	[0.2]
$Noun \rightarrow wumpus$	[0.1]
$VP \rightarrow Verb$	[0.4]
$Verb \rightarrow smells$	[0.1]



Analiza sintactica cu GICP

- Fiind data o GICP si o fraza s , fie $T(s)$ multimea de arbori de derivare asociati lui s
- Ne intereseaza sa gasim

$$\arg \max_{t \in T(s)} P(t)$$

Avem 2 posibilitati:

- 1) **Generez toti arborii posibili**; calculez probabilitatile asociate si aleg arborele cu cea mai mare probabilitate

Ineficienta datorita numarului mare de arbori posibili

- 2) **Bazata pe programare dinamica / chart parsers**

Analizez subsiruri si memorez analizele partiale

Analiza CKY (Cocke–Younger–Kasami)

- Analiza CKY este o analiza “bottom-up” in care dupa ce analizam un subsir, se memoreaza rezultatul intr-o tabela (**chart**), astfel incat sa il putem reutiliza - **chart parser**
- GIC – orice subsir/fraza analizata pe o ramura a arborelui sintactic poate fi utilizata pe alta ramura
- Chart parser-ul **CKY** (J. Cocke, D. Young, T. Kasami)
- Gramatica trebuie sa fie in **Forma Normala Chomsky (CNF)**; orice regula trebuie sa aiba una din doua forme:
 - $X \rightarrow a$ cu $a \in \Sigma$ (reguli lexicale)
 - $X \rightarrow Y Z$ cu $Y, Z \in N$ (reguli sintactice)

Transformarea in Chomsky Normal Form

Trebuie sa transformam urmatoare tipuri de reguli:

- Reguli care combina terminale cu neterminale

$X \rightarrow a Y [p]$ transform $X \rightarrow A Y [p]$ si $A \rightarrow a [1.0]$

- Reguli care au 1 singur neterminal in partea dreapta (unitare)

Se rescrie partea dreapta cu partea dreapta a tuturor regulilor neunitare

$X \rightarrow Y [p]$ si $Y \rightarrow a [p1]$ transform

$X \rightarrow a [p \times p1]$

Transformarea in CNF

- Reguli care au mai mult de 2 neterminale in dreapta

Introduc noi neterminale

$X \rightarrow Y Z T [p]$ transform in

$X1 \rightarrow Y Z [p]$

$X \rightarrow X1 T [1.0]$

- Orice GIC (nu neaparat GICP) poate fi astfel transformata in CNF

Analiza sintactica

- CKY pt GICP foloseste un spatiu de $O(N^2M)$, N – nr cuvinte (lungimea sirului de intrare), M – nr neterminale din gramatica, pentru a construi o **tabela de probabilitati P**
- Timp $O(N^3M)$
- Nu examineaza toti a.d. posibili ci calculeaza probabilitatea celui mai probabil a.d.
- Toti subarborii sunt implicit reprezentati in tabela P de unde se pot obtine daca dorim

Exemplul 1

The flight includes a meal

Det → the 0.5				
	N → flight 0.02			
		V → includes 0.05		
			Det → a 0.4	
				N → meal 0.01

Chart parser pas 1

- S → NP VP 0.8

NP → Det N 0.3

VP → V NP 0.2
- V → includes 0.05

Det → the 0.5

Det → a 0.4

N → meal 0.01

N → flight 0.02

Exemplul 1

The	flight	includes	a	meal
Det → the 0.5	NP → Det N 0.3 x 0.5 x 0.02 = 0.003			
	N → flight 0.02			
		V → includes 0.05		
			Det → a 0.4	NP → Det N 0.3 x 0.4 x 0.01 = 0.0012
				N → meal 0.01

Chart parser pas 2

S → NP VP 0.8
NP → Det N 0.3
VP → V NP 0.2

V → includes 0.05
Det → the 0.5
Det → a 0.4
N → meal 0.01
N → flight 0.02

Exemplul 1

The	flight	includes	a	meal
Det → the 0.5	NP → Det N 0.3 x 0.5 x 0.02 = 0.003			S → NP VP 0.8 x 0.003 x 0.000012 = 0.0000000288
	N → flight 0.02			
		V → includes 0.05		VP → V NP 0.2 x 0.05 x 0.000012 = 0.000012
			Det → a 0.4	NP → Det N 0.3 x 0.4 x 0.01 = 0.0012
				N → meal 0.01

Chart complet

- S → NP VP 0.8

NP → Det N 0.3

VP → V NP 0.2
- V → includes 0.05

Det → the 0.5

Det → a 0.4

N → meal 0.01

N → flight 0.02

Exemplul 2

G initiala

Fish people fish tanks

G transformata in CNF

S → NP VP	0.9	S → NP VP	0.9
S → VP	0.1	S → V NP	0.05
		S → people	0.001
		S → fish	0.006
VP → V NP	0.5	VP → V NP	0.5
VP → V	0.1	VP → people	0.01
		VP → fish	0.06
NP → NP NP	0.2	NP → NP NP	0.2
NP → N	0.7	NP → people	0.35
		NP → fish	0.14
		NP → tanks	0.14
N → people	0.5	N → people	0.5
N → fish	0.2	N → fish	0.2
N → tanks	0.2	N → tanks	0.2
V → people	0.1	V → people	0.1
V → fish	0.6	V → fish	0.6

Fish

people

fish

tanks

$S \rightarrow \text{fish } 0.006$
 $N \rightarrow \text{fish } 0.2$
 $V \rightarrow \text{fish } 0.6$
 $NP \rightarrow \text{fish } 0.14$
 $VP \rightarrow \text{fish } 0.06$

$S \rightarrow NP VP$
 $0.9 * 0.14 * 0.01 = 0.00126$
 $S \rightarrow V NP$
 $0.05 * 0.6 * 0.35 = 0.0105$
 $VP \rightarrow V NP$
 $0.5 * 0.6 * 0.35$
 $NP \rightarrow NP NP$
 $0.2 * 0.14 * 0.35$

$S \rightarrow \text{people } 0.001$
 $N \rightarrow \text{people } 0.5$
 $V \rightarrow \text{people } 0.1$
 $NP \rightarrow \text{people } 0.35$
 $VP \rightarrow \text{people } 0.01$

$S \rightarrow NP VP$
 $0.9 * 0.35 * 0.06 = 0.0189$
 $S \rightarrow V NP$
 $0.05 * 0.1 * 0.14 = 0.0007$
 $VP \rightarrow V NP$
 $0.5 * 0.1 * 0.14$
 $NP \rightarrow NP NP$
 $0.2 * 0.35 * 0.14$

$S \rightarrow \text{fish } 0.006$
 $N \rightarrow \text{fish } 0.2$
 $V \rightarrow \text{fish } 0.6$
 $NP \rightarrow \text{fish } 0.14$
 $VP \rightarrow \text{fish } 0.06$

$S \rightarrow V NP$
 $0.05 * 0.6 * 0.14 = 0.0042$
 $VP \rightarrow V NP$
 $0.5 * 0.6 * 0.14$
 $NP \rightarrow NP NP$
 $0.2 * 0.14 * 0.14$

$N \rightarrow \text{tanks } 0.2$
 $NP \rightarrow \text{tanks } 0.14$

$S \rightarrow NP VP$ 0.9
 $S \rightarrow V NP$ 0.05
 $S \rightarrow \text{people}$ 0.001
 $S \rightarrow \text{fish}$ 0.006
 $VP \rightarrow V NP$ 0.5
 $VP \rightarrow \text{people}$ 0.01
 $VP \rightarrow \text{fish}$ 0.06
 $NP \rightarrow NP NP$ 0.2
 $NP \rightarrow \text{people}$ 0.35
 $NP \rightarrow \text{fish}$ 0.14
 $NP \rightarrow \text{tanks}$ 0.14
 $N \rightarrow \text{people}$ 0.5
 $N \rightarrow \text{fish}$ 0.2
 $N \rightarrow \text{tanks}$ 0.2
 $V \rightarrow \text{people}$ 0.1
 $V \rightarrow \text{fish}$ 0.6

Exemplul 2

Fish

people

fish

tanks

$S \rightarrow \text{fish}$ 0.006 $N \rightarrow \text{fish}$ 0.2 $V \rightarrow \text{fish}$ 0.6 $NP \rightarrow \text{fish}$ 0.14 $VP \rightarrow \text{fish}$ 0.06	$S \rightarrow NP VP$ $0.9 * 0.14 * 0.01 = 0.00126$ $S \rightarrow V NP$ $0.05 * 0.6 * 0.35 = 0.0105$ $VP \rightarrow V NP$ 0.5 * 0.6 * 0.35 $NP \rightarrow NP NP$ 0.2 * 0.14 * 0.35	$S \rightarrow NP VP$ 0.9 * 0.14 * 0.007 $S \rightarrow V NP$ 0.05 * 0.6 * 0.0049	
	$S \rightarrow \text{people}$ 0.001 $N \rightarrow \text{people}$ 0.5 $V \rightarrow \text{people}$ 0.1 $NP \rightarrow \text{people}$ 0.35 $VP \rightarrow \text{people}$ 0.01	$S \rightarrow NP VP$ $0.9 * 0.35 * 0.06 = 0.0189$ $S \rightarrow V NP$ $0.05 * 0.1 * 0.14 = 0.0007$ $VP \rightarrow V NP$ 0.5 * 0.1 * 0.14 = 0.007 $NP \rightarrow NP NP$ 0.2 * 0.35 * 0.14 = 0.0049	
		$S \rightarrow \text{fish}$ 0.006 $N \rightarrow \text{fish}$ 0.2 $V \rightarrow \text{fish}$ 0.6 $NP \rightarrow \text{fish}$ 0.14 $VP \rightarrow \text{fish}$ 0.06	$S \rightarrow V NP$ $0.05 * 0.6 * 0.14 = 0.0042$ $VP \rightarrow V NP$ $0.5 * 0.6 * 0.14$ $NP \rightarrow NP NP$ $0.2 * 0.14 * 0.14$
			$N \rightarrow \text{tanks}$ 0.2 $NP \rightarrow \text{tanks}$ 0.14

$S \rightarrow NP VP$ 0.9
 $S \rightarrow V NP$ 0.05
 $S \rightarrow \text{people}$ 0.001
 $S \rightarrow \text{fish}$ 0.006
 $VP \rightarrow V NP$ 0.5
 $VP \rightarrow \text{people}$ 0.01
 $VP \rightarrow \text{fish}$ 0.06
 $NP \rightarrow NP NP$ 0.2
 $NP \rightarrow \text{people}$ 0.35
 $NP \rightarrow \text{fish}$ 0.14
 $NP \rightarrow \text{tanks}$ 0.14
 $N \rightarrow \text{people}$ 0.5
 $N \rightarrow \text{fish}$ 0.2
 $N \rightarrow \text{tanks}$ 0.2
 $V \rightarrow \text{people}$ 0.1
 $V \rightarrow \text{fish}$ 0.6

Exemplul 2

Fish

people

fish

tanks

$S \rightarrow \text{fish } 0.006$ $N \rightarrow \text{fish } 0.2$ $V \rightarrow \text{fish } 0.6$ $NP \rightarrow \text{fish } 0.14$ $VP \rightarrow \text{fish } 0.06$	$S \rightarrow NP VP$ $0.9 * 0.14 * 0.01 = 0.00126$ $S \rightarrow V NP$ $0.05 * 0.6 * 0.35 = 0.0105$ $VP \rightarrow V NP$ $0.5 * 0.6 * 0.35$ $NP \rightarrow NP NP$ $0.2 * 0.14 * 0.35$		
	$S \rightarrow \text{people } 0.001$ $N \rightarrow \text{people } 0.5$ $V \rightarrow \text{people } 0.1$ $NP \rightarrow \text{people } 0.35$ $VP \rightarrow \text{people } 0.01$	$S \rightarrow NP VP$ $0.9 * 0.35 * 0.06 = 0.0189$ $S \rightarrow V NP$ $0.05 * 0.1 * 0.14 = 0.0007$ $VP \rightarrow V NP$ $0.5 * 0.1 * 0.14$ $NP \rightarrow NP NP$ $0.2 * 0.35 * 0.14$	
		$S \rightarrow \text{fish } 0.006$ $N \rightarrow \text{fish } 0.2$ $V \rightarrow \text{fish } 0.6$ $NP \rightarrow \text{fish } 0.14$ $VP \rightarrow \text{fish } 0.06$	$S \rightarrow V NP$ $0.05 * 0.6 * 0.14 = 0.0042$ $VP \rightarrow V NP$ $0.5 * 0.6 * 0.14$ $NP \rightarrow NP NP$ $0.2 * 0.14 * 0.14$
			$N \rightarrow \text{tanks } 0.2$ $NP \rightarrow \text{tanks } 0.14$

$S \rightarrow NP VP$ 0.9
 $S \rightarrow V NP$ 0.05
 $S \rightarrow \text{people}$ 0.001
 $S \rightarrow \text{fish}$ 0.006
 $VP \rightarrow V NP$ 0.5
 $VP \rightarrow \text{people}$ 0.01
 $VP \rightarrow \text{fish}$ 0.06
 $NP \rightarrow NP NP$ 0.2
 $NP \rightarrow \text{people}$ 0.35
 $NP \rightarrow \text{fish}$ 0.14
 $NP \rightarrow \text{tanks}$ 0.14
 $N \rightarrow \text{people}$ 0.5
 $N \rightarrow \text{fish}$ 0.2
 $N \rightarrow \text{tanks}$ 0.2
 $V \rightarrow \text{people}$ 0.1
 $V \rightarrow \text{fish}$ 0.6

Exemplul 2

Chart incomplet

*De completat de catre studenti
pt examen*

De gasit cel mai probabil arbore de analiza

Analiza sintactica

- CYK pt GICP foloseste o **tabela de probabilitati P** de dimensiuni $M \times N \times N$ (M -nr neterm, N - nr cuv)
- **P(i,j,X)** - contine probabilitatea maxima ca subsirul de la **i** la **j** sa fie generat de neterminalul **X**
- Intoarce matricea P cu aceste probabilitati calculate
- Foloseste si o matrice **Back(i,j,X)** care indica pentru subsirul de la **i** la **j** generat de **X** $\rightarrow Y Z$ unde s-a impartit subsirul a.i. sa fie generat de **X**

algorithm CYK(sir, gramatica) **intoarce** **P**, o tabela de probabilitati,
si **T**, cel mai probabil arbore

$N \leftarrow \text{Lungime}(\text{sir})$ /* sir = $a_1 a_2 \dots a_N$ */

$M \leftarrow \text{nr de neterminale in gramatica}$

P $\leftarrow$ matrice de $N \times N \times M$, initial 0

Back $\leftarrow$ matrice $N+1 \times N+1 \times M$

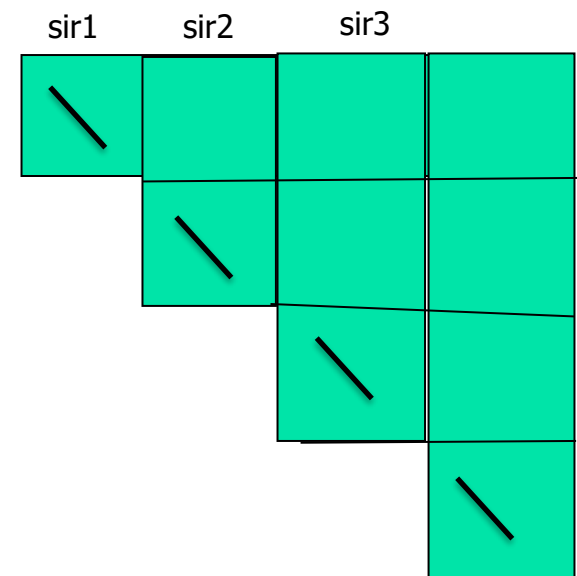
/ insereaza reguli lexicale pt fiecare cuvant */*

pentru $i=0 \dots N-1$ **repeta**

pentru fiecare regula de forma $(X \rightarrow \text{sir}_i, [p])$ **repeta**

$P[i, i+1, X] \leftarrow p$

(eventual si pointer catre regula)



algorithm CYK(sir, gramatica) **intoarce** **P**, o tabela de probabilitati, si **T**,
cel mai probabil arbore

/ combina primul si al doilea neterminal*

*din RHS a regulilor sintactice , incepand cu cele mai scurte */*

pentru span = 2 .. N **repeta** */* lungimea analizata */*

pentru begin = 0 .. N-span **repeta** */* inceput analiza */*

 end $\leftarrow$ begin + span

pentru split = begin+1 .. end-1 **repeta** */* partitie analizata */*

pentru fiecare regula de forma ($X \rightarrow Y Z$ [**p**]) **repeta**

daca **p** > P[begin, end, X] **atunci**

 P[begin, end, X] $\leftarrow$

 P[begin, split, Y] x P[split, end, Z] x **p**)

 (eventual si pointer catre regula)

 Back[begin,end, X] $\leftarrow$ (split, Y,Z)

intoarce P, construiesteArb(Back, P)

sfarsit

Observatii

- Algoritmul se poate modifica pentru a trata si productii unare ($A \rightarrow B$)
- Algoritmul se poate modifica si pentru a trata productii vide
- Existenta productiilor cu 2 terminale este esentiala
- Exista si alti algoritmi de tip “chart parser”:
 - Earley – top down
 - Chart parsing – un fel de agenda-based

Analiza sintactica partiala

- Multe taskuri NLP nu necesita arbori de analiza completi (si deci de complexitate mare)
- Pentru aceste taskuri este suficient sa se realizeze o analiza partiala (***partial parse*** or ***shallow parse***)
- *Sistemele de extragere a informatiei* nu extrag toata informatia posibila din text, ci identifica si analizeaza acele segmente de text care contin informatii de interes
- *Sistemele de regasire a informatiei* pot indexa documentele pe baza unei submultimi de constitienti din text

5 Definite Clause Grammar (DCG)

- Utilizarea Logicii Predicatelor pentru reprezentarea gramaticilor
- Gramatici cu clauze definite (DCG)
- Fac parte din categoria gramaticilor semantice
- Fiecare regula din gramatica poate fi vazuta ca o regula din GIC

Definite Clause Grammar (DCG)

- Fiecare **categorie sintactica** se reprezinta printr-un **predicat cu un argument sir**
- NP(s) este adevarat daca s este NP
$$S \rightarrow NP \ VP \text{ devine}$$
$$NP(s1) \wedge VP(s2) \Rightarrow S(\text{append}(s1,s2))$$
- Parsing = inferenta logica
- Bottom-up parsing – forward chaining
- Top-down parsing – backward chaining
- Aceeasi gramatica poate fi utilizata atat pentru analiza cat si pentru generare

In BNF

$S \rightarrow NP VP$

In LP / DGC

$NP(s_1) \wedge VP(s_2) \Rightarrow S(\text{Append}(s_1, s_2))$

BNF

$\text{Noun} \rightarrow \text{ball} \mid \text{book}$

In LP / DGC

$(s = \text{“ball”} \vee s = \text{“book”}) \Rightarrow \text{Noun}(s)$

BNF, DCG, Prolog

BNF	FOPL/DCG	PROLOG
$S \rightarrow NP \ VP$ $NP \rightarrow \text{Noun}$ $\text{Noun} \rightarrow \text{stench}$ $\text{Noun} \rightarrow \text{wumpus}$ $VP \rightarrow \text{Verb}$ $\text{Verb} \rightarrow \text{smells}$ $\text{Verb} \rightarrow \text{kills}$	$NP(s1) \wedge VP(s2) \Rightarrow S(\text{append}(s1,s2))$ $\text{Noun}(s) \Rightarrow NP(s)$ $\text{Verb}(s) \Rightarrow VP(s)$ $(s = \text{"stench"} \vee s = \text{"wumpus"}) \Rightarrow$ $\text{Noun}(s)$ $(v = \text{"smells"} \vee v = \text{"kills"}) \Rightarrow$ $\text{Verb}(v)$	$\text{sentence}([S1, S2]) :-$ $\text{np}(S1), \text{vp}(S2).$ $\text{np}(S) :- \text{noun}(S).$ $\text{vp}(S) :- \text{verb}(S).$ $\text{noun}(\text{stench}).$ $\text{noun}(\text{wumpus}).$ $\text{verb}(\text{smells}).$ $\text{verb}(\text{kills}).$ $?- \text{sentence}([\text{wumpus}, \text{smells}]).$ $?- \text{sentence}([S1, S2]).$

Imbogatire DCG

- Imbogatesc neterminale cu argumente suplimentare
- Argumentele suplimentare permit:
 - Verifica corectitudinea gramaticala
 - Ataseaza semantica
 - Adauga expresii / functii care se testeaza

Argument pt semantica

DCG	FOPL	PROLOG
$S(\text{sem}) \rightarrow NP(\text{sem1}) VP(\text{sem2})$ $\{ \text{compose}(\text{sem1}, \text{sem2}, \text{sem}) \}$	$NP(s1, \text{sem1}) \wedge VP(s2, \text{sem2}) \Rightarrow$ $S(\text{append}(s1, s2)),$ $\text{compose}(\text{sem1}, \text{sem2}, \text{sem})$	slide urmator

semantica compozitionala

The dog has legs.

(caine *parti* picioare)

The ball is yellow.

(minge *proprietate* galbena)

The ball is red.

(minge *proprietate* rosie)

The dog bites.

(caine *actiune* musca)

sentence(S, Sem) :- np(S1, Sem1), vp(S2, Sem2), append(S1, S2, S),
Sem = [Sem1 | Sem2].

np([S1, S2], Sem) :- article(S1), noun(S2, Sem).

vp([S], Sem) :- verb(S, Sem1), Sem = [*actiune*, Sem1].

vp([S1, S2], Sem) :- verb(S1, _), adjective(S2, Sem1), Sem = [*proprietate*, Sem1].

vp([S1, S2], Sem) :- verb(S1, _), noun(S2, Sem1), Sem = [*parti*, Sem1].

noun(dog,caine).

noun(ball,minge).

noun(legs,picioare).

verb(bytes,musca).

verb(is,este).

verb(has,are).

adjective(yellow,galbena).

adjective(red,rosie).

S(sem) → NP(sem1) VP(sem2)
{compose(sem1, sem2, sem)}

VP(sem) → verb(sem1)
{compose(*actiune*, sem1, sem)}

VP(sem) → verb,adjective(sem1)
{compose(*proprietate*, sem1, sem)}

VP(sem) → verb,noun(sem1)
{compose(*parti*, sem1, sem)}

The dog has legs.	(caine <i>parti</i> picioare)
The ball is yellow.	(minge <i>proprietate</i> galbena)
The ball is red.	(mine <i>proprietate</i> rosie)
The dog bites.	(caine <i>actiune</i> musca)

```
sentence(S, Sem) :- np(S1, Sem1), vp(S2, Sem2), append(S1, S2, S),
                    Sem = [Sem1 | Sem2].
```

```
np([S1, S2], Sem) :- article(S1), noun(S2, Sem).
```

```
vp([S], Sem) :- verb(S, Sem1), Sem = [actiune, Sem1].
```

```
vp([S1, S2], Sem) :- verb(S1, _), adjective(S2, Sem1), Sem = [proprietate, Sem1].
```

```
vp([S1, S2], Sem) :- verb(S1, _), noun(S2, Sem1), Sem = [parti, Sem1].
```

```
noun(dog,caine).
```

```
noun(ball,ball).
```

```
noun(legs,picioare).
```

```
verb(bytes,musca).
```

```
verb(is,este).
```

```
verb(has,are).
```

```
adjective(yellow,galbena).
```

```
adjective(red,rosie).
```

```
?- sentence([the,ball,is,yellow],Sem).
```

```
Sem = [minge, proprietate, galbena]
```

```
Yes
```

```
?- sentence([the,dog,bytes],Sem).
```

```
Sem = [caine, actiune, musca]
```

```
Yes
```

```
?- sentence([is,dog,bytes],Sem).
```

```
No
```

```
?- sentence([the,dog,has,legs],Sem).
```

```
Sem = [caine, parti, picioare]
```

```
Yes
```

Verificare corectitudine gramaticală

- Cazuri
 - Subcategorii verbe: complementul pe care îl poate accepta un verb
 - Acord subiect predicat
 - etc.
-
- Parametrizarea neterminalilor

Cazuri

Nominativ (subjective)

I take the bus

You take the bus

He takes the bus

Eu iau autobuzul

Tu iei autobuzul

El ia autobuzul

Acuzativ (objective)

He gives me the book

Imi da cartea

$S \rightarrow NP(\text{Subjective}) VP$

$NP(\text{case}) \rightarrow \text{Pronoun (case)} \mid \text{Noun} \mid \text{Article Noun}$

//

I

$VP \rightarrow VP NP(\text{Objective})$

//

believe him

$VP \rightarrow VP PP$

//

turn to the right

$VP \rightarrow VP \text{ Adjective}$

$VP \rightarrow \text{Verb}$

$PP \rightarrow \text{Preposition } NP(\text{Objective})$

$\text{Pronoun}(\text{Subjective}) \rightarrow I \mid \text{you} \mid \text{he} \mid \text{she}$

$\text{Pronoun}(\text{Objective}) \rightarrow \text{me} \mid \text{you} \mid \text{him} \mid \text{her}$

sentence(S) :- np(S1,subjective), vp(S2), append(S1, S2, S).

np([S], Case) :- pronoun(S, Case).

np([S], _) :- noun(S).

np([S1, S2], _) :- article(S1), noun(S2).

pronoun(i, subjective).

pronoun(you, _).

pronoun(he, subjective).

pronoun(she, subjective).

pronoun(me, objective).

pronoun(him, objective).

pronoun(her, objective).

noun(ball).

noun(stick).

article(a).

article(the).

$S \rightarrow NP(\text{Subjective}) VP$

$NP(\text{case}) \rightarrow \text{Pronoun}(\text{case}) \mid \text{Noun} \mid \text{Article Noun}$

$VP \rightarrow VP \ NP(\text{Objective})$

$VP \rightarrow VP \ PP$

$VP \rightarrow VP \ \text{Adjective}$

$VP \rightarrow \text{Verb}$

$PP \rightarrow \text{Preposition } NP(\text{Objective})$

$\text{Pronoun}(\text{Subjective}) \rightarrow I \mid \text{you} \mid \text{he} \mid \text{she}$

$\text{Pronoun}(\text{Objective}) \rightarrow \text{me} \mid \text{you} \mid \text{him} \mid \text{her}$

Subcategorii verbe

- Lista de subcategorii: ce complemente accepta verbul; depinde de verb

$S \rightarrow NP(\text{Subjective}) VP(\text{subcat})$

$VP(\text{subcat}) \rightarrow$ {subcat = np} VP(np) NP(Objective)
 | {subcat = adj} VP(adj) Adjective
 | {subcat = pp} VP (pp) PP
 | Verb

Subcategorii verbe

VP(subcat) → {subcat = np} VP(np) NP(Objective)
| {subcat = adj} VP(adj) Adjective
| {subcat = pp} VP (pp) PP
| Verb

smell	[NP]	smell a wumpus
	[Adjective]	smell awfull
	[PP]	smell like a wumpus

is	[Adjective]	is smelly
	[PP]	is in box
	[NP]	is a pit

give	[NP, PP]	give the gold in box to me
	[NP, NP]	give me the gold

died	[]	died
------	----	------

Combinat cazuri cu subcategorii verbe

$S \rightarrow \text{NP(Subjective)} \text{ VP(subcat)}$

$\text{NP(case)} \rightarrow \text{Pronoun (case)} \mid \text{Noun} \mid \text{Article Noun}$

$\text{Pronoun(Subjective)} \rightarrow \text{I} \mid \text{you} \mid \text{he} \mid \text{she}$

$\text{Pronoun(Objective)} \rightarrow \text{me} \mid \text{you} \mid \text{him} \mid \text{her}$

$\text{VP(subcat)} \rightarrow$ $\{\text{subcat} = \text{np}\} \text{ VP(np)} \text{ NP(Objective)}$
 $\mid \{\text{subcat} = \text{adj}\} \text{ VP(adj)} \text{ Adjective}$
 $\mid \{\text{subcat} = \text{pp}\} \text{ VP (pp)} \text{ PP}$
 $\mid \text{Verb}$
 $\mid \text{VP(subcat)} \text{ PP}$
 $\mid \text{VP(subcat)} \text{ Adverb}$

$S \rightarrow \text{NP(Subjective)} \text{ VP(subcat)}$
 $\text{VP(subcat)} \rightarrow \{\text{subcat} = \text{np}\} \text{ VP(np)} \text{ NP(Objective)}$
 | {subcat = adj} VP(adj) Adjective
 | {subcat = pp} VP (pp) PP
 | Verb
 | **VP(subcat) PP**
 | VP(subcat) Adverb

Reprezentare in Prolog

sentence(S) :- np(S1,subjective), vp(S2,Subcat), append(S1, S2, S).

... pt NP

vp([S],np) :- vp(S1,np), np(S2,objective), append(S1, S2, S).

vp([S],adj) :- vp(S1,adj), adjective(S2),append(S1, S2, S).

vp([S],pp) :- vp(S1,pp), pp(S2),append(S1, S2, S).

vp([S],_) :- verb(S).

vp([S],Subcat) :- vp(S1,Subcat), pp(S2),append(S1, S2, S).

vp([S],Subcat) :- vp(S1,Subcat), adverb(S2),append(S1, S2, S).

Probleme datorita recursivitatii

Trebuie eliminata recursivitatea pentru a evita cicluri infinite

Solutie pentru eliminare recursivitate

`vp([S],np) :- vp(S1,np), np(S2,objective), append(S1, S2, S).`

`vp([S],Subcat) :- vp(S1,Subcat), pp(S2),append(S1, S2, S).`

Se transforma in

`vp([S], Subcat) :- Subcat = np, vp1(S1, np), np(S2, objective),
append(S1, S2, S).`

`vp([S],Subcat) :- vp1(S1, Subcat), pp(S2), append(S1, S2, S).`

`vp1([S],np) :- verb(S).`

`verb(give).`

`verb(make).`

Gramatici semantice

- DCG au inspirat gramaticile semantice
- Gramaticile lingvistice (de ex GICP) utilizeaza neterminale sintactice (noun, verb, etc.)
- Gramaticile semantice au pe post de neterminale categorii semantice, specifice unui domeniu

Top 10 artists in US for the last 3 weeks

limit

entity

geo

date

6. Analiza semantica

- Elmentele de baza
 - Entitati / instante
 - Concepte / clase
 - Relatii – intre entitati si concepte
 - Predicate – verbe, roluri semantice

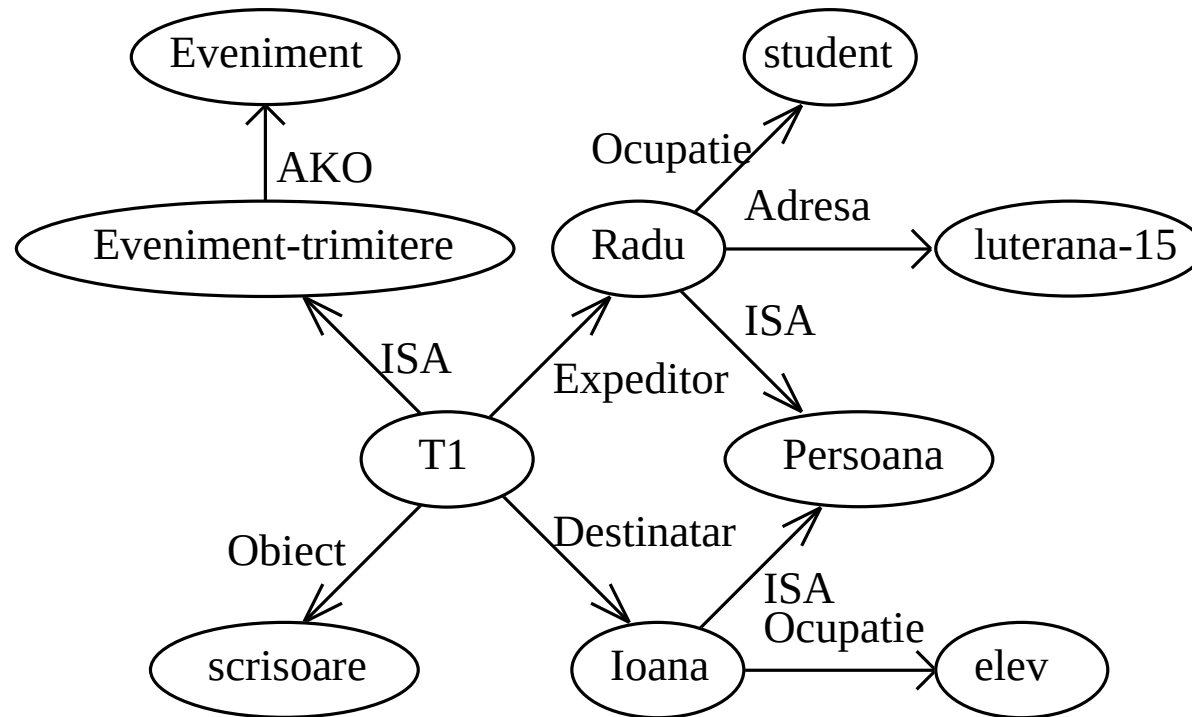
Analiza semantica - abordari

- Retele semantice
- Dependente conceptuale
- Gramatici de caz (Case Grammars)
- Grafuri conceptuale

Retele semantice

- Radu i-a trimis Ioanei o scrisoare.
- Radu este student.
- Ioana este eleva.
- Adresa lui Radu este Luterana, 15.

- ❑ Ocupatie (radu, student)
- ❑ Ocupatie (ioana, eleva)
- ❑ Trimite (radu, ioana, scrisoare)
- ❑ Adresa (radu, luterana - 15)



(b)

Retele semantice

■ Mostenirea proprietatilor (atributelor)

1) Mostenirii proprietatilor de la clasa la instanta:

Daca un obiect O_1 este o particularizare (legat prin relatia ISA) a unui obiect generic/ clasa O si clasa O are un atribut (proprietate) A , atunci si instanta O_1 are atributul A .

2) Aplicarea mostenirii proprietatilor intre o clasa si o superclasa, de-a lungul unei relatii sau a unui lant de relatii AKO

Daca o clasa C_1 este o subclasa a unei clase C (legata prin una sau mai multe relatii AKO) si clasa C are proprietatea A , atunci clasa C_1 are de asemenea proprietatea (atributul) A .

7 Analiza pragmatica

- Interpretare pragmatica – utilizare si efect asupra ascultatorului
- Indexical – refera situatia curenta, contextul (eu, acum, aici, atunci)
- Speech acts (comezi, asertiuni)
- Ambiguitati

Dezambiguizare

- Semantica lexicala
- **Elemente de ambiguitate lexicala**
 - Hiponimie – relatie între un termen generic și instanțe ale termenului generic (culoare / roșu galben)
 - Omonimie – aceeași ortografie dar sensuri diferite (broască)
 - Polisemie – cuvinte cu sensuri diferite dar corelate (banca)
 - Sinonimie – forme lexicale diferite dar același sens sau sensuri apropiate (vulnerabil, firav, slab, nerezistent)
 - Antinomie – simetrie a înțeleșului în fct de o axă, de ex proprietate, utilizare etc (agreabil / dezagreabil, adevărat/fals)

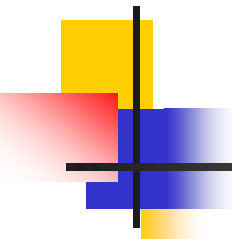
Dezambiguizare

- Sintactica – arbori diferiti de analiza
- Referentiala – referire la obiecte anterioare
(Mihai i-a spus lui Tudor ca a luat examenul)
- Pragmatica – referire la loc, timp
- Ambiguitati intre semnificatia uzuala si figurativa
 - Metonimie (a numi un obiect prin intermediul unui termen desemnând un alt obiect, legat cu primul printr-o relatie logica)
 - Metafora (figura de stil prin care se trece de la sensul obișnuit al unui cuvânt la alt sens, prin intermediul unei comparații subînțelese)

Inteligență Artificială

Universitatea Politehnica Bucuresti
Anul universitar 2022-2023

Adina Magda Florea

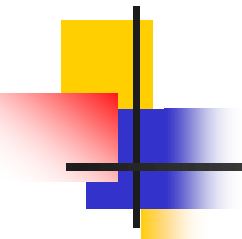


Curs nr. 13

Prelucrarea limbajului natural

**a) Prelucrare LN pentru comunicare
(curs 12)**

b) Prelucrare LN pt achizitia cunostintelor



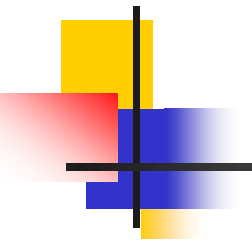
Modele ale limbajului

Modele de limbaj bazate pe structura
gramaticala

Modele n-Gram

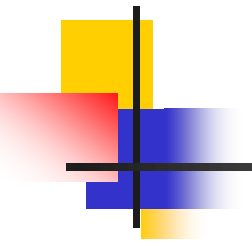
POS tagging

Word embedding



1 Modele N-gram

- **Model N-gram de caractere** – **distributie de probabilitate** peste secvente de caractere
- $P(c_{1:N})$ – probabilitatea unei secvente de N caractere, c_1 la c_N
$$P(\text{"the"}) = 0.27 \quad P(\text{"zgq"}) = 0.00000002$$
- O secventa de simboluri de lungime n – **n-gram**
 - unigram, bigram, trigram
- Un model N-gram este definit ca un lant Markov de ordin N-1 (probabilitatea unui caracter depinde de caracterele precedente)



Modele N-gram de caractere

■ Trigram

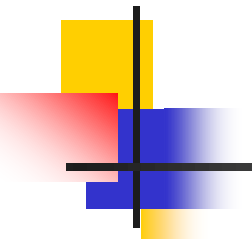
$$P(c_i|c_{1:i-1}) = P(c_i|c_{i-2:i-1})$$

$$P(c_{1:N}) = \prod_{i=1,N} P(c_i|c_{1:i-1}) = \prod_{i=1,N} P(c_i|c_{i-2:i-1})$$

Un model trigram a unui limbaj de 100 caractere

$$P(c_i|c_{i-2:i-1})$$

are 1 mil intrari



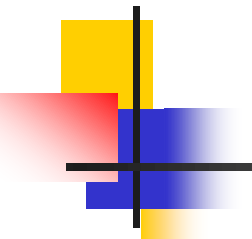
Modele N-gram de caractere

Ce putem face cu un astfel de model?

- **Identificarea limbajului** = fiind dat un text se determina in ce limba este scris (99%)
- Construiește un model trigram caracter pentru fiecare limbaj candidat l
- $P(c_i|c_{i-2:i-1},l)$; este nevoie de aprox 100 000 caractere pt fiecare limbaj

$$l^* = \underset{l}{\operatorname{argmax}} P(l|c_{1:N}) = \underset{l}{\operatorname{argmax}} P(l) * P(c_{1:N}|l) = \\ \underset{l}{\operatorname{argmax}} P(l) * \prod_{i=1,N} P(c_i|c_{i-2:i-1},l)$$

- **Alte aplicatii:**
 - verificare ortografie, clasificare texte in fct de tipuri, identificarea numelor proprii,...



Modele N-gram de caractere

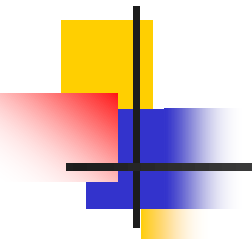
Omogenizarea modelelor

- *Problema*: pt secvente de caractere comune, cam orice corpus va da o estimare buna ($P(\text{" th"})=0.15$)
- Dar $P(\text{" ht"})=0$ http?
- *Solutie*
- Calculam N-gram si pentru secvente cu $P=0$ sau f mica
- Calculam N-1-gram + interpolare

$$P(c_i|c_{i-2:i-1},l) = \alpha_1 P(c_i|c_{i-2:i-1},l) + \alpha_2 P(c_i|c_{i-1},l) + \alpha_3 P(c_i)$$

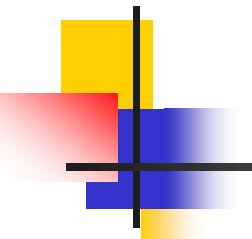
$$\text{cu } \alpha_1 + \alpha_2 + \alpha_3 = 1$$

Evaluarea modelelor – prin validare incrucisata



Modele N-gram de cuvinte

- In acest caz vocabularul este mult mai mare
- Daca max 100 car in cele mai multe limbaje, sute de mii, milioane de cuvinte
- Cuvinte noi
- Putem adauga un cuvant <NEC> in vocabular (sau mai multe)
- Bigramele si trigramele sunt mai bune decat unigramele



Modele N-gram de cuvinte - ex

Fie acest mic corpus

<s> I am Sam </s>

<s> Sam and I </s>

<s> I do not like green eggs and ham </s>

Calculul probabilitatilor unor bigrame pt acesta

$$P(I|<s>) = 2/3 = 0.67$$

$$P(\text{Sam}|<s>) = 1/3 = 0.33$$

$$P(\text{am}|I) = 1/3 = 0.33$$

$$P(</s>|\text{Sam}) = 1/2 = 0.5$$

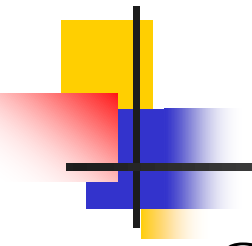
$$P(\text{and}|\text{Sam}) = 1/2 = 0.5$$

$$P(\text{do}|I) = 1/3 = 0.33$$

Daca avem probabilitatile tuturor bigramelor putem calcula

$$P(<s> \text{ Sam and I } </s>) =$$

$$P(<\text{Sam}|<s>)*P(\text{and}|\text{Sam})*P(I|\text{and})*P(</s>|I)$$



2 Clasificarea textelor

- Clasificare sau apartenenta la o clasa
- Identificare limbaj, clasificare tip text, analiza starii induse, detectie spam

Detectie spam

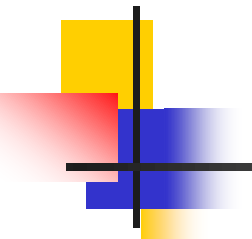
Not-spam (Ham)

Spam

$m_1, m_2, \dots$

$n_1, n_2, \dots$

- "for cheap" "you can buy" – n-gram de cuvinte
- "yo,u d-eserve" – n-gram de caractere



Detectie spam

- Calculez $P(\text{Mesaj}|\text{Spam})$ $P(\text{Mesaj}|\text{Ham})$
- Clasific mesaj nou

$$\underset{C \in \{\text{Spam}, \text{Ham}\}}{\operatorname{argmax}} P(C|\text{Mesaj}) = \underset{C \in \{\text{Spam}, \text{Ham}\}}{\operatorname{argmax}} P(\text{Mesaj}|C) * P(C)$$

unde $P(C)$ este estimat prin numararea nr de mesaje din Spam si Ham

- Se reprezinta mesajul ca o multime de caracteristici $(\text{car}_i, \text{val}_i)$ si se poate aplica un algoritm de clasificare (invatare)
- Caracteristici – cuvinte din vocabular
- Valori – nr de aparitii in mesaj
- Alg posibili: K-nearest-neigh, SVM, AD, Bayes naiv



3 Regasirea informatiei

- Scop: Gasirea documentelor (de obicei text nestructurat) care sunt relevante pentru o cerere utilizator, dintr-o colectie de documnte

Un sistem de IR (Information Retrieval) poate fi caracterizat de:

- **Un corpus de documente** – paragrafe, pagini, texte pe mai multe pagini
- **Interogarea (query) in limbajul de interogare** – lista cuvinte, cuvinte adiacente, op logici, op nelogici (near)
- **Multimea rezultat** – multimea de documente relevante pentru query
- **Prezentarea rezultatelor** – lista ordonata, grafic, etc.

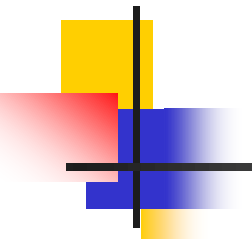


Evaluarea rezultatului

- **Precizie (precision)** – procentul de documente relevante din multimea obtinuta
- **Relevanta (recall)** – procentul de documente relevante din colectie care sunt in setul rezultat
- 100 documente cu 1 query pt care obtinem o multime de 40

	In multime	Nu in multime
Relevant	30	20
Nerelevant	10	40

- **Precision** = $30/(30+10) = 0.75$
- **Recall** = $30/(30+20) = 0.6$
- Recall este mai greu de calculat
- Se pot combina $2PR/(P+R)$



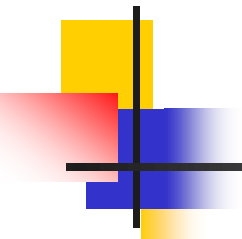
Regasirea Booleana

Primele sisteme IR – **Boolean keyword model**

- Fiecare cuvant din text tratat ca un flag
- Limbajul de interogare – expresii logice peste cuvinte
- Ce piese ale lui Shakespeare contin cuvintele ***Brutus AND Caesar*** but ***NOT Calpurnia***?

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

1 daca piesa continue cuvantul, 0 altfel



Regasirea Booleana

Vectori booleni (0/1)

Pentru a raspunde la query: se iau vectorii pt Brutus, Caesar si Calpurina

- 110100 *AND*
- 110111 *AND*
- 101111 =
- **100100**

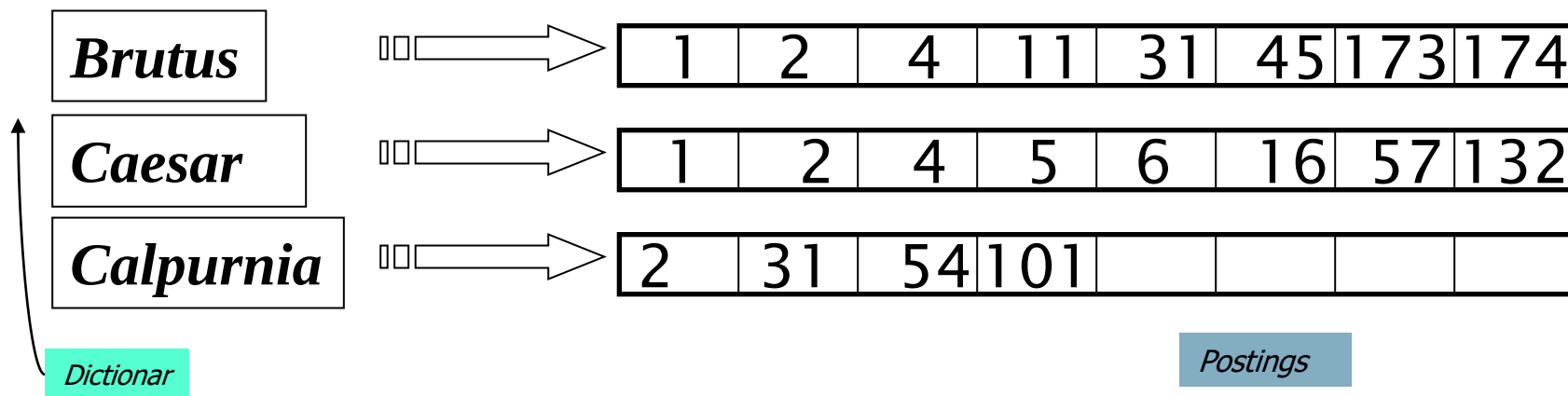
	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

Regasirea Booleana

Pt 1 milion de documente cu aprox 1000 cuvinte fiecare, rezulta o matrice f mare

O reprezentare mai buna: index inversat

Pt fiecare termen se memoreaza lista documentelor care contin t



Constructia indexului inversat

Documente de
indexat



Friends, Romans, countrymen.

⋮

Tokenizer

Tokens

Friends

Romans

Countrymen

Linguistic modules

Tokens
modificate

friend

roman

countryman

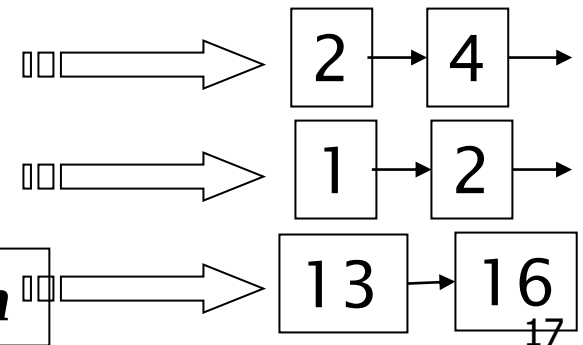
Indexer

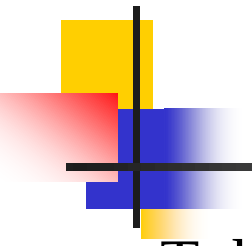
friend

roman

countryman

Index inversat





Etape initiale in prelucrarea textului

- Tokenization
 - Determina token (analiza lexicala)
 - *John's*
- Normalizare
 - Mapeaza textul si termenul din query la aceeasi forma
 - *U.S.A.* si *USA*
- Stemming
 - Forme diferite pt o aceeasi radacina sa se potriveasca
 - *authorize, authorization*
- Eliminarea cuvintelor comune (stop words)
 - *the, a, to, of*

Secventa tokens

Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

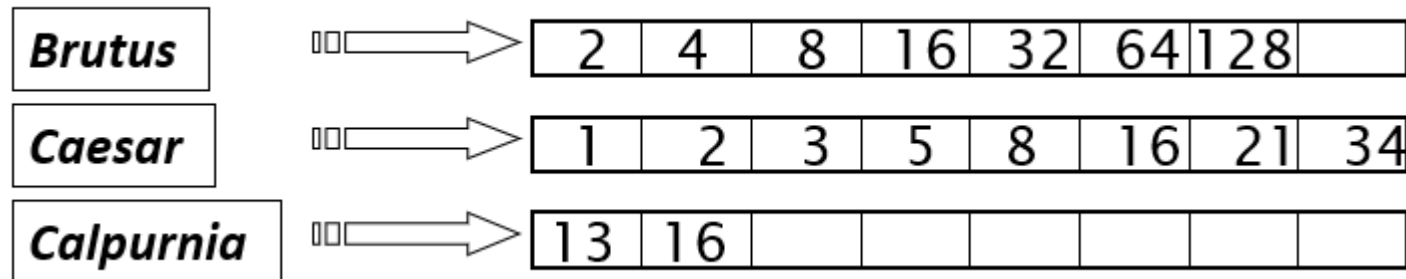
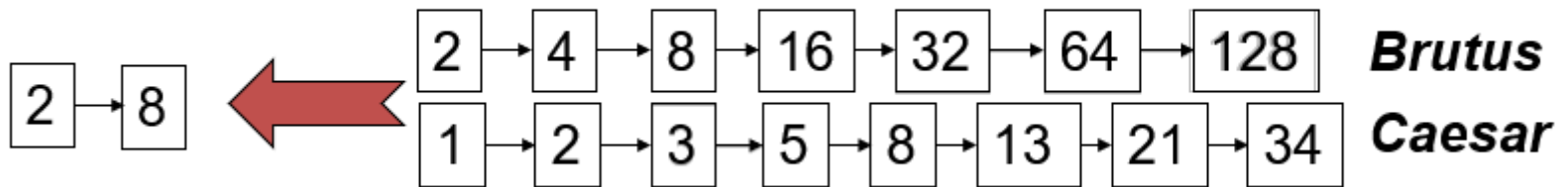
Secventa tokens sortata

Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

Dictionar si postings

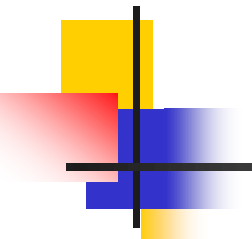
term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
i	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2

Prelucrare integrogari



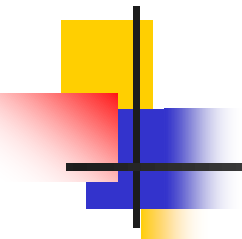
Brutus AND Caesar but NOT Calpurnia

(Calpurnia AND Brutus) AND Caesar



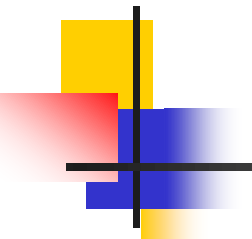
Regasirea pe baza de rang

- In regasirea Booleana documentele fie se potrivesc fie nu
- Utilizatorul trebuie sa introduca un query Boolean (uneori dificil)
- Numarul de documente gasite f mare sau 0
- **Regasirea bazata pe rang (ranked retrieval systems)** – se produce un set ordonat de documente relevante pentru query
- Nu mai folosesc un limbaj de interogare ci interogare in limbaj natural
- Ideea de baza – **frecventa termenilor (Term Frequency)** = numarul de aparitii ale unui termen in document
- Modele statistice bazate pe contoare de cuvinte



Regasirea pe baza de rang

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	157	73	0	0	0	0
Brutus	4	157	0	1	0	0
Caesar	232	227	0	2	1	1
Calpurnia	0	10	0	0	0	0
Cleopatra	57	0	0	0	0	0
mercy	2	0	3	5	5	1
worser	2	0	1	1	1	0

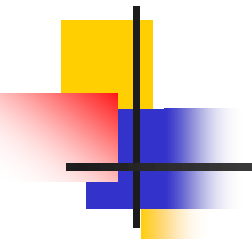


Term frequency (tf)

- **Frecventa termenilor (Term Frequency)**
- $f_{t,d}$ = numarul de aparitii ale termenului t in documentul d
- Relevanta unui document nu creste proportional cu frecventa termenilor (un document cu 10 aparitii a lui t este mai relevant decat un document cu 1 aparitie, dar nu de 10 ori mai relevant)
- Se defineste varianta logaritmica a **term frequency**

$$tf_{t,d} = 1 + \log_{10} f_{t,d} \quad \text{daca } f_{t,d} > 0$$

0 in caz contrar



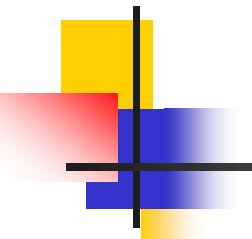
Term frequency (tf)

- **Frecventa augmentata** – impartita la frecventa cuvintului care apare cel mai des in document

$$\mathbf{tf}_{t,d} = 0.5 + 0.5 * f_{t,d} / \mathbf{max}(f_{t',d} : t' \in d)$$

- Scorul pt termenul **t** pt query **q** si document **d**

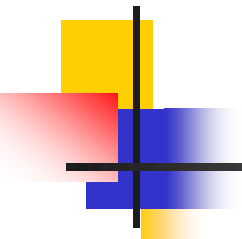
$$\mathbf{Scor} = \sum_{t \in q \cap d} \mathbf{tf}_{t,d}$$



Document frequency si idf

- Aparitia in document a unor cuvinte rare este mai relevanta decat cea a cuvintelor comune
- **Frecventa in documente** = numarul de documente in care apare t
- **d_{ft} – document frequency pt t**
- Este o masura inversa a cat de informativ este documentul si $d_{ft} \leq N$, cu $N=|D|$, nr total de documente din corpusul D
- **Inverse document frequency** pt t

$$idf_{t,D} = \log_{10} (N/df_t)$$



Inverse document frequency

$N = 1 \text{ milion}$

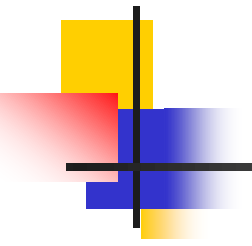
$$\text{idf}_{t,D} = \log_{10} (N/\text{df}_t)$$

term	df_t	idf_t
calpurnia	1	6
animal	100	4
sunday	1,000	3
fly	10,000	2
under	100,000	1
the	1,000,000	0

idf - *cata informatie ofera un cuvant*

scade ponderea cuvintelor care apar frecvent

in documente si creste ponderea celor care apar rar



tf si idf

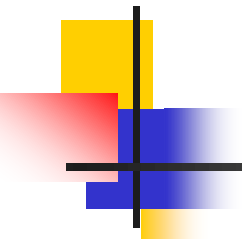
$$tf_{t,d} = 1 + \log_{10} t_{f,d}$$

$$idf_{t,D} = \log_{10} (N/df_t)$$

$$\mathbf{tfidf}_{t,d,D} = \mathbf{tf}_{t,d} * \mathbf{idf}_{t,D}$$

$\mathbf{tfidf}_{t,d,D}$

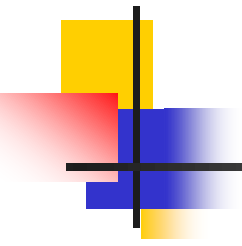
- Crește odată cu numărul de apariții în document
- Crește odată cu raritatea cuvintelor în colecție
- O valoare mare a $\mathbf{tfidf}_{t,d,D}$ se obține pentru o frecvență mare a cuvântului în document și o valoare mică a frecvenței cuvântului în colecția de documente (se încearcă eliminarea cuvintelor comune)



Matrice de ponderi

Antony	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
	5.25	3.18	0	0	0	0.35
Brutus	1.21	6.1	0	1	0	0
Caesar	8.59	2.54	0	1.51	0.25	0
Calpurnia	0	1.54	0	0	0	0
Cleopatra	2.85	0	0	0	0	0
mercy	1.51	0	1.9	0.12	5.25	0.88
worser	1.37	0	0.11	4.15	0.25	1.95

Fiecare document este reprezentat printr-un vector cu valori reale – ponderile **tfidf**



tf si idf

Ponderea (weight) a unui termen este

$$w_{t,d} = \log(1 + tf_{t,d}) \times \log_{10}(N / df_t)$$

Cea mai cunoscuta formula in IR

Creste cu nr de aparitii in document

Creste cu raritatea cuventelor in colectie

Exemplu sistemul Lucene

TF(q_i, d_j) pt **N documente** – nr de aparitii q_i in documentul d_j

DF(q_i) – **Document frequency counts** – nr de documente care contin cuvantul q_i

Fiind dat un document d_j si un query cu cuvintele $q_{1:N}$ avem

$$BM25(d_j, q_{1:N}) = \sum_{i=1}^N IDF(q_i) * \frac{TF(q_i, d_j) * (k+1)}{TF(q_i, d_j) + k(1-b + b \frac{|d_j|}{L})}$$

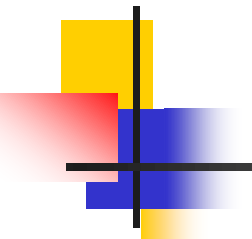
$|d_j|$ - nr cuvinte din documentul d_j

$L = \sum_i |d_i| / N$ – lungimea medie a documentelor din corpus

k, b – determinati prin validare incrucisata, valori tipice:

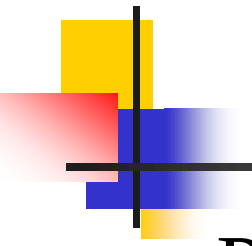
$$k=2.0, b=0.75$$

$$IDF(q_i) = \log \frac{N - DF(q_i) + 0.5}{DF(q_i) + 0.5}$$



Regasirea informatiei

- Dificil de aplicat tfidf fiecarui document din corpus
- **Hit list** = index creat anterior care refera pentru fiecare cuvant din vocabular documentele ce contin acel cuvant
- Pt un query, se face intersectia intre *hit list* si cuvintele din query si se face cautarea numai pe aceasta intersectie
- tfidf – model care trateaza cuvintele ca fiind independente
- **Imbunatatiri**
 - Corelatii
 - Cuvinte derivate, omonime

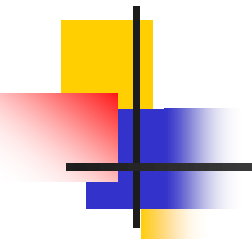


Regasirea informatiei

- Page Rank (Brin and Larry Page 1998 Google)
- In loc sa caute documente cu masuri IR cat mai mari, dau prioritate documentelor de calitate
- Paginile cu multe *in-links* au scor mare
- Page Rank – o distributie de probabilitate – persoana navigheaza pe Web sa ajunga la o anumita pagina

$$PR(p) = \sum_{in_i \in Bp} \frac{PR(in_i)}{C(in_i)}$$

- $PR(p)$ – rangul paginii p
- Bp – multimea paginilor care au link la pagina p
- in_i – paginile care au link la p (paginile din Bp)
- $C(in_i)$ – nr de *out-links* in pagina in_i
- Ex: A, B→A,C, C→A, D→A,B,C $PR(A) = PR(B)/2 + PR(C)/1 + PR(D)/3$

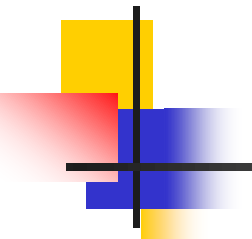


Regasirea informatiei

$$PR(p) = \frac{1-d}{N} + d \sum_{in_i \in Bp} \frac{PR(in_i)}{C(in_i)}$$

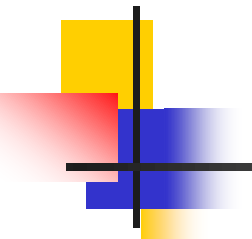
- $PR(p)$ – rangul paginii p
- Bp – multimea paginilor care au link la pagina p
- in_i – paginile care au link la p (paginile din Bp)
- $C(in_i)$ – nr de *out-links* in pagina in_i
- N – nr total de pagini in corpus
- d – damping factor – probabilitatea ca sa ramana pe aceeaasi pagina
- Calculat iterativ – se incepe cu paginile cu $PR(p,0)=1/N$ si itereaza actualizand rangurile pana la convergenta

$$PR(p, t+1) = \frac{1-d}{N} + d \sum_i \frac{PR(in_i, t)}{C(in_i)}$$



4 Extragerea informatiei

- **Sisteme IE scop:**
- Localizeaza si extrage parti relevante dintr-un text
- Achizitia cunostintelor prin analiza unui text cu focalizare pe aparitia unei clase particulare de obiecte si a relatiilor dintre aceste obiecte
- Produce o reprezentare structurata
 - Relatii (intr-o bd)
 - Baza de cunostinte, ontologii



Extragerea informatiei

- Exemple tipice
 - extragerea din pagini web a adreselor (cu campuri strada, nr, etc.)
 - meteo – temp, vat, precipitatii, etc.

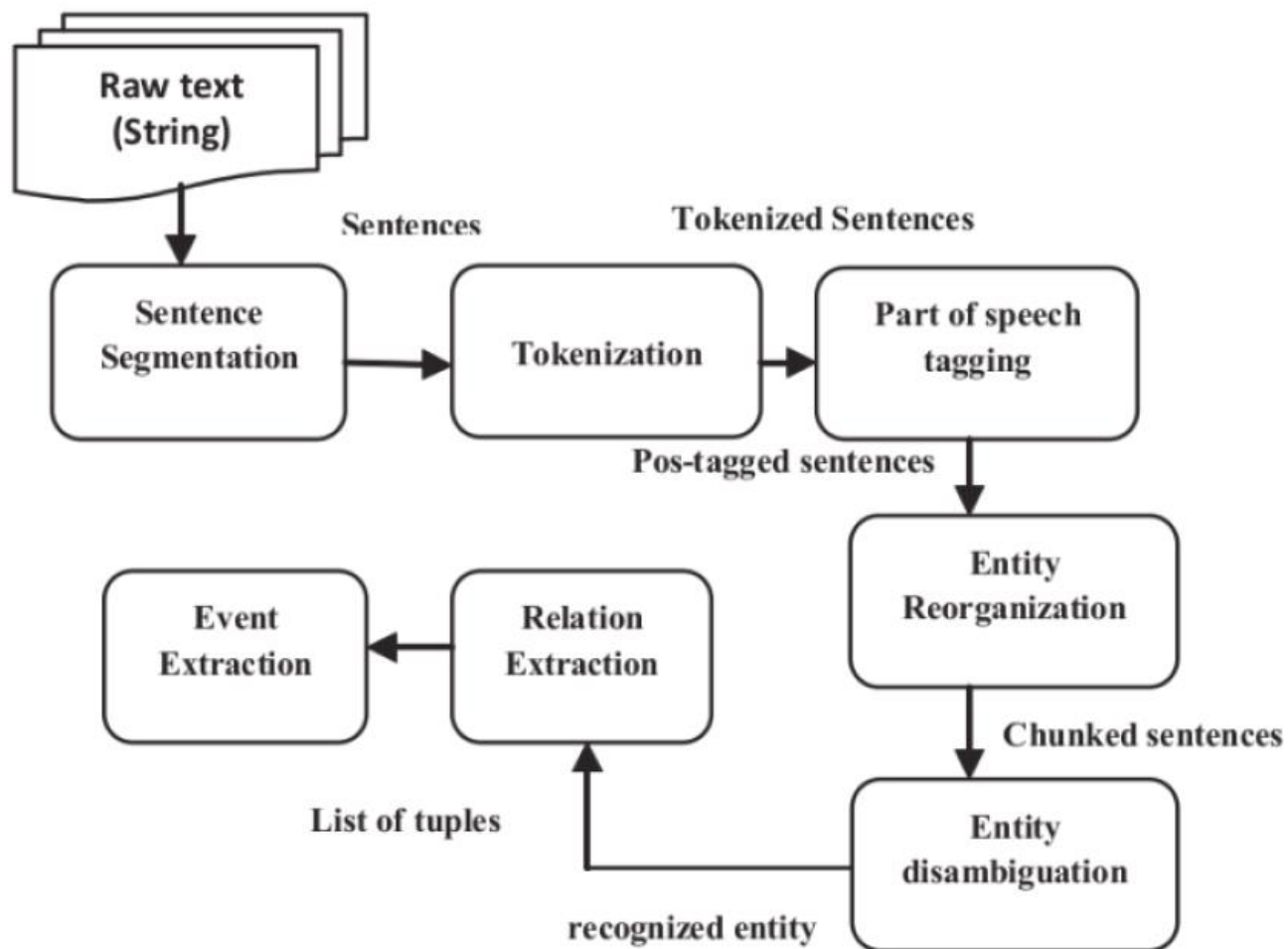
Capitala Iranului este orasul Teheran

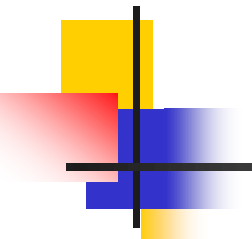
`capitala(Iran, teheran)`

- De ce este dificil?
- Ex: Pretul unui produs (un interval, sau was \$100 now \$120, offers in the high 700s..)

Extragerea informatiei

- Arhitectura tipica (constantino, 1997)





Named Entity Recognition (NER)

- Gasirea si clasificarea entitatilor nume, de ex nume persoane, organizatii, locatii (orase) sau expresii numerice de ex timp, data, bani, procent etc.
- **Viorel Salvador Caragea**, fost șef al **Inspectoratului Județean de Poliție Gorj**, a demisionat din **USR**, după ce cu numai o zi înainte filiala județeană a **USR** i-a acceptat adeziunea. (sursa Digi24)
- Mazare fina congelata 450gr **5,99** ron

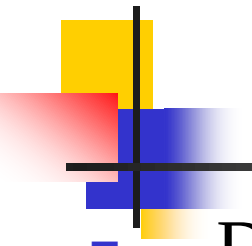


Evaluarea rezultatului

- **Precizie (precision)** – procentul de entitati selectate care sunt corecte
- **Relevanta (recall)** – procentul de entitati corecte care sunt selectate
- 100 documente din care selectez entitate nume companie XX

	Corecte	Incorecte
Selectate	30	20
Neselectate	10	40

- **Precision** = $30/(30+20) = 0.6$
- **Recall** = $30/(30+10) = 0.75$



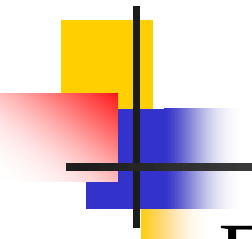
Evaluarea rezultatului

- De obicei o masura combinata (medie armonica)

$$F = \frac{1}{\alpha \frac{1}{P} + (1-\alpha) \frac{1}{R}} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

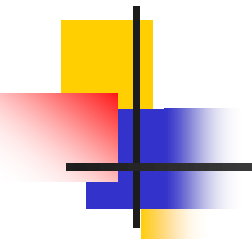
- Mai frecvent se utilizeaza cu $\beta=1$

$$F = 2PR/(P+R)$$



Abordari pentru NER

- Expresii regulate (automate finite)
- ML cu modele secventiale
 - Naive Bayes
 - Discriminative: modele Maxent
- Modele probabiliste
 - HMM
 - CRFs
- Utilizate si in combinatie cu POS tagging, analiza sintactica (NP, VP, PP) sau categorii semantice (din WordNet de ex)

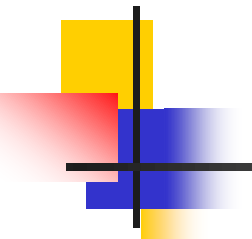


NER cu expresii regulate

■ Atomate finite

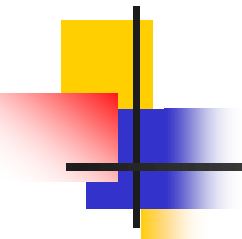
Templates

- Extrage informatii relevante unui obiect – valorile unor attribute predefinite
- Se defineste un template (pattern) pentru fiecare atribut de extras
- Template-ul este definit de un automat finit (expresii regulate)
- Template – prefix regex, **target regex**, postfix regex
- price prefix regex **[€][0-9]*[.][0-9][0-9]** postfix regex



NER cu expresii regulate

- Daca template-ul se potriveste 1 data – extrage target regex
- Daca nu se potriveste de loc – atribut lipsa
- Daca se potriveste de mai multe ori – prioritate, mai multe versiuni de template (prefix regex de ex)
- Intereseaza in special **Recall**



NER cu reguli de productie

- Ce persoana are ce pozitie intr-o organizatie

[person], [office] of [org]

Ian Flex, leader of Geen movement

[org] (named, appointed etc.) [person] Prep [office]

NATO appointed Mircea Geoana as Deputy Secretary General

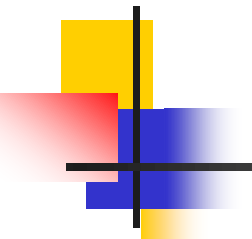
- Determinarea locatiei unei organizatii

[org] in [loc]

NATO headquarters in Brussels

[org] [loc] (division, branch, headquarters, etc.)

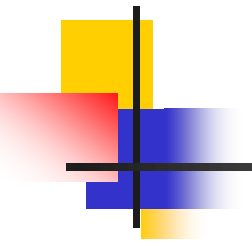
Google branch in Vancouver



ML cu modele secventiale

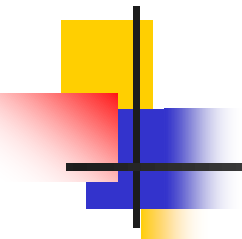
- Antrenare
 - Colecteaza set de documente reprezentative
 - Eticheteaza fiecare token cu clasa entitatii
 - Extrage attribute semnificative pentru clasa
 - Antreneaza modelul pt a prezice clase

- Testare
 - Colecteaza setul de documente de test
 - Ruleaza recunoasterea
 - Evalueaza rezultatele (R, P)



ML cu modele secventiale

- De ex "Named Entity Recognition and Classification with Scikit-Learn"
- Corpus adnotat cu IOB si POS tags
- IOB tags = Inside, Outside, Beginning
 - I- prefix - inaintea unui tag indica ca tagul este in interiorul unui chunk
 - B- prefix - inaintea unui tag indica ca un tag este la inceputul unui chunk.
 - O tag - indica ca un token nu apartine unui chunk (outside).



ML cu modele secventiale

- **Atribute ale entitatilor**

geo = Geographical Entity

org = Organization

per = Person

gpe = Geopolitical Entity

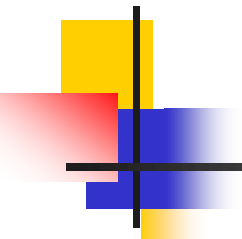
tim = Time indicator

art = Artifact

eve = Event

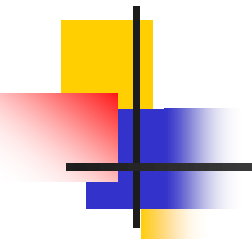
nat = Natural Phenomenon

A Sentence #	A Word	A POS	A Tag
Sentence: 1	Thousands	NNS	0
	of	IN	0
	demonstrators	NNS	0
	have	VBP	0
	marched	VBN	0
	through	IN	0
	London	NNP	B-geo
	to	TO	0
	protest	VB	0
	the	DT	0
	war	NN	0
	in	IN	0
	Iraq	NNP	B-geo
	and	CC	0
	demand	VB	0
	the	DT	0
	withdrawal	NN	0
	of	IN	0
	British	JJ	B-gpe
	troops	NNS	0
	from	IN	0
	that	DT	0
	country	NN	0
	.	.	0
Sentence: 2	Families	NNS	0
	of	IN	0
	soldiers	NNS	0



IR cu IE

- Anumite sisteme combina Information retrieval cu information extraction



5 POS tagging

- **Part-of-speech tagging** – un cuvânt sau un grup de cuvinte sunt etichetate cu o categorie gramaticală în funcție de semnificație și de context (cuvinte adiacente, fraza, paragraf).
- Substantiv, adj, etc
- Gen, acord, forme inflexionate etc
- Engleza – de la 50 la 150 PoS
- Metode: HMM, Logistic Regression, NN

POS tagging

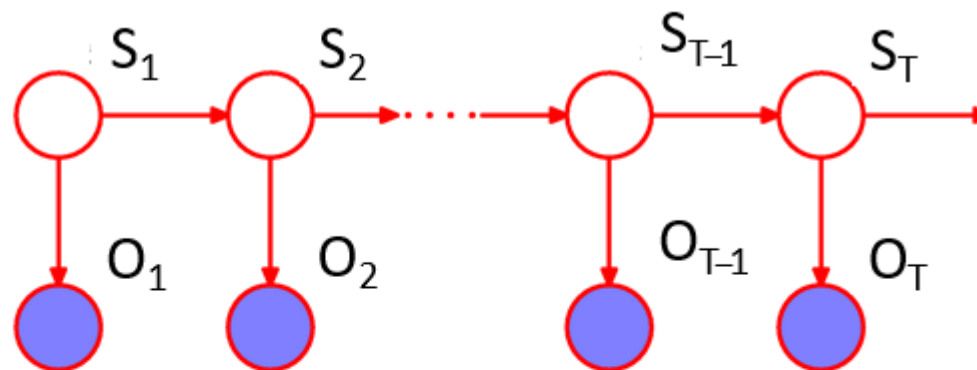
Penn

Tree bank

Tag	Word	Description	Tag	Word	Description
CC	<i>and</i>	Coordinating conjunction	PRPS	<i>your</i>	Possessive pronoun
CD	<i>three</i>	Cardinal number	RB	<i>quickly</i>	Adverb
DT	<i>the</i>	Determiner	RBR	<i>quicker</i>	Adverb, comparative
EX	<i>there</i>	Existential there	RBS	<i>quickest</i>	Adverb, superlative
FW	<i>per se</i>	Foreign word	RP	<i>off</i>	Particle
IN	<i>of</i>	Preposition	SYM	<i>+</i>	Symbol
JJ	<i>purple</i>	Adjective	TO	<i>to</i>	to
JJR	<i>better</i>	Adjective, comparative	UH	<i>eureka</i>	Interjection
JJS	<i>best</i>	Adjective, superlative	VB	<i>talk</i>	Verb, base form
LS	<i>I</i>	List item marker	VBD	<i>talked</i>	Verb, past tense
MD	<i>should</i>	Modal	VBG	<i>talking</i>	Verb, gerund
NN	<i>kitten</i>	Noun, singular or mass	VCN	<i>talked</i>	Verb, past participle
NNS	<i>kittens</i>	Noun, plural	VBP	<i>talk</i>	Verb, non-3rd-sing
NNP	<i>Ali</i>	Proper noun, singular	VBZ	<i>talks</i>	Verb, 3rd-sing
NNPS	<i>Fords</i>	Proper noun, plural	WDT	<i>which</i>	Wh-determiner
PDT	<i>all</i>	Predeterminer	WP	<i>who</i>	Wh-pronoun
POS	<i>'s</i>	Possessive ending	WPS	<i>whose</i>	Possessive wh-pronoun
PRP	<i>you</i>	Personal pronoun	WRB	<i>where</i>	Wh-adverb
\$	<i>\$</i>	Dollar sign	#	<i>#</i>	Pound sign
"	<i>'</i>	Left quote	"	<i>'</i>	Right quote
(	<i>[</i>	Left parenthesis	)	<i>]</i>	Right parenthesis
,	<i>,</i>	Comma	.	<i>!</i>	Sentence end
:	<i>;</i>	Mid-sentence punctuation			

POS tagging

HMM - Hidden Markov Models



Observation space

$$O_t \in \{x_1, x_2, \dots, x_K\}$$

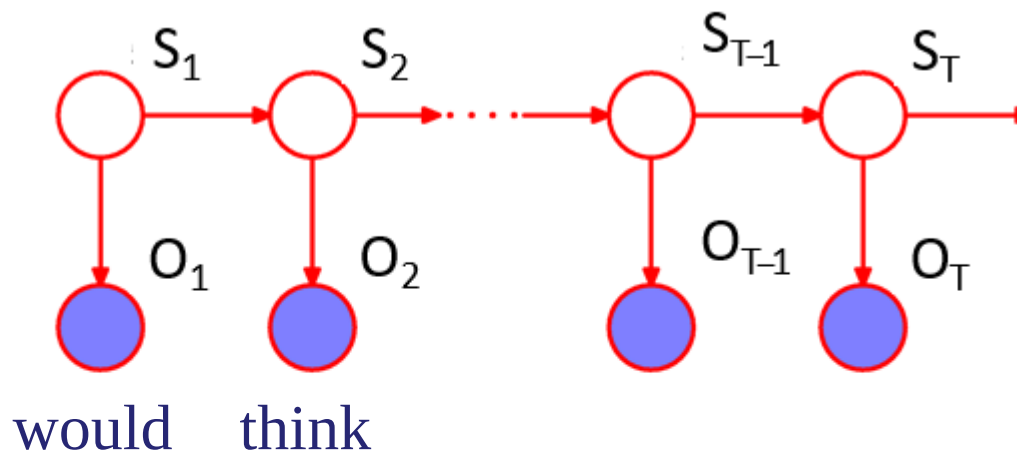
Hidden states

$$S_t \in \{S_1, \dots, S_M\}$$

POS tagging

$$P(C_t|C_{t-1})$$

$$P(W_t|C_t)$$

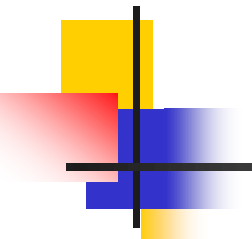


Observation space

$$O_t \in \{x_1, x_2, \dots, x_K\}$$

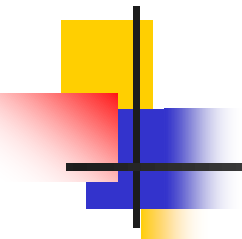
Hidden states

$$S_t \in \{S_1, \dots, S_M\}$$



6 Word Embedding

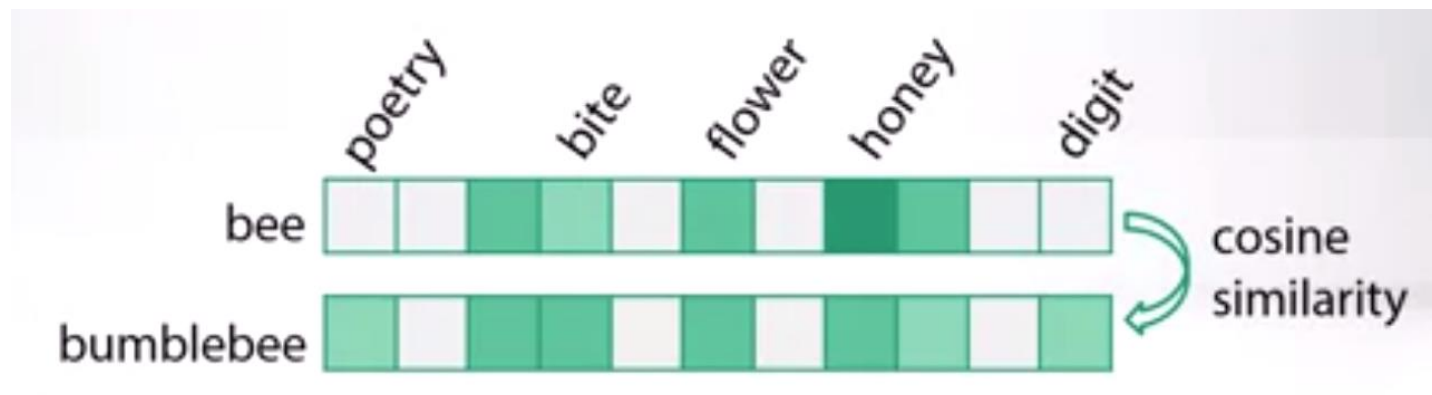
- Reprezentare a vocabularului care surprinde contextul cuvintelor – cuvinte care sunt legate/similare dpv sintactic (mic, frumos), sematic (pisica, pisoi), topica (insorit, ploios), sentiment (interesant, plictisitor)
- **Asocierea semantica** – co-aparitie in acelasi context (albina si miere)
- **Co-aparitie de ordinul I** (relatii sintagmatice – albina miere)
- **Co-aparitei de ordinul II** (relatii paradigmatiche sau de similaritate – albina bondar)
- Am putea reprezenta ca one-hot-vector sau ca n-grame de cuvinte – vectori f mari si rari



Word Embedding

- Am dori vectori de dimensiuni mai mici (de ex cateva sute), care pot fi determinati automat

Matrice de co-aparitie



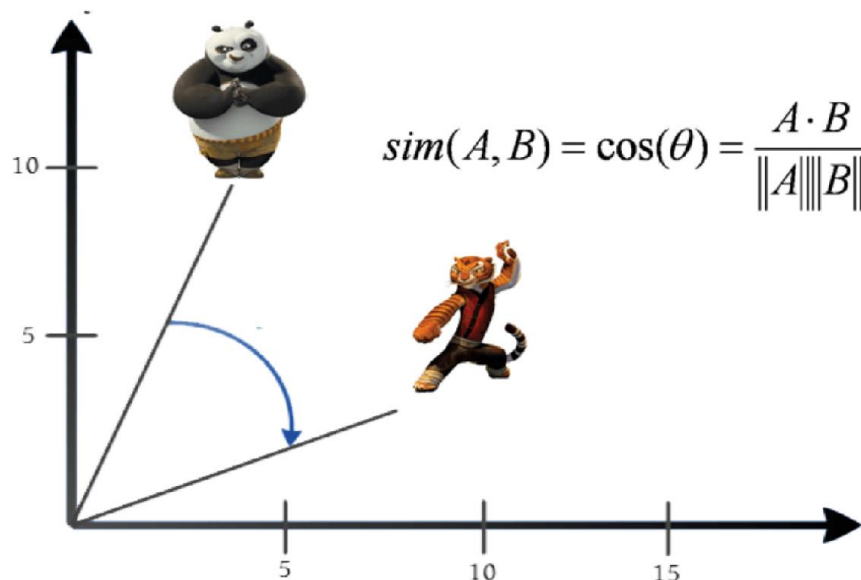
- Cum se calculeaza valorile din vector
- Pointwise Mutual Information

$$\text{PMI}(a,b) = \log (p(a,b) / p(a) p(b))$$

- Calcul dificil

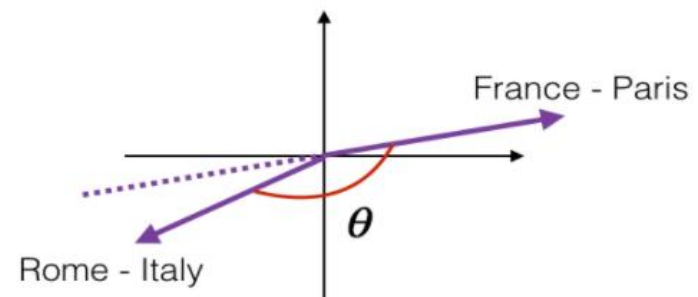
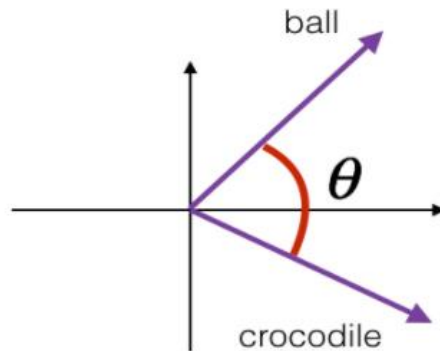
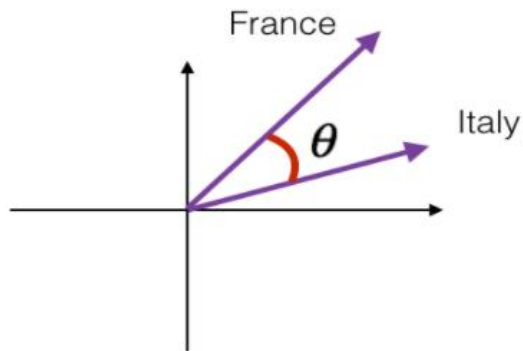
Cosine similarity

Cosine Similarity



Se determina numarul de aparitii ale cuvintelor
intr-o fereastră glisanta in text
Se calculeaza similaritatea cosina

Cosine similarity

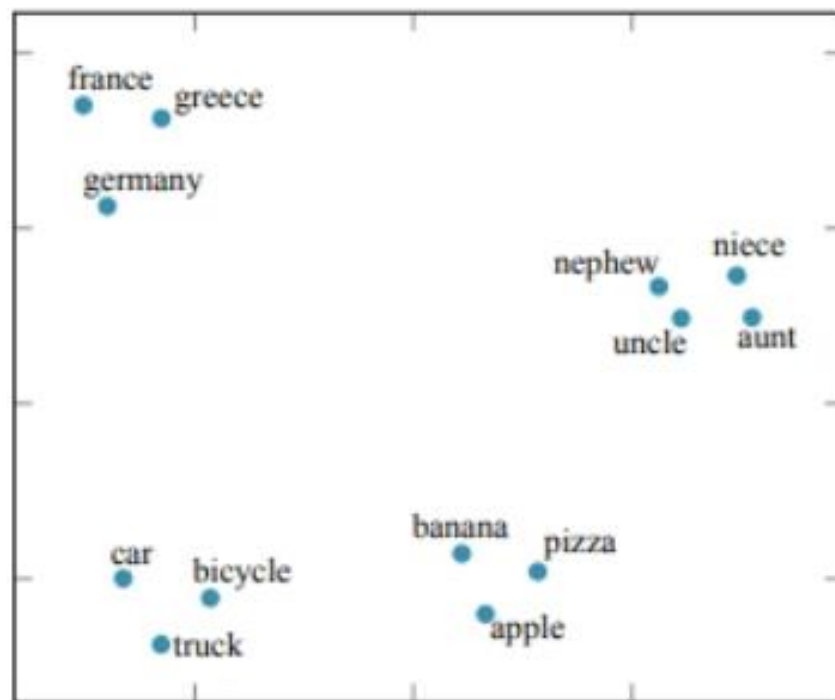


Word Embedding

- Am dori vectori de dimensiuni mai mici (de ex cateva sute), care pot fi invatati automat

“aardvark” = $[-0.7, +0.2, -3.2, \dots]$
“abacus” = $[+0.5, +0.9, -1.3, \dots]$
...
“zyzzyva” = $[-0.1, +0.8, -0.4, \dots]$.

Vectori word embedding calculati
cu alg. GloVe antrenat pe
6 miliarde de cuvinte din texte
(sursa AIMA)



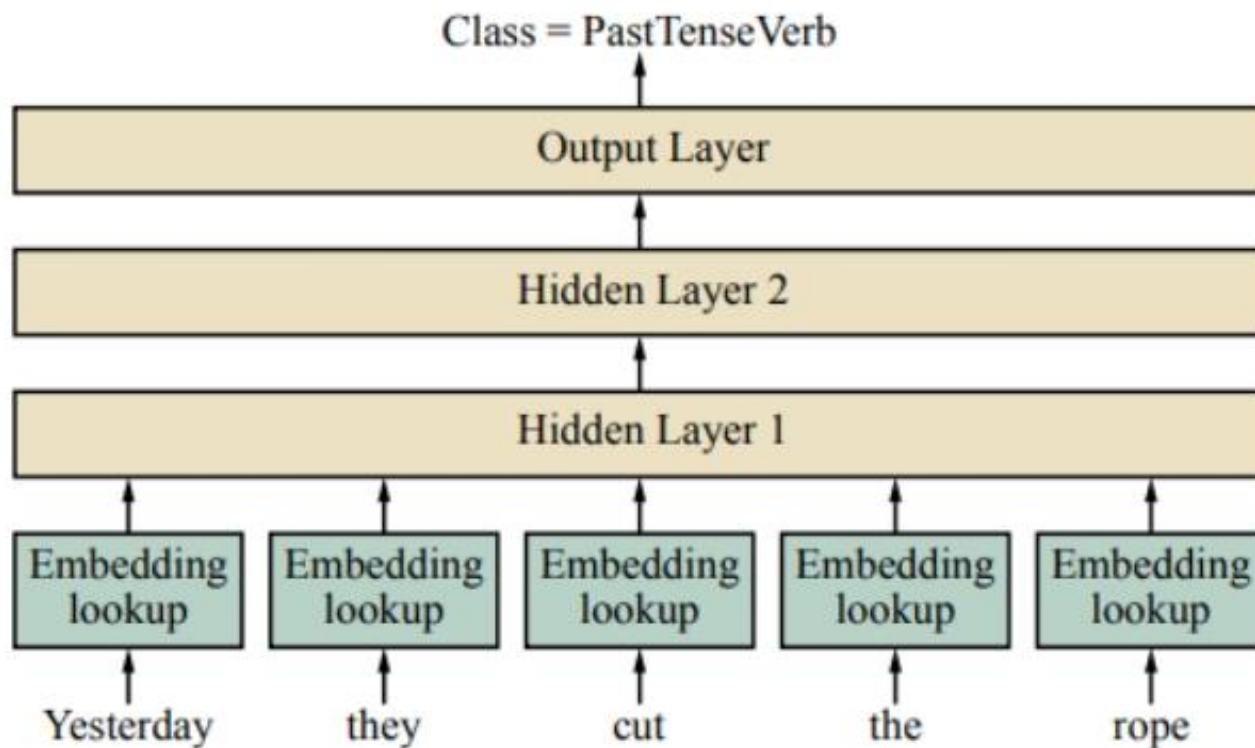
Word Embedding

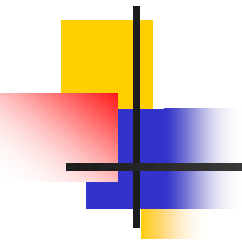
A	B	C	$D = C + (B - A)$	Relationship
Athens	Greece	Oslo	Norway	<i>Capital</i>
Astana	Kazakhstan	Harare	Zimbabwe	<i>Capital</i>
Angola	kwanza	Iran	rial	<i>Currency</i>
copper	Cu	gold	Au	<i>Atomic Symbol</i>
Microsoft	Windows	Google	Android	<i>Operating System</i>
New York	New York Times	Baltimore	Baltimore Sun	<i>Newspaper</i>
Berlusconi	Silvio	Obama	Barack	<i>First name</i>
Switzerland	Swiss	Cambodia	Cambodian	<i>Nationality</i>
Einstein	scientist	Picasso	painter	<i>Occupation</i>
brother	sister	grandson	granddaughter	<i>Family Relation</i>
Chicago	Illinois	Stockton	California	<i>State</i>
possibly	impossibly	ethical	unethical	<i>Negative</i>
mouse	mice	dollar	dollars	<i>Plural</i>
easy	easiest	lucky	luckiest	<i>Superlative</i>
walking	walked	swimming	swam	<i>Past tense</i>

Exista dictionare de vectori preantrenati: WORD2VEC, GloVe(Global Vectors), FASTTEXT

Word Embedding

Retea neurala pt word embedding si POS tagging





Slide-uri pct 3 bazate pe

CS 276 / LING 286: Information Retrieval and Web Search